CS229 机器学习讲义

2026 春季 · 中文译本

Tengyu Ma · Andrew Ng

原文更新：2026 年 8 月 23 日

目录

译本说明 **1**

第一部分　监督学习 **2**

第 1 章　线性回归 **4**
1.1 LMS 算法 . . . 5
1.2 正规方程 . . . 8
1.2.1 矩阵求导 . . . 8
1.2.2 再谈最小二乘 . . . 9
1.3 概率解释 . . . 10
1.4 局部加权线性回归 . . . 12

第 2 章　分类与逻辑回归 **14**
2.1 逻辑回归 . . . 14
2.2 题外话：感知机学习算法 . . . 17
2.3 多分类 . . . 18
2.4 最大化 $\ell(\theta)$ 的另一种算法 . . . 19

第 3 章　广义线性模型 **21**
3.1 指数族 . . . 21
3.2 构造 GLM . . . 22
3.2.1 普通最小二乘 . . . 23
3.2.2 逻辑回归 . . . 23

第 4 章　生成式学习算法 **24**
4.1 高斯判别分析 . . . 24
4.1.1 多元正态分布 . . . 25
4.1.2 高斯判别分析模型 . . . 27
4.1.3 讨论：GDA 与逻辑回归 . . . 28
4.2 朴素贝叶斯（选读） . . . 29
4.2.1 拉普拉斯平滑 . . . 31
4.2.2 文本分类的事件模型 . . . 33

第 5 章　核方法 **35**
5.1 特征映射 . . . 35
5.2 带特征的 LMS（最小均方）算法 . . . 35
5.3 使用核技巧的 LMS . . . 36
5.4 核的性质 . . . 38

第 6 章　支持向量机 **42**
6.1 间隔：直观理解 . . . 42
6.2 记号 . . . 42

6.3 函数间隔与几何间隔 . . . 43
6.4 最优间隔分类器 . . . 45
6.5 拉格朗日对偶性 . . . 46
6.6 最优间隔分类器：对偶形式 . . . 48
6.7 正则化与不可分情形 . . . 51
6.8 SMO 算法 . . . 52
6.8.1 坐标上升 . . . 53
6.8.2 SMO . . . 53

第二部分 深度学习 **56**

第 7 章 深度学习 **57**
7.1 使用非线性模型的监督学习 . . . 57
7.2 神经网络 . . . 59
7.3 现代神经网络中的模块 . . . 65
7.4 反向传播 . . . 69
7.4.1 偏导数预备知识 . . . 69
7.4.2 反向传播的总体策略 . . . 71
7.4.3 基本模块的反向函数 . . . 73
7.4.4 MLP 的反向传播 . . . 74
7.5 对训练样本进行向量化 . . . 75

第三部分 泛化与正则化 **77**

第 8 章 泛化 **78**
8.1 偏差—方差权衡 . . . 78
8.1.1 数学分解（针对回归） . . . 82
8.2 双下降现象 . . . 83
8.3 样本复杂度界 . . . 86
8.3.1 预备知识 . . . 86
8.3.2 有限假设函数类 . . . 87
8.3.3 无限假设函数类 . . . 88

第 9 章 正则化与模型选择 **92**
9.1 正则化 . . . 92
9.2 隐式正则化效应 . . . 93
9.3 通过交叉验证选择模型 . . . 94
9.4 贝叶斯统计与正则化 . . . 96

第四部分 无监督学习 **98**

第 10 章 聚类与 k-means 算法 **99**

第 11 章 EM 算法 **101**
11.1 高斯混合模型的 EM 算法 . . . 101
11.2 Jensen 不等式 . . . 103
11.3 一般的 EM 算法 . . . 104
11.3.1 ELBO 的其他解释 . . . 108
11.4 再看高斯混合模型 . . . 108
11.5 变分推断与变分自编码器 . . . 110

第 12 章 主成分分析 **113**

第 13 章　独立成分分析 **118**
13.1 ICA 的不确定性 . . . 118
13.2 密度与线性变换 . . . 119
13.3 ICA 算法 . . . 120

第五部分　生成模型与基础模型 **122**

第 14 章　扩散模型 **123**
14.1　扩散过程 . . . 123
14.2　反向过程的参数化 . . . 125
14.3　通过最大化 ELBO 训练扩散模型 . . . 126
14.4　反向扩散的连续时间视角 . . . 129

第 15 章　基础模型概述 **131**
15.1　表示学习中的线性探测与微调 . . . 132
15.2　低秩适配（LoRA） . . . 133

第 16 章　表示学习 **135**
16.1 监督预训练 . . . 135
16.2 对比学习 . . . 135
16.3 语义检索 . . . 136
16.4 检索增强生成 . . . 137

第 17 章　大语言模型 **139**
17.1 词元化 . . . 139
17.2 自回归模型与下一个词元预测损失 . . . 140
17.3 Transformer 架构 . . . 142
17.4 注意力的变体 . . . 146
17.5 混合专家层 . . . 147
17.6 上下文学习 . . . 148
17.7 零样本学习 / 提示 . . . 149
17.8 监督微调（SFT） . . . 149

第 18 章　大语言模型中的推理 **151**
18.1 思维链 . . . 151
18.2 采用长思维链推理的 RLVR . . . 151

第六部分 强化学习与控制 **155**

第 19 章　强化学习 **156**
19.1 马尔可夫决策过程 . . . 156
19.2 价值迭代与策略迭代 . . . 158
19.3 学习 MDP 的模型 . . . 159
19.4 连续状态 MDP . . . 160
19.4.1 离散化 . . . 160
19.4.2 价值函数近似 . . . 163
19.5 策略迭代与价值迭代的联系（选读） . . . 166

第 20 章　LQR、DDP 与 LQG **167**
20.1 有限时域 MDP . . . 167
20.2 线性二次调节（LQR） . . . 169
20.3 从非线性动力学到 LQR . . . 171
20.3.1 动力学的线性化 . . . 172
20.3.2 微分动态规划（DDP） . . . 172

20.4 线性二次高斯（LQG） 173

第 21 章　策略梯度及其变体 **176**
21.1 REINFORCE 176
21.2 PPO 179

附录 A　高斯分布与 KL 散度的基本结论 **182**
A.1 高斯分布与 KL 散度的基本恒等式 182

参考文献 **185**

译本说明

本书译自 Tengyu Ma 与 Andrew Ng 的 *CS229 Lecture Notes*，原文日期为 **2026 年 8 月 23 日**，正文页眉标注 **CS229 Spring 2026**。

译文保留原文的章节体系、公式与图表编号，并在各章标明原文印刷页码，便于与英文讲义对照。中文重排后的页码不同于英文原版。图表保留原图，图注及正文说明译为中文；参考文献保留原始书目信息，便于检索。少量明显的原文笔误或符号疑点以“译注”明确说明。

英文原文：https://cs229.stanford.edu/main_notes.pdf。本译本为辅助学习的非官方中文译本。

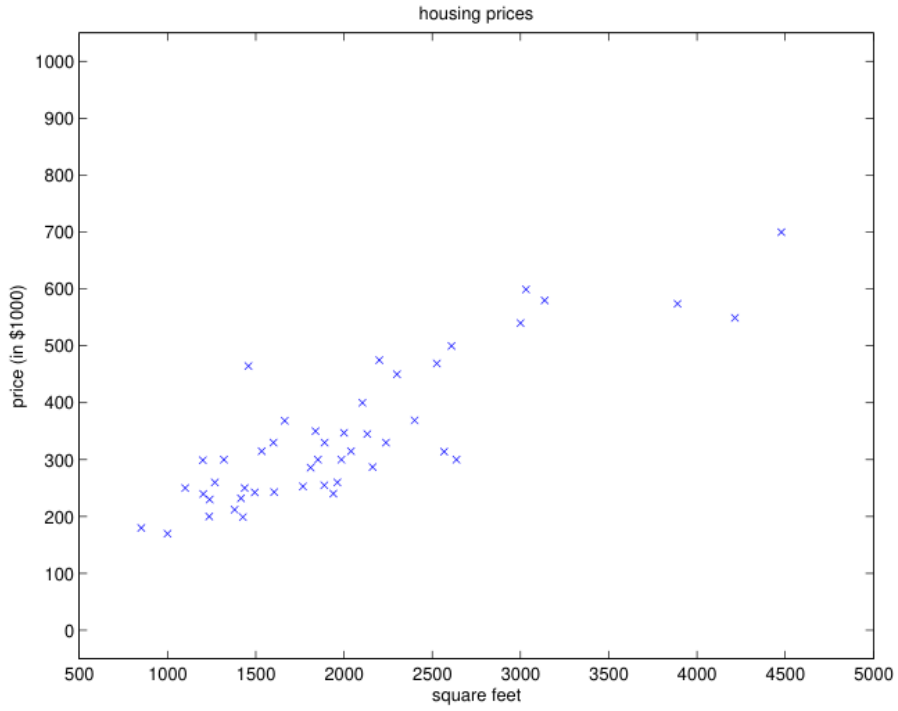
第一部分　监督学习

原文印刷页码：6–8。

先来看几个监督学习问题的例子。假设有一个数据集，记录了美国俄勒冈州波特兰市 47 套房屋的居住面积与价格：

居住面积（平方英尺）	价格（千美元）
2104	400
1600	330
2400	369
1416	232
3000	540
……	……

把这些数据绘制出来，得到下图：



房屋价格数据：横轴为面积（平方英尺），纵轴为价格（千美元）。

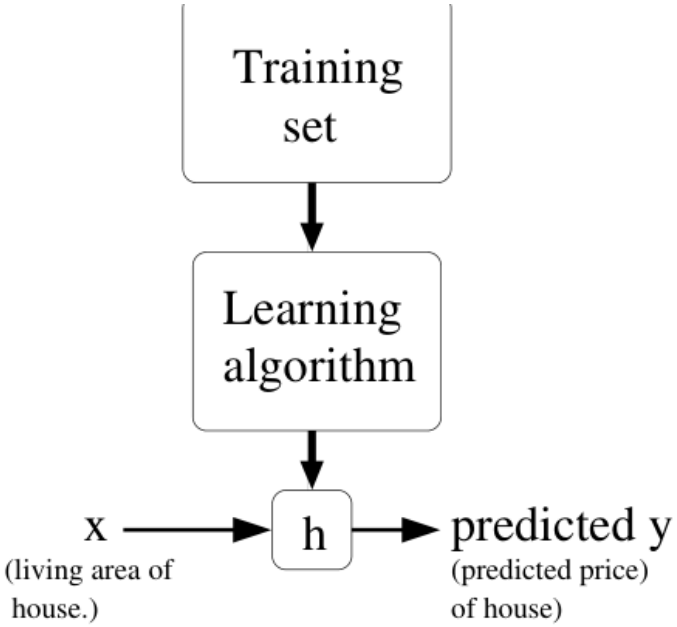
有了这样的数据，我们怎样学会根据居住面积来预测波特兰市其他房屋的价格呢？

先约定后面要用到的符号。用 $x^{(i)}$ 表示 “输入” 变量，在这个例子中就是居住面积，也称为**输入特征**；用 $y^{(i)}$ 表示希望预测的 “输出” 变量或**目标变量**，在这里就是价格。一对 $(x^{(i)}, y^{(i)})$ 称为**一个训练样本**。用于学习的数据集，即由 n 个训练样本组成的集合

$$\{(x^{(i)}, y^{(i)});\ i = 1, \dots, n\},$$

称为**训练集**。注意，上标 “(i)” 只是训练集中的样本编号，与乘方无关。还用 $\mathcal{X}$ 表示输入值空间，用 $\mathcal{Y}$ 表示输出值空间。在这个例子中，$\mathcal{X} = \mathcal{Y} = \mathbb{R}$。

更正式地说，监督学习的目标是：给定一个训练集，学习函数 $h : \mathcal{X} \mapsto \mathcal{Y}$，使 $h(x)$ 能够 “很好地” 预测相应的 y 值。由于历史原因，函数 h 被称为**假设函数**。整个过程可以用下图表示：



监督学习流程：训练集输入学习算法，得到假设函数 h；h 再把输入 x（房屋居住面积）映射为预测的 y（房屋价格）。

如果要预测的目标变量是连续的，就像上面的房价例子，我们把这一学习问题称为**回归问题**。如果 y 只能取少量离散值，例如希望根据居住面积判断某个住宅是独栋房屋还是公寓，我们就把它称为**分类问题**。

第 1 章　线性回归

原文印刷页码：9–20。

为了让房价的例子更有意思，我们考虑一个稍丰富的数据集，其中还记录了每所房屋的卧室数量：

居住面积（平方英尺）	卧室数	价格（千美元）
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
……	……	……

这里的 x 是 $\mathbb{R}^2$ 中的二维向量。例如，$x_1^{(i)}$ 是训练集中第 i 所房屋的居住面积，$x_2^{(i)}$ 是它的卧室数量。（一般来说，在设计学习问题时，需要由你决定选用哪些特征。因此，如果你正在波特兰收集房屋数据，也可以决定加入其他特征，例如每所房屋是否有壁炉、浴室数量等等。后面还会进一步讨论**特征选择**，目前先把特征视为给定的。）

要进行监督学习，我们必须决定如何在计算机中表示函数 / 假设函数 h。作为最初的选择，假定我们决定用 x 的线性函数来近似 y：

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2.$$

这里的 θ_i 是参数（也称为**权重**），它们对从 $\mathcal{X}$ 映射到 $\mathcal{Y}$ 的线性函数空间进行参数化。在不会引起混淆时，我们省略 $h_\theta(x)$ 中的下标 θ，简写为 $h(x)$。为简化记号，还约定 $x_0 = 1$（这是**截距项**），于是

$$h(x) = \sum_{i=0}^{d} \theta_i x_i = \theta^T x.$$

上式右侧将 θ 和 x 都视为向量，d 是输入变量的数量（不计 x_0）。

给定训练集，如何选择或学习参数 θ 呢？一种合理的方法似乎是让 $h(x)$ 尽量接近 y，至少在已有的训练样本上如此。为使这一点形式化，我们定义一个函数，衡量每组 θ 下，各个 $h(x^{(i)})$ 与对应的 $y^{(i)}$ 有多接近。定义**成本函数**：

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2.$$

如果你学过线性回归，就可能认出，这是熟悉的**最小二乘成本函数**，由此得到普通最小二乘回归模型。无论以前是否见过，我们都继续往下看，最终将说明它是一大类算法的特例。

1.1 LMS 算法

我们希望选择 θ 使 $J(\theta)$ 最小。为此，使用一种搜索算法：从对 θ 的某个 “初始猜测” 出发，反复修改 θ，让 $J(\theta)$ 变小，希望最终收敛到使其最小的 θ。具体来说，考虑**梯度下降**算法，它从某个初始 θ 出发，反复执行更新：

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$

（对所有 $j = 0, \ldots, d$ **同时**执行这一更新。）这里的 α 称为**学习率**。这个算法十分自然：反复朝 J 下降最快的方向迈出一步。

要实现算法，必须算出右侧的偏导数。先考虑只有一个训练样本 (x, y) 的情况，此时可省略 J 定义中的求和号。有

$$\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_\theta(x) - y)^2 \\
&= 2 \cdot \frac{1}{2} (h_\theta(x) - y) \frac{\partial}{\partial \theta_j} (h_\theta(x) - y) \\
&= (h_\theta(x) - y) \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{d} \theta_i x_i - y \right) \\
&= (h_\theta(x) - y) x_j.
\end{aligned}$$

对于单个训练样本，这给出更新规则：[1]

$$\theta_j := \theta_j + \alpha \big(y^{(i)} - h_\theta(x^{(i)}) \big) x_j^{(i)}.$$

这一规则称为 **LMS 更新规则**（LMS 是 “least mean squares”，即 “最小均方” 的缩写），也称 **Widrow–Hoff 学习规则**。它有一些自然且直观的性质。例如，更新幅度与误差项 $(y^{(i)} - h_\theta(x^{(i)}))$ 成正比。因此，遇到某个样本时，如果预测值几乎等于实际的 $y^{(i)}$，就几乎不必改变参数；反之，如果预测值 $h_\theta(x^{(i)})$ 误差很大（即离 $y^{(i)}$ 很远），参数的改动也就较大。

刚才推导 LMS 规则时只有一个训练样本。对包含多个样本的训练集，有两种修改方法。第一种是使用以下算法：

重复执行，直至收敛：对每个 j 同时更新

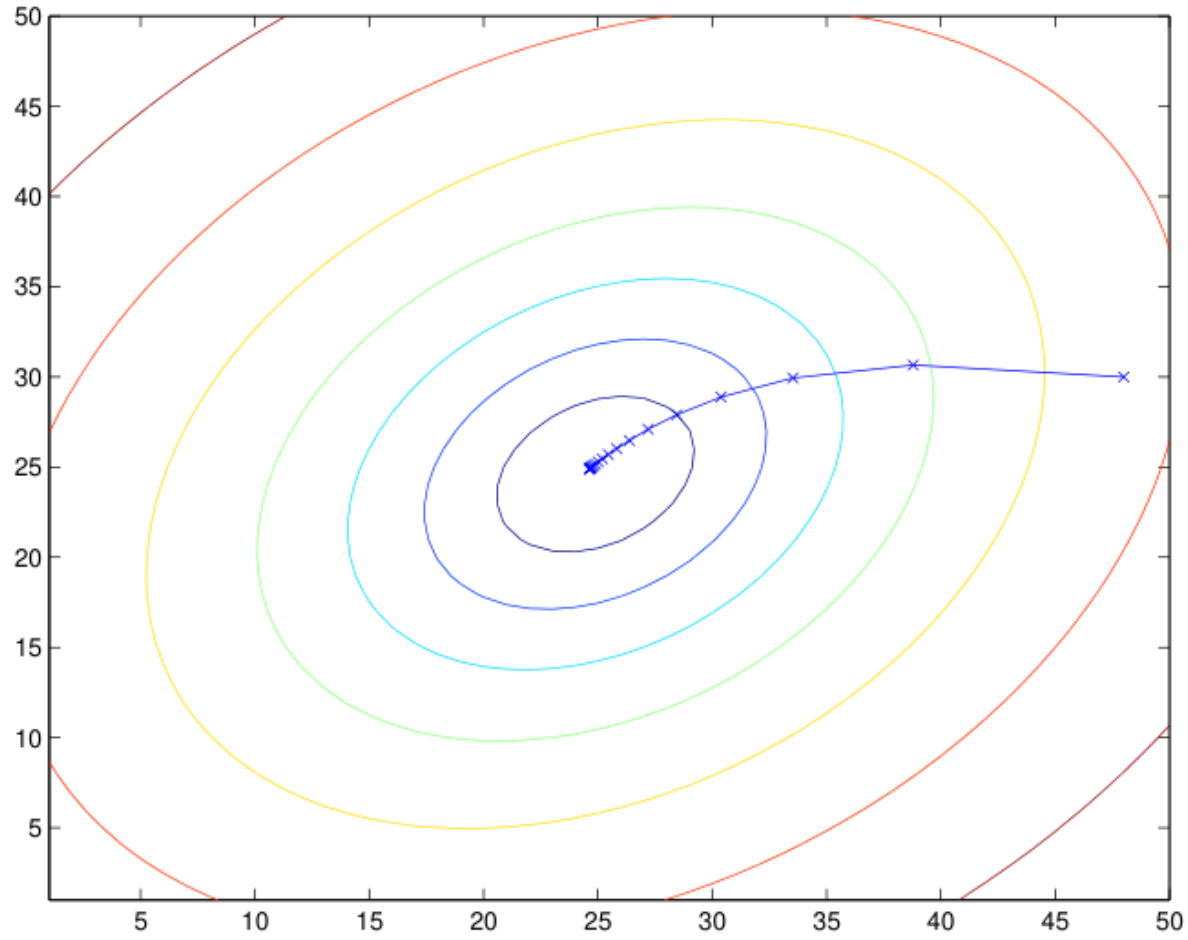
$$\theta_j := \theta_j + \alpha \sum_{i=1}^{n} \big(y^{(i)} - h_\theta(x^{(i)}) \big) x_j^{(i)}. \tag{1.1}$$

将各坐标的更新合并成向量 θ 的更新，可以把式 (1.1) 更简洁地写成

[1] 记号 “$a := b$” 表示计算机程序中的赋值操作，即把变量 a 设为 b 的值；换句话说，用 b 的值覆盖 a。而 “$a = b$” 是在陈述一个事实，即 a 的值等于 b 的值。

$$\theta := \theta + \alpha \sum_{i=1}^{n} \left(y^{(i)} - h_\theta(x^{(i)})\right) x^{(i)}.$$

读者很容易验证，上述更新规则中求和的量就是 $\partial J(\theta)/\partial \theta_j$（这里采用 J 最初的定义）。因此，这就是对原始成本函数 J 进行梯度下降。[2] 该方法每一步都会查看整个训练集中的全部样本，称为**批量梯度下降**。注意，一般的梯度下降可能受到局部极小值影响，但这里线性回归的优化问题只有一个全局最优点，没有其他局部最优点；所以，只要学习率 α 不太大，梯度下降总会收敛到全局最小值。事实上，J 是凸二次函数。[3] 下图展示了用梯度下降最小化二次函数的例子。



二次函数的等高线与梯度下降轨迹；初始点为 (48,30)，叉号表示 依次经过的参数值。

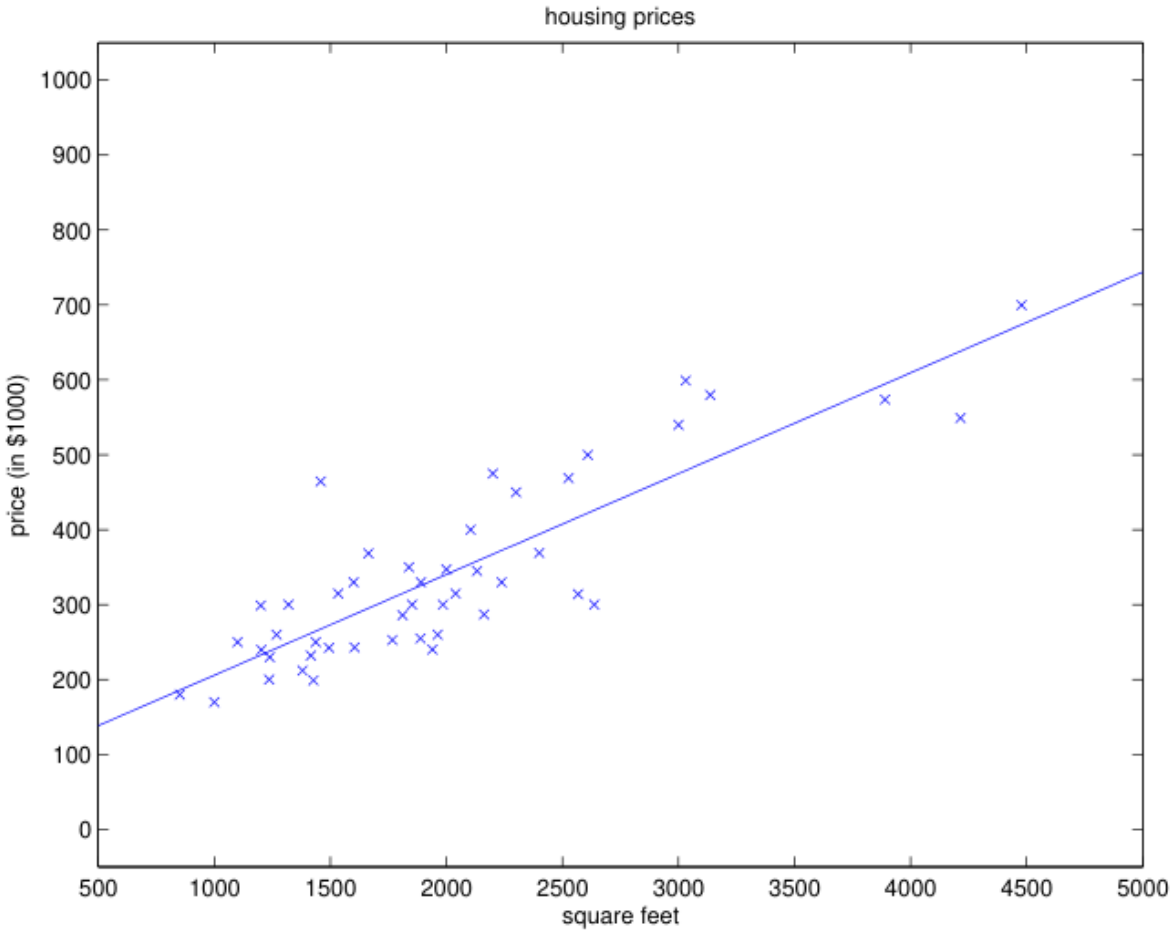
图中的椭圆是二次函数的等高线。图中也画出了从 $(48, 30)$ 初始化的梯度下降轨迹；用直线连接起来的叉号标记了梯度下降依次经过的 θ 值。

在前面的数据集上运行批量梯度下降来拟合 θ，学习由居住面积预测房价的函数，可得 $\theta_0 = 71.27, \theta_1 = 0.1345$。把 $h_\theta(x)$ 作为 x（面积）的函数，与训练数据画在一起，就得到下图：

若把卧室数也加入输入特征，则得到 $\theta_0 = 89.60, \theta_1 = 0.1392, \theta_2 = -8.738$。

[2]译注：原文此处将求和项称为偏导数；严格说，坐标形式的求和项是该偏导数的负数，向量形式则是负梯度。上面的更新公式本身正确。

[3]译注：原文关于最优点唯一的表述隐含设计矩阵满列秩。一般情况下，凸性保证局部最小值也是全局最小值，但参数解不一定唯一。



房价与居住面积的训练数据及线性回归拟合；横轴为平方英尺，纵轴为千美元。

上面的结果由批量梯度下降得到。还有一种效果很好的替代算法。考虑以下过程：

循环：对 $i = 1, \ldots, n$ 依次执行以下操作，对每个 j 同时更新：

$$\theta_j := \theta_j + \alpha\big(y^{(i)} - h_\theta(x^{(i)})\big)x_j^{(i)}. \tag{1.2}$$

将坐标更新合并为向量更新，式 (1.2) 可以更简洁地写为

$$\theta := \theta + \alpha\big(y^{(i)} - h_\theta(x^{(i)})\big)x^{(i)}.$$

这个算法反复遍历训练集，每遇到一个训练样本，就只根据该样本误差的梯度来更新参数。它称为**随机梯度下降**（也称**增量梯度下降**）。批量梯度下降必须扫描整个训练集才能迈出一步，当 n 很大时，这个操作代价高昂；而随机梯度下降可以立即开始取得进展，并且每查看一个样本都继续前进。随机梯度下降通常比批量梯度下降更快地让 θ "接近" 最小值。（但要注意，它可能永远不会"收敛" 到最小值，参数 θ 会持续在 $J(\theta)$ 的最小值附近振荡；不过实践中，最小值附近的大多数值都能合理地近似真正的最小值。[4]）因此，尤其当训练集很大时，通常更倾向于采用随机梯度下降。

1.2 正规方程

梯度下降给出了最小化 J 的一种方法。现在讨论第二种方法：直接求出最小值，不使用迭代算法。具体做法是显式计算 J 对各个 θ_j 的导数，并令它们为零。为了不必写下大量代数式和满页的导数矩阵，先引入一些矩阵微积分记号。

1.2.1 矩阵求导

对于从 $n \times d$ 矩阵映射到实数的函数 $f : \mathbb{R}^{n\times d} \mapsto \mathbb{R}$，定义 f 对 A 的导数为：

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1d}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{n1}} & \cdots & \frac{\partial f}{\partial A_{nd}} \end{bmatrix}.$$

因此，梯度 $\nabla_A f(A)$ 本身是一个 $n \times d$ 矩阵，其第 (i, j) 个元素为 $\partial f/\partial A_{ij}$。例如，假设

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

是一个 2×2 矩阵，函数 $f : \mathbb{R}^{2\times 2} \mapsto \mathbb{R}$ 为

$$f(A) = \frac{3}{2}A_{11} + 5A_{12}^2 + A_{21}A_{22}.$$

这里 A_{ij} 表示矩阵 A 的第 (i, j) 个元素。于是

[4] 在算法运行过程中，让学习率 α 缓慢减小到零，也可以保证参数收敛到全局最小值，而不只是围绕最小值振荡。

$$\nabla_A f(A) = \begin{bmatrix} \frac{3}{2} & 10A_{12} \\ A_{22} & A_{21} \end{bmatrix}.$$

1.2.2 再谈最小二乘

有了矩阵求导工具，现在来求使 $J(\theta)$ 最小的 θ 的闭式解。首先把 J 改写成矩阵向量形式。

给定训练集，定义**设计矩阵** X 为一个 $n \times d$ 矩阵（如果包括截距项，实际上是 $n \times (d+1)$），它的每一行包含一个训练样本的输入值：

$$X = \begin{bmatrix} — (x^{(1)})^T — \\ — (x^{(2)})^T — \\ \vdots \\ — (x^{(n)})^T — \end{bmatrix}.$$

再令 $\vec{y}$ 为包含训练集中所有目标值的 n 维向量：

$$\vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}.$$

由于 $h_\theta(x^{(i)}) = (x^{(i)})^T\theta$，容易验证

$$\begin{aligned} X\theta - \vec{y} &= \begin{bmatrix} (x^{(1)})^T\theta \\ \vdots \\ (x^{(n)})^T\theta \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} \\ &= \begin{bmatrix} h_\theta(x^{(1)}) - y^{(1)} \\ \vdots \\ h_\theta(x^{(n)}) - y^{(n)} \end{bmatrix}. \end{aligned}$$

利用向量 z 满足 $z^T z = \sum_i z_i^2$ 这一事实，得到

$$\frac{1}{2}(X\theta - \vec{y})^T(X\theta - \vec{y}) = \frac{1}{2}\sum_{i=1}^{n}(h_\theta(x^{(i)}) - y^{(i)})^2 = J(\theta).$$

最后，为最小化 J，求其关于 θ 的导数：

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta \frac{1}{2}(X\theta - \vec{y})^T(X\theta - \vec{y}) \\ &= \frac{1}{2}\nabla_\theta\left[(X\theta)^T X\theta - (X\theta)^T\vec{y} - \vec{y}^T(X\theta) + \vec{y}^T\vec{y}\right] \\ &= \frac{1}{2}\nabla_\theta\left[\theta^T(X^TX)\theta - \vec{y}^T(X\theta) - \vec{y}^T(X\theta)\right] \\ &= \frac{1}{2}\nabla_\theta\left[\theta^T(X^TX)\theta - 2(X^T\vec{y})^T\theta\right] \\ &= \frac{1}{2}\left[2X^TX\theta - 2X^T\vec{y}\right] \\ &= X^TX\theta - X^T\vec{y}.\end{aligned}$$

第三步使用了 $a^Tb = b^Ta$；第五步使用了 $\nabla_x b^Tx = b$，以及对称矩阵 A 满足 $\nabla_x x^TAx = 2Ax$（详情见《线性代数复习与参考》第 4.3 节）。为了最小化 J，令导数为零，得到**正规方程**：

$$X^TX\theta = X^T\vec{y}.$$

因此，使 $J(\theta)$ 最小的 θ 的闭式解为

$$\theta = (X^TX)^{-1}X^T\vec{y}.$$

[5]

1.3 概率解释

面对回归问题，为什么线性回归会是合理的选择，尤其是为什么最小二乘成本函数 J 合理？本节将给出一组概率假设，在这些假设下，最小二乘回归是一个很自然的算法。

假设目标变量和输入通过以下方程联系起来：

$$y^{(i)} = \theta^Tx^{(i)} + \epsilon^{(i)},$$

其中 $\epsilon^{(i)}$ 是误差项，包含未建模的影响（例如，某些与房价预测密切相关的特征没有纳入回归），或随机噪声。进一步假设 $\epsilon^{(i)}$ **独立同分布**（IID），服从均值为零、方差为 σ^2 的高斯分布（也称正态分布）。记为 $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$，即其密度为

$$p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right).$$

这意味着

$$p(y^{(i)} \mid x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(y^{(i)} - \theta^Tx^{(i)})^2}{2\sigma^2}\right).$$

记号 $p(y^{(i)} \mid x^{(i)}; \theta)$ 表示给定 $x^{(i)}$、由 θ 参数化的 $y^{(i)}$ 分布。注意，不应把 θ 放进条件中，写成

[5] 上一步隐含假设 X^TX 可逆。求逆前可以先检查这一点。如果线性无关的样本数量小于特征数量，或特征并非线性无关，则 X^TX 不可逆。即使如此，也可以用其他技术“修复”这种情况；为简洁起见，这里略去。

$p(y^{(i)} \mid x^{(i)}, \theta)$，因为 θ 不是随机变量。也可以写成 $y^{(i)} \mid x^{(i)}; \theta \sim \mathcal{N}(\theta^T x^{(i)}, \sigma^2)$。

给定 X（包含所有 $x^{(i)}$ 的设计矩阵）和 θ，各个 $y^{(i)}$ 的分布是什么？数据的概率由 $p(\vec{y} \mid X; \theta)$ 给出。通常将这个量看作在固定 θ 下，关于 $\vec{y}$（可能也关于 X）的函数。若明确把它看作 θ 的函数，则称为**似然函数**：

$$L(\theta) = L(\theta; X, \vec{y}) = p(\vec{y} \mid X; \theta).$$

根据 $\epsilon^{(i)}$ 的独立性假设（因此给定各个 $x^{(i)}$ 后，$y^{(i)}$ 也相互独立），还可以写成

$$\begin{aligned} L(\theta) &= \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}; \theta) \\ &= \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right). \end{aligned}$$

给定这个联系 $y^{(i)}$ 与 $x^{(i)}$ 的概率模型，怎样合理地选择参数 θ 的最佳猜测？**最大似然原则**要求选择 θ，使观察到这些数据的概率尽可能高，即选择使 $L(\theta)$ 最大的 θ。

不直接最大化 $L(\theta)$，而最大化它的任何严格递增函数，也能得到相同结果。尤其是，最大化**对数似然** $\ell(\theta)$ 会使推导更简单：

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \log \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\ &= \sum_{i=1}^{n} \log\left[\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)\right] \\ &= n \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{\sigma^2} \cdot \frac{1}{2} \sum_{i=1}^{n} (y^{(i)} - \theta^T x^{(i)})^2. \end{aligned}$$

所以，最大化 $\ell(\theta)$ 与最小化下式得到相同答案：

$$\frac{1}{2} \sum_{i=1}^{n} (y^{(i)} - \theta^T x^{(i)})^2.$$

这正是最初的最小二乘成本函数 $J(\theta)$。

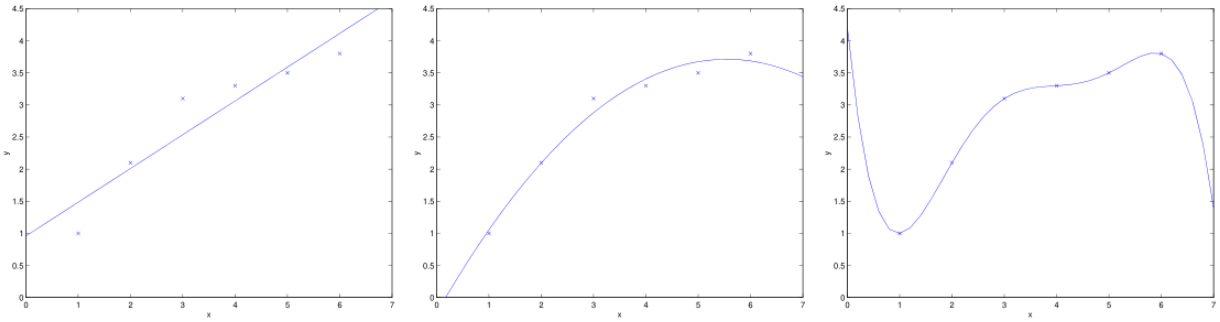
因此，在前面对数据所作的概率假设下，最小二乘回归对应于求 θ 的最大似然估计。这是一组使最小二乘回归显得十分自然的假设：它只是在做最大似然估计。（不过，最小二乘要成为合理、有效的方法，并不一定需要这些概率假设；还可能——而且确实——存在其他自然的假设来说明它的合理性。）

还要注意，前面最终选出的 θ 不依赖 σ^2 的值，即使 σ^2 未知，我们也会得到同样的结果。后面讨论指数族和广义线性模型时，还会用到这一事实。

1.4 局部加权线性回归

考虑由 $x \in \mathbb{R}$ 预测 y。下图最左侧显示了在数据集上拟合 $y = \theta_0 + \theta_1 x$ 的结果。数据并不真正落在直线上，因此拟合不太好。

如果增加一个特征 x^2，拟合 $y = \theta_0 + \theta_1 x + \theta_2 x^2$，对数据的拟合就会稍好一些（见中图）。



从左至右：线性拟合、二次多项式拟合和五次多项式拟合，分别展示欠拟合、较合理的拟合与过拟合。

直觉上似乎特征越多越好，但增加太多特征也有危险：最右侧是拟合五次多项式 $y = \sum_{j=0}^{5} \theta_j x^j$ 的结果。尽管拟合曲线完美穿过数据，我们也不认为它能很好地预测不同居住面积 x 对应的房价 y。暂不正式定义这些术语，我们把左图称为**欠拟合**的例子：数据明显具有模型没有捕捉到的结构；右图则是**过拟合**的例子。（后面讲学习理论时，将形式化其中一些概念，并更严谨地定义一个假设函数好坏的含义。）

如前所述，也如上例所示，特征的选择对学习算法取得良好性能很重要。（讨论模型选择时，还会看到自动选择一组好特征的算法。）本节简要介绍**局部加权线性回归**（LWR）：在训练数据充足的前提下，它使特征选择不再那么关键。这里只作简短介绍，因为你会在作业中亲自探索 LWR 的一些性质。

原来的线性回归算法要在查询点 x 处预测（即计算 $h(x)$），会：

1. 拟合 θ，使 $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$ 最小。
2. 输出 $\theta^T x$。

而局部加权线性回归会：

1. 拟合 θ，使 $\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$ 最小。
2. 输出 $\theta^T x$。

这里的 $w^{(i)}$ 是非负权重。直观地说，如果某个 i 对应的 $w^{(i)}$ 很大，选取 θ 时就会努力让 $(y^{(i)} - \theta^T x^{(i)})^2$ 较小；若 $w^{(i)}$ 很小，这个误差项在拟合中就基本会被忽略。

一种相当常用的权重选择是[6]

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right).$$

注意，权重依赖于我们希望求值的特定点 x。若 $|x^{(i)} - x|$ 很小，则 $w^{(i)}$ 接近 1；若 $|x^{(i)} - x|$ 很大，则 $w^{(i)}$ 很小。因此，选择 θ 时，会给靠近查询点 x 的训练样本（上的误差）高得多的“权重”。（还应注意，虽然权重公式在外观上类似高斯密度，但 $w^{(i)}$ 与高斯分布没有直接关系；尤其是，$w^{(i)}$ 不

[6] 如果 x 是向量，可推广为 $w^{(i)} = \exp(-(x^{(i)} - x)^T (x^{(i)} - x)/(2\tau^2))$，或者对适当的 τ 或 Σ，取 $w^{(i)} = \exp(-(x^{(i)} - x)^T \Sigma^{-1} (x^{(i)} - x)/(2\tau^2))$。

是随机变量，更谈不上服从正态分布或其他分布。）参数 τ 控制训练样本的权重随 $x^{(i)}$ 到查询点 x 的距离增大而下降的速度，称为**带宽参数**；你也会在作业中对它进行实验。

局部加权线性回归是我们遇到的第一个**非参数算法**。先前介绍的无权重线性回归属于**参数学习算法**，因为它用固定且有限数量的参数（各个 θ_i）拟合数据。拟合并存储这些 θ_i 后，未来做预测便不再需要保留训练数据。相反，使用局部加权线性回归预测时，必须保留整个训练集。“非参数”大致指的是：为了表示假设函数 h，需要保存的信息量随着训练集规模线性增长。

第 2 章　分类与逻辑回归

原文印刷页码：21–29。

现在讨论分类问题。它与回归问题类似，只是要预测的 y 只取少量离散值。目前先关注**二分类**：y 只能取 0 和 1。（这里的大部分内容都可以推广到多分类。）例如，建立垃圾邮件分类器时，$x^{(i)}$ 可以是某封邮件的特征；若是垃圾邮件，则 $y=1$，否则 $y=0$。0 也称**负类**，1 称**正类**，有时分别用 “–” 和 “+” 表示。给定 $x^{(i)}$，对应的 $y^{(i)}$ 也称为该训练样本的**标签**。

2.1 逻辑回归

我们可以忽略 y 是离散值这一事实，沿用线性回归算法，由 x 预测 y。然而，很容易构造出这种方法表现很差的例子。直观上，既然知道 $y\in\{0,1\}$，让 $h_\theta(x)$ 大于 1 或小于 0 也没有道理。

为解决这一问题，改变假设函数 $h_\theta(x)$ 的形式，选择

$$h_\theta(x)=g(\theta^T x)=\frac{1}{1+e^{-\theta^T x}},$$

其中

$$g(z)=\frac{1}{1+e^{-z}}$$

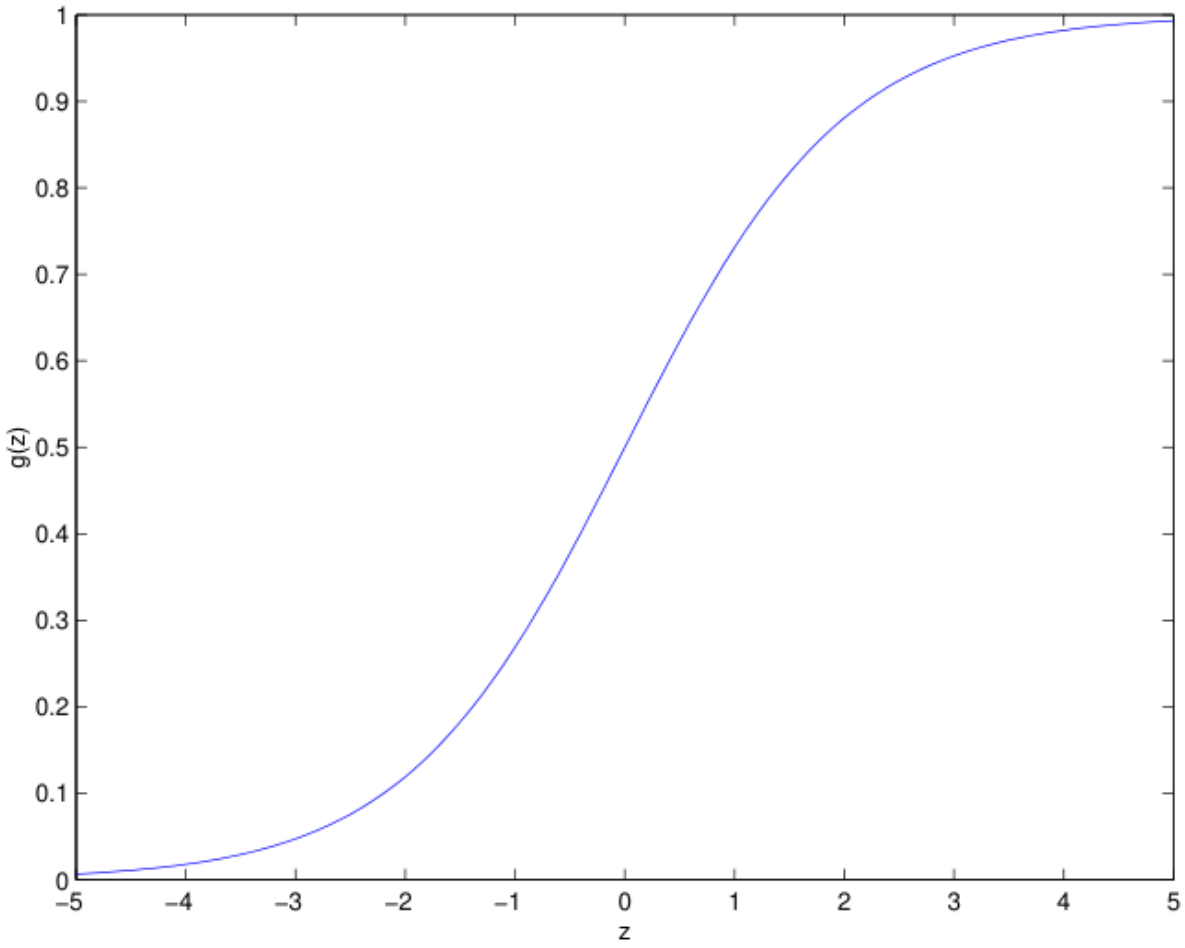
称为 **logistic 函数**或 **sigmoid 函数**。它的图像如下：

注意，当 $z\to\infty$ 时，$g(z)$ 趋于 1；当 $z\to-\infty$ 时，$g(z)$ 趋于 0。此外，$g(z)$，因而也包括 $h(x)$，始终介于 0 与 1 之间。与之前一样，约定 $x_0=1$，于是 $\theta^T x=\theta_0+\sum_{j=1}^{d}\theta_j x_j$。

目前先把选择 g 这件事当作给定的。也可以使用其他从 0 平滑上升到 1 的函数；但后面讨论 GLM 和生成式学习算法时会看到一些理由，说明 logistic 函数是很自然的选择。继续之前，先给出 sigmoid 导数 g' 的一个有用性质：

$$\begin{aligned}
g'(z)&=\frac{d}{dz}\frac{1}{1+e^{-z}}\\
&=\frac{1}{(1+e^{-z})^2}e^{-z}\\
&=\frac{1}{1+e^{-z}}\left(1-\frac{1}{1+e^{-z}}\right)\\
&=g(z)(1-g(z)).
\end{aligned}$$

给定逻辑回归模型，怎样拟合 θ？先前，我们在一组假设下将最小二乘回归推导为最大似然估计。



sigmoid 函数 g(z) 的图像。

沿用这一思路，为分类模型赋予一组概率假设，再通过最大似然拟合参数。

假设

$$P(y = 1 \mid x; \theta) = h_\theta(x),$$
$$P(y = 0 \mid x; \theta) = 1 - h_\theta(x).$$

可以更紧凑地写成

$$p(y \mid x; \theta) = h_\theta(x)^y (1 - h_\theta(x))^{1-y}.$$

假设 n 个训练样本相互独立地产生，则参数的似然为

$$\begin{aligned} L(\theta) &= p(\vec{y} \mid X; \theta) \\ &= \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}; \theta) \\ &= \prod_{i=1}^{n} h_\theta(x^{(i)})^{y^{(i)}} \left(1 - h_\theta(x^{(i)})\right)^{1-y^{(i)}}. \end{aligned}$$

与前面一样，最大化对数似然更容易：

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{n} \left[y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)})) \right]. \tag{2.1}$$

怎样最大化似然？与线性回归的推导类似，可以使用梯度上升。向量形式的更新为 $\theta := \theta + \alpha \nabla_\theta \ell(\theta)$。（注意，这里使用加号而非减号，因为现在要最大化函数。）先考虑单个训练样本 (x, y)，通过求导推导随机梯度上升规则：

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \ell(\theta) &= \left[y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right] \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ &= \left[y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right] g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\ &= \left[y(1 - g(\theta^T x)) - (1 - y) g(\theta^T x) \right] x_j \\ &= (y - h_\theta(x)) x_j. \end{aligned} \tag{2.2}$$

这里使用了 $g'(z) = g(z)(1 - g(z))$。于是得到随机梯度上升规则

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)}) \right) x_j^{(i)}.$$

与 LMS 更新规则比较，会发现它们看起来完全相同；但这不是同一个算法，因为现在的 $h_\theta(x^{(i)})$ 是 $\theta^T x^{(i)}$ 的非线性函数。尽管如此，在相当不同的算法和学习问题中得到相同的更新规则，仍然令人惊讶。这是巧合，还是有更深层的原因？讨论 GLM 时将给出答案。

注 2.1.1。用另一种记号来看同一损失函数也很有用，尤其在第 7.1 节研究非线性模型时。定义逻辑损失 $\ell_{\text{logistic}} : \mathbb{R} \times \{0, 1\} \to \mathbb{R}_{\geq 0}$：

$$\ell_{\text{logistic}}(t, y) \triangleq y \log(1 + \exp(-t)) + (1 - y) \log(1 + \exp(t)). \tag{2.3}$$

代入 $h_\theta(x) = 1/(1 + e^{-\theta^\top x})$，可验证负对数似然（式 (2.1) 中 $\ell(\theta)$ 的相反数）可改写为

$$-\ell(\theta) = \ell_{\text{logistic}}(\theta^\top x, y). \tag{2.4}$$

这里 $\theta^\top x$ 或 t 通常称为 **logit**。[7] 基本微积分给出

$$\frac{\partial \ell_{\text{logistic}}(t, y)}{\partial t} = y \frac{-\exp(-t)}{1 + \exp(-t)} + (1 - y) \frac{1}{1 + \exp(-t)} \tag{2.5}$$

$$= \frac{1}{1 + \exp(-t)} - y. \tag{2.6}$$

再根据链式法则，

$$\frac{\partial}{\partial \theta_j} \ell(\theta) = -\frac{\partial \ell_{\text{logistic}}(t, y)}{\partial t} \cdot \frac{\partial t}{\partial \theta_j} \tag{2.7}$$

$$= \left(y - \frac{1}{1 + \exp(-t)} \right) x_j = (y - h_\theta(x)) x_j, \tag{2.8}$$

与式 (2.2) 的推导一致。第 7.1 节将说明这种观点如何推广到非线性模型。

2.2 题外话：感知机学习算法

现在简要介绍一个具有历史意义的算法，后面讨论学习理论时还会回到它。考虑修改逻辑回归，“强制” 它精确输出 0 或 1。为此，很自然的做法是把 g 改为阈值函数：

$$g(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

使用这个修改后的 g，仍令 $h_\theta(x) = g(\theta^T x)$，再使用更新规则

$$\theta_j := \theta_j + \alpha \big(y^{(i)} - h_\theta(x^{(i)}) \big) x_j^{(i)},$$

就得到**感知机学习算法**。

20 世纪 60 年代，人们认为这种 “感知机” 是对大脑单个神经元工作方式的粗略建模。由于算法很简单，它也将成为本课程后面学习理论分析的起点。但要注意，虽然感知机在形式上可能类似我们讨论过的其他算法，其类型实际上与逻辑回归、最小二乘线性回归有很大不同；尤其难以给感知机的预测赋予有意义的概率解释，或将它推导为最大似然估计算法。

[7] 译注：式 (2.4) 及其后的推导针对单个样本；对于式 (2.1) 的整个训练集，右侧应对所有样本求和。原文在这里沿用了同一个 $\ell(\theta)$ 记号。

2.3 多分类

考虑响应变量 y 可以取 k 个值中任意一个的分类问题，即 $y \in \{1, 2, \dots, k\}$。例如，不把邮件分成垃圾邮件和非垃圾邮件两类（二分类），而希望分成垃圾邮件、私人邮件和工作邮件三类。标签 / 响应变量仍是离散的，但现在可以取两个以上的值。因此，我们将其建模为服从多项分布。

这里 $p(y \mid x; \theta)$ 是 k 个可能离散结果上的分布，因而是多项分布。回忆一下，多项分布用 k 个数 $\phi_1, \dots, \phi_k$ 指定每个结果的概率，必须满足 $\sum_{i=1}^{k} \phi_i = 1$。我们将设计一个参数化模型，给定输入 x 后，输出满足此约束的 $\phi_1, \dots, \phi_k$。

引入 k 组参数 $\theta_1, \dots, \theta_k$，每一组都是 $\mathbb{R}^d$ 中的向量。直观上，希望用 $\theta_1^\top x, \dots, \theta_k^\top x$ 来表示 $\phi_1, \dots, \phi_k$，即概率 $P(y = 1 \mid x; \theta), \dots, P(y = k \mid x; \theta)$。但直接这样做有两个问题：第一，$\theta_j^\top x$ 不一定落在 $[0, 1]$ 内；第二，这些数之和不一定等于 1。因此，使用 softmax 函数，把 $(\theta_1^\top x, \dots, \theta_k^\top x)$ 转换为各元素非负且总和为 1 的概率向量。

定义 softmax 函数 $\mathrm{softmax} : \mathbb{R}^k \to \mathbb{R}^k$：

$$\mathrm{softmax}(t_1, \dots, t_k) = \begin{bmatrix} \frac{\exp(t_1)}{\sum_{j=1}^{k} \exp(t_j)} \\ \vdots \\ \frac{\exp(t_k)}{\sum_{j=1}^{k} \exp(t_j)} \end{bmatrix}. \tag{2.9}$$

softmax 的输入，也就是这里的向量 t，通常称为 **logits**。由定义可知，softmax 的输出始终是一个元素非负、总和为 1 的概率向量。

令 $(t_1, \dots, t_k) = (\theta_1^\top x, \dots, \theta_k^\top x)$，将 softmax 作用于这个向量，并把输出作为 $P(y = 1 \mid x; \theta), \dots, P(y = k \mid x; \theta)$，得到概率模型：

$$\begin{bmatrix} P(y = 1 \mid x; \theta) \\ \vdots \\ P(y = k \mid x; \theta) \end{bmatrix} = \mathrm{softmax}(t_1, \dots, t_k) = \begin{bmatrix} \frac{\exp(\theta_1^\top x)}{\sum_{j=1}^{k} \exp(\theta_j^\top x)} \\ \vdots \\ \frac{\exp(\theta_k^\top x)}{\sum_{j=1}^{k} \exp(\theta_j^\top x)} \end{bmatrix}. \tag{2.10}$$

为方便记号，令 $\phi_i = \exp(t_i) / \sum_{j=1}^{k} \exp(t_j)$，上式可更简洁地写成

$$P(y = i \mid x; \theta) = \phi_i = \frac{\exp(t_i)}{\sum_{j=1}^{k} \exp(t_j)} = \frac{\exp(\theta_i^\top x)}{\sum_{j=1}^{k} \exp(\theta_j^\top x)}. \tag{2.11}$$

下面计算单个样本 (x, y) 的负对数似然：

$$-\log p(y \mid x, \theta) = -\log\left(\frac{\exp(t_y)}{\sum_{j=1}^{k} \exp(t_j)}\right) = -\log\left(\frac{\exp(\theta_y^\top x)}{\sum_{j=1}^{k} \exp(\theta_j^\top x)}\right). \tag{2.12}$$

因此，训练数据的负对数似然，即损失函数，为

$$\ell(\theta) = \sum_{i=1}^{n} -\log\left(\frac{\exp(\theta_{y^{(i)}}^\top x^{(i)})}{\sum_{j=1}^{k} \exp(\theta_j^\top x^{(i)})}\right). \tag{2.13}$$

定义**交叉熵损失** $\ell_{\text{ce}} : \mathbb{R}^k \times \{1, \dots, k\} \to \mathbb{R}_{\geq 0}$，可以把上面复杂的公式模块化：[8]

$$\ell_{\text{ce}}((t_1, \dots, t_k), y) = -\log\left(\frac{\exp(t_y)}{\sum_{j=1}^{k} \exp(t_j)}\right). \tag{2.14}$$

采用此记号，式 (2.13) 可简单写成

$$\ell(\theta) = \sum_{i=1}^{n} \ell_{\text{ce}}((\theta_1^\top x^{(i)}, \dots, \theta_k^\top x^{(i)}), y^{(i)}). \tag{2.15}$$

交叉熵损失的梯度也很简单。令 $t = (t_1, \dots, t_k)$，回忆 $\phi_i = \exp(t_i)/\sum_{j=1}^{k} \exp(t_j)$，由基本微积分可得

$$\frac{\partial \ell_{\text{ce}}(t, y)}{\partial t_i} = \phi_i - 1\{y = i\}, \tag{2.16}$$

其中 $1\{\cdot\}$ 是**指示函数**：当 $y = i$ 时，$1\{y = i\} = 1$；否则为 0。用向量记号，则为

$$\frac{\partial \ell_{\text{ce}}(t, y)}{\partial t} = \phi - e_y. \tag{2.17}$$

这种形式在第 7 章很有用。这里 $e_s \in \mathbb{R}^k$ 是第 s 个标准基向量，即第 s 个元素为 1，其余元素均为 0。利用链式法则，

$$\frac{\partial \ell_{\text{ce}}((\theta_1^\top x, \dots, \theta_k^\top x), y)}{\partial \theta_i} = \frac{\partial \ell_{\text{ce}}(t, y)}{\partial t_i} \cdot \frac{\partial t_i}{\partial \theta_i} = (\phi_i - 1\{y = i\})x. \tag{2.18}$$

因此，损失对参数块 θ_i 的梯度是

$$\frac{\partial \ell(\theta)}{\partial \theta_i} = \sum_{j=1}^{n} (\phi_i^{(j)} - 1\{y^{(j)} = i\})x^{(j)}, \tag{2.19}$$

其中 $\phi_i^{(j)} = \exp(\theta_i^\top x^{(j)})/\sum_{s=1}^{k} \exp(\theta_s^\top x^{(j)})$ 是模型对样本 $x^{(j)}$ 预测类别 i 的概率。有了上述梯度，即可实现（随机）梯度下降，最小化损失函数 $\ell(\theta)$。

2.4 最大化 $\ell(\theta)$ 的另一种算法

回到 $g(z)$ 是 sigmoid 函数的逻辑回归，讨论最大化 $\ell(\theta)$ 的另一种算法。

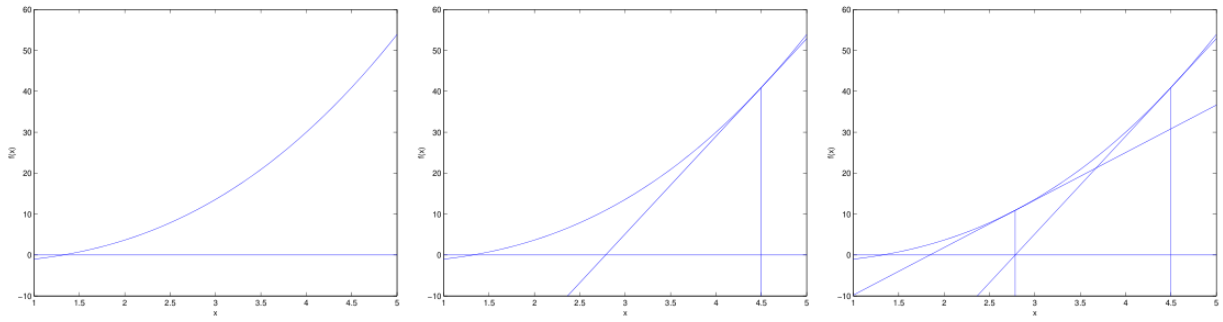
先考虑用牛顿法求函数的零点。设有函数 $f : \mathbb{R} \mapsto \mathbb{R}$，希望找到使 $f(\theta) = 0$ 的 θ，其中 $\theta \in \mathbb{R}$ 是实数。牛顿法执行更新

$$\theta := \theta - \frac{f(\theta)}{f'(\theta)}.$$

这个方法有自然的解释：用在当前猜测值 θ 处与 f 相切的线性函数来近似 f，求这个线性函数等

[8] 此处命名存在一些歧义。有些人把从概率向量（这里的 ϕ）和标签 y 映射到最终实数的函数称为交叉熵损失，把本讲义的版本称为 softmax 交叉熵损失。这里采用当前命名，是为了与 PyTorch、Jax 等多数现代深度学习库保持一致。

于零的位置，并把该位置作为下一个 θ 的猜测值。



牛顿法求零点的过程：初始值 4.5，经切线更新到约 2.8，再到约 1.8，逐渐接近约 1.3 的零点。

最左图画出了 f 和直线 $y = 0$。我们要找满足 $f(\theta) = 0$ 的 θ，这个值约为 1.3。假定从 $\theta = 4.5$ 初始化。牛顿法在 $\theta = 4.5$ 处作 f 的切线，求该直线取值为零的位置（中图），得到下一个猜测值，约为 2.8。右图显示再运行一次迭代，将 θ 更新到约 1.8。再迭代几次，就会迅速接近 $\theta = 1.3$。

牛顿法提供了求解 $f(\theta) = 0$ 的方法。如果想用它最大化函数 ℓ 呢？ℓ 的极大值对应于一阶导数 $\ell'(\theta)$ 为零的点。因此，令 $f(\theta) = \ell'(\theta)$，就能使用同一算法来最大化 ℓ，得到

$$\theta := \theta - \frac{\ell'(\theta)}{\ell''(\theta)}.$$

（思考：如果想用牛顿法最小化而非最大化函数，会有什么变化？）

最后，逻辑回归中的 θ 是向量，所以需要推广到多维情形。多维牛顿法，也称 **Newton–Raphson 方法**，为

$$\theta := \theta - H^{-1}\nabla_\theta \ell(\theta).$$

其中，$\nabla_\theta \ell(\theta)$ 仍是 $\ell(\theta)$ 对各个 θ_i 的偏导数所组成的向量；H 是一个 $d \times d$ 矩阵（包括截距项时实际为 $(d+1) \times (d+1)$），称为 **Hessian 矩阵**，其元素为

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}.$$

牛顿法通常比（批量）梯度下降收敛得更快，达到非常接近最小值的位置所需迭代次数少得多。不过，一次牛顿迭代可能比一次梯度下降更昂贵，因为需要求出一个 $d \times d$ 的 Hessian 矩阵并求逆；但只要 d 不太大，整体通常快得多。将牛顿法用于最大化逻辑回归的对数似然 $\ell(\theta)$ 时，得到的方法也称为 **Fisher 评分法**。

第 3 章　广义线性模型

原文印刷页码：30–34。

到目前为止，我们看过一个回归例子和一个分类例子。在回归例子中，$y \mid x;\theta \sim \mathcal{N}(\mu,\sigma^2)$；在分类例子中，$y \mid x;\theta \sim \text{Bernoulli}(\phi)$，其中 μ 和 ϕ 都适当地定义为 x 与 θ 的函数。本章将说明，这两种方法都是一个更大模型族的特例，称为**广义线性模型**（Generalized Linear Models，GLM）。[9] 我们也会说明怎样推导 GLM 族中的其他模型，并将其用于其他分类与回归问题。

3.1 指数族

为逐步引出 GLM，先定义**指数族分布**。如果一类分布可以写成

$$p(y;\eta) = b(y)\exp\big(\eta^T T(y) - a(\eta)\big), \tag{3.1}$$

就说它属于指数族。

这里，η 称为分布的**自然参数**（也称**典范参数**）；$T(y)$ 是**充分统计量**（在我们考虑的分布中，常有 $T(y)=y$）；$a(\eta)$ 是**对数配分函数**。$e^{-a(\eta)}$ 本质上起归一化常数的作用，确保 $p(y;\eta)$ 对 y 求和或积分为 1。

固定选择 T, a, b，就定义了一个由 η 参数化的分布族（或分布集合）；改变 η，就得到该族内的不同分布。

下面说明伯努利分布和高斯分布都是指数族的例子。均值为 ϕ 的伯努利分布记为 $\text{Bernoulli}(\phi)$，它是 $y \in \{0,1\}$ 上的分布，满足 $p(y=1;\phi)=\phi$，$p(y=0;\phi)=1-\phi$。改变 ϕ，就得到均值不同的伯努利分布。现在证明，这一类由改变 ϕ 得到的伯努利分布属于指数族，即存在一组 T, a, b，使式 (3.1) 恰好成为伯努利分布族。

将伯努利分布写成

$$\begin{aligned} p(y;\phi) &= \phi^y (1-\phi)^{1-y} \\ &= \exp\big(y\log\phi + (1-y)\log(1-\phi)\big) \\ &= \exp\left[\left(\log\frac{\phi}{1-\phi}\right)y + \log(1-\phi)\right]. \end{aligned}$$

因此，自然参数为 $\eta = \log(\phi/(1-\phi))$。有趣的是，如果反解这一关系，将 ϕ 表示为 η 的函数，就得到 $\phi = 1/(1+e^{-\eta})$。这正是熟悉的 sigmoid 函数！把逻辑回归推导为 GLM 时，它还会再次出现。为

[9] 本章的讲述受到 Michael I. Jordan 的 *Learning in Graphical Models*（未出版书稿），以及 McCullagh 与 Nelder 的 *Generalized Linear Models*（第 2 版）的启发。

了完成伯努利分布的指数族表示，还需取

$$\begin{aligned} T(y) &= y, \\ a(\eta) &= -\log(1-\phi) = \log(1+e^{\eta}), \\ b(y) &= 1. \end{aligned}$$

由此可见，通过适当选择 T, a, b，伯努利分布确实可以写成式 (3.1)。

接着考虑高斯分布。回忆推导线性回归时，σ^2 的取值并不影响最终选出的 θ 和 $h_\theta(x)$。因此可以任意指定 σ^2，而不改变结果。为简化下面的推导，令 $\sigma^2 = 1$。[10] 于是

$$\begin{aligned} p(y;\mu) &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y-\mu)^2\right) \\ &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right) \exp\left(\mu y - \frac{1}{2}\mu^2\right). \end{aligned}$$

因此高斯分布属于指数族，其中

$$\begin{aligned} \eta &= \mu, \\ T(y) &= y, \\ a(\eta) &= \mu^2/2 = \eta^2/2, \\ b(y) &= \frac{1}{\sqrt{2\pi}} \exp(-y^2/2). \end{aligned}$$

许多其他分布也属于指数族：多项分布（后面会看到）、泊松分布（用于计数数据建模，也见习题）、伽马分布与指数分布（用于时间间隔等连续非负随机变量）、贝塔分布与狄利克雷分布（用于概率上的分布），等等。下一节将介绍一种构造模型的通用“配方”，让给定 x 和 θ 的 y 服从这些分布中的任意一种。

3.2 构造 GLM

假设你想根据商店促销、近期广告、天气、星期几等特征 x，建立模型，估计某一小时进入商店的顾客人数 y（或网站的页面浏览量）。我们知道，泊松分布通常能很好地对访客数量建模。知道这一点后，该如何为问题构建模型？幸运的是，泊松分布属于指数族，因此可以应用 GLM。本节介绍如何为这类问题构造 GLM。

更一般地，考虑一个分类或回归问题：我们希望把某个随机变量 y 的取值预测为 x 的函数。为这个问题推导 GLM，需要对给定 x 的 y 的条件分布以及模型作三个假设：

1. $y \mid x;\theta \sim \text{ExponentialFamily}(\eta)$。也就是说，给定 x 与 θ，y 服从某个参数为 η 的指数族分布。
2. 给定 x，目标是预测 $T(y)$ 在给定 x 下的期望。在大多数例子中，$T(y) = y$，所以希望学习到的假设函数 h 输出的预测满足 $h(x) = \mathbb{E}[y \mid x]$。（逻辑回归和线性回归对 $h_\theta(x)$ 的选择都满足这一假设。例如逻辑回归中，$h_\theta(x) = p(y=1 \mid x;\theta) = 0 \cdot p(y=0 \mid x;\theta) + 1 \cdot p(y=1 \mid x;\theta) = \mathbb{E}[y \mid x;\theta]$。）

[10] 如果保留 σ^2 为变量，也可证明高斯分布属于指数族；此时 $\eta \in \mathbb{R}^2$ 是依赖于 μ 与 σ 的二维向量。就 GLM 而言，还可以采用更一般的指数族定义来处理 σ^2：$p(y;\eta,\tau) = b(a,\tau)\exp((\eta^T T(y) - a(\eta))/c(\tau))$。这里 τ 称为**离散参数**，对高斯分布有 $c(\tau) = \sigma^2$。不过，由于上面的简化，本章例子不需要这一更一般定义。译注：原文写作 $b(a,\tau)$；通常这一基准项记为 $b(y,\tau)$，这里保留原式并指出记号疑点。

3. 自然参数 η 与输入 x 呈线性关系：$\eta = \theta^T x$。（若 η 是向量，则 $\eta_i = \theta_i^T x$。）

第三个假设看起来可能最缺少依据；与其说是一个假设，不如把它视为构造 GLM 配方中的“设计选择”。这三个假设 / 设计选择导出一类十分优雅的学习算法，即 GLM，具有容易学习等许多理想性质。此外，得到的模型通常能有效地刻画 y 的不同类型分布；例如，马上就会证明，逻辑回归和普通最小二乘都能推导为 GLM。

3.2.1 普通最小二乘

为了说明普通最小二乘是 GLM 模型族的特例，考虑目标变量 y（GLM 术语中也称**响应变量**）连续的情形，将给定 x 时 y 的条件分布建模为高斯分布 $\mathcal{N}(\mu, \sigma^2)$。（这里 μ 可以依赖 x。）即令上面的 ExponentialFamily(η) 为高斯分布。前面已说明，在高斯分布的指数族表示中，$\mu = \eta$，于是

$$\begin{aligned} h_\theta(x) &= \mathbb{E}[y \mid x; \theta] \\ &= \mu \\ &= \eta \\ &= \theta^T x. \end{aligned}$$

第一个等号来自假设 2；第二个等号来自 $y \mid x; \theta \sim \mathcal{N}(\mu, \sigma^2)$，其期望为 μ；第三个等号来自假设 1，以及前面关于高斯分布指数族表示中 $\mu = \eta$ 的推导；最后一个等号来自假设 3。

3.2.2 逻辑回归

现在考虑逻辑回归。我们关注二分类，所以 $y \in \{0, 1\}$。既然 y 是二值的，自然就选择伯努利分布族来建模给定 x 时 y 的条件分布。在伯努利分布的指数族表示中，$\phi = 1/(1 + e^{-\eta})$。另外，若 $y \mid x; \theta \sim \text{Bernoulli}(\phi)$，则 $\mathbb{E}[y \mid x; \theta] = \phi$。与普通最小二乘的推导类似，得到

$$\begin{aligned} h_\theta(x) &= \mathbb{E}[y \mid x; \theta] \\ &= \phi \\ &= 1/(1 + e^{-\eta}) \\ &= 1/(1 + e^{-\theta^T x}). \end{aligned}$$

于是得到 $h_\theta(x) = 1/(1 + e^{-\theta^T x})$ 这种形式的假设函数。如果你先前好奇 logistic 函数 $1/(1 + e^{-z})$ 是怎样想到的，这就给出了一个答案：一旦假设给定 x 的 y 服从伯努利分布，它就自然来自 GLM 和指数族分布的定义。

再引入一些术语。将自然参数映射到分布均值的函数 g，即 $g(\eta) = \mathbb{E}[T(y); \eta]$，称为**典范响应函数**；它的逆函数 g^{-1} 称为**典范连接函数**。因此，高斯族的典范响应函数就是恒等函数，而伯努利族的典范响应函数是 logistic 函数。[11]

[11] 许多教材用 g 表示连接函数，用 g^{-1} 表示响应函数；这里的记号沿袭早期机器学习文献，以便与本课程其余部分一致。

第 4 章　生成式学习算法

原文印刷页码：35–48。

到目前为止，我们主要讨论的是对 $p(y \mid x;\theta)$，即给定 x 时 y 的条件分布，进行建模的学习算法。例如，逻辑回归将 $p(y \mid x;\theta)$ 建模为 $h_\theta(x) = g(\theta^T x)$，其中 g 是 sigmoid 函数。本章将讨论另一类学习算法。

考虑这样一个分类问题：根据动物的一些特征，学习区分大象（$y = 1$）和狗（$y = 0$）。给定训练集，逻辑回归或感知机之类的算法，基本上是在寻找一条直线，也就是决策边界，把大象和狗分开。然后，要将一只新动物分类为大象或狗，只需检查它位于决策边界的哪一侧，再据此预测。

还有另一种做法。首先，观察大象，建立一个描述大象外观的模型。然后观察狗，单独建立一个描述狗外观的模型。最后，要对新动物分类，就分别将它与大象模型和狗模型匹配，看看它更像训练集中见过的大象，还是更像狗。

试图直接学习 $p(y \mid x)$ 的算法（例如逻辑回归），或者试图直接学习从输入空间 $\mathcal{X}$ 到标签 $\{0,1\}$ 的映射的算法（例如感知机），称为**判别式学习算法**。这里要讨论的算法则尝试建模 $p(x \mid y)$（以及 $p(y)$），称为**生成式学习算法**。例如，如果 y 表示一个样本是狗（0）还是大象（1），那么 $p(x \mid y = 0)$ 描述狗的特征分布，$p(x \mid y = 1)$ 描述大象的特征分布。

建立 $p(y)$（称为**类别先验**）和 $p(x \mid y)$ 的模型后，算法就能利用贝叶斯公式推导给定 x 时 y 的后验分布：

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

这里，分母为 $p(x) = p(x \mid y = 1)p(y = 1) + p(x \mid y = 0)p(y = 0)$（你应该能根据概率的基本性质验证这一点），所以也可以用已学到的 $p(x \mid y)$ 和 $p(y)$ 表示。实际上，如果计算 $p(y \mid x)$ 是为了进行预测，那么不必计算分母，因为

$$\begin{aligned}\arg\max_y p(y \mid x) &= \arg\max_y \frac{p(x \mid y)p(y)}{p(x)} \\ &= \arg\max_y p(x \mid y)p(y).\end{aligned}$$

4.1 高斯判别分析

我们首先介绍的生成式学习算法是**高斯判别分析**（Gaussian Discriminant Analysis，GDA）。在这个模型中，假设 $p(x \mid y)$ 服从多元正态分布。在介绍 GDA 模型本身之前，先简要讨论多元正态分布的性质。

4.1.1 多元正态分布

d 维多元正态分布，也称为**多元高斯分布**，由均值向量 $\mu \in \mathbb{R}^d$ 和协方差矩阵 $\Sigma \in \mathbb{R}^{d\times d}$ 参数化，其中 $\Sigma \geq 0$ 为对称半正定矩阵。它也记为 $\mathcal{N}(\mu, \Sigma)$，其密度为

$$p(x;\mu,\Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right).$$

上式中的 $|\Sigma|$ 表示矩阵 Σ 的行列式。

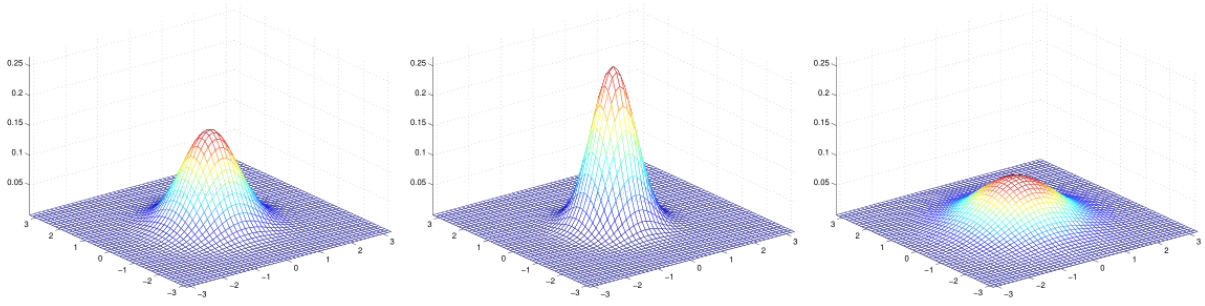
对服从 $\mathcal{N}(\mu, \Sigma)$ 的随机变量 X，其均值毫不意外地为 μ：

$$\mathbb{E}[X] = \int_x x\, p(x;\mu,\Sigma)\, dx = \mu.$$

向量值随机变量 Z 的**协方差**定义为 $\mathrm{Cov}(Z) = \mathbb{E}[(Z-\mathbb{E}[Z])(Z-\mathbb{E}[Z])^T]$。它推广了实值随机变量的方差概念。协方差也可以定义为 $\mathrm{Cov}(Z) = \mathbb{E}[ZZ^T] - (\mathbb{E}[Z])(\mathbb{E}[Z])^T$。（你应该能自行证明这两个定义等价。）若 $X \sim \mathcal{N}(\mu, \Sigma)$，则

$$\mathrm{Cov}(X) = \Sigma.$$

下面是高斯分布密度的几个例子：



零均值、不同尺度协方差矩阵下的高斯密度。

最左图为均值为零（即 2×1 零向量）、协方差矩阵 $\Sigma = I$（2×2 单位矩阵）的高斯分布。均值为零、协方差为单位矩阵的高斯分布也称为**标准正态分布**。中图为均值为零、$\Sigma = 0.6I$ 的高斯密度；最右图则取 $\Sigma = 2I$。可以看到，Σ 越大，高斯分布就越“分散”；Σ 越小，分布就越“压缩”。

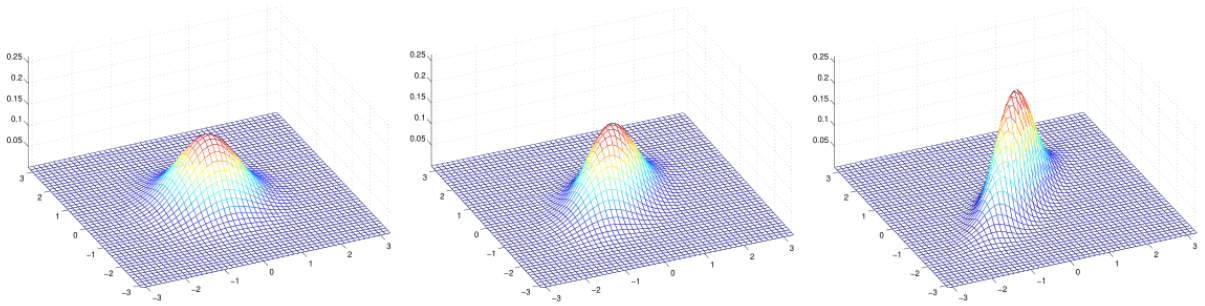
再看几个例子。

上图中各高斯分布的均值均为 0，协方差矩阵分别为

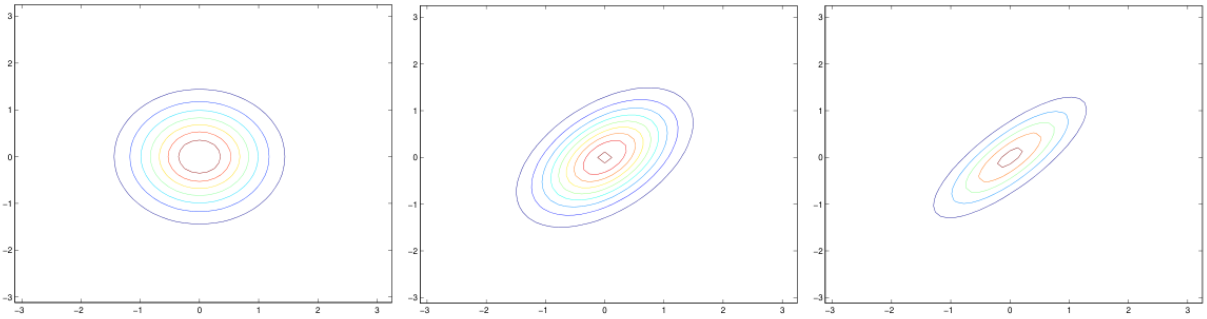
$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$

最左图是熟悉的标准正态分布。可以看到，随着 Σ 的非对角元素增大，密度越来越朝 45° 直线（即 $x_1 = x_2$）“压缩”。观察这三个密度的等高线，可以看得更清楚：

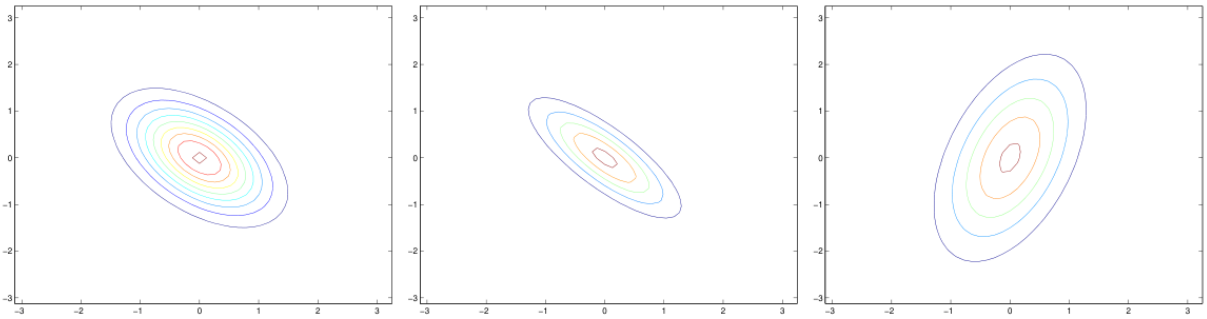
下面是最后一组通过改变 Σ 得到的例子：



增加协方差矩阵的非对角元素对高斯密度的影响。



上面三个高斯密度的等高线。



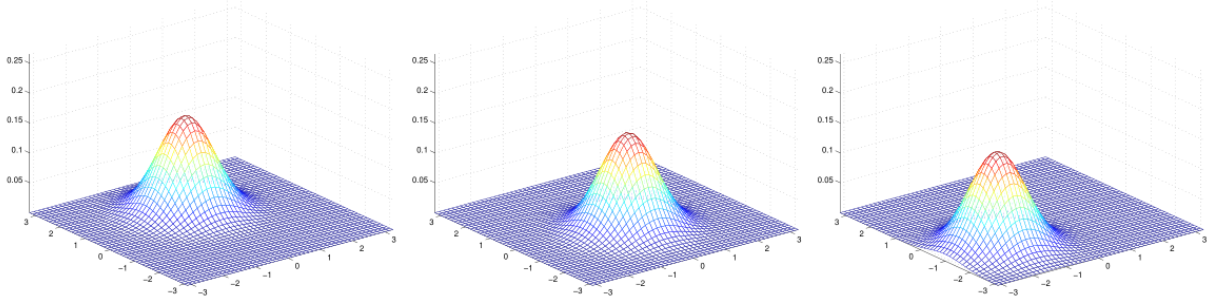
负相关与不同边际方差下的高斯等高线。

上图依次使用

$$\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$

从最左图和中图可以看到，减小协方差矩阵的非对角元素，密度也会再次“压缩”，但方向相反。最后，一般而言，改变参数会得到椭圆形等高线，最右图就是一例。

最后一组例子固定 $\Sigma = I$，通过改变 μ 移动密度的均值位置。



保持单位协方差矩阵、改变均值位置的高斯密度。

上图使用 $\Sigma = I$，均值分别为

$$\mu = \begin{bmatrix} 1 \\ 0 \end{bmatrix}; \quad \mu = \begin{bmatrix} -0.5 \\ 0 \end{bmatrix}; \quad \mu = \begin{bmatrix} -1 \\ -1.5 \end{bmatrix}.$$

4.1.2 高斯判别分析模型

当分类问题的输入特征 x 是连续值随机变量时，可以用高斯判别分析（GDA）模型，通过多元正态分布建模 $p(x \mid y)$。模型为

$$\begin{aligned} y &\sim \text{Bernoulli}(\phi), \\ x \mid y = 0 &\sim \mathcal{N}(\mu_0, \Sigma), \\ x \mid y = 1 &\sim \mathcal{N}(\mu_1, \Sigma). \end{aligned}$$

展开写出这些分布：

$$\begin{aligned} p(y) &= \phi^y (1-\phi)^{1-y}, \\ p(x \mid y = 0) &= \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x - \mu_0)^T \Sigma^{-1} (x - \mu_0) \right), \\ p(x \mid y = 1) &= \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x - \mu_1)^T \Sigma^{-1} (x - \mu_1) \right). \end{aligned}$$

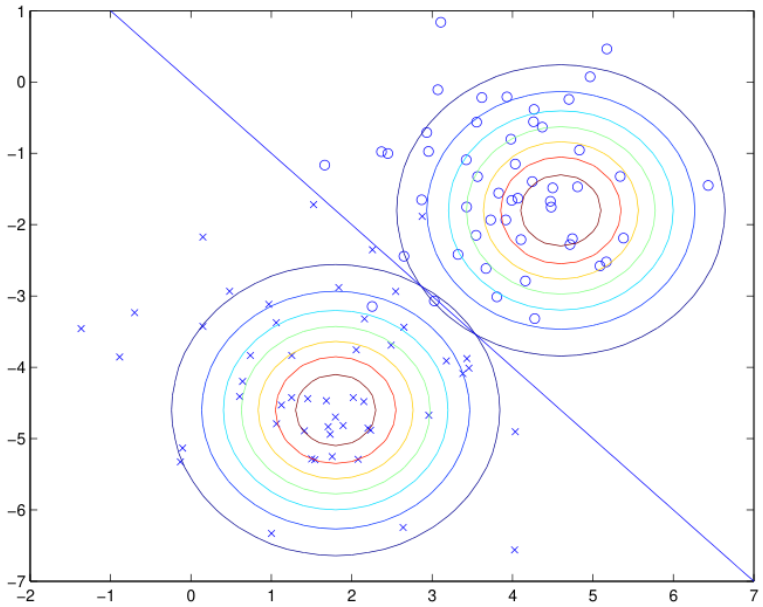
模型参数为 $\phi, \Sigma, \mu_0, \mu_1$。（注意，虽然有两个不同的均值向量 μ_0, μ_1，但这个模型通常只使用一个协方差矩阵 Σ。）数据的对数似然为

$$\begin{aligned}\ell(\phi,\mu_0,\mu_1,\Sigma) &= \log\prod_{i=1}^{n} p(x^{(i)},y^{(i)};\phi,\mu_0,\mu_1,\Sigma)\\ &= \log\prod_{i=1}^{n} p(x^{(i)}\mid y^{(i)};\mu_0,\mu_1,\Sigma)p(y^{(i)};\phi).\end{aligned}$$

对参数最大化 ℓ，可以得到参数的最大似然估计（见习题集 1）：

$$\begin{aligned}\phi &= \frac{1}{n}\sum_{i=1}^{n}\mathbf{1}\{y^{(i)}=1\},\\ \mu_0 &= \frac{\sum_{i=1}^{n}\mathbf{1}\{y^{(i)}=0\}x^{(i)}}{\sum_{i=1}^{n}\mathbf{1}\{y^{(i)}=0\}},\\ \mu_1 &= \frac{\sum_{i=1}^{n}\mathbf{1}\{y^{(i)}=1\}x^{(i)}}{\sum_{i=1}^{n}\mathbf{1}\{y^{(i)}=1\}},\\ \Sigma &= \frac{1}{n}\sum_{i=1}^{n}(x^{(i)}-\mu_{y^{(i)}})(x^{(i)}-\mu_{y^{(i)}})^T.\end{aligned}$$

可以用下图直观展示算法在做什么：



GDA 拟合的两个高斯分布及线性决策边界。

图中展示了训练集，以及分别对两类数据拟合的两个高斯分布的等高线。注意，因为共享协方差矩阵 Σ，两个高斯分布的等高线形状和方向相同，但均值 μ_0,μ_1 不同。图中还画出了满足 $p(y=1\mid x)=0.5$ 的直线决策边界。在边界一侧，我们预测 $y=1$ 是最可能的结果；另一侧则预测 $y=0$。

4.1.3 讨论：GDA 与逻辑回归

GDA 模型与逻辑回归有一种有趣的关系。如果把 $p(y=1\mid x;\phi,\mu_0,\mu_1,\Sigma)$ 看作 x 的函数，会发现它可以写成

$$p(y=1 \mid x;\phi,\Sigma,\mu_0,\mu_1)=\frac{1}{1+\exp(-\theta^T x)},$$

其中 θ 是 ϕ,Σ,μ_0,μ_1 的某个适当函数。[12] 这正是判别式算法逻辑回归用来建模 $p(y=1 \mid x)$ 的形式。

什么时候应该优先选择其中一个模型？通常，GDA 和逻辑回归在同一个数据集上训练，会得到不同的决策边界。哪一个更好？

刚才我们说明，如果 $p(x \mid y)$ 是多元高斯分布（共享 Σ），那么 $p(y \mid x)$ 必定是逻辑函数。但反过来不成立：$p(y \mid x)$ 是逻辑函数，并不意味着 $p(x \mid y)$ 是多元高斯分布。这说明 GDA 对数据作出的建模假设比逻辑回归更强。当这些建模假设正确时，GDA 会更好地拟合数据，是更好的模型。具体而言，当 $p(x \mid y)$ 确实是高斯分布（共享 Σ）时，GDA 是**渐近有效的**。不严格地说，这意味着在训练集极大（n 很大）的极限下，没有任何算法严格优于 GDA，例如在估计 $p(y \mid x)$ 的准确程度方面。尤其可以证明，在这种情况下，GDA 比逻辑回归更好；更一般地，即使训练集较小，我们通常也预期 GDA 表现更好。

相反，逻辑回归作出的假设弱得多，因此也更稳健，对错误的建模假设更不敏感。有许多不同的假设都能导出逻辑函数形式的 $p(y \mid x)$。例如，如果 $x \mid y=0 \sim \text{Poisson}(\lambda_0)$，$x \mid y=1 \sim \text{Poisson}(\lambda_1)$，那么 $p(y \mid x)$ 就是逻辑函数。逻辑回归在这样的泊松数据上也能表现良好。但若对这些数据使用 GDA，即用高斯分布拟合非高斯数据，结果就较难预料：GDA 可能表现好，也可能不好。

概括来说，GDA 作出更强的建模假设；当假设正确或至少近似正确时，它的数据效率更高（即只需要更少的训练数据就能学得“好”）。逻辑回归的假设更弱，在数据偏离建模假设时明显更稳健。具体而言，当数据确实不是高斯分布时，在大数据集的极限下，逻辑回归几乎总是优于 GDA。因此，实践中逻辑回归比 GDA 更常用。（关于判别式模型和生成式模型的一些类似考量，也适用于下面要讨论的朴素贝叶斯算法，但朴素贝叶斯仍被认为是一种非常好的分类算法，而且确实十分流行。）

4.2 朴素贝叶斯（选读）

在 GDA 中，特征向量 x 是连续的实值向量。下面讨论另一种学习算法，其中 x_j 取离散值。

作为引例，考虑用机器学习构建电子邮件垃圾邮件过滤器。我们希望把邮件分为未经请求的商业邮件（垃圾邮件）和非垃圾邮件。学会分类后，就能让邮件阅读器自动过滤垃圾邮件，也许把它们放入单独的文件夹。邮件分类是更广泛的**文本分类**问题的一种。

假设有一个训练集，即一组已标注为垃圾或非垃圾的邮件。构建垃圾邮件过滤器的第一步，是指定用来表示邮件的特征 x_j。

我们用一个长度等于词典词数的特征向量表示邮件。具体地，如果邮件包含词典中的第 j 个词，就令 $x_j=1$；否则令 $x_j=0$。例如，向量

[12] 这里沿用在右侧给 $x^{(i)}$ 增加额外坐标 $x_0^{(i)}=1$、将其重新定义为 $(d+1)$ 维向量的约定；参见习题集 1。

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

表示一封包含 “a” 和 “buy”，但不包含 “aardvark” “aardwolf” 或 “zygmurgy” 的邮件。[13] 编码进特征向量的词的集合称为**词汇表**，所以 x 的维度等于词汇表大小。

选好特征向量后，我们希望建立一个生成式模型，因此必须建模 $p(x \mid y)$。但如果词汇表有 50000 个词，那么 $x \in \{0,1\}^{50000}$，也就是由 0 和 1 组成的 50000 维向量。如果使用覆盖 2^{50000} 种可能结果的多项分布显式建模 x，就会得到一个 $2^{50000} - 1$ 维参数向量。显然，参数太多了。

因此，为了建模 $p(x \mid y)$，我们要作一个很强的假设：给定 y 后，各 x_i **条件独立**。这个假设称为**朴素贝叶斯（NB）假设**，得到的算法称为**朴素贝叶斯分类器**。例如，若 $y = 1$ 表示垃圾邮件，“buy” 为第 2087 个词，“price” 为第 39831 个词，那么这一假设就是说：如果告诉你 $y = 1$（某封邮件是垃圾邮件），那么知道 x_{2087}（即知道 “buy” 是否出现在邮件中），不会改变你对 x_{39831}（即 “price” 是否出现）的判断。形式化地，$p(x_{2087} \mid y) = p(x_{2087} \mid y, x_{39831})$。（注意，这并不是说 x_{2087} 和 x_{39831} 独立；独立应写为 $p(x_{2087}) = p(x_{2087} \mid x_{39831})$。这里只假设给定 y 时两者条件独立。）

于是有

$$\begin{aligned} p(x_1, \ldots, x_{50000} \mid y) &= p(x_1 \mid y)p(x_2 \mid y, x_1)p(x_3 \mid y, x_1, x_2) \cdots p(x_{50000} \mid y, x_1, \ldots, x_{49999}) \\ &= p(x_1 \mid y)p(x_2 \mid y)p(x_3 \mid y) \cdots p(x_{50000} \mid y) \\ &= \prod_{j=1}^{d} p(x_j \mid y). \end{aligned}$$

第一个等式直接来自概率的基本性质，第二个等式使用了 NB 假设。值得指出，尽管朴素贝叶斯假设非常强，得到的算法在许多问题上仍表现良好。

模型的参数为 $\phi_{j|y=1} = p(x_j = 1 \mid y = 1)$、$\phi_{j|y=0} = p(x_j = 1 \mid y = 0)$ 和 $\phi_y = p(y = 1)$。与往常一样，给定训练集 $\{(x^{(i)}, y^{(i)}); i = 1, \ldots, n\}$，可以写出数据的联合似然：

$$L(\phi_y, \phi_{j|y=0}, \phi_{j|y=1}) = \prod_{i=1}^{n} p(x^{(i)}, y^{(i)}).$$

关于 $\phi_y, \phi_{j|y=0}, \phi_{j|y=1}$ 最大化此式，得到最大似然估计：

[13] 实际中，比起查阅英语词典来列出所有英文单词，更常见的是遍历训练集，只把至少出现过一次的词编码到特征向量中。这样除了减少建模的词数，从而减少计算和存储需求之外，还有一个优点：可以建模或纳入许多可能出现在邮件中、但词典中找不到的词，例如 “cs229”。有时（如本课程作业），还会排除频率特别高的词，例如 “the” “of” “and”；这些高频但 “不含内容” 的词称为**停用词**。它们出现在太多文档中，几乎不能帮助判断邮件是否为垃圾邮件。

$$\phi_{j|y=1} = \frac{\sum_{i=1}^{n} \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 1\}},$$
$$\phi_{j|y=0} = \frac{\sum_{i=1}^{n} \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 0\}},$$
$$\phi_y = \frac{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 1\}}{n}.$$

上式中的 $\wedge$ 表示“且”。这些参数有非常自然的解释。例如，$\phi_{j|y=1}$ 就是在所有垃圾邮件（$y = 1$）中，出现了第 j 个词的邮件所占的比例。

拟合这些参数后，要对特征为 x 的新样本预测，只需计算

$$\begin{aligned} p(y = 1 \mid x) &= \frac{p(x \mid y = 1)p(y = 1)}{p(x)} \\ &= \frac{\left(\prod_{j=1}^{d} p(x_j \mid y = 1)\right) p(y = 1)}{\left(\prod_{j=1}^{d} p(x_j \mid y = 1)\right) p(y = 1) + \left(\prod_{j=1}^{d} p(x_j \mid y = 0)\right) p(y = 0)}, \end{aligned}$$

并选择后验概率较大的类别。

最后，虽然我们主要针对特征 x_j 为二元值的情况推导了朴素贝叶斯算法，但把它推广到 $x_j \in \{1, 2, \ldots, k_j\}$ 非常直接：只需用多项分布而不是伯努利分布建模 $p(x_j \mid y)$。事实上，即使某些原始输入属性是连续值（例如前面例子中的房屋居住面积），也经常会把它**离散化**，即转换为少量离散值，再应用朴素贝叶斯。例如，若用特征 x_j 表示居住面积，可以按如下方式离散化：

居住面积（平方英尺）	< 400	400—800	800—1200	1200—1600	> 1600
x_j	1	2	3	4	5

因此，对居住面积 890 平方英尺的房屋，对应特征 x_j 设为 3。然后就可以应用朴素贝叶斯，用多项分布建模 $p(x_j \mid y)$，如前所述。当原始连续属性不能被多元正态分布很好地建模时，离散化特征并使用朴素贝叶斯而不是 GDA，往往能得到更好的分类器。

4.2.1 拉普拉斯平滑

上述朴素贝叶斯算法在许多问题上表现不错，但一个简单改动就能显著改善效果，尤其是在文本分类中。先简要讨论当前算法的一个问题，再介绍修正方法。

考虑垃圾邮件分类。假设现在是 20xx 年，你完成了 CS229，并在课程项目中做出了出色的工作，于是决定在 20xx 年 5 月左右把成果投稿到 NeurIPS 会议。[14] 因为你在邮件中讨论这个会议，也开始收到包含 “neurips” 的邮件。但这是你的第一篇 NeurIPS 论文，此前从未见过包含 “neurips” 的邮件；特别地，“neurips” 从未在垃圾 / 非垃圾邮件训练集中出现。假设它是词典中第 35000 个词，那么朴素贝叶斯垃圾邮件过滤器选出的参数 $\phi_{35000|y}$ 的最大似然估计为

[14] NeurIPS 是顶级机器学习会议之一，论文投稿截止时间通常在 5—6 月。

$$\phi_{35000|y=1} = \frac{\sum_{i=1}^{n} \mathbf{1}\{x_{35000}^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 1\}} = 0,$$
$$\phi_{35000|y=0} = \frac{\sum_{i=1}^{n} \mathbf{1}\{x_{35000}^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 0\}} = 0.$$

也就是说，因为此前在两类训练样本中都没有见过 “neurips”，算法认为在任一类邮件中看到它的概率都是零。因此，尝试判断这些包含 “neurips” 的邮件是否为垃圾邮件时，它计算类别后验概率，得到

$$\begin{aligned} p(y = 1 \mid x) &= \frac{\prod_{j=1}^{d} p(x_j \mid y = 1)p(y = 1)}{\prod_{j=1}^{d} p(x_j \mid y = 1)p(y = 1) + \prod_{j=1}^{d} p(x_j \mid y = 0)p(y = 0)} \\ &= \frac{0}{0}. \end{aligned}$$

这是因为每个乘积 $\prod_{j=1}^{d} p(x_j \mid y)$ 都乘上了 $p(x_{35000} \mid y) = 0$ 这一项。于是算法得到 0/0，不知道如何预测。

更一般地说，仅仅因为有限训练集中没见过某个事件，就把它的概率估计为零，从统计角度看并不是好主意。考虑估计取值于 $\{1, \ldots, k\}$ 的多项随机变量 z 的均值。可以用 $\phi_j = p(z = j)$ 参数化这个多项分布。给定 n 个独立观测 $\{z^{(1)}, \ldots, z^{(n)}\}$，最大似然估计为

$$\phi_j = \frac{\sum_{i=1}^{n} \mathbf{1}\{z^{(i)} = j\}}{n}.$$

如前所见，使用这些最大似然估计时，某些 ϕ_j 可能为零，因而造成问题。为避免这一点，可以使用**拉普拉斯平滑**，把上述估计替换为

$$\phi_j = \frac{1 + \sum_{i=1}^{n} \mathbf{1}\{z^{(i)} = j\}}{k + n}.$$

这里给分子加了 1，给分母加了 k。注意，$\sum_{j=1}^{k} \phi_j = 1$ 仍然成立（请自行验证 !），这是我们希望保持的性质，因为这些 ϕ_j 是概率估计，和必须为 1。此外，对所有 j，$\phi_j \neq 0$，解决了概率被估计为零的问题。在某些可以说相当强的条件下，可以证明拉普拉斯平滑实际上给出了 ϕ_j 的最优估计量。

回到朴素贝叶斯分类器，采用拉普拉斯平滑后，得到以下参数估计：

$$\phi_{j|y=1} = \frac{1 + \sum_{i=1}^{n} \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{2 + \sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 1\}},$$
$$\phi_{j|y=0} = \frac{1 + \sum_{i=1}^{n} \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{2 + \sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 0\}}.$$

（实践中，是否也对 ϕ_y 应用拉普拉斯平滑通常影响不大，因为两类邮件通常都占相当比例，因而 ϕ_y 会是 $p(y = 1)$ 的合理估计，本来就不会接近 0。）

4.2.2 文本分类的事件模型

作为生成式学习算法讨论的结尾，再介绍一种专门用于文本分类的模型。前面介绍的朴素贝叶斯在许多分类问题上表现良好，但对文本分类而言，有一个相关模型表现更好。

在文本分类这一特定语境中，前面介绍的朴素贝叶斯使用所谓的**伯努利事件模型**（有时称为多元伯努利事件模型）。模型假设一封邮件按如下方式生成：首先，根据类别先验 $p(y)$ 随机决定下一封邮件来自垃圾邮件发送者还是普通发送者。随后，发送者遍历词典，根据概率 $p(x_j = 1 \mid y) = \phi_{j|y}$，独立决定是否将每个词 j 放入邮件。因此，一封邮件的概率为 $p(y)\prod_{j=1}^{d} p(x_j \mid y)$。

另一种模型称为**多项事件模型**。为描述这个模型，我们采用不同的记号和特征来表示邮件。令 x_j 表示邮件中第 j 个词的词典编号。因此，x_j 现在是取值于 $\{1,\dots,|V|\}$ 的整数，其中 $|V|$ 是词汇表（词典）大小。包含 d 个词的邮件表示为长度为 d 的向量 $(x_1, x_2, \dots, x_d)$；注意，不同文档的 d 可以不同。例如，若邮件以 “A NeurIPS…” 开头，则 $x_1 = 1$（“a” 是词典中第一个词），$x_2 = 35000$（假设 “neurips” 是词典中第 35000 个词）。

在多项事件模型中，邮件由一个随机过程生成：首先像之前一样，根据 $p(y)$ 决定它是不是垃圾邮件。然后，发送者先从某个定义在词汇上的多项分布 $p(x_1 \mid y)$ 生成 x_1。接着，从同一个多项分布中独立于 x_1 选出第二个词 x_2，x_3, x_4 等依此类推，直到生成全部 d 个词。因此，一封邮件的总概率为 $p(y)\prod_{j=1}^{d} p(x_j \mid y)$。注意，这个公式看起来与伯努利事件模型中的邮件概率相同，但各项的含义已完全不同。特别地，$x_j \mid y$ 现在服从多项分布，而不是伯努利分布。

新模型的参数仍包括 $\phi_y = p(y)$，以及 $\phi_{k|y=1} = p(x_j = k \mid y = 1)$（任意 j）和 $\phi_{k|y=0} = p(x_j = k \mid y = 0)$。注意，我们假设所有位置 j 的 $p(x_j \mid y)$ 都相同，即生成某个词所用的分布不依赖于它在邮件中的位置 j。

给定训练集 $\{(x^{(i)}, y^{(i)}); i = 1, \dots, n\}$，其中 $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_{d_i}^{(i)})$，$d_i$ 为第 i 个训练样本中的词数，则数据似然为

$$
\begin{aligned}
L(\phi_y, \phi_{k|y=0}, \phi_{k|y=1}) &= \prod_{i=1}^{n} p(x^{(i)}, y^{(i)}) \\
&= \prod_{i=1}^{n} \left(\prod_{j=1}^{d_i} p(x_j^{(i)} \mid y^{(i)}; \phi_{k|y=0}, \phi_{k|y=1}) \right) p(y^{(i)}; \phi_y).
\end{aligned}
$$

最大化这个式子，得到参数的最大似然估计：

$$
\begin{aligned}
\phi_{k|y=1} &= \frac{\sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}\{x_j^{(i)} = k \wedge y^{(i)} = 1\}}{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 1\} d_i}, \\
\phi_{k|y=0} &= \frac{\sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}\{x_j^{(i)} = k \wedge y^{(i)} = 0\}}{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 0\} d_i}, \\
\phi_y &= \frac{\sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 1\}}{n}.
\end{aligned}
$$

如果在估计 $\phi_{k|y=0}$ 和 $\phi_{k|y=1}$ 时使用拉普拉斯平滑（实践中要取得良好性能，需要这样做），则给分子加 1、分母加 $|V|$，得到

$$\phi_{k|y=1} = \frac{1 + \sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}\{x_j^{(i)} = k \wedge y^{(i)} = 1\}}{|V| + \sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 1\} d_i},$$
$$\phi_{k|y=0} = \frac{1 + \sum_{i=1}^{n} \sum_{j=1}^{d_i} \mathbf{1}\{x_j^{(i)} = k \wedge y^{(i)} = 0\}}{|V| + \sum_{i=1}^{n} \mathbf{1}\{y^{(i)} = 0\} d_i}.$$

朴素贝叶斯分类器未必是最好的分类算法，但常常表现得出乎意料地好。由于它简单而且容易实现，通常也非常适合作为 “首先尝试” 的方法。

第 5 章　核方法

原文印刷页码：49–59。

5.1 特征映射

回顾我们讨论线性回归时考虑的问题：根据房屋的居住面积（记为 x）预测房价（记为 y），并用关于 x 的线性函数拟合训练数据。如果用 x 的非线性函数能够更准确地表示价格 y，该怎么办？这时，我们需要比线性模型更有表达能力的模型族。

我们先考虑拟合三次函数 $y = \theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0$。原来，我们可以把三次函数看作另一组特征变量（定义如下）的线性函数。具体来说，定义函数 $\phi : \mathbb{R} \to \mathbb{R}^4$：

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix} \in \mathbb{R}^4. \tag{5.1}$$

令 $\theta \in \mathbb{R}^4$ 为由 $\theta_0, \theta_1, \theta_2, \theta_3$ 组成的向量。于是可以把关于 x 的三次函数改写为

$$\theta_3 x^3 + \theta_2 x^2 + \theta_1 x + \theta_0 = \theta^T \phi(x).$$

因此，变量 x 的三次函数可以看作变量 $\phi(x)$ 的线性函数。为区分这两组变量，在核方法的语境中，我们把 “原始” 输入值称为问题的**输入属性**（本例中为居住面积 x）。当原始输入被映射为一组新的量 $\phi(x)$ 时，我们把这些新量称为**特征变量**。（遗憾的是，不同作者在不同语境下使用的术语并不相同。）我们把将属性映射到特征的 ϕ 称为**特征映射**。

5.2 带特征的 LMS（最小均方）算法

下面推导拟合模型 $\theta^T \phi(x)$ 的梯度下降算法。首先回顾，在拟合 $\theta^T x$ 的普通最小二乘问题中，批量梯度下降的更新为（推导见第一份讲义）：

$$\begin{aligned} \theta &:= \theta + \alpha \sum_{i=1}^{n} \left(y^{(i)} - h_\theta(x^{(i)})\right) x^{(i)} \\ &:= \theta + \alpha \sum_{i=1}^{n} \left(y^{(i)} - \theta^T x^{(i)}\right) x^{(i)}. \end{aligned} \tag{5.2}$$

设 $\phi : \mathbb{R}^d \to \mathbb{R}^p$ 是一个特征映射，把 $\mathbb{R}^d$ 中的属性 x 映射到 $\mathbb{R}^p$ 中的特征 $\phi(x)$。（上一节的引例中，$d = 1, p = 4$。）现在我们的目标是拟合函数 $\theta^T\phi(x)$，其中 θ 是 $\mathbb{R}^p$ 中而非 $\mathbb{R}^d$ 中的向量。把上述算法中所有的 $x^{(i)}$ 替换为 $\phi(x^{(i)})$，便得到新的更新规则：

$$\theta := \theta + \alpha \sum_{i=1}^{n} \big(y^{(i)} - \theta^T\phi(x^{(i)})\big)\phi(x^{(i)}). \tag{5.3}$$

类似地，对应的随机梯度下降更新规则为

$$\theta := \theta + \alpha\big(y^{(i)} - \theta^T\phi(x^{(i)})\big)\phi(x^{(i)}). \tag{5.4}$$

5.3 使用核技巧的 LMS

当特征 $\phi(x)$ 的维度很高时，上述梯度下降或随机梯度更新的计算代价会很高。例如，把式 (5.1) 中的特征映射直接推广到高维输入 x：假设 $x \in \mathbb{R}^d$，令 $\phi(x)$ 是包含 x 的所有次数不超过 3 的单项式的向量：

$$\phi(x) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_1^2 \\ x_1x_2 \\ x_1x_3 \\ \vdots \\ x_2x_1 \\ \vdots \\ x_1^3 \\ x_1^2x_2 \\ \vdots \end{bmatrix}. \tag{5.5}$$

特征 $\phi(x)$ 的维度为 d^3 的量级。[15] 从计算角度看，这个向量长得令人难以承受：当 $d = 1000$ 时，每次更新至少需要计算和存储一个 $1000^3 = 10^9$ 维向量，比普通最小二乘的更新规则 (5.2) 慢 10^6 倍。

乍看起来，每次更新所需的 d^3 量级运行时间和内存开销似乎不可避免，因为向量 θ 本身的维度就是 $p \approx d^3$，而我们可能需要更新并存储它的每个分量。不过，下面将介绍**核技巧**：使用它就不必显式存储 θ，运行时间也能大幅改善。

为简便起见，把 θ 初始化为 0，并集中讨论迭代更新 (5.3)。关键观察是：在任意时刻，θ 都可以表示为向量 $\phi(x^{(1)}), \dots, \phi(x^{(n)})$ 的线性组合。事实上，这可以用归纳法证明。初始化时，$\theta = 0 = \sum_{i=1}^{n} 0 \cdot \phi(x^{(i)})$。假设在某一时刻，对某些 $\beta_1, \dots, \beta_n \in \mathbb{R}$，可以写成

[15] 为了简便，这里保留所有单项式的重复项，例如 $x_1x_2x_3$ 和 $x_2x_3x_1$ 都出现在 $\phi(x)$ 中。因此，$\phi(x)$ 总共有 $1+d+d^2+d^3$ 个分量。

$$\theta = \sum_{i=1}^{n} \beta_i \phi(x^{(i)}). \tag{5.6}$$

我们断言，下一轮中的 θ 仍是 $\phi(x^{(1)}), \dots, \phi(x^{(n)})$ 的线性组合，因为

$$\begin{aligned} \theta &:= \theta + \alpha \sum_{i=1}^{n} \big(y^{(i)} - \theta^T \phi(x^{(i)})\big)\phi(x^{(i)}) \\ &= \sum_{i=1}^{n} \beta_i \phi(x^{(i)}) + \alpha \sum_{i=1}^{n} \big(y^{(i)} - \theta^T \phi(x^{(i)})\big)\phi(x^{(i)}) \\ &= \sum_{i=1}^{n} \underbrace{\big(\beta_i + \alpha(y^{(i)} - \theta^T \phi(x^{(i)}))\big)}_{\text{新的}\beta_i} \phi(x^{(i)}). \end{aligned} \tag{5.7}$$

你可能已经意识到，我们的总体策略是用一组系数 $\beta_1, \dots, \beta_n$ 隐式表示 p 维向量 θ。为此，我们推导这些系数的更新规则。由上式可知，新 β_i 与旧值的关系为

$$\beta_i := \beta_i + \alpha\big(y^{(i)} - \theta^T \phi(x^{(i)})\big). \tag{5.8}$$

这里等式右侧仍含有旧的 θ。将其替换为 $\theta = \sum_{j=1}^{n} \beta_j \phi(x^{(j)})$，得到

$$\forall i \in \{1, \dots, n\}, \quad \beta_i := \beta_i + \alpha\left(y^{(i)} - \sum_{j=1}^{n} \beta_j \phi(x^{(j)})^T \phi(x^{(i)})\right).$$

我们常把 $\phi(x^{(j)})^T \phi(x^{(i)})$ 写作 $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$，以强调它是两个特征向量的内积。把这些 β_i 看作 θ 的新表示，我们便成功地将批量梯度下降转化为迭代更新 β 的算法。看起来，每次迭代仍需对所有 i, j 计算 $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$，每次计算可能需要约 $O(p)$ 次运算。不过，下面两个重要性质可以解决这一困难：

1. 在循环开始前，可以预先计算所有 i, j 对应的成对内积 $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$。
2. 对式 (5.5) 定义的特征映射 ϕ（以及许多其他有用的特征映射），可以高效计算 $\langle \phi(x^{(j)}), \phi(x^{(i)}) \rangle$，不一定需要显式计算 $\phi(x^{(i)})$。这是因为：

$$\begin{aligned} \langle \phi(x), \phi(z) \rangle &= 1 + \sum_{i=1}^{d} x_i z_i + \sum_{i,j \in \{1,\dots,d\}} x_i x_j z_i z_j \\ &\quad + \sum_{i,j,k \in \{1,\dots,d\}} x_i x_j x_k z_i z_j z_k \\ &= 1 + \sum_{i=1}^{d} x_i z_i + \left(\sum_{i=1}^{d} x_i z_i\right)^2 + \left(\sum_{i=1}^{d} x_i z_i\right)^3 \\ &= 1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3. \end{aligned} \tag{5.9}$$

因此，为计算 $\langle \phi(x), \phi(z) \rangle$，可以先用 $O(d)$ 时间计算 $\langle x, z \rangle$，再用常数次运算计算 $1 + \langle x, z \rangle + \langle x, z \rangle^2 + \langle x, z \rangle^3$。

可以看到，特征之间的内积 $\langle \phi(x), \phi(z) \rangle$ 在这里至关重要。我们把对应于特征映射 ϕ 的核定义为从

$\mathcal{X} \times \mathcal{X}$ 映射到 $\mathbb{R}$ 的函数，并要求它满足：[16]

$$K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle. \tag{5.10}$$

总结以上讨论，最终算法如下：

1. 对所有 $i, j \in \{1, \dots, n\}$，使用式 (5.9) 计算 $K(x^{(i)}, x^{(j)}) \triangleq \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$。令 $\beta := 0$。
2. 循环更新：

$$\forall i \in \{1, \dots, n\}, \quad \beta_i := \beta_i + \alpha \left(y^{(i)} - \sum_{j=1}^{n} \beta_j K(x^{(i)}, x^{(j)}) \right). \tag{5.11}$$

或者，采用向量记号，令 K 为 $n \times n$ 矩阵，其元素 $K_{ij} = K(x^{(i)}, x^{(j)})$，则有

$$\beta := \beta + \alpha(\vec{y} - K\beta).$$

使用上述算法，我们可以高效地更新向量 θ 的表示 β，每次更新需要 $O(n)$ 时间。最后，需要说明仅知道表示 β 就足以计算预测值 $\theta^T \phi(x)$。事实上，

$$\theta^T \phi(x) = \sum_{i=1}^{n} \beta_i \phi(x^{(i)})^T \phi(x) = \sum_{i=1}^{n} \beta_i K(x^{(i)}, x). \tag{5.12}$$

你可能已经发现，从根本上说，我们需要了解的关于特征映射 $\phi(\cdot)$ 的全部信息，都封装在对应的核函数 $K(\cdot, \cdot)$ 中。下一节将进一步讨论这一点。

5.4 核的性质

上一节从显式定义的特征映射 ϕ 出发，由它诱导核函数 $K(x, z) \triangleq \langle \phi(x), \phi(z) \rangle$。随后我们看到，核函数如此关键，以至于只要定义了核函数，整个训练算法就可以完全用核来表述，无须提及特征映射 ϕ；对测试样本 x 进行预测时也一样，见式 (5.12)。

因此，我们自然会想定义其他核函数 $K(\cdot, \cdot)$，再运行算法 (5.11)。注意，算法 (5.11) 不需要显式访问特征映射 ϕ，所以只需保证 ϕ 存在，不一定需要显式写出它。

什么样的函数 $K(\cdot, \cdot)$ 能对应于某个特征映射 ϕ？换言之，我们能否判断是否存在某个 ϕ，使得对所有 x, z 都有 $K(x, z) = \phi(x)^T \phi(z)$？

如果能精确刻画有效核函数，从而回答这个问题，就可以把选择特征映射 ϕ 的方式完全改为选择核函数 K。具体来说，我们可以选取函数 K，验证它满足这一刻画条件（因此存在对应的特征映射 ϕ），再运行更新规则 (5.11)。其好处是无需能够计算 ϕ 或写出它的解析表达式，只需知道它存在即可。我们将在介绍几个具体的核的例子后，于本节末尾回答这个问题。

设 $x, z \in \mathbb{R}^d$，首先考虑函数

$$K(x, z) = (x^T z)^2.$$

[16] 回顾，$\mathcal{X}$ 是输入 x 的空间。在贯穿本节的例子中，$\mathcal{X} = \mathbb{R}^d$。

也可以写成

$$\begin{aligned} K(x,z) &= \left(\sum_{i=1}^{d} x_i z_i\right)\left(\sum_{j=1}^{d} x_j z_j\right) \\ &= \sum_{i=1}^{d}\sum_{j=1}^{d} x_i x_j z_i z_j \\ &= \sum_{i,j=1}^{d} (x_i x_j)(z_i z_j). \end{aligned}$$

因此，$K(x,z) = \langle \phi(x), \phi(z) \rangle$ 是对应于如下特征映射 ϕ 的核函数（这里展示 $d = 3$ 的情形）：

$$\phi(x) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \end{bmatrix}.$$

再从计算效率的角度看核：计算高维的 $\phi(x)$ 需要 $O(d^2)$ 时间，而计算 $K(x,z)$ 只需 $O(d)$ 时间，即与输入属性的维度成线性关系。

另一个相关例子是

$$\begin{aligned} K(x,z) &= (x^T z + c)^2 \\ &= \sum_{i,j=1}^{d} (x_i x_j)(z_i z_j) + \sum_{i=1}^{d} (\sqrt{2c}\, x_i)(\sqrt{2c}\, z_i) + c^2. \end{aligned}$$

（请自行验证。）这个函数 K 对应的特征映射为（仍展示 $d = 3$ 的情形）

$$\phi(x) = \begin{bmatrix} x_1x_1 \\ x_1x_2 \\ x_1x_3 \\ x_2x_1 \\ x_2x_2 \\ x_2x_3 \\ x_3x_1 \\ x_3x_2 \\ x_3x_3 \\ \sqrt{2c}\,x_1 \\ \sqrt{2c}\,x_2 \\ \sqrt{2c}\,x_3 \\ c \end{bmatrix},$$

参数 c 控制 x_i（一阶）项和 x_ix_j（二阶）项之间的相对权重。

更一般地，核 $K(x,z) = (x^Tz + c)^k$ 对应的特征映射将输入映射到一个 $\binom{d+k}{k}$ 维特征空间，其特征对应次数不超过 k 的所有形如 $x_{i_1}x_{i_2}\cdots x_{i_k}$ 的单项式。然而，尽管我们在这个 $O(d^k)$ 维空间中工作，计算 $K(x,z)$ 仍然只需 $O(d)$ 时间，所以始终不必显式表示这个极高维特征空间中的特征向量。

把核看作相似性度量。下面从一个略有不同的角度看核。直观地说（这个直觉有不完全正确的地方，但暂且不论），如果 $\phi(x)$ 和 $\phi(z)$ 彼此接近，我们可能期望 $K(x,z) = \phi(x)^T\phi(z)$ 较大。反过来，如果 $\phi(x)$ 和 $\phi(z)$ 相距很远，例如几乎互相正交，那么 $K(x,z) = \phi(x)^T\phi(z)$ 就会较小。因此，可以把 $K(x,z)$ 看作对 $\phi(x)$ 与 $\phi(z)$、或者 x 与 z 的相似程度的某种度量。

基于这种直觉，假设对正在处理的某个学习问题，你想到了一个函数 $K(x,z)$，认为它可以合理地度量 x 与 z 的相似程度。例如，你可能选择

$$K(x,z) = \exp\left(-\frac{\|x-z\|^2}{2\sigma^2}\right).$$

这确实是一个合理的相似性度量：当 x 与 z 接近时，它接近 1；相距很远时，它接近 0。是否存在特征映射 ϕ，使这个核满足 $K(x,z) = \phi(x)^T\phi(z)$？对这个具体例子，答案是肯定的。这个核称为**高斯核**，对应一个无限维的特征映射 ϕ。下面将精确刻画函数 K 要成为对应某个特征映射 ϕ 的有效核函数，必须满足哪些性质。

有效核的必要条件。暂时假设 K 确实是对应某个特征映射 ϕ 的有效核，先看看它满足哪些性质。考虑由 n 个点组成的任意有限集合 $\{x^{(1)},\dots,x^{(n)}\}$（不一定是训练集），并定义 $n\times n$ 方阵 K，其第 (i,j) 个元素为 $K_{ij} = K(x^{(i)},x^{(j)})$。这个矩阵称为**核矩阵**。注意，由于核函数与核矩阵显然密切相关，我们重用了记号 K，同时用它表示两者。

如果 K 是有效核，那么

$$K_{ij} = K(x^{(i)},x^{(j)}) = \phi(x^{(i)})^T\phi(x^{(j)}) = \phi(x^{(j)})^T\phi(x^{(i)}) = K(x^{(j)},x^{(i)}) = K_{ji},$$

所以 K 必须对称。此外，令 $\phi_k(x)$ 表示向量 $\phi(x)$ 的第 k 个坐标，则对任意向量 z，有

$$
\begin{aligned}
z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\
&= \sum_i \sum_j z_i \phi(x^{(i)})^T \phi(x^{(j)}) z_j \\
&= \sum_i \sum_j z_i \sum_k \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\
&= \sum_k \sum_i \sum_j z_i \phi_k(x^{(i)}) \phi_k(x^{(j)}) z_j \\
&= \sum_k \left(\sum_i z_i \phi_k(x^{(i)}) \right)^2 \\
&\geq 0.
\end{aligned}
$$

倒数第二步使用了 $\sum_{i,j} a_i a_j = (\sum_i a_i)^2$，其中 $a_i = z_i \phi_k(x^{(i)})$。由于 z 是任意的，这说明 K 是半正定的（$K \geq 0$）。

因此，我们证明了：如果 K 是有效核（即对应某个特征映射 ϕ），那么相应核矩阵 $K \in \mathbb{R}^{n \times n}$ 必定对称半正定。

有效核的充分条件。更一般地，上述条件实际上不仅是 K 成为有效核（也称为 Mercer 核）的必要条件，也是充分条件。以下结果由 Mercer 给出。[17]

定理（Mercer）。给定 $K : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$。$K$ 是有效的（Mercer）核，当且仅当对任意 $\{x^{(1)}, \ldots, x^{(n)}\}$（$n < \infty$），对应的核矩阵都是对称半正定的。

因此，对于给定函数 K，除了尝试寻找对应的特征映射 ϕ，这个定理提供了另一种检验它是否为有效核的方法。在习题集 2 中，你还将有机会进一步运用这些思想。

课堂上，我们还简要讨论了另外几个核的例子。例如，数字识别问题要求根据一幅手写数字（0—9）的 16×16 像素图像，判断它是哪个数字。使用简单的多项式核 $K(x, z) = (x^T z)^k$ 或高斯核，支持向量机都能在这个问题上取得极好的表现。这尤其令人惊讶，因为输入属性 x 仅仅是由图像像素强度组成的 256 维向量，系统没有任何视觉先验知识，甚至不知道哪些像素彼此相邻。

课堂上简要讨论的另一个例子是：如果要分类的对象 x 是字符串（例如 x 是一列氨基酸，连接起来构成蛋白质），那么对大多数学习算法来说，构造一组合理而且“小”的特征似乎很困难，尤其当不同字符串长度不同时。不过，可以令 $\phi(x)$ 为一个特征向量，统计每个长度为 k 的子串在 x 中出现的次数。如果考虑由英文字母组成的字符串，这样的子串共有 26^k 个。所以 $\phi(x)$ 是一个 26^k 维向量；即使 k 只是中等大小，它也可能大到无法高效处理（例如 $26^4 \approx 460000$）。然而，使用类似动态规划的字符串匹配算法，可以高效计算 $K(x, z) = \phi(x)^T \phi(z)$，从而隐式地在这个 26^k 维特征空间中工作，而不必显式计算其中的特征向量。

核方法的应用。我们已经看到了核在线性回归中的应用。下一部分将介绍可以直接应用核的支持向量机，这里不再过多展开。事实上，核的思想远不限于线性回归和支持向量机。具体而言，如果某个学习算法可以完全通过输入属性向量之间的内积 $\langle x, z \rangle$ 来表述，那么只要用核 $K(x, z)$ 替换这些内积，就能“神奇地”使算法在 K 对应的高维特征空间中高效运行。例如，可以把核技巧用于感知机，得到核感知机算法。本课程后面介绍的许多算法也适用于这种方法，这就是所谓的“核技巧”。

[17] 许多教材以涉及 L^2 函数的稍复杂形式陈述 Mercer 定理；当输入属性取值于 $\mathbb{R}^d$ 时，这里的版本与之等价。

第 6 章　支持向量机

原文印刷页码：60–78。

本章介绍**支持向量机**（Support Vector Machine，SVM）学习算法。SVM 是最好的 “开箱即用” 监督学习算法之一，许多人甚至认为它就是最好的。要讲清楚 SVM，首先需要讨论间隔，以及用较大 “空隙” 分隔数据的思想。接下来介绍最优间隔分类器，并由此暂时转向拉格朗日对偶性。我们还将涉及核：它使 SVM 可以在极高维、甚至无限维特征空间中高效运行。最后介绍能够高效实现 SVM 的 SMO 算法。

6.1 间隔：直观理解

我们从间隔开始介绍 SVM。本节给出关于间隔以及预测 “置信程度” 的直觉；第 6.3 节将把这些思想形式化。

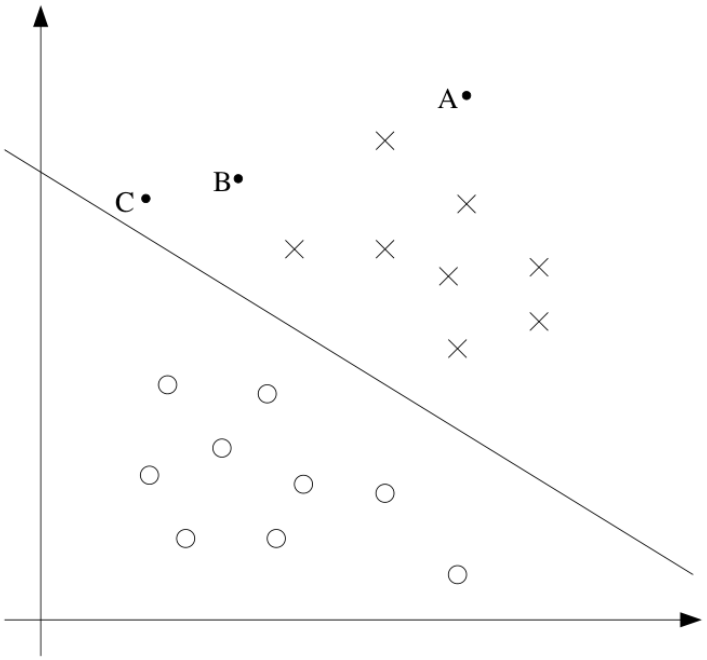
考虑逻辑回归，它用 $h_\theta(x) = g(\theta^T x)$ 建模概率 $p(y=1 \mid x;\theta)$。当且仅当 $h_\theta(x) \geq 0.5$，等价地，当且仅当 $\theta^T x \geq 0$ 时，对输入 x 预测为 “1”。考虑一个正训练样本（$y=1$）。$\theta^T x$ 越大，$h_\theta(x) = p(y=1 \mid x;\theta)$ 也越大，因此我们对标签为 1 的 “信心” 越强。于是，不严格地说，若 $\theta^T x \gg 0$，就可以认为预测非常确信 $y=1$。类似地，若 $\theta^T x \ll 0$，则认为逻辑回归十分确信地预测 $y=0$。给定训练集，如果能找到 θ，使 $y^{(i)}=1$ 时总有 $\theta^T x^{(i)} \gg 0$，而 $y^{(i)}=0$ 时总有 $\theta^T x^{(i)} \ll 0$，那么直观上似乎就很好地拟合了训练数据，因为这意味着对所有训练样本的分类都十分确信，而且正确。这似乎是一个值得追求的目标，稍后会用函数间隔的概念将它形式化。

再看另一种直觉。下图中，叉号表示正训练样本，圆圈表示负训练样本，图中还画出了决策边界，即满足 $\theta^T x = 0$ 的直线，也称为**分离超平面**，并标出了 A、B、C 三个点。

注意，点 A 离决策边界很远。如果要预测 A 处的 y 值，我们似乎应该十分确信它是 1。相反，点 C 离边界很近，虽然它位于我们预测 $y=1$ 的一侧，但只要稍微改变决策边界，就很容易使预测变成 $y=0$。所以，我们对 A 的预测远比对 C 的预测有信心。点 B 介于两者之间。更一般地，如果一个点远离分离超平面，我们对预测可能会更有信心。直观上，给定训练集，如果能找到一个决策边界，使所有训练样本的预测都正确且确信（即远离边界），那会很好。稍后会用几何间隔将这个想法形式化。

6.2 记号

为便于讨论 SVM，先引入一套新的分类记号。我们考虑一个二分类问题的线性分类器，标签为 y，特征为 x。从现在起，用 $y \in \{-1, 1\}$ 而不是 $\{0, 1\}$ 表示类别标签。同时，不再用向量 θ 参数化线性分类器，而改用参数 w, b，写作



决策边界与距离不同的 A、B、C 三个点。

$$h_{w,b}(x) = g(w^T x + b).$$

这里，$z \geq 0$ 时 $g(z) = 1$，否则 $g(z) = -1$。这种 w, b 记号使我们能够明确地把截距 b 与其他参数分开处理。（同时取消在输入特征向量中增加坐标 $x_0 = 1$ 的约定。）因此，b 对应先前的 θ_0，w 对应 $[\theta_1 \ \cdots \ \theta_d]^T$。

还要注意，根据上面的 g 的定义，分类器直接预测 1 或 -1（可与感知机算法比较），不经过先估计 $p(y = 1)$ 的中间步骤，而逻辑回归有这一步。

6.3 函数间隔与几何间隔

下面正式定义函数间隔与几何间隔。给定训练样本 $(x^{(i)}, y^{(i)})$，定义 (w, b) 相对于该样本的**函数间隔**为

$$\hat{\gamma}^{(i)} = y^{(i)}(w^T x^{(i)} + b).$$

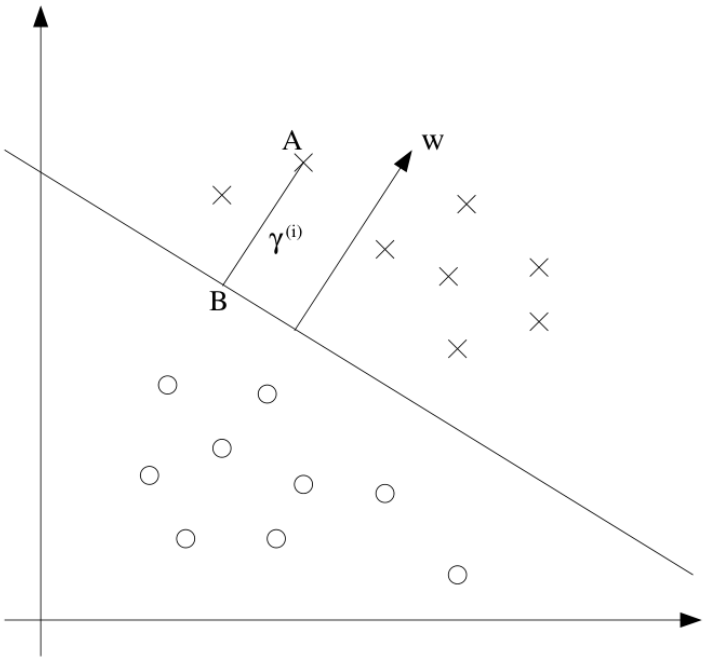
注意，若 $y^{(i)} = 1$，要使函数间隔大（即预测确信而且正确），$w^T x^{(i)} + b$ 必须是一个较大的正数。相反，若 $y^{(i)} = -1$，要使函数间隔大，$w^T x^{(i)} + b$ 就必须是一个较大的负数。此外，若 $y^{(i)}(w^T x^{(i)} + b) > 0$，则对该样本的预测是正确的。（请自行验证。）因此，大的函数间隔代表确信且正确的预测。

不过，对于采用上述 g（取值于 $\{-1, 1\}$）的线性分类器，函数间隔有一个性质，使它并不是很好的置信程度度量。因为 $g(w^T x + b) = g(2w^T x + 2b)$，把 w 换为 $2w$、b 换为 $2b$，完全不会改变 $h_{w,b}(x)$。也就是说，g 以及 $h_{w,b}(x)$ 只依赖于 $w^T x + b$ 的正负号，而不依赖其大小。然而，把 (w, b) 换为 $(2w, 2b)$，却会把函数间隔乘以 2。所以，利用缩放 w, b 的自由度，可以让函数间隔任意大，却没有改变任何有意义的东西。直观上，施加某种归一化条件，例如 $\|w\|_2 = 1$，可能是合理的；也就是说，可以把 (w, b) 替换为 $(w/\|w\|_2, b/\|w\|_2)$，然后考虑后者的函数间隔。稍后还会回到这一点。

给定训练集 $S = \{(x^{(i)}, y^{(i)}); i = 1, \ldots, n\}$，还定义 (w, b) 相对于 S 的函数间隔为所有训练样本函数间隔的最小值。记为 $\hat{\gamma}$，即

$$\hat{\gamma} = \min_{i=1,\ldots,n} \hat{\gamma}^{(i)}.$$

接下来讨论**几何间隔**。考虑下图：



几何间隔为样本到分离超平面的有符号距离。

图中画出了 (w, b) 对应的决策边界，以及向量 w。注意，w 与分离超平面正交，即夹角为 90°。（请自行确认这一点。）考虑点 A，它代表标签 $y^{(i)} = 1$ 的某个训练样本的输入 $x^{(i)}$。它到决策边界的距离 $\gamma^{(i)}$ 由线段 AB 给出。

如何计算 $\gamma^{(i)}$？$w/\|w\|$ 是沿 w 方向的单位向量。由于 A 表示 $x^{(i)}$，所以 B 为 $x^{(i)} - \gamma^{(i)} w/\|w\|$。B 位于决策边界，而边界上所有点 x 都满足 $w^T x + b = 0$，所以

$$w^T \left(x^{(i)} - \gamma^{(i)} \frac{w}{\|w\|} \right) + b = 0.$$

解得

$$\gamma^{(i)} = \frac{w^T x^{(i)} + b}{\|w\|} = \left(\frac{w}{\|w\|} \right)^T x^{(i)} + \frac{b}{\|w\|}.$$

这是针对图中 A 为正训练样本的情形推导的，此时位于决策边界的“正”侧是正确的。更一般地，定义 (w, b) 相对于训练样本 $(x^{(i)}, y^{(i)})$ 的几何间隔为

$$\gamma^{(i)} = y^{(i)} \left(\left(\frac{w}{\|w\|} \right)^T x^{(i)} + \frac{b}{\|w\|} \right).$$

注意，若 $\|w\| = 1$，函数间隔就等于几何间隔，这把两种间隔联系起来。此外，几何间隔在参数缩放下不变；例如，把 w 换为 $2w$、b 换为 $2b$，几何间隔不变。这个性质稍后会很有用。具体而言，因为几何间隔不受参数缩放影响，在用训练数据拟合 w, b 时，可以对 w 施加任意缩放约束，而不改变重要的东西。例如，可以要求 $\|w\| = 1$，或 $|w_1| = 5$，或 $|w_1 + b| + |w_2| = 2$；只要缩放 w, b，就可以满足这些条件。

最后，给定训练集 $S = \{(x^{(i)}, y^{(i)}); i = 1, \dots, n\}$，定义 (w, b) 相对于 S 的几何间隔为所有训练样本几何间隔的最小值：

$$\gamma = \min_{i=1,\dots,n} \gamma^{(i)}.$$

6.4 最优间隔分类器

根据前面的讨论，给定训练集，一个自然的目标是寻找使几何间隔最大的决策边界，因为这意味着对训练集作出十分确信的预测，并且很好地 “拟合” 训练数据。具体来说，这将得到一个在正负训练样本之间留出 “空隙”（几何间隔）的分类器。

暂时假设训练集线性可分，即存在某个分离超平面可以把正负样本分开。如何找到几何间隔最大的那个？可以提出以下优化问题：

$$\begin{aligned} \max_{\gamma, w, b} \quad & \gamma \\ \text{满足} \quad & y^{(i)}(w^T x^{(i)} + b) \geq \gamma, \quad i = 1, \dots, n, \\ & \|w\| = 1. \end{aligned}$$

即在每个训练样本的函数间隔至少为 γ 的约束下，最大化 γ。约束 $\|w\| = 1$ 保证函数间隔等于几何间隔，因此所有几何间隔也至少为 γ。所以，解这个问题会得到使训练集几何间隔尽可能大的 (w, b)。

如果能够解这个优化问题，任务就完成了。但 $\|w\| = 1$ 是一个棘手的非凸约束，这个问题显然不是能直接交给标准优化软件求解的形式。因此，尝试把它转化为更好的形式。考虑

$$\begin{aligned} \max_{\hat{\gamma}, w, b} \quad & \frac{\hat{\gamma}}{\|w\|} \\ \text{满足} \quad & y^{(i)}(w^T x^{(i)} + b) \geq \hat{\gamma}, \quad i = 1, \dots, n. \end{aligned}$$

这里，在所有函数间隔至少为 $\hat{\gamma}$ 的条件下，最大化 $\hat{\gamma}/\|w\|$。因为几何间隔与函数间隔的关系为 $\gamma = \hat{\gamma}/\|w\|$，所以这会得到想要的答案。并且我们去掉了不喜欢的约束 $\|w\| = 1$。缺点是现在目标函数 $\hat{\gamma}/\|w\|$ 又变得棘手，同样非凸，仍没有现成的软件可求解这种形式的优化问题。

继续变换。回顾前面说过，可以给 w, b 增加任意缩放约束而不改变重要的东西。这就是现在要用的关键思想。我们引入缩放约束，要求 w, b 相对于训练集的函数间隔为 1：

$$\hat{\gamma} = 1.$$

因为 w, b 乘以某个常数，函数间隔也会乘以同一个常数，所以这确实是一个可以通过缩放 w, b 满足的约束。把它代入上面的问题，并注意最大化 $\hat{\gamma}/\|w\| = 1/\|w\|$ 等价于最小化 $\|w\|^2$，得到

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 \\ \text{满足} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad i = 1, \ldots, n. \end{aligned}$$

现在，问题已变为能够高效求解的形式：目标是凸二次函数，约束全部线性。它的解给出**最优间隔分类器**。这个优化问题可以用商业二次规划（QP）软件求解。[18]

虽然到这里可以认为问题已经解决，但我们接下来会暂时转向讨论拉格朗日对偶性，由此得到优化问题的对偶形式。对偶形式是利用核让最优间隔分类器在极高维空间中高效工作的关键，也将使我们能够推导出求解上述优化问题的高效算法，通常比通用 QP 软件表现好得多。

6.5 拉格朗日对偶性

暂时放下 SVM 和最大间隔分类器，讨论如何求解约束优化问题。

考虑如下形式的问题：

$$\begin{aligned} \min_{w} \quad & f(w) \\ \text{满足} \quad & h_i(w) = 0, \quad i = 1, \ldots, l. \end{aligned}$$

有些读者可能记得如何用拉格朗日乘数法求解（没见过也不必担心）。该方法定义**拉格朗日函数**为

$$\mathcal{L}(w, \beta) = f(w) + \sum_{i=1}^{l} \beta_i h_i(w).$$

其中 β_i 称为**拉格朗日乘子**。随后求出 $\mathcal{L}$ 的偏导数并令其为零：

$$\frac{\partial \mathcal{L}}{\partial w_i} = 0; \qquad \frac{\partial \mathcal{L}}{\partial \beta_i} = 0,$$

再求解 w, β。

本节将推广到同时含有不等式和等式约束的优化问题。受时间限制，课程无法完整深入地讲授拉格朗日对偶理论，[19] 但会给出主要思想和结论，再应用于最优间隔分类器的优化问题。

考虑如下问题，称为**原始优化问题**：

$$\begin{aligned} \min_{w} \quad & f(w) \\ \text{满足} \quad & g_i(w) \leq 0, \quad i = 1, \ldots, k, \\ & h_i(w) = 0, \quad i = 1, \ldots, l. \end{aligned}$$

为求解它，先定义**广义拉格朗日函数**

[18] 你可能熟悉线性规划，它求解目标和约束均线性的优化问题。二次规划软件也广泛可得，允许使用凸二次目标和线性约束。

[19] 希望进一步了解的读者可阅读，例如 R. T. Rockafellar（1970），《Convex Analysis》，Princeton University Press。

$$\mathcal{L}(w,\alpha,\beta) = f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w).$$

其中 α_i, β_i 为拉格朗日乘子。考虑

$$\theta_{\mathcal{P}}(w) = \max_{\alpha,\beta:\alpha_i\geq 0} \mathcal{L}(w,\alpha,\beta).$$

下标 $\mathcal{P}$ 表示 “原始”。给定某个 w，若它违反任何原始约束，即存在某个 i 使 $g_i(w) > 0$ 或 $h_i(w) \neq 0$，则可以验证

$$\theta_{\mathcal{P}}(w) = \max_{\alpha,\beta:\alpha_i\geq 0} \left[f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w) \right]. \tag{6.1}$$

$$\theta_{\mathcal{P}}(w) = \infty. \tag{6.2}$$

相反，如果某个 w 满足约束，则 $\theta_{\mathcal{P}}(w) = f(w)$。因此，

$$\theta_{\mathcal{P}}(w) = \begin{cases} f(w), & w \text{ 满足原始约束,} \\ \infty, & \text{否则.} \end{cases}$$

所以，对于所有满足原始约束的 w，$\theta_{\mathcal{P}}$ 与问题的目标函数相同；违反约束时则为正无穷。因此，最小化问题

$$\min_w \theta_{\mathcal{P}}(w) = \min_w \max_{\alpha,\beta:\alpha_i\geq 0} \mathcal{L}(w,\alpha,\beta)$$

与最初的原始问题相同，即具有相同的解。为了后面使用，定义目标函数最优值 $p^* = \min_w \theta_{\mathcal{P}}(w)$，称为原始问题的值。

现在看一个略有不同的问题。定义

$$\theta_{\mathcal{D}}(\alpha,\beta) = \min_w \mathcal{L}(w,\alpha,\beta).$$

下标 $\mathcal{D}$ 表示 “对偶”。注意，定义 $\theta_{\mathcal{P}}$ 时是对 α, β 优化（最大化），这里则是对 w 最小化。

由此提出**对偶优化问题**：

$$\max_{\alpha,\beta:\alpha_i\geq 0} \theta_{\mathcal{D}}(\alpha,\beta) = \max_{\alpha,\beta:\alpha_i\geq 0} \min_w \mathcal{L}(w,\alpha,\beta).$$

这与上面的原始问题完全一样，区别仅在于交换了 “max” 和 “min” 的次序。定义对偶问题的最优目标值为 $d^* = \max_{\alpha,\beta:\alpha_i\geq 0} \theta_{\mathcal{D}}(\alpha,\beta)$。

原始问题与对偶问题有何关系？容易证明

$$d^* = \max_{\alpha,\beta:\alpha_i\geq 0} \min_w \mathcal{L}(w,\alpha,\beta) \leq \min_w \max_{\alpha,\beta:\alpha_i\geq 0} \mathcal{L}(w,\alpha,\beta) = p^*.$$

请自行确认这一点；它来自一个函数的 “max min” 总是不大于 “min max”。但在某些条件下，有

$$d^* = p^*,$$

这时就能以求解对偶问题代替求解原始问题。下面看这些条件是什么。

假设 f 和所有 g_i 为凸函数，[20] 所有 h_i 为仿射函数。[21] 再假设约束 g_i 严格可行，即存在某个 w，使所有 i 都满足 $g_i(w) < 0$。

在上述假设下，必存在 w^*, α^*, β^*，使 w^* 为原始问题的解，α^*, β^* 为对偶问题的解，并且 $p^* = d^* = \mathcal{L}(w^*, \alpha^*, \beta^*)$。此外，它们满足以下 **Karush–Kuhn–Tucker（KKT）条件**：

$$\frac{\partial}{\partial w_i}\mathcal{L}(w^*,\alpha^*,\beta^*) = 0, \quad i = 1,\dots,d. \tag{6.3}$$

$$\frac{\partial}{\partial \beta_i}\mathcal{L}(w^*,\alpha^*,\beta^*) = 0, \quad i = 1,\dots,l. \tag{6.4}$$

$$\alpha_i^* g_i(w^*) = 0, \quad i = 1,\dots,k. \tag{6.5}$$

$$g_i(w^*) \leq 0, \quad i = 1,\dots,k. \tag{6.6}$$

$$\alpha_i^* \geq 0, \quad i = 1,\dots,k. \tag{6.7}$$

另外，如果某组 w^*, α^*, β^* 满足 KKT 条件，那么 w^* 就是原始问题的解，α^*, β^* 是对偶问题的解。

特别注意式 (6.5)，它称为 **KKT 对偶互补条件**。它意味着：如果 $\alpha_i^* > 0$，那么 $g_i(w^*) = 0$。也就是说，约束 $g_i(w) \leq 0$ 是活跃的，即取等号而不是严格不等号。稍后，这将是证明 SVM 只有少量 “支持向量” 的关键；讨论 SMO 算法时，它也将给出收敛判据。

6.6 最优间隔分类器：对偶形式

注意：本节最重要的结论是优化问题 (6.8) 与 (6.12) 的等价性，以及式 (6.10) 中原始变量与对偶变量的关系。

前面，为寻找最优间隔分类器，我们提出了以下原始优化问题：

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 \\ \text{满足} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad i = 1,\dots,n. \end{aligned} \tag{6.8}$$

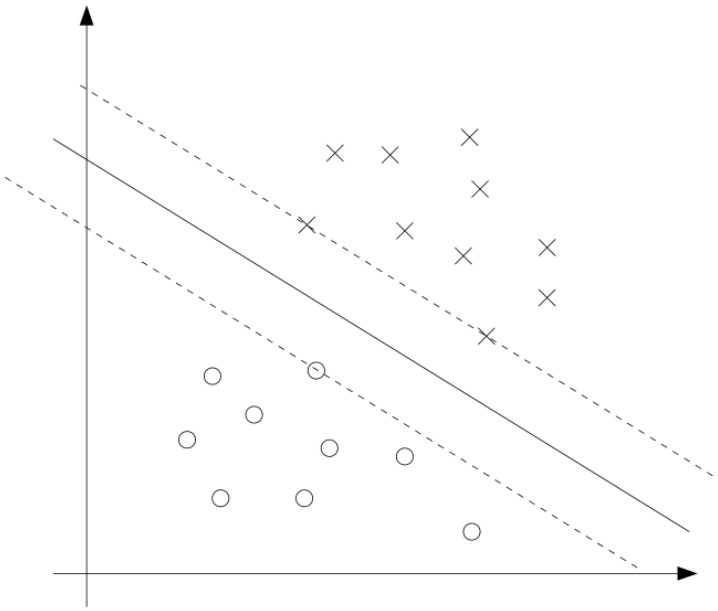
[20] 当 f 存在 Hessian 矩阵时，它为凸函数当且仅当 Hessian 半正定。例如，$f(w) = w^T w$ 是凸函数；所有线性函数和仿射函数也是凸的。（函数也可以不可微而仍为凸函数，不过这里不需要更一般的凸性定义。）

[21] 即存在 a_i, b_i，使 $h_i(w) = a_i^T w + b_i$。“仿射” 与线性相似，只是额外允许截距项 b_i。

可以把约束写成

$$g_i(w) = -y^{(i)}(w^T x^{(i)} + b) + 1 \le 0.$$

每个训练样本对应一个这样的约束。根据 KKT 对偶互补条件，只有函数间隔恰好等于 1 的训练样本，才可能有 $\alpha_i > 0$，即对应的约束取等号 $g_i(w) = 0$。考虑下图，其中实线表示最大间隔分离超平面。



最大间隔超平面与位于间隔边界上的支持向量。

间隔最小的点恰好是距离决策边界最近的点。这里是位于与决策边界平行的虚线上的三个点：一个负样本、两个正样本。因此，在优化问题的最优解处，只有对应这三个训练样本的三个 α_i 非零。这三个点称为该问题的**支持向量**。支持向量的数量可能远小于训练集大小，这一点稍后很有用。

继续推导。接下来推导对偶形式时，需要留意一个关键思想：我们将尝试只用输入特征空间中点之间的内积 $\langle x^{(i)}, x^{(j)} \rangle$（可以理解为 $(x^{(i)})^T x^{(j)}$）来写出算法。能够用这些内积表示算法，是应用核技巧的关键。

构造该优化问题的拉格朗日函数：

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{n} \alpha_i \big[y^{(i)}(w^T x^{(i)} + b) - 1 \big]. \tag{6.9}$$

注意，只有 α_i 而没有 β_i 乘子，因为问题仅包含不等式约束。

为得到对偶形式，首先在固定 α 时，对 w, b 最小化 $\mathcal{L}(w, b, \alpha)$，得到 $\theta_{\mathcal{D}}$。方法是令其关于 w, b 的导数为零。有

$$\nabla_w \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)} = 0,$$

因此

$$w = \sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)}. \tag{6.10}$$

关于 b 的导数给出

$$\frac{\partial}{\partial b}\mathcal{L}(w, b, \alpha) = -\sum_{i=1}^{n} \alpha_i y^{(i)} = 0. \tag{6.11}$$

译注（式 6.11）：原文将关于 b 的导数写为 $\sum_{i=1}^{n} \alpha_i y^{(i)} = 0$，漏写了负号。由式 (6.9) 求导应得 $-\sum_{i=1}^{n} \alpha_i y^{(i)} = 0$，此处补正；两者给出的等式约束相同。

将式 (6.10) 中的 w 代回拉格朗日函数 (6.9)，化简得到

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)} - b \sum_{i=1}^{n} \alpha_i y^{(i)}.$$

由式 (6.11)，最后一项必须为零，故

$$\mathcal{L}(w, b, \alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j (x^{(i)})^T x^{(j)}.$$

回顾，上式是通过对 w, b 最小化 $\mathcal{L}$ 得到的。结合一直存在的约束 $\alpha_i \geq 0$ 以及约束 (6.11)，得到以下对偶优化问题：

$$\begin{aligned} \max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \\ \text{满足} \quad & \alpha_i \geq 0, \quad i = 1, \ldots, n, \\ & \sum_{i=1}^{n} \alpha_i y^{(i)} = 0. \end{aligned} \tag{6.12}$$

你也应该能够验证，在这个优化问题中，使 $p^* = d^*$ 以及 KKT 条件 (6.3)—(6.7) 成立的条件确实得到满足。因此，可以求解对偶问题来代替原始问题。具体来说，上面的对偶问题是一个参数为 α_i 的最大化问题。稍后将介绍求解它的具体算法。如果确实能求解，即找到在约束下使 $W(\alpha)$ 最大的 α，就可以利用式 (6.10) 根据 α 求出最优的 w。找到 w^* 后，考虑原始问题，也很容易得到截距最优值

$$b^* = -\frac{\max_{i:y^{(i)}=-1} (w^*)^T x^{(i)} + \min_{i:y^{(i)}=1} (w^*)^T x^{(i)}}{2}. \tag{6.13}$$

请自行验证。

继续之前，再仔细看式 (6.10)，它根据最优 α 给出最优 w。假设已经用训练集拟合了参数，现在要对新输入 x 预测。我们计算 $w^T x + b$，当且仅当它大于零时预测 $y = 1$。利用式 (6.10)，这个量还可以写为

$$w^T x + b = \left(\sum_{i=1}^{n} \alpha_i y^{(i)} x^{(i)} \right)^T x + b. \tag{6.14}$$

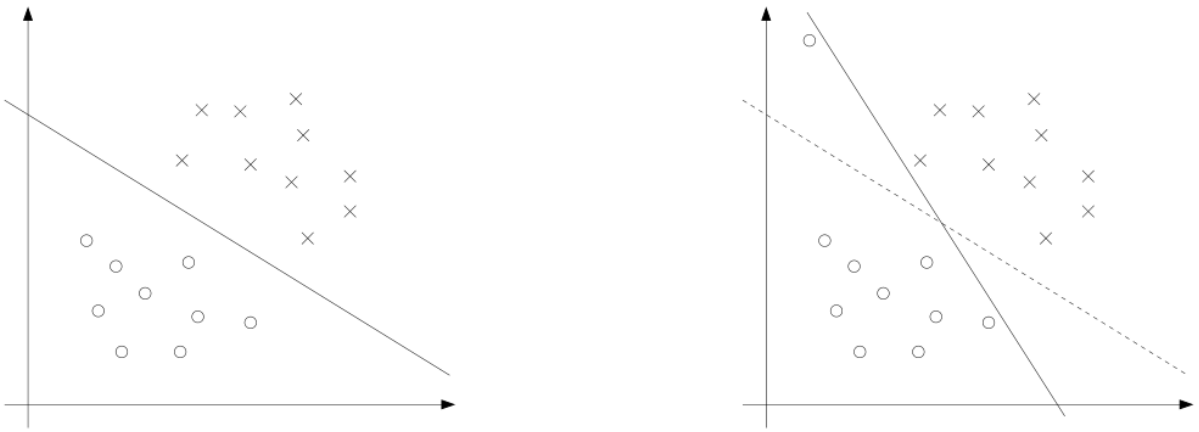
$$w^T x + b = \sum_{i=1}^{n} \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b. \tag{6.15}$$

因此，如果已知 α_i，预测时只需计算一个仅依赖于 x 与训练集各点内积的量。而且，前面看到，除了支持向量，对应的 α_i 全部为零。因此上式求和中的许多项为零，实际上只需计算 x 与支持向量（通常数量很少）之间的内积，就能计算 (6.15) 并预测。

考察优化问题的对偶形式，使我们深入了解了问题结构，也让整个算法能够仅用输入特征向量之间的内积表示。下一节将利用这一性质，把核应用于分类问题。得到的支持向量机算法将能够在极高维空间中高效学习。

6.7 正则化与不可分情形

到目前为止，SVM 的推导假设数据线性可分。虽然通过 ϕ 将数据映射到高维特征空间通常会增加可分的可能性，但无法保证总是可分。另外，某些情况下，寻找分离超平面也未必恰好是我们想做的，因为它可能容易受离群点影响。例如，下方左图为最优间隔分类器；在左上方增加一个离群点后（右图），决策边界发生了大幅旋转，分类器的间隔也小得多。



增加一个离群点前后，最优间隔决策边界的变化。

为了让算法适用于非线性可分的数据集，并降低对离群点的敏感程度，使用 ℓ_1 正则化，将优化问题改写为

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i \\ \text{满足} \quad & y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \ldots, n, \\ & \xi_i \geq 0, \quad i = 1, \ldots, n. \end{aligned}$$

译注（软间隔优化变量）：原文最小化符号下写为 $\min_{\gamma,w,b}$，但该问题中没有变量 γ，而松弛变量 ξ_i 需要共同优化，因此改为 $\min_{w,b,\xi}$。

现在允许样本的函数间隔小于 1。如果某个样本的函数间隔为 $1-\xi_i$，其中 $\xi_i > 0$，就会使目标函数增加 $C\xi_i$，作为相应代价。参数 C 控制两个目标之间的相对权重：一是使 $\|w\|^2$ 小（前面看到，这使间隔增大），二是确保大多数样本的函数间隔至少为 1。

与此前一样，构造拉格朗日函数：

$$\begin{aligned}\mathcal{L}(w,b,\xi,\alpha,r) = \frac{1}{2}w^T w + C\sum_{i=1}^{n}\xi_i \\ -\sum_{i=1}^{n}\alpha_i\big[y^{(i)}((x^{(i)})^T w + b) - 1 + \xi_i\big] - \sum_{i=1}^{n} r_i\xi_i.\end{aligned}$$

译注（软间隔拉格朗日函数）：原文求和项写为 $y^{(i)}(x^T w + b)$，省略了输入的样本上标。每项对应第 i 个训练样本，因此补为 $y^{(i)}((x^{(i)})^T w + b)$，以与前面的约束一致。

这里 α_i, r_i 是拉格朗日乘子，均约束为非负。我们不再详细重复对偶推导；与此前一样，令关于 w, b 的导数为零，代回并化简，得到

$$\begin{aligned}\max_{\alpha} \quad & W(\alpha) = \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i,j=1}^{n} y^{(i)}y^{(j)}\alpha_i\alpha_j\langle x^{(i)}, x^{(j)}\rangle \\ \text{满足} \quad & 0 \le \alpha_i \le C, \quad i = 1, \dots, n, \\ & \sum_{i=1}^{n}\alpha_i y^{(i)} = 0.\end{aligned}$$

与此前一样，w 仍可按式 (6.10) 用 α_i 表示，所以解出对偶问题后，还能继续用式 (6.15) 预测。略微令人惊讶的是，加入 ℓ_1 正则化后，对偶问题唯一的变化，就是原来的约束 $0 \le \alpha_i$ 变成了 $0 \le \alpha_i \le C$。b^* 的计算也需要修改，式 (6.13) 不再有效；参见下一节的说明或 Platt 的论文。

此外，KKT 对偶互补条件如下，下一节会用它们检测 SMO 算法是否收敛：

$$\alpha_i = 0 \implies y^{(i)}(w^T x^{(i)} + b) \ge 1. \tag{6.16}$$

$$\alpha_i = C \implies y^{(i)}(w^T x^{(i)} + b) \le 1. \tag{6.17}$$

$$0 < \alpha_i < C \implies y^{(i)}(w^T x^{(i)} + b) = 1. \tag{6.18}$$

现在，剩下的就是给出实际求解对偶问题的算法，下一节将介绍。

6.8 SMO 算法

John Platt 提出的 **SMO（序列最小优化，sequential minimal optimization）**算法，能够高效求解 SVM 推导中出现的对偶问题。为了引出 SMO，同时也因为其本身值得研究，我们先暂时讨论坐标上升算法。

6.8.1 坐标上升

考虑求解无约束优化问题

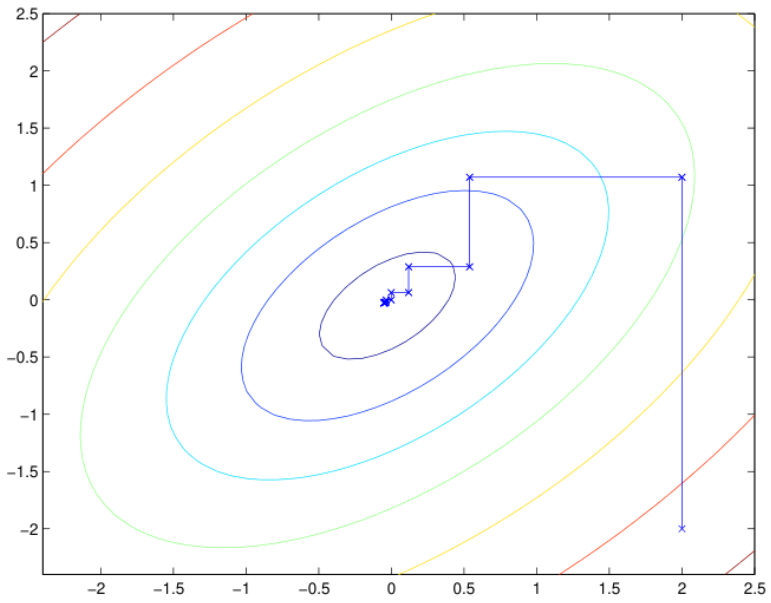
$$\max_{\alpha} W(\alpha_1, \alpha_2, \ldots, \alpha_n).$$

这里把 W 看作参数 α_i 的某个函数，暂时忽略它与 SVM 的关系。我们已经见过梯度上升和牛顿法两种优化算法。现在介绍的新算法称为**坐标上升**：

> 循环直到收敛：
> 对 $i = 1, \ldots, n$，依次执行
> $\alpha_i := \arg\max_{\hat{\alpha}_i} W(\alpha_1, \ldots, \alpha_{i-1}, \hat{\alpha}_i, \alpha_{i+1}, \ldots, \alpha_n)$。

在最内层循环中，固定除某个 α_i 以外的所有变量，只对 α_i 重新优化 W。这里给出的版本依次按 $\alpha_1, \alpha_2, \ldots, \alpha_n, \alpha_1, \alpha_2, \ldots$ 的顺序优化。（更复杂的版本可以选择其他次序；例如，根据预期哪个变量能使 $W(\alpha)$ 增加最多，决定下一个更新哪个变量。）

如果 W 的形式恰好使内层循环中的 arg max 能够高效计算，那么坐标上升也会相当高效。下面是坐标上升的运行示意图：



坐标上升沿各坐标轴方向逐步到达二次函数的全局最大值。

图中椭圆是待优化二次函数的等高线。坐标上升从 $(2, -2)$ 初始化，图中也画出了到达全局最大值所经过的路径。注意，每一步都平行于某一坐标轴，因为每次只优化一个变量。

6.8.2 SMO

我们通过概述 SMO 算法的推导，结束对 SVM 的讨论。

要求解的对偶优化问题为

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{n} y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle. \tag{6.19}$$

约束为

$$0 \le \alpha_i \le C, \quad i = 1, \dots, n, \tag{6.20}$$

$$\sum_{i=1}^{n} \alpha_i y^{(i)} = 0. \tag{6.21}$$

假设当前有一组满足约束 (6.20)—(6.21) 的 α_i。如果希望固定 $\alpha_2, \dots, \alpha_n$，执行一步坐标上升，只对 α_1 重新优化，能够有所进展吗？不能，因为约束 (6.21) 保证

$$\alpha_1 y^{(1)} = -\sum_{i=2}^{n} \alpha_i y^{(i)}.$$

两边乘以 $y^{(1)}$，等价地有

$$\alpha_1 = -y^{(1)} \sum_{i=2}^{n} \alpha_i y^{(i)}.$$

这里使用了 $y^{(1)} \in \{-1, 1\}$，从而 $(y^{(1)})^2 = 1$。因此，α_1 由其余 α_i 完全确定。如果固定 $\alpha_2, \dots, \alpha_n$，就不能对 α_1 作任何改变而不违反约束 (6.21)。

所以，如果要更新部分 α_i，就必须同时更新至少两个，才能始终满足约束。由此得到 SMO 算法，其步骤很简单：

重复以下步骤直至收敛：

1. 选择接下来要更新的一对 α_i, α_j，使用启发式方法，尽量选择能够让我们朝全局最大值前进最多的两个。
2. 固定所有其余 α_k（$k \neq i, j$），只对 α_i, α_j 重新优化 $W(\alpha)$。

要检验算法是否收敛，可以检查 KKT 条件 (6.16)—(6.18) 是否在某个容差 tol 内满足。这里 tol 是收敛容差参数，通常设为约 0.01 到 0.001。细节见论文及伪代码。

SMO 高效的关键原因，是对 α_i, α_j 的更新可以非常高效地计算。下面简要介绍推导这一高效更新的主要思想。

假设当前 α_i 满足约束 (6.20)—(6.21)，我们决定固定 $\alpha_3, \dots, \alpha_n$，在约束下对 α_1, α_2 重新优化 $W(\alpha_1, \alpha_2, \dots, \alpha_n)$。由 (6.21)，必须满足

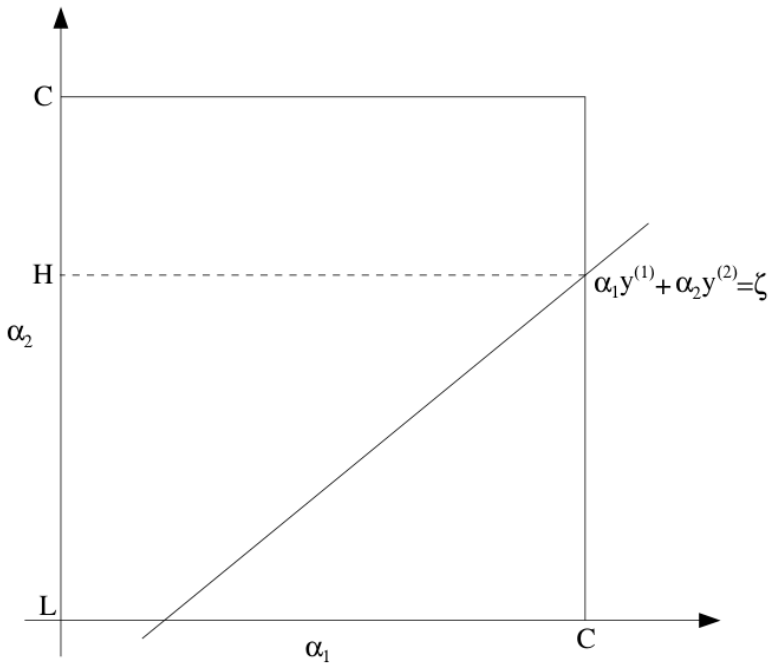
$$\alpha_1 y^{(1)} + \alpha_2 y^{(2)} = -\sum_{i=3}^{n} \alpha_i y^{(i)}.$$

因为右侧固定，记它为常数 ζ：

$$\alpha_1 y^{(1)} + \alpha_2 y^{(2)} = \zeta. \tag{6.22}$$

因此，可以如下图表示对 α_1, α_2 的约束：

由约束 (6.20)，α_1, α_2 必须位于图示的方框 $[0, C] \times [0, C]$ 内。同时，它们必须位于图示直线 $\alpha_1 y^{(1)} +$



SMO 两变量更新的箱约束、等式约束以及可行区间。

$\alpha_2 y^{(2)} = \zeta$ 上。这些约束还说明，$L \le \alpha_2 \le H$；否则 (α_1, α_2) 无法同时满足方框与直线约束。本例中 $L = 0$，但这取决于直线的位置，一般不一定如此。总之，α_2 的允许取值有某个下界 L 和上界 H，确保 α_1, α_2 位于方框 $[0, C] \times [0, C]$ 中。

利用式 (6.22)，还能把 α_1 写成 α_2 的函数：

$$\alpha_1 = (\zeta - \alpha_2 y^{(2)}) y^{(1)}.$$

请自行验证；这里再次使用 $y^{(1)} \in \{-1, 1\}$，所以 $(y^{(1)})^2 = 1$。因此，目标函数可以写为

$$W(\alpha_1, \alpha_2, \ldots, \alpha_n) = W((\zeta - \alpha_2 y^{(2)}) y^{(1)}, \alpha_2, \ldots, \alpha_n).$$

把 $\alpha_3, \ldots, \alpha_n$ 看作常数，可以验证这只是关于 α_2 的某个二次函数，即存在适当的 a, b, c，使其写成 $a\alpha_2^2 + b\alpha_2 + c$。如果忽略箱约束 (6.20)，等价地忽略 $L \le \alpha_2 \le H$，只需令导数为零并求解，就能容易地最大化这个二次函数。把得到的 α_2 记为 $\alpha_2^{\text{new,unclipped}}$。还可以确认：如果要在箱约束下对 α_2 最大化 W，只需把 $\alpha_2^{\text{new,unclipped}}$ “截断” 到区间 $[L, H]$ 内，即

$$\alpha_2^{\text{new}} = \begin{cases} H, & \alpha_2^{\text{new,unclipped}} > H, \\ \alpha_2^{\text{new,unclipped}}, & L \le \alpha_2^{\text{new,unclipped}} \le H, \\ L, & \alpha_2^{\text{new,unclipped}} < L. \end{cases}$$

最后，得到 α_2^{new} 后，可利用式 (6.22) 求回 α_1^{new} 的最优值。

还有几个相当简单的细节，留给读者自行阅读 Platt 的论文：其一是如何选择启发式规则来确定下一对更新的 α_i, α_j；其二是在 SMO 运行过程中如何更新 b。

第二部分 深度学习

第 7 章　深度学习

原文印刷页码：79–113。

现在开始学习深度学习。本章概述神经网络，讨论向量化，以及利用反向传播训练神经网络。

7.1 使用非线性模型的监督学习

在监督学习中（由输入 x 预测 y），设模型 / 假设函数为 $h_\theta(x)$。此前讨论了 $h_\theta(x)=\theta^\top x$（线性回归）和 $h_\theta(x)=\theta^\top\phi(x)$（$\phi(x)$ 是特征映射）两种情况。两者都对参数 θ 呈线性。接下来研究一类对参数 θ 和输入 x 都呈非线性的更一般模型。最常见的非线性模型是神经网络，下一节将开始定义。本节只需将 $h_\theta(x)$ 看成抽象的非线性模型即可。[22]

设训练样本为 $\{(x^{(i)},y^{(i)})\}_{i=1}^n$。下面定义非线性模型及其学习所用的损失 / 成本函数。

回归问题。为简单起见，先考虑输出为实数，即 $y^{(i)}\in\mathbb{R}$，模型也输出实数 $h_\theta(x)\in\mathbb{R}$。第 i 个样本 $(x^{(i)},y^{(i)})$ 的最小二乘成本函数定义为

$$J^{(i)}(\theta)=\frac{1}{2}\big(h_\theta(x^{(i)})-y^{(i)}\big)^2. \tag{7.1}$$

数据集的均方成本函数定义为

$$J(\theta)=\frac{1}{n}\sum_{i=1}^{n}J^{(i)}(\theta). \tag{7.2}$$

除为遵循惯例在前面增加常数 $1/n$ 外，它与线性回归相同。成本函数乘以标量不改变其局部或全局极小点。虽然成本函数仍是均方损失，但 $h_\theta(x)$ 的底层参数化与线性回归不同。本讲义交替使用“损失”和“代价”。

二分类。下面定义二分类模型和损失函数。设输入 $x\in\mathbb{R}^d$，$\bar{h}_\theta:\mathbb{R}^d\to\mathbb{R}$ 是参数化模型（对应逻辑线性回归中的 $\theta^\top x$）。称输出 $\bar{h}_\theta(x)\in\mathbb{R}$ 为 logit。类似第 2.1 节，用逻辑函数 $g(\cdot)$ 将 logit 转为概率 $h_\theta(x)\in[0,1]$：

$$h_\theta(x)=g(\bar{h}_\theta(x))=\frac{1}{1+\exp(-\bar{h}_\theta(x))}. \tag{7.3}$$

给定 x,θ 时，y 的条件分布建模为

[22] 如果具体例子有帮助，本小节可以想象模型 $h_\theta(x)=\theta_1^2x_1^2+\theta_2^2x_2^2+\cdots+\theta_d^2x_d^2$，尽管它不是神经网络。

$$P(y=1 \mid x;\theta) = h_\theta(x), \qquad P(y=0 \mid x;\theta) = 1 - h_\theta(x).$$

沿用第 2.1 节和注 2.1.1 的推导，负对数似然损失为

$$J^{(i)}(\theta) = -\log p(y^{(i)} \mid x^{(i)};\theta) = \ell_{\text{logistic}}(\bar{h}_\theta(x^{(i)}), y^{(i)}). \tag{7.4}$$

与式（7.2）一样，总损失定义为逐样本损失的平均值 $J(\theta) = \frac{1}{n}\sum_{i=1}^{n} J^{(i)}(\theta)$。

多分类。沿用第 2.3 节，响应变量 y 可取 k 个值，即 $y \in \{1, 2, \dots, k\}$。设 $\bar{h}_\theta : \mathbb{R}^d \to \mathbb{R}^k$ 为参数化模型，称输出 $\bar{h}_\theta(x) \in \mathbb{R}^k$ 为 logits，每一项对应一个类别的预测。类似第 2.3 节，用 softmax 将 logits 转为各项非负且总和为 1 的概率向量：

$$P(y=j \mid x;\theta) = \frac{\exp(\bar{h}_\theta(x)_j)}{\sum_{s=1}^{k} \exp(\bar{h}_\theta(x)_s)}. \tag{7.5}$$

其中 $\bar{h}_\theta(x)_s$ 表示第 s 个坐标。单个训练样本的损失同样为其负对数似然：

$$J^{(i)}(\theta) = -\log p(y^{(i)} \mid x^{(i)};\theta) = -\log\left(\frac{\exp(\bar{h}_\theta(x^{(i)})_{y^{(i)}})}{\sum_{s=1}^{k} \exp(\bar{h}_\theta(x^{(i)})_s)}\right). \tag{7.6}$$

用第 2.3 节的记号，可抽象地简写为

$$J^{(i)}(\theta) = \ell_{\text{ce}}(\bar{h}_\theta(x^{(i)}), y^{(i)}). \tag{7.7}$$

总损失同样取平均 $J(\theta) = \frac{1}{n}\sum_{i=1}^{n} J^{(i)}(\theta)$。上述方法还能推广到任意条件概率模型：$y$ 服从指数族分布 Exponential-family$(y;\eta)$，其中 $\eta = \bar{h}_\theta(x)$ 是 x 的参数化非线性函数。不过，最广泛使用的仍是前述三种情况。

优化器（SGD）。通常用梯度下降（GD）、随机梯度下降（SGD）及其变体优化 $J(\theta)$。GD 的更新规则为[23]

$$\theta := \theta - \alpha \nabla_\theta J(\theta), \tag{7.8}$$

其中 $\alpha > 0$ 称为学习率或步长。下面介绍略不同于第一份讲义的 SGD 版本（算法 1）。

算法 1 随机梯度下降

1. 超参数：学习率 α，总迭代次数 n_{iter}。
2. 随机初始化 θ。
3. 对 $i = 1, \dots, n_{\text{iter}}$，从 $\{1, \dots, n\}$ 均匀抽样 j，更新

$$\theta := \theta - \alpha \nabla_\theta J^{(j)}(\theta). \tag{7.9}$$

由于硬件并行，同时计算 B 个样本关于 θ 的梯度，往往比分别计算 B 次更快。因此，深度学习最

[23]按之前的约定，“$a := b$” 表示计算机程序中的赋值操作：把变量 a 的值设为 b 的值，即以 b 覆盖 a。而 “$a = b$” 表示关于数值相等的事实陈述。

常用的是算法 2 所示的小批量 SGD。SGD 和小批量 SGD 还有采用略有不同抽样方案的其他变体。

算法 2 小批量随机梯度下降

1. 超参数：学习率 α，批量大小 B，迭代次数 n_{iter}。
2. 随机初始化 θ。
3. 对 $i = 1, \dots, n_{\text{iter}}$，从 $\{1, \dots, n\}$ 均匀、无放回地抽取 B 个样本 $j_1, \dots, j_B$，更新

$$\theta := \theta - \frac{\alpha}{B} \sum_{k=1}^{B} \nabla_\theta J^{(j_k)}(\theta). \tag{7.10}$$

使用这些通用算法，典型的深度学习过程如下：1. 定义神经网络参数化 $h_\theta(x)$（第 7.2 节）；2. 编写反向传播算法，高效计算损失 $J^{(j)}(\theta)$ 的梯度（第 7.4 节）；3. 对损失 $J(\theta)$ 运行 SGD、小批量 SGD 或其他基于梯度的优化器。

7.2 神经网络

神经网络泛指一类非线性模型 / 参数化 $\bar{h}_\theta(x)$，由矩阵乘法和其他逐元素非线性运算组合而成。为统一处理回归与分类，这里将 $\bar{h}_\theta(x)$ 视为网络输出。回归时最终预测为 $h_\theta(x) = \bar{h}_\theta(x)$；分类时 $\bar{h}_\theta(x)$ 是 logits，二分类的预测概率为 $h_\theta(x) = 1/(1 + \exp(-\bar{h}_\theta(x)))$（式 7.3），多分类则为 $h_\theta(x) = \text{softmax}(\bar{h}_\theta(x))$（式 7.5）。我们从简单情况开始，逐步构建网络。

只有一个神经元的神经网络。回顾房价预测：给定房屋面积，预测价格。本小节一直使用这个例子。之前在面积与价格的图上拟合直线；现在希望把最低价格设为零，避免负房价。这样，图上会出现一个 “折点”（图 7.1）。如何用含未知参数的 $\bar{h}_\theta(x)$ 表示这种只有一个折点的函数？做到这一点后，就可应用第 7.1 节的方法。

定义输入为 x、参数为 θ、输出房价 y 的函数 $\bar{h}_\theta : x \mapsto y$。最简单的参数化之一是

$$\bar{h}_\theta(x) = \max(wx + b, 0), \qquad \theta = (w, b) \in \mathbb{R}^2. \tag{7.11}$$

它返回 $wx + b$ 与零中的较大者。在神经网络中，$\max\{t, 0\}$ 称为 ReLU（读作 “ray-lu”），即修正线性单元，常记为 $\text{ReLU}(t) \triangleq \max\{t, 0\}$。

一般而言，ReLU 这类从 $\mathbb{R}$ 到 $\mathbb{R}$ 的一维非线性函数称为激活函数。该模型称为单神经元模型，部分原因是它只有一个非线性激活函数。稍后将进一步解释这一名称。输入 $x \in \mathbb{R}^d$ 为多维时，单神经元网络可写为

$$\bar{h}_\theta(x) = \text{ReLU}(w^\top x + b), \qquad w \in \mathbb{R}^d,\ b \in \mathbb{R},\ \theta = (w, b). \tag{7.12}$$

b 常称为偏置，w 称为权重向量。该网络有 1 层；后面将定义多层的含义。

堆叠神经元。更复杂的网络可以将上述单神经元 “堆叠” 起来，一个神经元的输出作为下一个的输入，从而构成更复杂的函数。

进一步考虑房价例子。除面积外，假设还知道卧室数量、邮政编码、社区富裕程度。搭建网络像搭乐高：把单块积木堆成复杂结构。同样，我们把单个神经元堆成复杂网络。

给定面积、卧室数、邮编、富裕程度这些特征，可以认为房价取决于房屋能容纳的最大家庭规模。

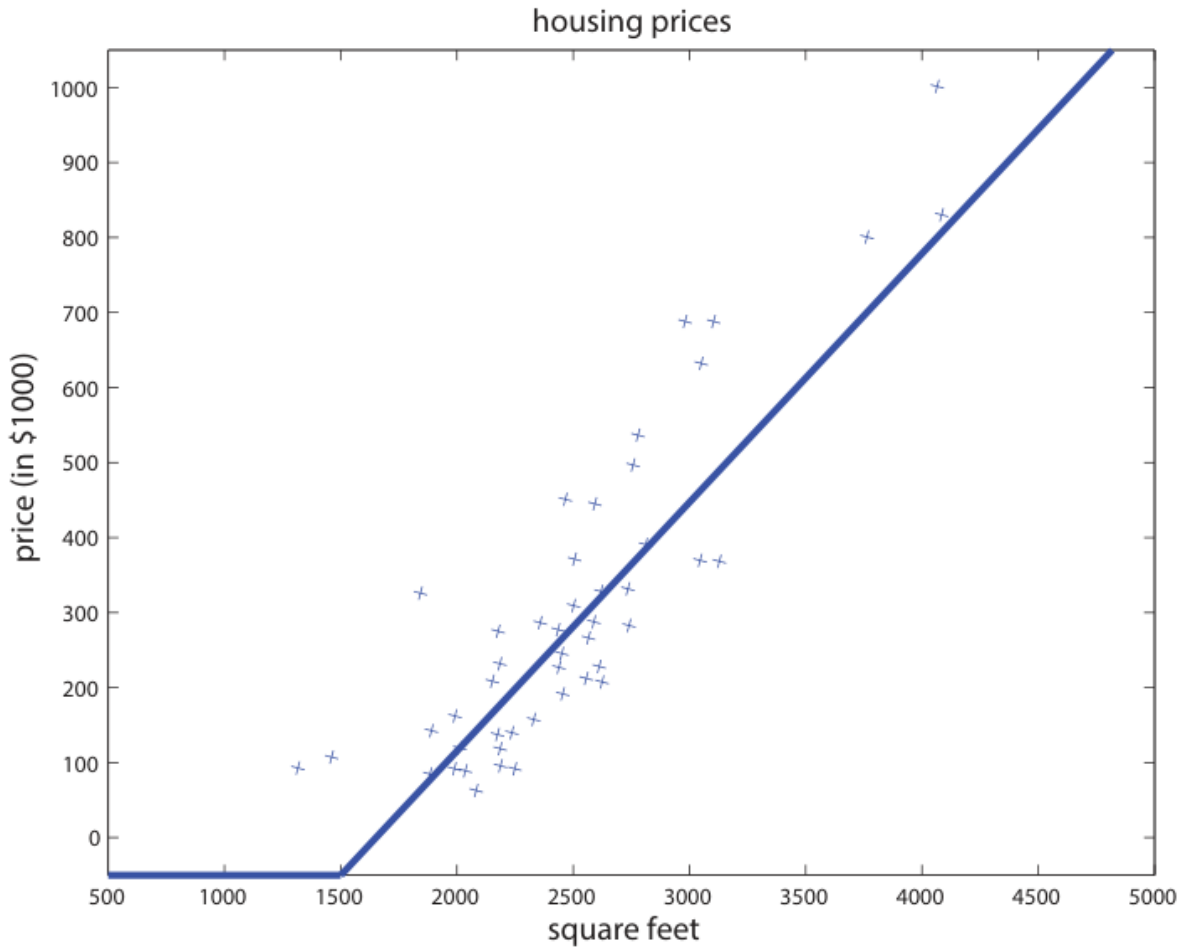


图 7.1：房价曲线中的 “折点”。

设家庭规模由面积和卧室数量决定（图 7.2）。邮编可能提供社区步行便利程度等信息，例如能否步行买菜，还是去哪儿都得开车。邮编与社区富裕程度结合，可能预测当地小学质量。由家庭规模、步行便利程度、学校质量这三个衍生特征，可以认为房价最终取决于它们。

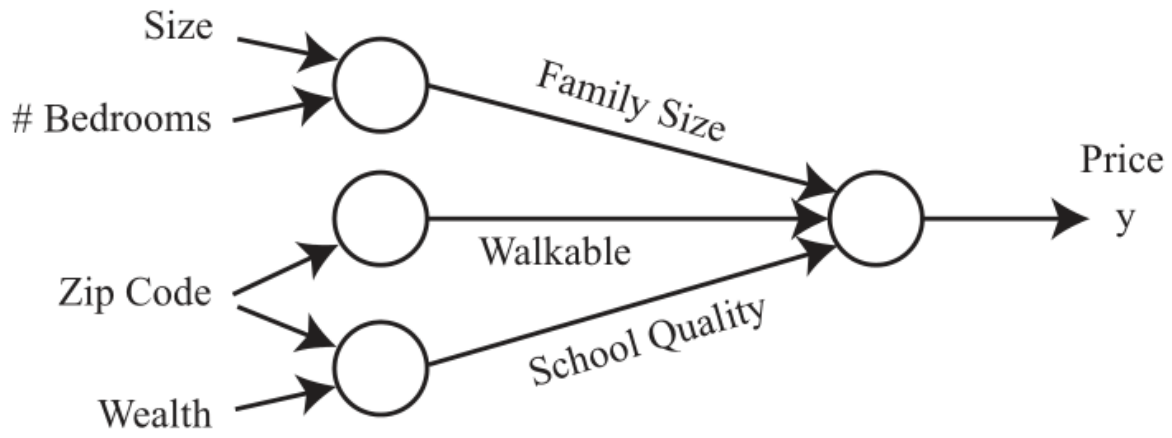


图 7.2：用于预测房价的小型神经网络示意图。

形式上，输入特征为 x_1, x_2, x_3, x_4；“家庭规模”“步行便利程度”“学校质量”的中间变量记为 a_1, a_2, a_3，它们常称为隐藏单元或隐藏神经元。每个 a_i 用一个单神经元网络表示，以 $x_1, \dots, x_4$ 的子集为输入。因此，如图 7.1 所示，得到参数化

$$
\begin{aligned}
a_1 &= \mathrm{ReLU}(\theta_1 x_1 + \theta_2 x_2 + \theta_3), \\
a_2 &= \mathrm{ReLU}(\theta_4 x_3 + \theta_5), \\
a_3 &= \mathrm{ReLU}(\theta_6 x_3 + \theta_7 x_4 + \theta_8).
\end{aligned}
$$

其中 $\theta_1, \dots, \theta_8$ 为参数。再把最终输出表示成以 a_1, a_2, a_3 为输入的线性函数，得到[24]

$$
\bar{h}_\theta(x) = \theta_9 a_1 + \theta_{10} a_2 + \theta_{11} a_3 + \theta_{12}. \tag{7.13}
$$

这里 θ 包含全部参数 $\theta_1, \dots, \theta_{12}$。现在输出已表示为关于 x 的较复杂函数，可用第 7.1 节的方法学习其参数。

来自生物神经网络的启发。顾名思义，人工神经网络受生物神经网络启发。隐藏单元 $a_1, \dots, a_m$ 对应生物神经元，参数 θ_i 对应突触。但现代深度人工网络与生物网络究竟有多相似并不清楚。例如，可能很少有神经科学家认为生物网络具有 1000 层，而一些现代人工网络确实如此（后面会详细说明层的概念）。此外，人脑更新网络的方式是否类似计算机科学家学习人工网络的方式——下一节介绍的反向传播——仍是开放问题。

两层全连接神经网络。式（7.13）用了大量关于输入如何决定“家庭规模”“步行便利程度”“学校质量”的先验知识或信念。我们隐含假设：家庭规模是重要量，且只由面积和卧室数决定。但其他应用未必有这些知识。采用通用参数化会更灵活、更普适。简单做法是让 a_1 等中间变量依赖所有 $x_1, \dots, x_4$：

[24] 多层网络接近输出的末端通常不应用 ReLU，尤其是在输出未必为正时。

$$\begin{aligned} a_1 &= \mathrm{ReLU}(w_1^\top x + b_1), \quad w_1 \in \mathbb{R}^4,\ b_1 \in \mathbb{R}, \\ a_2 &= \mathrm{ReLU}(w_2^\top x + b_2), \quad w_2 \in \mathbb{R}^4,\ b_2 \in \mathbb{R}, \\ a_3 &= \mathrm{ReLU}(w_3^\top x + b_3), \quad w_3 \in \mathbb{R}^4,\ b_3 \in \mathbb{R}. \end{aligned} \tag{7.14}$$

仍按式（7.13）定义 $\bar{h}_\theta$。因为每个中间变量都依赖所有输入，这称为全连接神经网络。一般而言，含 m 个隐藏单元、输入为 $x \in \mathbb{R}^d$ 的两层全连接网络定义为

$$\forall j \in \{1, \dots, m\}, \quad z_j = (w_j^{[1]})^\top x + b_j^{[1]}, \quad w_j^{[1]} \in \mathbb{R}^d,\ b_j^{[1]} \in \mathbb{R}. \tag{7.15}$$

$$a_j = \mathrm{ReLU}(z_j), \qquad a = [a_1, \dots, a_m]^\top \in \mathbb{R}^m,$$

$$\bar{h}_\theta(x) = (w^{[2]})^\top a + b^{[2]}, \quad w^{[2]} \in \mathbb{R}^m,\ b^{[2]} \in \mathbb{R}. \tag{7.16}$$

默认向量为列向量，尤其 a 是分量为 $a_1, \dots, a_m$ 的列向量。[1]、[2] 用来区分两组参数：每个 $w_j^{[1]} \in \mathbb{R}^d$，而 $w^{[2]} \in \mathbb{R}^m$。后面还会用到更多这样的下标。

向量化。在介绍更深、更复杂的网络前，先用矩阵和向量记号简化表达式。另一个重要动机是实现速度。要高效实现神经网络，必须谨慎使用 for 循环。式（7.15）最自然的代码实现也许是 for 循环，但实际输入和隐藏单元维度很高，循环会使运行很慢。利用 GPU 并行性一直对深度学习的发展至关重要。

这催生了向量化：用矩阵代数与高度优化的数值线性代数库（如 BLAS）取代循环，使计算更快。深度学习兴起前，较小数据集上的循环也许足够，但现代深度网络和先进数据集用循环处理将不可行。

把两层全连接网络向量化：将所有 $w_j^{[1]}$ 按以下方式拼成权重矩阵 $W^{[1]} \in \mathbb{R}^{m \times d}$：

$$W^{[1]} = \begin{bmatrix} — & (w_1^{[1]})^\top & — \\ — & (w_2^{[1]})^\top & — \\ & \vdots & \\ — & (w_m^{[1]})^\top & — \end{bmatrix} \in \mathbb{R}^{m \times d}. \tag{7.17}$$

按矩阵向量乘法定义，$z = [z_1, \dots, z_m]^\top$ 可写为

$$\underbrace{\begin{bmatrix} z_1 \\ \vdots \\ z_m \end{bmatrix}}_{z \in \mathbb{R}^{m \times 1}} = \underbrace{\begin{bmatrix} (w_1^{[1]})^\top \\ \vdots \\ (w_m^{[1]})^\top \end{bmatrix}}_{W^{[1]} \in \mathbb{R}^{m \times d}} \underbrace{\begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix}}_{x \in \mathbb{R}^{d \times 1}} + \underbrace{\begin{bmatrix} b_1^{[1]} \\ \vdots \\ b_m^{[1]} \end{bmatrix}}_{b^{[1]} \in \mathbb{R}^{m \times 1}}. \tag{7.18}$$

简写为

$$z = W^{[1]} x + b^{[1]}. \tag{7.19}$$

再次强调，本讲义按之前约定自动把 $\mathbb{R}^d$ 中的向量看成列向量，也视为 $d \times 1$ 矩阵。这与 NumPy 广

播时将向量看成行向量的情况不同。

从 z 计算 a 要逐元素应用非线性 ReLU，可高效并行计算。复用 ReLU 记号表示逐元素运算，即对 $t \in \mathbb{R}^d$，$\mathrm{ReLU}(t)_i = \mathrm{ReLU}(t_i)$，于是

$$a = \mathrm{ReLU}(z). \tag{7.20}$$

类似定义 $W^{[2]} = [(w^{[2]})^\top] \in \mathbb{R}^{1\times m}$，式（7.16）可汇总为

$$a = \mathrm{ReLU}(W^{[1]}x + b^{[1]}), \qquad \bar{h}_\theta(x) = W^{[2]}a + b^{[2]}. \tag{7.21}$$

参数 θ 包含权重矩阵 $W^{[1]}, W^{[2]}$ 和偏置 $b^{[1]}, b^{[2]}$。$W^{[1]}, b^{[1]}$ 合称第一层，$W^{[2]}, b^{[2]}$ 合称第二层；激活 a 称为隐藏层。两层网络也称单隐藏层网络。

多层全连接神经网络。用此简洁记号可继续堆叠层。令 r 为层数（权重矩阵数），各层权重和偏置为 $W^{[1]}, \dots, W^{[r]}, b^{[1]}, \dots, b^{[r]}$，则

$$\begin{aligned} a^{[1]} &= \mathrm{ReLU}(W^{[1]}x + b^{[1]}), \\ a^{[2]} &= \mathrm{ReLU}(W^{[2]}a^{[1]} + b^{[2]}), \\ &\cdots \\ a^{[r-1]} &= \mathrm{ReLU}(W^{[r-1]}a^{[r-2]} + b^{[r-1]}), \\ \bar{h}_\theta(x) &= W^{[r]}a^{[r-1]} + b^{[r]}. \end{aligned} \tag{7.22}$$

各矩阵、偏置的维度必须匹配。若 $a^{[k]}$ 维度为 m_k，则 $W^{[k]}$ 的维度为 $m_k \times m_{k-1}$，$b^{[k]} \in \mathbb{R}^{m_k}$。此外 $W^{[1]} \in \mathbb{R}^{m_1\times d}$，$W^{[r]} \in \mathbb{R}^{1\times m_{r-1}}$。

网络总神经元数为 $m_1 + \cdots + m_r$，总参数数为 $(d+1)m_1 + (m_1+1)m_2 + \cdots + (m_{r-1}+1)m_r$。有时为统一记号，写作 $a^{[0]} = x$、$a^{[r]} = h_\theta(x)$，得到递推

$$a^{[k]} = \mathrm{ReLU}(W^{[k]}a^{[k-1]} + b^{[k]}), \quad k = 1, \dots, r-1. \tag{7.23}$$

若式（7.22）多一个末端 ReLU，该式对 $k = r$ 也成立。但通常最后一层设为线性（无 ReLU），以允许负输出，也便于把最后一层解释成线性模型。更多解释见本节“与核方法的联系”。

其他激活函数。 ReLU 可替换为许多从 $\mathbb{R}$ 到 $\mathbb{R}$ 的非线性函数 σ，例如

$$\sigma(z) = \frac{1}{1+e^{-z}} \quad \text{（sigmoid）}, \tag{7.24}$$

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad \text{（tanh）}, \tag{7.25}$$

$$\sigma(z) = \max\{z, \gamma z\}, \quad \gamma \in (0,1) \quad \text{（leaky ReLU）}, \tag{7.26}$$

$$\sigma(z) = \frac{z}{1+e^{-\beta z}}, \quad \beta > 0 \quad (\mathrm{Swish}_\beta), \tag{7.27}$$

$$\sigma(z) = \frac{z}{2}\left(1 + \operatorname{erf}\left(\frac{z}{\sqrt{2}}\right)\right) \quad \text{（GELU）}, \tag{7.28}$$

$$\sigma(z) = \max\{z, 0\}^2 \quad \text{（ReLU}^2\text{）}, \tag{7.29}$$

$$\sigma(z) = \frac{1}{\beta}\log(1 + \exp(\beta z)), \quad \beta > 0 \quad \text{（Softplus）}. \tag{7.30}$$

图 7.3 绘出了这些函数。sigmoid 和 tanh 如今越来越少被单独用作隐藏层激活，部分原因是它们两端有界，z 趋向正、负无穷时梯度均消失；上述其他函数在输入趋向正无穷时仍有梯度。不过，sigmoid 仍是重要的门控非线性，例如用于一些混合专家路由器 [Nguyen et al., 2025] 和门控注意力机制 [Qiu et al., 2025]。Softplus 实际也不常用，可看成 ReLU 的平滑版本，从而具有适当的二阶导数。Swish$_\beta$ 由 Ramachandran et al. [2017] 提出，也常称 SiLU，尤其当 $\beta = 1$ 时；EfficientNet [Tan and Le, 2019] 等架构使用它。GELU 由 Hendrycks and Gimpel [2016] 提出，广泛用于 BERT [Devlin et al., 2019] 等 Transformer 语言模型，也用于 Hunyuan-DiT [Li et al., 2024] 等扩散 Transformer。ReLU^2 是 ReLU 的简单高阶变体，用于 Primer [So et al., 2021]，后来发现它能改善稀疏大语言模型的稀疏性 [Zhang et al., 2024]。

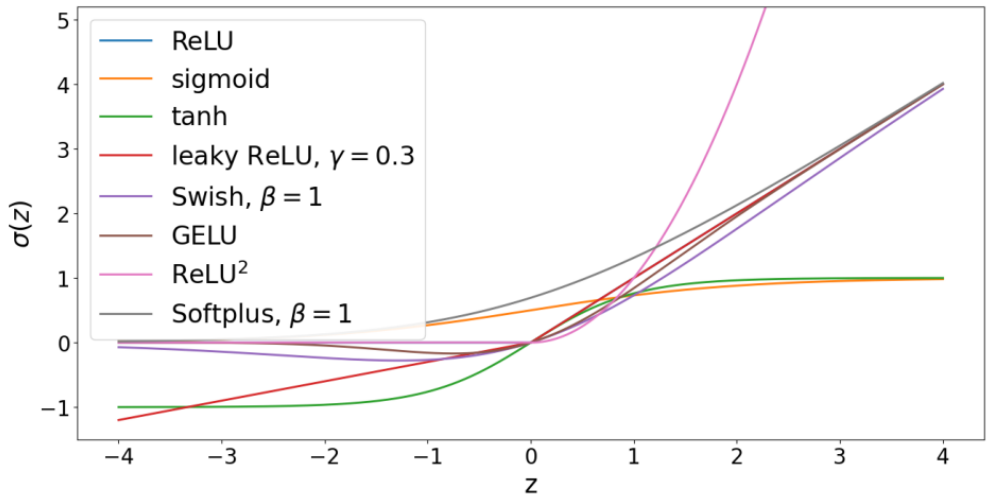


图 7.3：深度学习中的激活函数。为便于显示，纵轴上限截在 5。

另一类实践中很重要、尤其适用于现代序列模型的函数是门控激活。门控线性单元（GLU）对同一输入 h 作两次仿射变换，以其中一个结果控制另一个：

$$\mathrm{GLU}(h) = (W_1 h + b_1) \odot g(W_2 h + b_2). \tag{7.31}$$

g 通常是逻辑 sigmoid，$\odot$ 为逐元素乘法 [Dauphin et al., 2016]。因此，与前面的标量激活不同，GLU 不是 $\sigma : \mathbb{R} \to \mathbb{R}$，而是通过乘法门控组合两次矩阵向量乘法结果的小模块。现代 Transformer 的前馈层常采用 SwiGLU 变体 [Shazeer, 2020, Touvron et al., 2023, Yang et al., 2025, OpenAI, 2025]。

为什么不让 $\sigma(z)$ 为恒等函数？即为什么不取 $\sigma(z) = z$？为讨论方便，设 $b^{[1]} = b^{[2]} = 0$。那么两层网络满足

$$\bar{h}_\theta(x) = W^{[2]}a^{[1]} \tag{7.32}$$

$$= W^{[2]}\sigma(z^{[1]}) \quad \text{（根据定义）} \tag{7.33}$$

$$= W^{[2]}z^{[1]} \quad \text{（因为}\sigma(z)=z\text{）} \tag{7.34}$$

$$= W^{[2]}W^{[1]}x \quad \text{（由式 7.18）} \tag{7.35}$$

$$= \widetilde{W}x, \qquad \widetilde{W} = W^{[2]}W^{[1]}. \tag{7.36}$$

注意两个权重矩阵合并成了 $\widetilde{W}$。这是因为线性函数与线性函数复合仍是原输入的线性函数：可构造 $\widetilde{W}$ 使 $\widetilde{W}x = W^{[2]}W^{[1]}x$。而待预测输出与输入往往是非线性关系，因此这会丧失网络的大量表示能力。没有非线性激活，网络就只是在做线性回归。

与核方法的联系。前面介绍过特征映射。其主要动机是用 $\theta^\top\phi(x)$ 表示关于输入 x 的非线性函数，其中 θ 是参数，$\phi(x)$ 是人工设计的、关于原始输入的非线性函数。学习算法的表现可能强烈依赖特征映射的选择。人们常用领域知识为特定应用设计 $\phi(x)$，此过程称为特征工程。

深度学习可看成自动学习适当特征映射（有时也称“表示”）的方法。令 β 包含式（7.22）全连接网络中除最后一层外的所有参数，则可将 $a^{[r-1]}$ 抽象写成 $a^{[r-1]} = \phi_\beta(x)$。于是

$$\bar{h}_\theta(x) = W^{[r]}\phi_\beta(x) + b^{[r]}. \tag{7.37}$$

固定 β 时，ϕ_β 是特征映射，而 $\bar{h}_\theta$ 只是特征上的线性模型。但训练网络时，$\beta, W^{[r]}, b^{[r]}$ 都被优化，因而既学习特征空间中的线性模型，也学习好的特征映射本身，使线性模型能基于它准确预测。因此，深度学习通常较少依赖具体应用的领域知识，所需特征工程较少。倒数第二层 $a^{[r]}$ 常被非正式地称为学到的特征或表示。〔译注：原文此处写作 $a^{[r]}$；按本节定义，倒数第二层应为 $a^{[r-1]}$。〕

房价例子中，全连接网络无需我们指定“家庭规模”等中间量，可能自动在倒数第二层 $a^{[r-1]}$ 发现有用特征，并用它们线性预测房价。一个数据集得到的表示 / 特征映射 ϕ_β 往往也适用于其他数据集，说明其中包含数据的关键信息。但网络发现的复杂特征虽有助于预测，人类却可能很难理解或解释。这也是有人将神经网络称为黑箱的原因。

7.3 现代神经网络中的模块

第 7.2 节式（7.22）的多层网络如今常称为多层感知机（MLP）。实际使用的现代网络通常复杂得多，由多个构建模块或多层模块组成。本节介绍其他模块及其组合方式。

首先，每次矩阵乘法可视为一个模块。考虑参数为 (W, b)、输入为 z 的运算，其中 W 是权重矩阵，b 是偏置向量：

$$\mathrm{MM}_{W,b}(z) = Wz + b. \tag{7.38}$$

隐含假设所有维度相容。若上下文清楚，或参数对讨论不重要，为方便也会省略 MM 的下标。MLP 可写成多个矩阵乘法模块与非线性激活模块的复合：

$$\mathrm{MLP}(x) = \mathrm{MM}_{W^{[r]},b^{[r]}}\big(\sigma(\mathrm{MM}_{W^{[r-1]},b^{[r-1]}}(\sigma(\cdots \mathrm{MM}_{W^{[1]},b^{[1]}}(x))))\big). \tag{7.39}$$

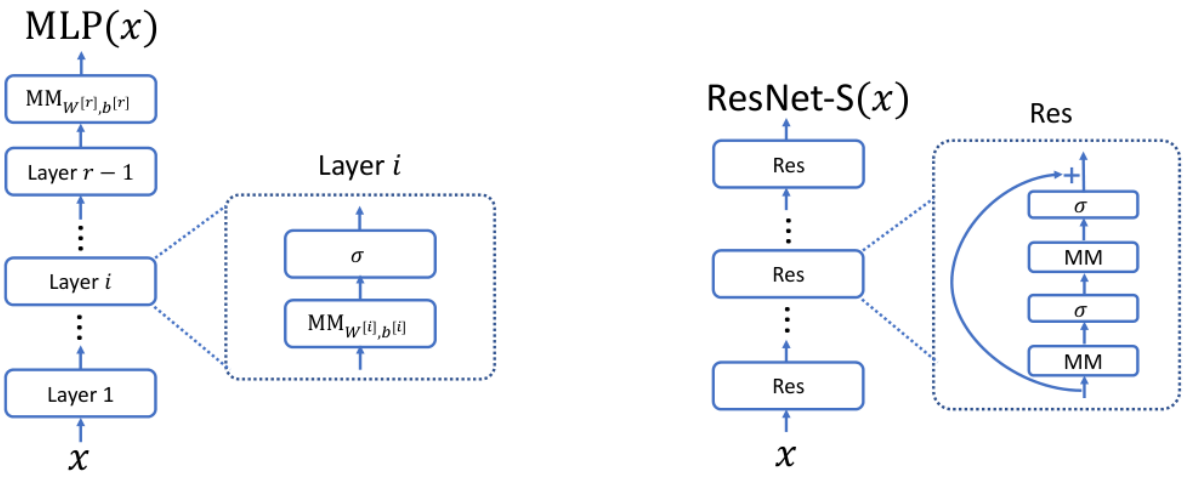


图 7.4：架构示意图。左：含 r 层的 MLP。右：残差网络。

省略参数下标，可写为

$$\mathrm{MLP}(x) = \mathrm{MM}(\sigma(\mathrm{MM}(\sigma(\cdots \mathrm{MM}(x))))). \tag{7.40}$$

本讲义默认每个模块具有不同的一组参数，且参数维度保证复合有意义。也可用小模块定义大模块，例如许多论文将一个激活层 σ 与一个矩阵乘法层 MM 合称为一“层”。人们常在架构图中用依赖关系连接基本模块，图 7.4 左侧就是 MLP 的例子。

残差连接。 ResNet 是视觉领域极具影响力的架构，其残差连接如今几乎用于所有大规模深度学习架构。使用上述记号，高度简化的残差块定义为

$$\mathrm{Res}(z) = z + \sigma(\mathrm{MM}(\sigma(\mathrm{MM}(z)))). \tag{7.41}$$

高度简化的 ResNet 是多个残差块复合，后接矩阵乘法：

$$\text{ResNet-S}(x) = \mathrm{MM}(\mathrm{Res}(\mathrm{Res}(\cdots \mathrm{Res}(x)))). \tag{7.42}$$

图 7.4 右侧也画出了这些模块的依赖关系。ResNet-S 与奠基论文 [He et al., 2016] 的 ResNet 仍不同，因为后者用卷积层取代普通矩阵乘法，并在卷积与激活之间加入批归一化。下面将介绍卷积层和若干批归一化变体。ResNet-S 和层归一化都是现代大语言模型广泛使用的 Transformer 架构的组成部分。

层归一化。本文记为 LN 的层归一化，将 $z \in \mathbb{R}^m$ 映射为归一化程度更高的向量 $\mathrm{LN}(z) \in \mathbb{R}^m$。它常用于非线性激活之后。先定义子模块 LN-S：

$$\text{LN-S}(z) = \begin{bmatrix} (z_1 - \hat{\mu})/\hat{\sigma} \\ (z_2 - \hat{\mu})/\hat{\sigma} \\ \vdots \\ (z_m - \hat{\mu})/\hat{\sigma} \end{bmatrix}, \tag{7.43}$$

其中 $\hat{\mu} = \frac{1}{m}\sum_{i=1}^{m} z_i$ 是向量的经验均值，$\hat{\sigma} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(z_i - \hat{\mu})^2}$ 是各分量的经验标准差。[25] 直观上，LN-S 输出经验均值为 0、经验标准差为 1 的向量。

零均值、单位标准差未必最理想，因此层归一化引入可学习标量 β, γ 作为期望的均值、标准差，通过仿射变换得到

$$\mathrm{LN}(z) = \beta + \gamma\,\text{LN-S}(z) = \begin{bmatrix} \beta + \gamma(z_1 - \hat{\mu})/\hat{\sigma} \\ \beta + \gamma(z_2 - \hat{\mu})/\hat{\sigma} \\ \vdots \\ \beta + \gamma(z_m - \hat{\mu})/\hat{\sigma} \end{bmatrix}. \tag{7.44}$$

左侧第一次出现的 β 严格说应理解为各分量均为 β 的向量。$\hat{\mu}, \hat{\sigma}$ 也是 z 的函数，求导时不能当常数。β, γ 可学习，因此层归一化是含参数模块，与不含参数的激活层不同。〔译注：原文把输出标准差写作 γ；若不约束其符号，标准差严格应为 $|\gamma|$。〕

尺度不变性。层归一化使模型对参数缩放具有如下不变性。把 LN 与 $\mathrm{MM}_{W,b}$ 复合，所得子网络在 MM 参数整体缩放时输出不变：

$$\mathrm{LN}(\mathrm{MM}_{\alpha W, \alpha b}(z)) = \mathrm{LN}(\mathrm{MM}_{W,b}(z)), \quad \forall \alpha > 0. \tag{7.45}$$

首先，LN-S 本身尺度不变：

$$\text{LN-S}(\alpha z) = \begin{bmatrix} (\alpha z_1 - \alpha\hat{\mu})/(\alpha\hat{\sigma}) \\ (\alpha z_2 - \alpha\hat{\mu})/(\alpha\hat{\sigma}) \\ \vdots \\ (\alpha z_m - \alpha\hat{\mu})/(\alpha\hat{\sigma}) \end{bmatrix} = \begin{bmatrix} (z_1 - \hat{\mu})/\hat{\sigma} \\ (z_2 - \hat{\mu})/\hat{\sigma} \\ \vdots \\ (z_m - \hat{\mu})/\hat{\sigma} \end{bmatrix} = \text{LN-S}(z). \tag{7.46}$$

因此

$$\mathrm{LN}(\mathrm{MM}_{\alpha W, \alpha b}(z)) = \beta + \gamma\,\text{LN-S}(\mathrm{MM}_{\alpha W, \alpha b}(z)) \tag{7.47}$$

$$= \beta + \gamma\,\text{LN-S}(\alpha\,\mathrm{MM}_{W,b}(z)) \tag{7.48}$$

$$= \beta + \gamma\,\text{LN-S}(\mathrm{MM}_{W,b}(z)) \tag{7.49}$$

$$= \mathrm{LN}(\mathrm{MM}_{W,b}(z)). \tag{7.50}$$

因此，大多数用于大规模视觉和语言任务的现代深度学习架构，对非最后一层权重具有尺度不变性。设最后一层权重为 W_{last}，其余权重统记为 W，则对所有 $\alpha > 0$，有 $f_{W_{\text{last}}, \alpha W}(x) = f_{W_{\text{last}}, W}(x)$。最后一层特殊，是因为其后通常不接层归一化或批归一化。

其他归一化层。其他归一化方法也希望将中间层缩放控制在更固定的范围，例如批归一化 [Ioffe and Szegedy, 2015] 和组归一化 [Wu and He, 2018]。两者较常用于视觉，而层归一化较常用于语言。早期 Transformer 主要采用层归一化，后来许多仅解码器 Transformer 改用 RMSNorm（均方根层归一化），用坐标的均方根重新缩放向量，省略显式减均值步骤 [Zhang and Sennrich, 2019]。对 $z \in \mathbb{R}^m$，

[25] 这里除以 m 而非 $m-1$，因为目的在于使 LN-S 输出的经验方差为 1，等价于平方和为 m，而非统计学意义上估计标准差。

$$\text{RMSNorm}(z) = \gamma \frac{z}{\sqrt{\frac{1}{m}\sum_{i=1}^{m} z_i^2}}, \tag{7.51}$$

其中 γ 为可学习尺度参数。LLaMA、Qwen 等现代大语言模型采用此方法 [Touvron et al., 2023, Yang et al., 2025]。

卷积层。卷积神经网络由卷积层及其他模块组成，尤其适合视觉。为便于说明，本文着重一维卷积，仅在本小节末简要介绍二维卷积。二维卷积更适合具有两个空间维度的图像；一维卷积也用于自然语言处理。

先介绍简化一维卷积层 Conv1D-S，它是有特殊结构的矩阵乘法层。参数为滤波器向量 $w \in \mathbb{R}^k$ 及偏置标量 b，k 为滤波器大小，通常 $k \ll m$。滤波器也常称为核，但与核方法的核没有太大关系。为简单起见，设 $k = 2\ell + 1$ 为奇数。先在输入两端补零：$z_{1-\ell} = \cdots = z_0 = 0$，$z_{m+1} = \cdots = z_{m+\ell} = 0$，将 z 看成 $m + 2\ell$ 维向量。输出为 m 维，每个坐标是部分输入坐标的线性组合，系数来自 w：

$$\text{Conv1D-S}(z)_i = w_1 z_{i-\ell} + w_2 z_{i-\ell+1} + \cdots + w_{2\ell+1} z_{i+\ell} = \sum_{j=1}^{2\ell+1} w_j z_{i-\ell+(j-1)}. \tag{7.52}$$

因此，它是参数共享的矩阵乘法 $\text{Conv1D-S}(z) = Qz$，其中

$$Q = \begin{bmatrix} w_{\ell+1} & \cdots & w_{2\ell+1} & 0 & \cdots & 0 \\ w_\ell & \cdots & w_{2\ell} & w_{2\ell+1} & \cdots & 0 \\ \vdots & & \ddots & \ddots & & \vdots \\ w_1 & \cdots & w_{\ell+1} & \cdots & w_{2\ell+1} & 0 \\ 0 & w_1 & \cdots & \cdots & \cdots & w_{2\ell+1} \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \cdots & 0 & w_1 & \cdots & w_{\ell+1} \end{bmatrix}. \tag{7.53}$$

这里矩阵的精确定义为：$|j - i| \le \ell$ 时 $Q_{ij} = w_{j-i+\ell+1}$，否则为零。对于所有 $i, j \in \{2, \ldots, m\}$，有 $Q_{ij} = Q_{i-1,j-1}$，因此是参数共享的矩阵乘法。卷积只需 $O(km)$ 时间，普通矩阵乘法需 $O(m^2)$；卷积有 k 个参数，普通矩阵乘法有 m^2 个。因此只要这种附加结构不妨碍拟合数据的灵活性，卷积就应高效得多。〔译注：原文先列出偏置 b，但式（7.52）起的公式和参数计数均未计入偏置；此处保留原文。〕

实际卷积有许多变体，例如不同补零方式，或输出维度不同于输入。为简单起见略去这些细节。

实际卷积还有多个“通道”，以上只是单通道版本。一般 Conv1D 输入 C 个向量 $z_1, \ldots, z_C \in \mathbb{R}^m$，$C$ 称为输入通道数；等价地，输入是这些向量拼成的 $m \times C$ 矩阵。它输出 C' 个 m 维向量 $\text{Conv1D}(z)_1, \ldots, \text{Conv1D}(z)_{C'}$，等价于 $m \times C'$ 矩阵，C' 为输出通道数。每个输出是各输入通道简化卷积之和：

$$\forall i \in \{1, \ldots, C'\}, \quad \text{Conv1D}(z)_i = \sum_{j=1}^{C} \text{Conv1D-S}_{i,j}(z_j). \tag{7.54}$$

每个 $\text{Conv1D-S}_{i,j}$ 都有不同参数，总参数数为 k（单模块参数数）乘以 CC'（模块数），即 kCC'。相比之下，从 $\mathbb{R}^{m \times C}$ 到 $\mathbb{R}^{m \times C'}$ 的一般线性映射有 $m^2 CC'$ 个参数。卷积参数也可表示成 $k \times C \times C'$ 的

三维张量。

二维卷积（简述）。单通道二维卷积 Conv2D-S 类似 Conv1D-S，但输入 $z \in \mathbb{R}^{m\times m}$，滤波器大小为 $k\times k$，输出仍在 $\mathbb{R}^{m\times m}$ 中。完整二维卷积 Conv2D 输入矩阵序列 $z_1,\dots,z_C \in \mathbb{R}^{m\times m}$，等价于三维张量 $z \in \mathbb{R}^{m\times m\times C}$，输出矩阵序列 $\text{Conv2D}(z)_1,\dots,\text{Conv2D}(z)_{C'}$，等价于 $\mathbb{R}^{m\times m\times C'}$ 中的张量。各输出通道是所有输入通道经 Conv2D-S 后的结果之和：

$$\forall i \in \{1,\dots,C'\},\quad \text{Conv2D}(z)_i = \sum_{j=1}^{C} \text{Conv2D-S}_{i,j}(z_j). \tag{7.55}$$

共有 CC' 个模块，每个有 k^2 个参数，因此总参数数为 $CC'k^2$；也可视为 $C\times C'\times k\times k$ 的四维张量。

进一步阅读。 Mazet [2026] 讨论基本性质、卷积定理等理论性质，以及图像处理中的相关与卷积之间的联系。

7.4 反向传播

本节介绍反向传播或自动微分，用来高效计算损失梯度 $\nabla J(\theta)$。先给出一个非正式定理：只要实值函数 f 可由可微网络或电路高效计算，其梯度也能在相近时间内高效计算。再给出神经网络中的具体方法。

一般定理的严格表述不是重点，因此术语采用非正式定义。可微电路或可微网络，是一系列可微算术运算（加、减、乘、除等）与初等可微函数（ReLU、exp、log、sin、cos 等）的复合。电路大小指这些运算和函数的总数。假设每个运算、函数及其导数 / 偏导数均可在 $O(1)$ 时间内计算。

定理 7.4.1（反向传播／自动微分，非正式表述）。若大小为 N 的可微电路计算实值函数 $f:\mathbb{R}^\ell \to \mathbb{R}$，则梯度 ∇f 可由大小为 $O(N)$ 的电路在 $O(N)$ 时间内计算。[26]

第 j 个样本的损失 $J^{(j)}(\theta)$ 确实可由加、减、乘和非线性激活等运算计算。因此定理表明，计算其梯度所需时间应与计算损失本身相近。这不仅适用于第 7.2 节的全连接网络，也适用于使用更复杂模块的许多其他网络。

TensorFlow、PyTorch 等所有深度学习库都实现了自动微分 / 反向传播，因此实际中研究人员大多不必自己编写反向传播算法。但理解它对于洞察深度学习的工作原理很有帮助。

一个有用推论是，梯度的任何标量函数（如 $s(\nabla f(x))$）仍可具有可高效计算的梯度，即使其中隐含涉及 f 的二阶导数。例如可高效计算 $\ell(\theta-\eta\nabla\ell(\theta))$ 关于 θ 的梯度。另一个推论：在相同条件下，若基本运算二次可微，对任意 $v\in\mathbb{R}^\ell$，Hessian 与向量的乘积 $\nabla^2 f(x)v$ 可在 $O(N+\ell)$ 时间内计算。令 $g(x)=\langle\nabla f(x),v\rangle$，由定理，$g$ 可在 $O(N+\ell)$ 时间求出；再应用一次定理，$\nabla g(x)=\nabla^2 f(x)v$ 也可在同样时间内求出。这些事实是二阶优化、元学习等许多二阶方法的基础，本课程不涉及这些主题。

本节余下内容的安排。 第 7.4.1 节从有助理解反向传播的新视角复习链式法则；第 7.4.2 节介绍总体策略；第 7.4.3 节计算基本模块的“反向函数”；第 7.4.4 节整合这些结果，给出 MLP 的具体算法。

7.4.1 偏导数预备知识

若标量 J 依赖变量 z（可为标量、矩阵或高阶张量），以 $\partial J/\partial z$ 表示相应偏导数。约定它与 z 维度完全相同。例如 $z\in\mathbb{R}^{m\times n}$ 时，$\partial J/\partial z\in\mathbb{R}^{m\times n}$，其 (i,j) 项为 $\partial J/\partial z_{ij}$。

[26] 如果 f 不依赖某些输入坐标，相应梯度默认置零。这里的运行时间记账不计这些置零操作。因此当 $N\le\ell$ 时，计算梯度的 $O(N)$ 时间可能甚至小于 ℓ。

注 7.4.2。如果 J, z 都不是标量，导数就是矩阵或张量，记号较棘手。除了数学和记号困难，这些多元输出函数的导数计算、存储也很昂贵，实践中很少显式构造。作者的经验是，只考虑标量函数关于向量、矩阵、张量的导数通常更有效。因此，本讲义不处理多元输出函数的导数。

链式法则。从更适合自动微分的视角和概念复习链式法则。考虑由 f, g 复合得到的标量 J：

$$z \in \mathbb{R}^m, \qquad u = g(z) \in \mathbb{R}^n, \qquad J = f(u) \in \mathbb{R}. \tag{7.56}$$

下面的推导易推广到 z, u 为矩阵或张量的情况，但最终的 J 必须是标量（另见注 7.4.2）。令 $u = (u_1, \dots, u_n)$，$g(z) = (g_1(z), \dots, g_n(z))$。标准链式法则给出

$$\forall i \in \{1, \dots, m\}, \quad \frac{\partial J}{\partial z_i} = \sum_{j=1}^{n} \frac{\partial J}{\partial u_j} \frac{\partial g_j}{\partial z_i}. \tag{7.57}$$

若两者均为向量，向量化形式为

$$\frac{\partial J}{\partial z} = \begin{bmatrix} \frac{\partial g_1}{\partial z_1} & \cdots & \frac{\partial g_n}{\partial z_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial g_1}{\partial z_m} & \cdots & \frac{\partial g_n}{\partial z_m} \end{bmatrix} \frac{\partial J}{\partial u}. \tag{7.58}$$

也就是说，反向函数始终是从 $\partial J/\partial u$ 到 $\partial J/\partial z$ 的线性映射，尽管该映射可能以复杂方式依赖 z。右侧矩阵其实是 g 的 Jacobian 的转置。为避免复杂化，这里不深入讨论 Jacobian；部分原因是，当 z 为矩阵或张量，要写出类似式（7.58）的公式，必须将其展平成向量，或引入张量与矩阵乘积记号。因此式（7.57）适用范围更广、更方便。例如 $z \in \mathbb{R}^{r \times s}$ 时，容易改写为

$$\forall i, k, \quad \frac{\partial J}{\partial z_{ik}} = \sum_{j=1}^{n} \frac{\partial J}{\partial u_j} \frac{\partial g_j}{\partial z_{ik}}. \tag{7.59}$$

第 7.4.3 节部分推导会用到它。

链式法则的关键解释。可以把式（7.57）或（7.58）看成由 $\partial J/\partial u$ 计算 $\partial J/\partial z$ 的方法。设 J 按式（7.56）通过 u 依赖 z。即使不知道 f，或 f 很复杂，只要已知 $\partial J/\partial u$ 的值，就可用式（7.58）计算

$$\frac{\partial J}{\partial u} \xrightarrow[\text{只需要} g(\cdot) \text{和} z \text{的信息}]{\text{链式法则，式（7.58）}} \frac{\partial J}{\partial z}. \tag{7.60}$$

公式只涉及 g 的信息，更具体地说是 $\partial g_j/\partial z_i$。当 g 是复杂网络 f 的构建模块时，我们会反复利用这一点。实践中把式（7.57）或（7.58）的映射模块化为黑箱很有用；数学上也便于定义记号。用 $B[g, z]$ 表示该映射：

$$\frac{\partial J}{\partial z} = B[g, z]\left(\frac{\partial J}{\partial u}\right). \tag{7.61}$$

称 $B[g, z]$ 为模块 g 的反向函数。z 固定时，它只是 $\mathbb{R}^n \to \mathbb{R}^m$ 的线性映射。由式（7.57），

$$(B[g, z](v))_i = \sum_{j=1}^{m} \frac{\partial g_j}{\partial z_i} v_j. \tag{7.62}$$

〔译注：原文求和上界为 m；按式（7.56）、（7.57）的维度，应为 n。〕

由式（7.58），向量化形式为

$$B[g,z](v)=\begin{bmatrix}\frac{\partial g_1}{\partial z_1} & \cdots & \frac{\partial g_n}{\partial z_1}\\ \vdots & \ddots & \vdots\\ \frac{\partial g_1}{\partial z_m} & \cdots & \frac{\partial g_n}{\partial z_m}\end{bmatrix}v. \tag{7.63}$$

所以它可视为一个矩阵。但实际中 z 变化，通常固定 g，需要对不同的 z 重算反向映射。因此实践中常把 $B[g,z](v)$ 视为一个输入 z,v 的函数：z 是 g 的输入，v 是某标量 J 关于 g 输出的梯度；返回的向量则是 J 关于 z 的梯度。

7.4.2 反向传播的总体策略

先建立自动微分的整体认识，再具体化为神经网络算法。我们将网络视为第 7.3 节中 MM、σ、Conv2D、LN 等小模块的复杂复合。均方损失、交叉熵损失等也可视为额外模块。因此单样本 (x,y) 的损失可抽象写为[27]

$$J=M_k(M_{k-1}(\cdots M_1(x))). \tag{7.64}$$

例如，采用式（7.39）、（7.40）MLP 的二分类损失可以如此表示：$M_1=\mathrm{MM}_{W^{[1]},b^{[1]}}$，$M_2=\sigma$，$M_3=\mathrm{MM}_{W^{[2]},b^{[2]}}$，依此类推，$M_{k-1}=\mathrm{MM}_{W^{[r]},b^{[r]}}$，$M_k=\ell_{\text{logistic}}$。

有些模块含参数，有些仅执行固定运算。一般地，设每个 M_i 具有参数集 $\theta^{[i]}$；若模块是非线性激活等固定运算，该集合可为空。稍后讨论模块化粒度，暂且假设每个模块都足够简单。

引入式（7.64）的中间变量：

$$\begin{aligned}u^{[0]}&=x,\\ u^{[1]}&=M_1(u^{[0]}),\\ u^{[2]}&=M_2(u^{[1]}),\\ &\vdots\\ J=u^{[k]}&=M_k(u^{[k-1]}).\end{aligned} \tag{F}$$

反向传播分前向、反向两遍。前向按 $i=1,\dots,k$ 用（F）顺次计算 $u^{[1]},\dots,u^{[k]}$，将全部中间变量保存在内存中。

反向先按逆序计算中间变量导数 $\partial J/\partial u^{[k]},\dots,\partial J/\partial u^{[1]}$，然后由 $\partial J/\partial u^{[i]}$ 和 $u^{[i-1]}$ 计算参数导数 $\partial J/\partial\theta^{[i]}$。两类计算也可以交错进行，因为参数导数只依赖这两个量，不依赖更早层的导数 $\partial J/\partial u^{[k]}$（$k<i$）。

先看如何由 $\partial J/\partial u^{[i]}$ 和 $u^{[i-1]}$ 高效求出 $\partial J/\partial u^{[i-1]}$。应用第 7.4.1 节链式法则，令 $u=u^{[i]}$，$z=u^{[i-1]}$，$f(u)=M_k(M_{k-1}(\cdots M_{i+1}(u)))$，$g=M_i$。尽管 f 很复杂，却不需要其具体信息。式（7.60）成为

$$\frac{\partial J}{\partial u^{[i]}}\xrightarrow[\text{只需要}M_i(\cdot)\text{和}u^{[i-1]}\text{的信息}]{\text{链式法则}}\frac{\partial J}{\partial u^{[i-1]}}. \tag{7.65}$$

[27] 严格说应写 $J=M_k(M_{k-1}(\cdots M_1(x)),y)$。但关于参数求导时 y 是常量，因此为简洁把它视为 M_k 的一部分。

更准确地，由式（7.61）

$$\frac{\partial J}{\partial u^{[i-1]}} = B[M_i, u^{[i-1]}]\left(\frac{\partial J}{\partial u^{[i]}}\right). \tag{B1}$$

同样令 $z = \theta^{[i]}$、$u = u^{[i]}$，得到

$$\frac{\partial J}{\partial \theta^{[i]}} = B[M_i, \theta^{[i]}]\left(\frac{\partial J}{\partial u^{[i]}}\right). \tag{B2}$$

图 7.5 展示了这一算法。

注 7.4.3（计算效率与模块粒度）。把复杂网络视为小模块复合，主要目的是小模块的反向函数通常容易高效实现。实际上，加法、乘法、ReLU 等原子模块的反向函数，在常数倍意义下与模块求值同样高效。因此，将网络视为原子运算的复合，并应用上述反向传播，就能证明定理 7.4.1。但实践中，以矩阵乘法、层归一化等为模块通常更方便。后面会看到，这些运算的反向函数用直接实现也能达到与前向求值相同的时间复杂度。

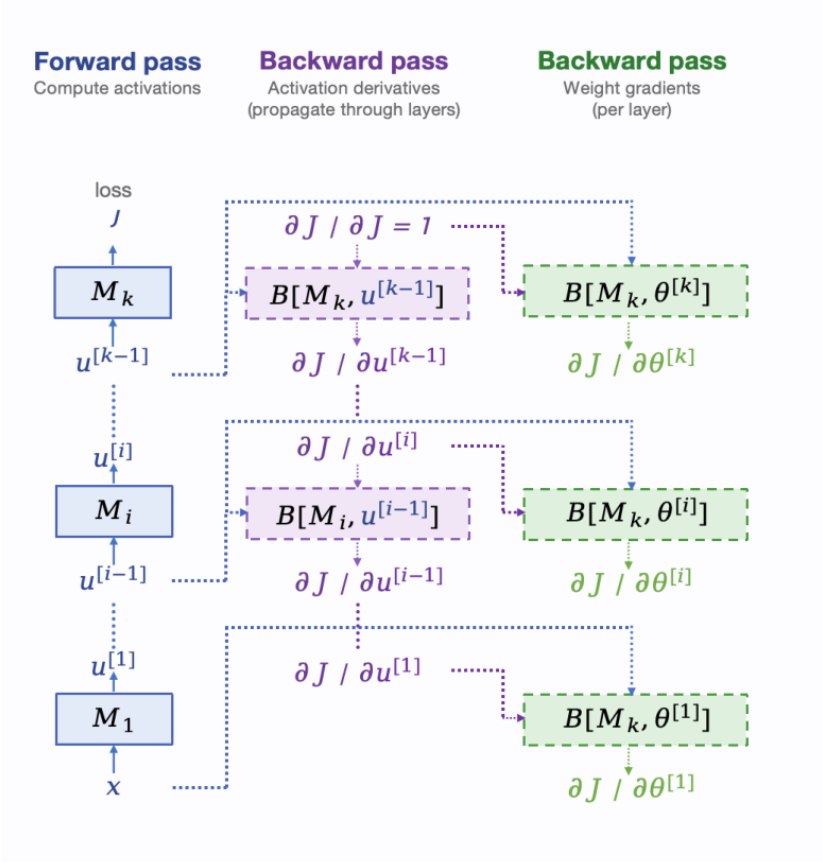


图 7.5：在模块复合中进行反向传播。

图 7.5 展示模块 $M_1, \dots, M_k$ 的逐层前向与反向计算。左侧前向从 $x = u^{[0]}$ 向上传播，产生 $u^{[1]}, u^{[2]}, \dots, u^{[k-1]}$，最终在顶端得到损失 J。中间的激活导数反向传播从 $\partial J/\partial J = 1$ 开始，递归应用局部反向算子 $B[M_\ell, u^{[\ell-1]}]$，把上游导数 $\partial J/\partial u^{[\ell]}$ 转为前一激活的导数 $\partial J/\partial u^{[\ell-1]}$。右侧的参数梯度反向过程使用各层相同的局部信息，通过 $B[M_\ell, \theta^{[\ell]}]$ 得到 $\partial J/\partial \theta^{[\ell]}$。

7.4.3 基本模块的反向函数

按第 7.4.2 节策略，只须求出网络中每个 M_i 的反向函数。本节计算 MM、激活 σ 和损失函数的反向函数。

MM 的反向函数。设 $\mathrm{MM}_{W,b}(z) = Wz + b$，其中 $z \in \mathbb{R}^m$，$W \in \mathbb{R}^{n\times m}$。由式（7.63），对 $v \in \mathbb{R}^n$，

$$B[\mathrm{MM}, z](v) = \begin{bmatrix} \frac{\partial (Wz+b)_1}{\partial z_1} & \cdots & \frac{\partial (Wz+b)_n}{\partial z_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial (Wz+b)_1}{\partial z_m} & \cdots & \frac{\partial (Wz+b)_n}{\partial z_m} \end{bmatrix} v. \tag{7.66}$$

因为对 $i \in [m], j \in [n]$，$\partial (Wz+b)_j / \partial z_i = \partial (b_j + \sum_{k=1}^m W_{jk} z_k)/\partial z_i = W_{ji}$，故

$$B[\mathrm{MM}, z](v) = W^\top v \in \mathbb{R}^m. \tag{7.67}$$

上面把 MM 视为 z 的函数，也可视为 W, b 的函数，求参数变量的反向函数。由于 W 是矩阵，用式（7.63）不够方便，原文称其中的矩阵会变成难于数学书写的四阶张量；因此改用（7.62）：

$$(B[\mathrm{MM}, W](v))_{ij} = \sum_{k=1}^m \frac{\partial (Wz+b)_k}{\partial W_{ij}} v_k = \sum_{k=1}^m \frac{\partial \sum_{s=1}^m W_{ks} z_s}{\partial W_{ij}} v_k = v_i z_j. \tag{7.68}$$

〔译注：原文两处外层求和上界均写 m；输出维度为 n，应为 n。〕向量化为

$$B[\mathrm{MM}, W](v) = v z^\top \in \mathbb{R}^{n\times m}. \tag{7.69}$$

对 b 用式（7.63），得到

$$B[\mathrm{MM}, b](v) = \begin{bmatrix} \frac{\partial (Wz+b)_1}{\partial b_1} & \cdots & \frac{\partial (Wz+b)_n}{\partial b_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial (Wz+b)_1}{\partial b_n} & \cdots & \frac{\partial (Wz+b)_n}{\partial b_n} \end{bmatrix} v = v. \tag{7.70}$$

这里用到：$i \neq j$ 时 $\partial (Wz+b)_j / \partial b_i = 0$，$i = j$ 时为 1。反向函数计算复杂度为 $O(mn)$，在常数倍意义下与矩阵乘法求值相同。

激活函数的反向函数。设 $M(z) = \sigma(z)$，其中 σ 逐元素作用，$z \in \mathbb{R}^m$。由式（7.63），

$$B[\sigma, z](v) = \begin{bmatrix} \frac{\partial \sigma(z_1)}{\partial z_1} & \cdots & \frac{\partial \sigma(z_m)}{\partial z_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial \sigma(z_1)}{\partial z_m} & \cdots & \frac{\partial \sigma(z_m)}{\partial z_m} \end{bmatrix} v \tag{7.71}$$

$$= \mathrm{diag}(\sigma'(z_1), \ldots, \sigma'(z_m)) v \tag{7.72}$$

$$= \sigma'(z) \odot v \in \mathbb{R}^m. \tag{7.73}$$

利用了 $j \neq i$ 时 $\partial \sigma(z_j)/\partial z_i = 0$；$\mathrm{diag}(\lambda_1, \ldots, \lambda_m)$ 是以这些数为对角元的对角矩阵，$\odot$ 表示同维向量逐元素乘法，σ' 表示逐元素应用激活函数的导数。

乍看（7.71）似乎需要 $O(m^2)$ 时间，但（7.73）表明可在 $O(m)$ 时间实现，与前向求值相同。这个化简不应令人惊讶：若使用更小模块，把向量到向量的激活看成 m 个标量到标量的激活，就更容易看出前后向时间应相近。

损失函数的反向函数。模块 M 输入向量 z、输出标量时，由式（7.63），反向函数输入标量 v、输出向量，其坐标为 $(B[M,z](v))_i = (\partial M/\partial z_i)v$。因此 $B[M,z](v) = (\partial M/\partial z)v$。

回顾平方损失 $\ell_{\text{MSE}}(z,y) = \frac{1}{2}(z-y)^2$，于是 $B[\ell_{\text{MSE}},z](v) = (z-y)v$。对于逻辑损失，由式（2.6），

$$B[\ell_{\text{logistic}},t](v) = \frac{\partial \ell_{\text{logistic}}(t,y)}{\partial t}v = \left(\frac{1}{1+\exp(-t)} - y\right)v. \tag{7.74}$$

对于交叉熵损失，由式（2.17），

$$B[\ell_{\text{ce}},t](v) = \frac{\partial \ell_{\text{ce}}(t,y)}{\partial t}v = (\phi - e_y)v, \quad \phi = \text{softmax}(t). \tag{7.75}$$

7.4.4 MLP 的反向传播

已知计算 MLP 损失所需各模块的反向函数，就能按第 7.4.2 节求隐藏激活和参数的梯度。考虑采用逻辑损失的 r 层 MLP。前向过程为

$$\begin{aligned}
z^{[1]} &= \text{MM}_{W^{[1]},b^{[1]}}(x),\\
a^{[1]} &= \sigma(z^{[1]}),\\
z^{[2]} &= \text{MM}_{W^{[2]},b^{[2]}}(a^{[1]}),\\
a^{[2]} &= \sigma(z^{[2]}),\\
&\vdots\\
z^{[r]} &= \text{MM}_{W^{[r]},b^{[r]}}(a^{[r-1]}),\\
J &= \ell_{\text{logistic}}(z^{[r]},y).
\end{aligned} \tag{7.76}$$

按逆序应用反向函数，首先

$$\frac{\partial J}{\partial z^{[r]}} = B[\ell_{\text{logistic}},z^{[r]}]\left(\frac{\partial J}{\partial J}\right) = B[\ell_{\text{logistic}},z^{[r]}](1). \tag{7.77}$$

再反复调用链式法则（7.62），迭代计算

$$\begin{aligned}
\frac{\partial J}{\partial a^{[r-1]}} &= B[\text{MM},a^{[r-1]}]\left(\frac{\partial J}{\partial z^{[r]}}\right),\\
\frac{\partial J}{\partial z^{[r-1]}} &= B[\sigma,z^{[r-1]}]\left(\frac{\partial J}{\partial a^{[r-1]}}\right),\\
&\vdots\\
\frac{\partial J}{\partial z^{[1]}} &= B[\sigma,z^{[1]}]\left(\frac{\partial J}{\partial a^{[1]}}\right).
\end{aligned} \tag{7.78}$$

数值上，对不同变量反复使用（7.73）和（7.67）即可。由于式（7.78）需要中间值 $a^{[i]},z^{[i]}$，前向后必须将其保留在内存。

然后用（7.69）、（7.70）计算参数梯度：

$$
\begin{aligned}
\frac{\partial J}{\partial W^{[r]}} &= B[\mathrm{MM}, W^{[r]}]\left(\frac{\partial J}{\partial z^{[r]}}\right),\\
\frac{\partial J}{\partial b^{[r]}} &= B[\mathrm{MM}, b^{[r]}]\left(\frac{\partial J}{\partial z^{[r]}}\right),\\
&\vdots\\
\frac{\partial J}{\partial W^{[1]}} &= B[\mathrm{MM}, W^{[1]}]\left(\frac{\partial J}{\partial z^{[1]}}\right),\\
\frac{\partial J}{\partial b^{[1]}} &= B[\mathrm{MM}, b^{[1]}]\left(\frac{\partial J}{\partial z^{[1]}}\right).
\end{aligned}
\tag{7.79}
$$

（7.79）与（7.78）的计算可交错：一旦求出 $\partial J/\partial z^{[i]}$，就能求出对应的 $W^{[i]}, b^{[i]}$ 梯度。结合（7.76）、（7.78）、（7.79），得到算法 3。

算法 3 多层神经网络的反向传播

1. 前向：由（7.76）计算并存储 $a^{[k]}, z^{[k]}, J$。
2. 反向：计算损失关于 $z^{[r]}$ 的梯度：

$$
\frac{\partial J}{\partial z^{[r]}} = B[\ell_{\text{logistic}}, z^{[r]}](1) = \frac{1}{1+\exp(-z^{[r]})} - y. \tag{7.80}
$$

3. 对 $k = r-1, r-2, \ldots, 0$，计算 $W^{[k+1]}, b^{[k+1]}$ 的梯度：

$$
\frac{\partial J}{\partial W^{[k+1]}} = B[\mathrm{MM}, W^{[k+1]}]\left(\frac{\partial J}{\partial z^{[k+1]}}\right) = \frac{\partial J}{\partial z^{[k+1]}}(a^{[k]})^{\mathsf{T}}. \tag{7.81}
$$

$$
\frac{\partial J}{\partial b^{[k+1]}} = B[\mathrm{MM}, b^{[k+1]}]\left(\frac{\partial J}{\partial z^{[k+1]}}\right) = \frac{\partial J}{\partial z^{[k+1]}}. \tag{7.82}
$$

4. 在上述第 3 步的每次循环内，若 $k \geq 1$，还计算

$$
\frac{\partial J}{\partial a^{[k]}} = B[\sigma, a^{[k]}]\left(\frac{\partial J}{\partial z^{[k+1]}}\right) = (W^{[k+1]})^{\mathsf{T}}\frac{\partial J}{\partial z^{[k+1]}}. \tag{7.83}
$$

$$
\frac{\partial J}{\partial z^{[k]}} = B[\sigma, z^{[k]}]\left(\frac{\partial J}{\partial a^{[k]}}\right) = \sigma'(z^{[k]}) \odot \frac{\partial J}{\partial a^{[k]}}. \tag{7.84}
$$

〔译注：式（7.83）原文写 $B[\sigma, a^{[k]}]$，但其右端以及（7.78）对应的是矩阵乘法模块，故此处应为 $B[\mathrm{MM}, a^{[k]}]$。〕

7.5 对训练样本进行向量化

如第 7.1 节所述，实现网络时要利用多个样本之间的并行性，因此需用矩阵记号同时表示多个样本的前向求值和反向传播。

基本思想。假设训练集有三个样本 $x^{(1)}, x^{(2)}, x^{(3)}$。各样本第一层为

$$
z^{[1](1)} = W^{[1]}x^{(1)} + b^{[1]}, \quad z^{[1](2)} = W^{[1]}x^{(2)} + b^{[1]}, \quad z^{[1](3)} = W^{[1]}x^{(3)} + b^{[1]}.
$$

方括号 $[\cdot]$ 表示层数，圆括号 $(\cdot)$ 表示样本编号。直觉上会用循环实现，但这些运算也能向量化。先

定义

$$X = \begin{bmatrix} | & | & | \\ x^{(1)} & x^{(2)} & x^{(3)} \\ | & | & | \end{bmatrix} \in \mathbb{R}^{d\times 3}. \tag{7.85}$$

注意样本按列而非按行堆叠。于是统一写为

$$Z^{[1]} = \begin{bmatrix} | & | & | \\ z^{[1](1)} & z^{[1](2)} & z^{[1](3)} \\ | & | & | \end{bmatrix} = W^{[1]}X + b^{[1]}. \tag{7.86}$$

这里试图将 $b^{[1]} \in \mathbb{R}^{4\times 1}$ 加到 $W^{[1]}X \in \mathbb{R}^{4\times 3}$ 上，严格按线性代数规则不允许。但实践中用广播完成：构造中间矩阵

$$\tilde{b}^{[1]} = \begin{bmatrix} | & | & | \\ b^{[1]} & b^{[1]} & b^{[1]} \\ | & | & | \end{bmatrix} \in \mathbb{R}^{4\times 3}, \tag{7.87}$$

然后计算 $Z^{[1]} = W^{[1]}X + \tilde{b}^{[1]}$。通常无须显式构造它，看（7.86）的维度就可以假定偏置被正确广播。此矩阵化方法易推广到多层，但有如下细节。

实现中的复杂之处／细节。所有深度学习软件包或实现都将数据点放在数据矩阵的行中；若数据点本身是矩阵或张量，则沿第零维堆叠。但大多数论文像本讲义一样把数据点当列向量。[28] 转换方法很简单：实现时，列向量变行向量，行向量变列向量，所有矩阵转置，并反转矩阵相乘的顺序。上例采用按行放置样本的约定时，$X \in \mathbb{R}^{3\times d}$，第一层权重为 $d \times m$（而非两层网络小节中的 $m \times d$），偏置 $b^{[1]} \in \mathbb{R}^{1\times m}$。隐藏激活计算变为

$$Z^{[1]} = XW^{[1]} + b^{[1]} \in \mathbb{R}^{3\times m}. \tag{7.88}$$

[28] 讲师推测主要原因是，数学上通常习惯把矩阵放在向量左边相乘。

第三部分 泛化与正则化

第 8 章　泛化

原文印刷页码：114–136。

本章讨论分析、理解机器学习模型泛化的工具，即研究模型在未见测试样本上的表现。回顾监督学习：给定训练集 $\{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$，通常通过最小化损失 / 成本函数 $J(\theta)$ 学习模型 h_θ，促使它拟合数据。例如，最小二乘损失（也称均方误差）为 $J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - h_\theta(x^{(i)}))^2$。这种用于训练的损失常称为训练损失 / 误差 / 代价。

但最小化训练损失并非最终目标，只是学习预测模型的手段。最重要的评价指标是未见测试样本上的损失，常称测试误差。形式上，从测试分布 $\mathcal{D}$ 抽取 (x, y)，用诸如均方误差 $(h_\theta(x) - y)^2$ 衡量预测误差。对测试样本随机性取期望，得到测试损失 / 误差：[29]

$$L(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[(y - h_\theta(x))^2]. \tag{8.1}$$

此误差涉及期望，实践中用许多抽样测试样本上的平均误差近似，这些样本称为测试集。训练集和测试集的关键区别在于测试样本是“未见”的：训练过程没有使用它们。经典统计学习中，训练样本也来自相同的 $\mathcal{D}$，但学习过程见过训练样本，没有见过测试样本。[30]

由于此区别，即使两者来自同一分布，测试误差也未必接近训练误差。[31] 因此，成功最小化训练误差未必产生小测试误差。若模型在训练集预测准确，却无法很好泛化，即训练误差小、测试误差大，称为过拟合。训练误差相对较大时，称为欠拟合；[32] 此时测试误差通常也较大。

本章研究学习过程、尤其模型参数化如何影响测试误差。将误差分为“偏差”和“方差”，研究参数化对两者的影响及权衡；据此讨论何时会发生、如何避免过拟合与欠拟合。第 8.2 节讨论双下降现象，第 8.3 节讨论一些经典理论结果。

8.1 偏差—方差权衡

以图 8.1 中的训练集、测试集为例。训练输入 $x^{(i)}$ 随机选取，输出按 $y^{(i)} = h^\star(x^{(i)}) + \xi^{(i)}$ 生成，其中 $h^\star$ 是图中实线所示二次函数，观测噪声 $\xi^{(i)} \sim \mathcal{N}(0, \sigma^2)$。测试样本也满足 $y = h^\star(x) + \xi$，$\xi \sim \mathcal{N}(0, \sigma^2)$。噪声不可能预测，因此本质目标是恢复 $h^\star$。

我们考察不同模型的测试误差。在线性回归中曾讨论该拟合“简单”的 $y = \theta_0 + \theta_1 x$，还是“复杂”的多项式 $y = \theta_0 + \theta_1 x + \cdots + \theta_5 x^5$。

[29] 理论与统计文献常将训练集上的均匀分布记为 $\widehat{\mathcal{D}}$，称经验分布，将 $\mathcal{D}$ 称总体分布。因此训练损失又称经验损失 / 风险 / 误差，测试损失又称总体损失 / 风险 / 误差。

[30] 近年来研究人员越来越关注“领域偏移”，即训练分布不同于测试分布。

[31] 测试误差与训练误差之差称泛化间隙。有些文献的“泛化误差”指测试误差，另一些则指泛化间隙。

[32] 例如，在回归中高于数据的固有噪声水平。

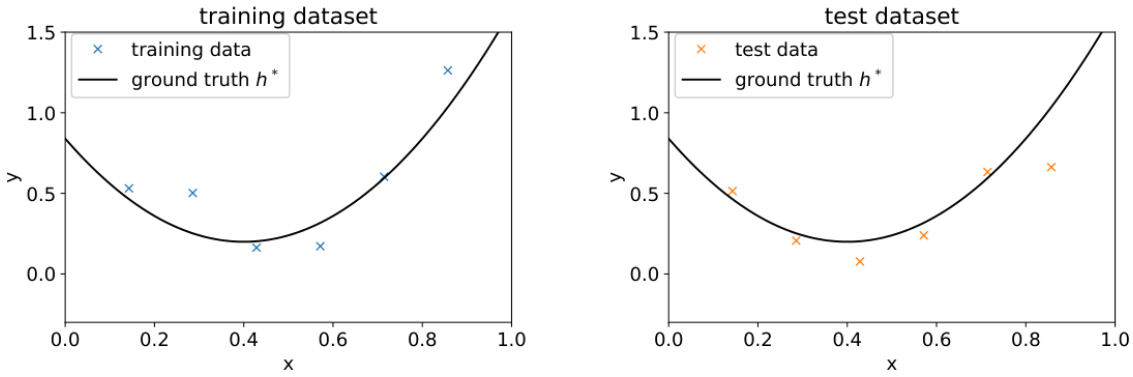


图 8.1：本节贯穿使用的训练集与测试集例子。

先拟合线性模型（图 8.2）。即使训练集上，最佳线性拟合也不能由 x 准确预测 y，更不用说测试集。因为真实关系非线性，任何线性模型都远离 $h^\star$。因此训练误差大，是典型欠拟合。

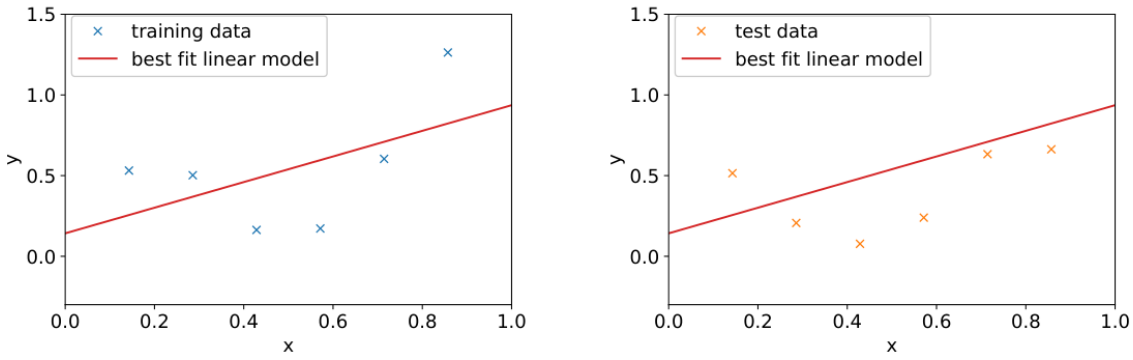


图 8.2：最佳线性拟合的训练误差、测试误差都大。

增加数据无法缓解：即使样本极多乃至无限，最佳线性拟合仍不准确，不能捕捉数据结构（图 8.3）。即使训练数据无噪声，问题依旧（图 8.4）。根本瓶颈不是缺数据，而是线性模型族无法表示真实二次函数。非正式地，将模型即使用极大、例如无限大的训练集拟合后仍有的测试误差称为偏差。因此，线性模型在此具有高偏差，欠拟合数据、无法捕捉其结构。

再拟合五次多项式（图 8.5），仍未学得好模型，但失败方式不同。尽管对训练样本的预测很好，测试样本上却表现差，即不善于泛化，测试误差高。与线性模型不同，五次多项式偏差小：若用极大数据集拟合，结果会接近真实二次函数、预测准确（图 8.6）。因为五次多项式族包含所有二次函数，令 $\theta_5 = \theta_4 = \theta_3 = 0$ 即可，因此原则上有能力捕捉数据结构。

五次多项式拟合的失败可由测试误差的另一部分——拟合过程的方差——刻画。图 8.7 中，我们很可能拟合了有限小训练集中偶然出现、却不反映 x, y 总体关系的模式。这些“伪模式”主要源自观测噪声，拟合它们会造成大测试误差。此时称模型方差大。

直观上，方差可用在同一底层分布的多个训练集上学到的模型变化程度刻画；第 8.1.1 节将给出数学形式。伪模式取决于特定数据集的随机噪声和输入，因此各数据集不同。分别过拟合这些模式会产生很不同的模型。图 8.7 正如此：三个模型各自过拟合所在数据集的伪模式。

偏差与方差通常存在权衡。模型太简单、参数太少，可能偏差大、方差小，并容易欠拟合；模型太复杂、参数太多，可能方差大、偏差较小，并容易过拟合。图 8.8 给出典型权衡。

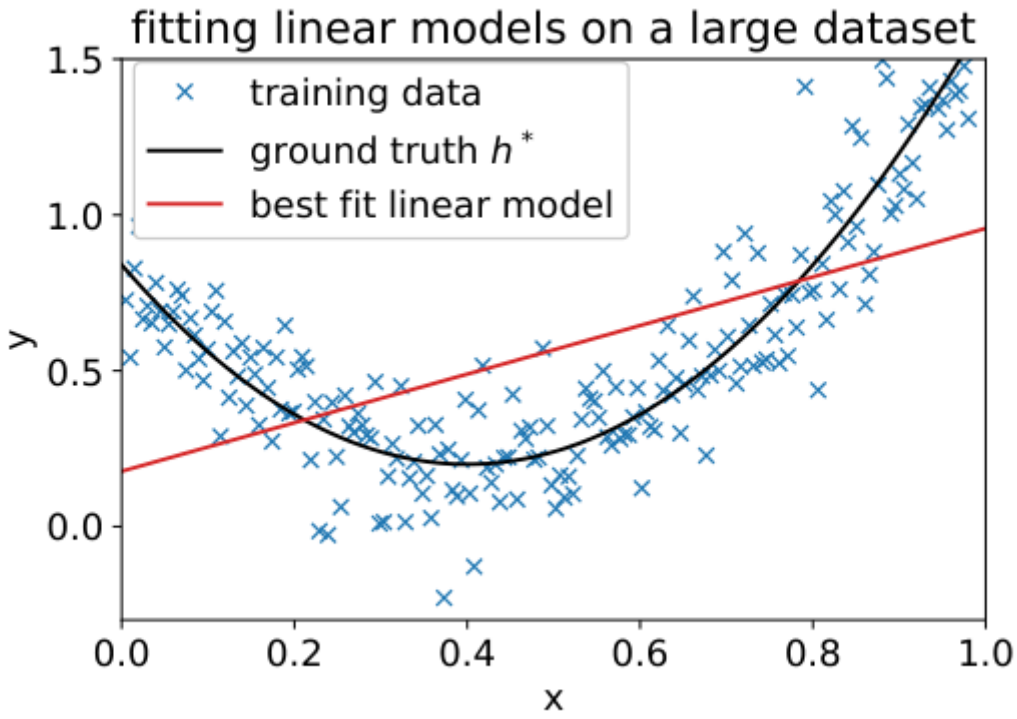


图 8.3：在大得多的数据集上，最佳线性拟合仍有较大训练误差。

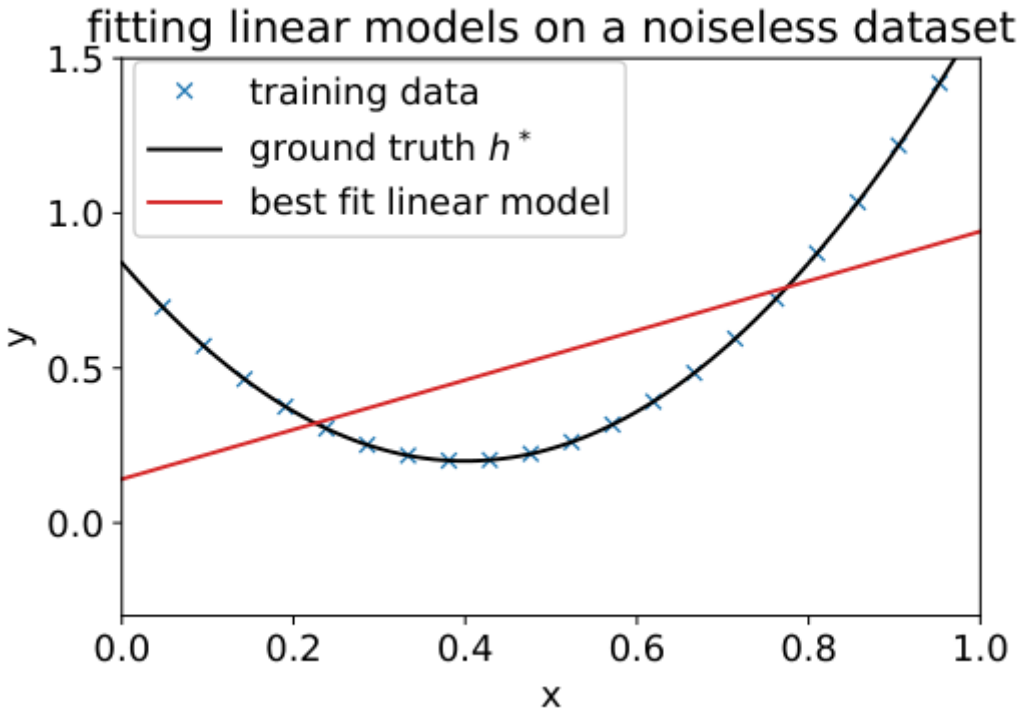


图 8.4：无噪声数据集上的最佳线性拟合，训练 / 测试误差仍大。

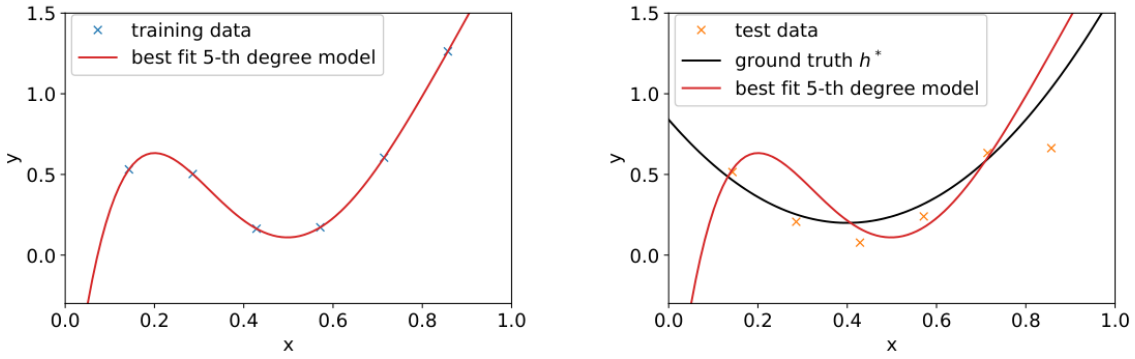


图 8.5：最佳五次多项式拟合的训练误差为零，但测试误差仍大，未恢复真实函数。这是经典过拟合。

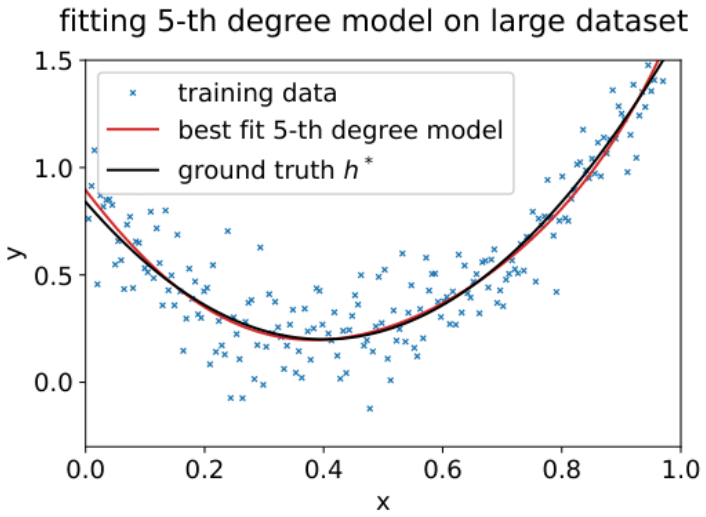


图 8.6：巨大数据集上的最佳五次多项式几乎恢复真实函数，说明图 8.5 的问题在于方差（缺数据），而不是偏差。

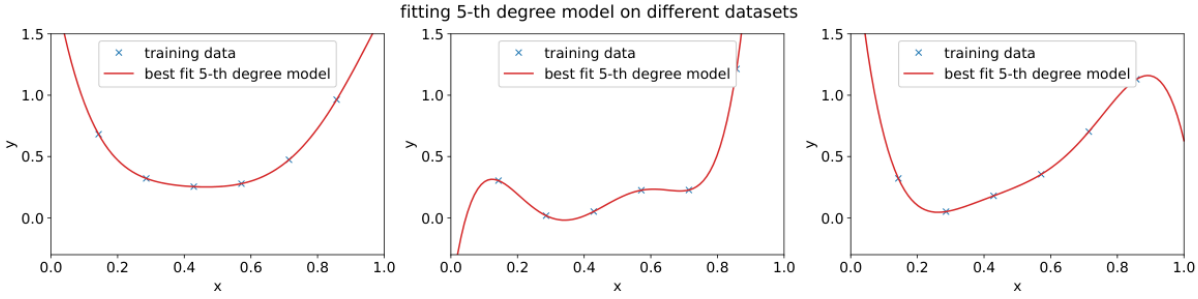


图 8.7：从同一分布生成的三个不同数据集上，最佳五次模型差异显著，说明方差大。

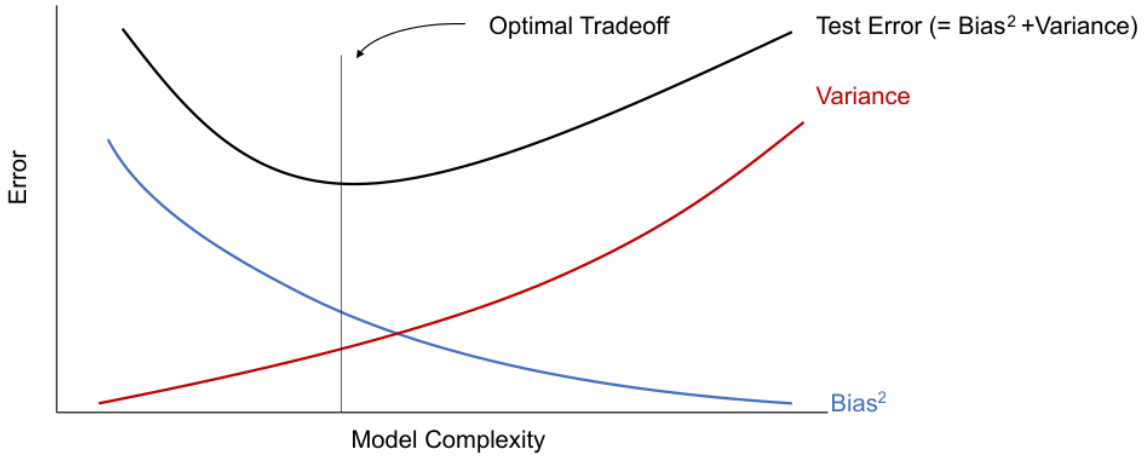


图 8.8：典型的偏差—方差权衡示意。

第 8.1.1 节会正式说明，测试误差可分解成偏差与方差之和。这意味着随模型复杂度增加，测试误差呈凸形曲线，实践中应调节复杂度取得最佳权衡。例如，拟合二次函数优于一次和五次这两个极端（图 8.9）。〔译注：这里的“凸形曲线”是原文对典型情形的直观描述，偏差—方差分解本身并不保证数学上的凸性。〕

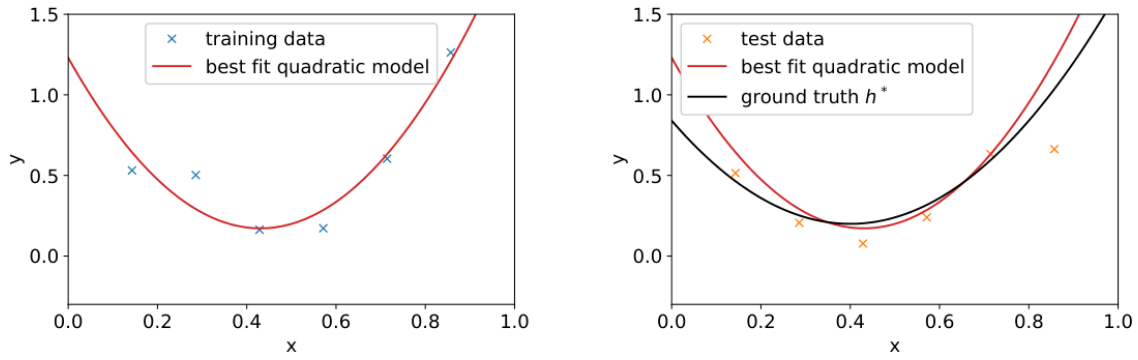


图 8.9：二次模型取得较好的权衡，因此训练和测试误差均小。

有趣的是，偏差—方差或测试误差曲线并非总如图 8.8，至少用参数数目衡量复杂度时不是如此；第 8.2 节将讨论双下降。尽管如此，分析、预测测试误差时，偏差—方差权衡仍通常是首先考虑的原则。

8.1.1 数学分解（针对回归）

为正式说明，考虑如下设置，是第 8.1 节开头的扩展：

- 抽取训练集 $S = \{x^{(i)}, y^{(i)}\}_{i=1}^{n}$，满足 $y^{(i)} = h^\star(x^{(i)}) + \xi^{(i)}$，$\xi^{(i)} \sim \mathcal{N}(0, \sigma^2)$。
- 在 S 上训练模型，记为 $\hat{h}_S$。
- 取测试样本 (x, y)，其中 $y = h^\star(x) + \xi$，$\xi \sim \mathcal{N}(0, \sigma^2)$；对随机训练集 S 和噪声 ξ 取平均，衡量期望测试误差：[3334]

[33] 为简单起见，测试输入 x 固定；对 x 的选取也取平均时，概念性结论不变。
[34] 期望下标强调取期望时哪些变量被视为随机量。

$$\mathrm{MSE}(x) = \mathbb{E}_{S,\xi}[(y - h_S(x))^2]. \tag{8.2}$$

这里沿用原文后续以 h_S 表示已训练模型的简写。将 MSE 分解前，先给出下面会使用两次的简单工具。

断言 8.1.1。若实随机变量 A, B 独立且 $\mathbb{E}[A] = 0$，则 $\mathbb{E}[(A+B)^2] = \mathbb{E}[A^2] + \mathbb{E}[B^2]$。特别地，随机变量与常数 c 独立，因此 $\mathbb{E}[(A+c)^2] = \mathbb{E}[A^2] + c^2$。

证明只需展开平方：$\mathbb{E}[(A+B)^2] = \mathbb{E}[A^2] + \mathbb{E}[B^2] + 2\mathbb{E}[AB]$；独立性给出 $\mathbb{E}[AB] = \mathbb{E}[A]\mathbb{E}[B] = 0$。

取 $A = \xi$、$B = h^\star(x) - \hat{h}_S(x)$，得到

$$\mathrm{MSE}(x) = \mathbb{E}[(y - h_S(x))^2] = \mathbb{E}[(\xi + (h^\star(x) - h_S(x)))^2] \tag{8.3}$$

$$= \mathbb{E}[\xi^2] + \mathbb{E}[(h^\star(x) - h_S(x))^2] = \sigma^2 + \mathbb{E}[(h^\star(x) - h_S(x))^2]. \tag{8.4}$$

定义平均模型 $h_{\mathrm{avg}}(x) = \mathbb{E}_S[h_S(x)]$：抽取无限多个数据集，分别训练，再平均其在 x 上的预测。这只是分析用的假想模型，现实无法取得无限多个数据集。许多情况下，它近似等于在单个无限大数据集上训练的模型，因此也可如此理解，与前面对偏差的直观定义一致。

取 $c = h^\star(x) - h_{\mathrm{avg}}(x)$（不依赖 S 的常数），$A = h_{\mathrm{avg}}(x) - h_S(x)$，应用断言的推论：

$$\mathrm{MSE}(x) = \sigma^2 + \mathbb{E}[(h^\star(x) - h_S(x))^2] \tag{8.5}$$

$$= \sigma^2 + (h^\star(x) - h_{\mathrm{avg}}(x))^2 + \mathbb{E}[(h_{\mathrm{avg}}(x) - h_S(x))^2] \tag{8.6}$$

$$= \underbrace{\sigma^2}_{\text{不可避免}} + \underbrace{(h^\star(x) - h_{\mathrm{avg}}(x))^2}_{\text{偏差平方}} + \underbrace{\mathrm{var}(h_S(x))}_{\text{方差}}. \tag{8.7}$$

第二项称偏差平方，第三项称方差。偏差刻画模型表达能力不足产生的误差。可把平均模型看成无限数据也只能学到的最佳模型，因此偏差并非缺数据，而是模型族根本无法近似 $h^\star$。如图 8.2，任何线性模型不能近似真实二次函数，平均模型也不能，故偏差必大。

方差刻画有限数据集的随机性给模型带来的误差，衡量模型对数据随机性的敏感程度，通常随数据量增大而下降。第一项 σ^2 无法消除，因为按定义噪声 ξ 不可预测。

分类的偏差—方差分解远不如回归明确。已有若干方案，但何种形式“正确”或最有用，尚无共识。

8.2 双下降现象

随模型规模的双下降。近期研究表明，线性模型和深度网络等多类模型的测试误差会出现“双下降”。[35] 第 8.1 节的传统认识是，复杂度增加时，测试误差先降后升（图 8.8）。但很多实际情况出现第二次下降：先下降，再在模型大到恰好能很好拟合全部训练数据附近达到峰值，然后在参数数超过数据点数的“过参数化”区间再次下降。图 8.10 给出以参数数衡量复杂度的典型曲线。某种程度上，第二次下降的过参数化区间对机器学习界较新，部分因为轻度正则化的过参数化模型直到深度学习时代才被广泛使用。实践启示是不要因此回避扩展和尝试过参数化模型，因为误差可能再次下降，甚至低于之前最低点。实际上，许多情况下，更大的过参数化模型总有更佳测试表

[35] 该现象的发现或可追溯至 Opper [1995, 2001]，近年来由 Belkin et al. [2020]、Hastie et al. [2019] 等推广。

现，即第二次下降后不会再次上升。

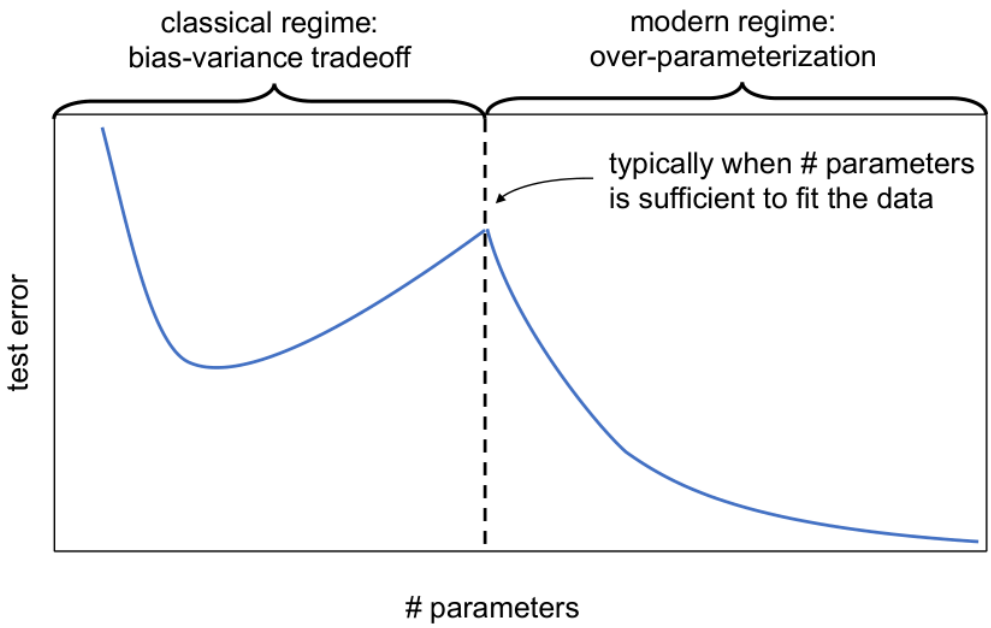


图 8.10：典型的随模型规模双下降。参数数小于训练样本数时，增加参数先使测试误差下降；进入过参数化区间后，误差再次下降。

随样本数的双下降。按先验直觉，样本越多应总使测试误差越小，因为算法可用的信息严格增加。但 Nakkiran [2019] 观察到，增加样本量时测试误差并非单调下降：如图 8.11，先下降，再在样本数 n 接近参数数 d 时上升至峰值，然后再次下降。这称为随样本数的双下降。它与随模型规模的双下降某种程度上描述了相似现象：$n \approx d$ 时误差达峰值。

解释及缓解策略。随样本数的双下降，尤其 $n \approx d$ 处的峰值，说明被评测的训练算法在该区域远非最优。丢弃部分样本，用较小样本量避开峰值反而更好。换言之，原则上存在能取得较小误差的算法，但实验中算法未做到。学习过程的次优性似乎是两类双下降峰值的根源。

确实，采用最优调节的正则化（第 9 章详述），可显著改善 $n \approx d$ 时误差，同时缓解两种双下降，见图 8.11。

上述直觉只解释峰值，未解释随模型规模的第二次下降，即过参数化模型为何泛化如此好。其理论理解是活跃研究领域，近来进展很多。典型解释是梯度下降等常用优化器产生隐式正则化（第 9.2 节详述）。即使损失无显式正则项，过参数化模型仍受到隐式正则化，因而优于任意一个能拟合数据的解。例如，线性模型中当 $n \ll d$，零初始化的梯度下降找到拟合数据的最小范数解，而非任意拟合解。最小范数正则化在过参数化区间足够好，但在 $n \approx d$ 时不够好，因而出现误差峰值。

图 8.11 转载自 Nakkiran et al. [2020] 图 1；Hastie et al. [2022]、Mei and Montanari [2022] 也观察到类似现象。

最后，多数双下降是在用参数数衡量复杂度时观察到的。参数数是否、何时是最佳复杂度指标尚不明确。很多情形用模型范数衡量复杂度。图 8.12 右侧的线性例子中，画出测试误差对学得模型范数的关系时，就不再出现双下降。部分原因是模型范数也在 $n \approx d$ 附近达峰值（图 8.12 中间；另见 Belkin et al. [2019]、Mei and Montanari [2022]、James et al. [2021] 第 10.8 节）。深度网络合适的复杂度指标更难确定。双下降仍是活跃的研究主题。

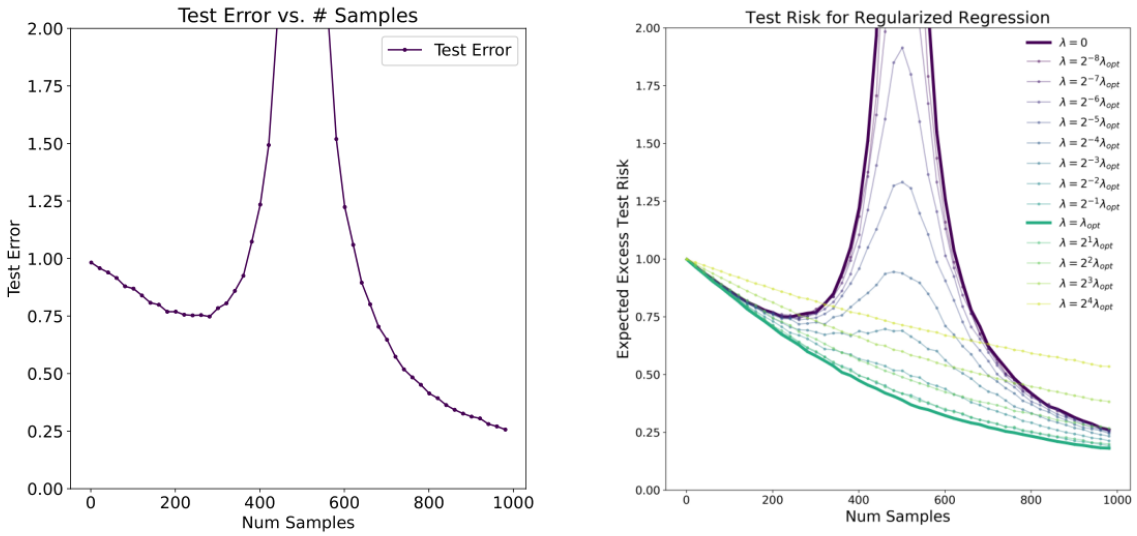


图 8.11：左为线性模型随样本数的双下降；右为不同正则化强度下的情况。对每个 n 最优调节 λ（绿色实线）可缓解双下降。设置：$x \sim \mathcal{N}(0, I_d)$，$y \sim x^\top \beta + \mathcal{N}(0, \sigma^2)$，$d = 500$、$\sigma = 0.5$、$\|\beta\|_2 = 1$。

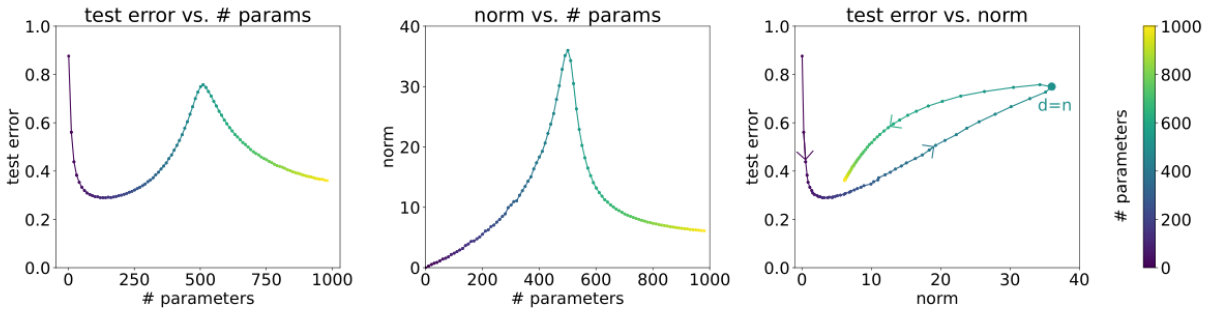


图 8.12：左为以参数数衡量复杂度的双下降；中为模型范数在 $n \approx d$ 处达峰值；右为测试误差对模型范数的关系，颜色表示参数数，箭头表示模型增大方向。右图更接近传统认识。设置：固定 $n = 500$ 的线性回归，输入是 Fashion-MNIST 的随机 ReLU 特征，输出 $y \in \mathbb{R}^{10}$ 为独热标签，与 Nakkiran et al. [2020] 第 5.2 节相同。

8.3 样本复杂度界

8.3.1 预备知识

现在开始探索学习理论。它本身有趣且富有启发，也能帮助我们深化直觉，形成在不同场景应用算法的经验法则。我们还希望回答：首先，能否形式化刚才的偏差—方差权衡？这最终也会引出自动决定拟合多项式阶数等模型选择方法。其次，真正关心的是泛化误差，然而多数算法在训练集上拟合；为什么训练集表现好能说明泛化好？能否联系两种误差？最后，是否有条件能证明学习算法表现良好？

先给出两个简单而有用的引理。

引理（联合界）。对不必独立的事件 $A_1, \dots, A_k$，

$$P(A_1 \cup \cdots \cup A_k) \le P(A_1) + \cdots + P(A_k).$$

原文称概率论中通常将此作为公理，因此不证明。直觉也很清楚：至少一个事件发生的概率不超过各事件概率总和。〔译注：联合界通常可由概率公理推出，并非独立的公理。〕

引理（Hoeffding 不等式）。设 $Z_1, \dots, Z_n$ 独立同分布，服从 Bernoulli(ϕ)，即 $P(Z_i = 1) = \phi$，$P(Z_i = 0) = 1 - \phi$。令 $\hat{\phi} = \frac{1}{n}\sum_{i=1}^{n} Z_i$，固定任意 $\gamma > 0$，则

$$P(|\phi - \hat{\phi}| > \gamma) \le 2\exp(-2\gamma^2 n).$$

这个引理在学习理论中也称 Chernoff 界。它说明，若以 n 个伯努利变量的平均值估计 ϕ，只要 n 足够大，估计偏离真值很远的概率就小。换言之，对正面概率为 ϕ 的偏置硬币投掷 n 次，正面比例以高概率是良好估计。

仅靠这两个引理，就能证明学习理论中一些深刻、重要的结果。为简洁，只考虑 $y \in \{0, 1\}$ 的二分类；讨论可推广至回归、多分类等。

给定大小为 n 的训练集 $S = \{(x^{(i)}, y^{(i)}) : i = 1, \dots, n\}$，样本从概率分布 $\mathcal{D}$ 独立同分布抽取。对假设函数 h，定义训练误差（经验风险 / 经验误差）

$$\hat{\varepsilon}(h) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{h(x^{(i)}) \neq y^{(i)}\}.$$

它就是训练样本错分比例。需强调依赖 S 时，写 $\hat{\varepsilon}_S(h)$。定义泛化误差为

$$\varepsilon(h) = P_{(x,y)\sim\mathcal{D}}(h(x) \neq y),$$

即从分布重新抽样后，h 错分它的概率。假设训练数据来自评价泛化误差所用的同一分布，这也称 PAC 假设之一。[36]

考虑线性分类 $h_\theta(x) = \mathbf{1}\{\theta^\top x \ge 0\}$，如何合理拟合参数？一种办法是最小化训练误差，选择

[36] PAC 即 “可能近似正确”（probably approximately correct），是证明大量学习理论结果的框架和假设集合。其中最重要的是训练与测试同分布，以及训练样本独立抽取。

$$\hat{\theta} = \arg\min_{\theta} \hat{\varepsilon}(h_\theta).$$

此过程称经验风险最小化（ERM），输出 $\hat{h} = h_{\hat{\theta}}$。我们把它看成最 “基本” 的学习算法，并重点研究。逻辑回归等也可视为 ERM 的近似。

学习理论中，将具体参数化、是否使用线性分类器等细节抽象掉很有用。假设函数类 $\mathcal{H}$ 是算法考虑的所有分类器的集合。线性分类中，$\mathcal{H} = \{h_\theta : h_\theta(x) = \mathbf{1}\{\theta^\top x \geq 0\}, \theta \in \mathbb{R}^{d+1}\}$，即输入域 $\mathcal{X}$ 上所有线性决策边界分类器。若研究神经网络，则可令 $\mathcal{H}$ 为某架构能表示的所有分类器。

ERM 因而就是在函数类上选择

$$\hat{h} = \arg\min_{h \in \mathcal{H}} \hat{\varepsilon}(h).$$

8.3.2 有限假设函数类

先设 $\mathcal{H} = \{h_1, \ldots, h_k\}$ 有限，包含 k 个从 $\mathcal{X}$ 到 $\{0,1\}$ 的函数。ERM 选取其中训练误差最小者。我们分两步保证 $\hat{h}$ 的泛化误差：先证明训练误差对所有 h 都可靠估计真实误差，再据此推导上界。

固定一个 h_i。从 $\mathcal{D}$ 抽样 (x, y)，定义伯努利变量 $Z = \mathbf{1}\{h_i(x) \neq y\}$，即指示它是否错分。类似定义 $Z_j = \mathbf{1}\{h_i(x^{(j)}) \neq y^{(j)}\}$。因为训练数据独立同分布，$Z, Z_j$ 分布相同，期望正好是错分概率 $\varepsilon(h_i)$。并且

$$\hat{\varepsilon}(h_i) = \frac{1}{n} \sum_{j=1}^{n} Z_j.$$

因此训练误差是均值为 $\varepsilon(h_i)$ 的 n 个独立伯努利变量的平均。由 Hoeffding 不等式，

$$P(|\varepsilon(h_i) - \hat{\varepsilon}(h_i)| > \gamma) \leq 2\exp(-2\gamma^2 n).$$

这说明 n 足够大时，对固定的 h_i 两种误差以高概率接近。但我们要对所有 $h \in \mathcal{H}$ 同时成立。令 A_i 为 $|\varepsilon(h_i) - \hat{\varepsilon}(h_i)| > \gamma$ 的事件。每个 $P(A_i) \leq 2\exp(-2\gamma^2 n)$，由联合界

$$\begin{aligned} P(\exists h \in \mathcal{H} : |\varepsilon(h) - \hat{\varepsilon}(h)| > \gamma) &= P(A_1 \cup \cdots \cup A_k) \\ &\leq \sum_{i=1}^{k} P(A_i) \\ &\leq \sum_{i=1}^{k} 2\exp(-2\gamma^2 n) \\ &= 2k\exp(-2\gamma^2 n). \end{aligned}$$

两边取补事件：

$$\begin{aligned} P(\neg\exists h \in \mathcal{H} : |\varepsilon(h) - \hat{\varepsilon}(h)| > \gamma) &= P(\forall h \in \mathcal{H} : |\varepsilon(h) - \hat{\varepsilon}(h)| \leq \gamma) \\ &\geq 1 - 2k\exp(-2\gamma^2 n). \end{aligned}$$

符号 $\neg$ 表示 “非”。因此，至少以 $1 - 2k\exp(-2\gamma^2 n)$ 的概率，所有函数的两种误差相差至多 γ。这称

为一致收敛，因为界同时适用于全部假设函数，而不是一个。

这里给定 n,γ，界定某个函数偏差超过 γ 的概率。关心的量有三个：n、γ、失败概率；可由任两个约束第三个。例如，给定 γ 与 $\delta>0$，需要多大 n 才能至少以 $1-\delta$ 的概率保证误差差不超过 γ？设 $\delta = 2k\exp(-2\gamma^2 n)$ 并解出 n（请自行确认这样做正确），得到只要

$$n \geq \frac{1}{2\gamma^2}\log\frac{2k}{\delta},$$

就至少以 $1-\delta$ 的概率，对所有 h 有 $|\varepsilon(h)-\hat{\varepsilon}(h)|\leq\gamma$。等价地，存在某个函数违反此条件的概率至多 δ。实现某性能水平所需的训练集大小，称为算法的样本复杂度。

关键在于所需样本数仅按函数数目 k 的对数增长，这在后面很重要。同样，固定 n,δ，解出 γ（也请自行确认），以至少 $1-\delta$ 的概率，对所有 $h\in\mathcal{H}$，

$$|\hat{\varepsilon}(h)-\varepsilon(h)|\leq\sqrt{\frac{1}{2n}\log\frac{2k}{\delta}}.$$

现在假设一致收敛成立，即所有函数误差差至多 γ。对 ERM 的泛化能说什么？定义 $h^*=\arg\min_{h\in\mathcal{H}}\varepsilon(h)$ 为类中最佳函数。既然限制使用该函数类，它就是可做到的最好结果，故与之比较合理。我们有

$$\varepsilon(\hat{h})\leq\hat{\varepsilon}(\hat{h})+\gamma\leq\hat{\varepsilon}(h^*)+\gamma\leq\varepsilon(h^*)+2\gamma.$$

第一步用一致收敛；第二步因 $\hat{h}$ 最小化训练误差，特别有 $\hat{\varepsilon}(\hat{h})\leq\hat{\varepsilon}(h^*)$；第三步再次用一致收敛。因此，若一致收敛成立，学到的模型比类中最优模型至多差 2γ！汇总为：

定理。设 $|\mathcal{H}|=k$，固定任意 n,δ。至少以 $1-\delta$ 的概率，

$$\varepsilon(\hat{h})\leq\min_{h\in\mathcal{H}}\varepsilon(h)+2\sqrt{\frac{1}{2n}\log\frac{2k}{\delta}}.$$

证明是令 γ 等于根号项，利用之前“一致收敛至少以 $1-\delta$ 概率成立”的结果，再使用刚证明的 2γ 界。

它也量化了模型选择中的偏差—方差权衡。如果从 $\mathcal{H}$ 换到更大的 $\mathcal{H}'\supseteq\mathcal{H}$，第一项只能减小，因为在更大集合取最小值，故“偏差”只会下降；但 k 增大使第二项增大，对应“方差”上升。固定 γ,δ 求解 n，还得到：

推论。设 $|\mathcal{H}|=k$，固定 δ,γ。若希望至少以 $1-\delta$ 的概率有 $\varepsilon(\hat{h})\leq\min_{h\in\mathcal{H}}\varepsilon(h)+2\gamma$，只需

$$n\geq\frac{1}{2\gamma^2}\log\frac{2k}{\delta}=O\left(\frac{1}{\gamma^2}\log\frac{k}{\delta}\right).$$

8.3.3 无限假设函数类

上面针对有限类得到有用定理，但线性分类等以实数为参数的类包含无限多个函数。能否证明类似结果？先看一个不算“正确”的论证。更好、更一般的论证存在，但这个有助建立直觉。

设 $\mathcal{H}$ 由 d 个实数参数化。计算机用 IEEE 双精度浮点数（C 的 double）存储每个实数需 64 位，因此算法实际由 $64d$ 位参数化，函数数至多 $k=2^{64d}$。由上一节推论，要至少以 $1-\delta$ 的概率保证

$\varepsilon(\hat{h}) \le \varepsilon(h^*) + 2\gamma$，只需

$$n \ge O\left(\frac{1}{\gamma^2}\log\frac{2^{64d}}{\delta}\right) = O\left(\frac{d}{\gamma^2}\log\frac{1}{\delta}\right) = O_{\gamma,\delta}(d).$$

下标表示隐藏常数可依赖 γ, δ。所以所需样本数至多线性依赖参数数。依赖 64 位浮点的论证不完全令人满意，但结论大体正确：若最小化训练误差，要用 d 参数的假设类学得“好”，一般需要数量级与 d 线性相关的样本。

这些结果针对 ERM。样本复杂度与 d 的线性关系通常适用于试图最小化训练误差或其近似的多数判别式算法，但原文接着说这些结论并不总能直接用于判别式算法。〔译注：原文此处重复了“discriminative”，前后表述疑有笔误；保留其意，不推定作者原拟用词。〕为许多非 ERM 算法提供良好理论保证，仍是活跃研究领域。

另一不足是依赖参数化方式；直觉上这不应重要。线性分类器可写 $h_\theta(x) = \mathbf{1}\{\theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d \ge 0\}$，用 $d+1$ 个参数；也可写

$$h_{u,v}(x) = \mathbf{1}\{(u_0^2 - v_0^2) + (u_1^2 - v_1^2)x_1 + \cdots + (u_d^2 - v_d^2)x_d \ge 0\},$$

用 $2d+2$ 个参数。但两者是同一个函数类：d 维空间的线性分类器。〔译注：原文第一种参数个数写为 $n+1$；按列举应为 $d+1$。〕

为得到更令人满意的论证，引入定义。给定点集 $S = \{x^{(1)}, \ldots, x^{(D)}\}$（与训练集无关），若 $\mathcal{H}$ 能实现其任意标记，则称它“打散” S：对任意 $y^{(1)}, \ldots, y^{(D)}$，存在 $h \in \mathcal{H}$，使所有 i 均满足 $h(x^{(i)}) = y^{(i)}$。

Vapnik–Chervonenkis 维数，记 $\mathrm{VC}(\mathcal{H})$，定义为能被该类打散的最大点集的大小。若可打散任意大集合，则维数为无穷。例如，下列三个点：

二维线性分类器 $h(x) = \mathbf{1}\{\theta_0 + \theta_1 x_1 + \theta_2 x_2 \ge 0\}$ 能否打散它们？答案是能。八种可能标记中的每一种都可找到“零训练误差”的线性分类器：

还可证明，任何四点集合都不能被此类打散，因此最大大小为 3，$\mathrm{VC}(\mathcal{H}) = 3$。但这不意味着每个三点集合都可被打散。若三点共线（下图左），则下图右的标记无法由线性分隔实现：

因此，证明 VC 维至少 D，只需找到至少一个可被打散的 D 点集合。

Vapnik 给出了如下定理，许多人认为它是整个学习理论最重要的定理。

定理。给定 $\mathcal{H}$，令 $D = \mathrm{VC}(\mathcal{H})$。至少以 $1-\delta$ 的概率，对所有 $h \in \mathcal{H}$，

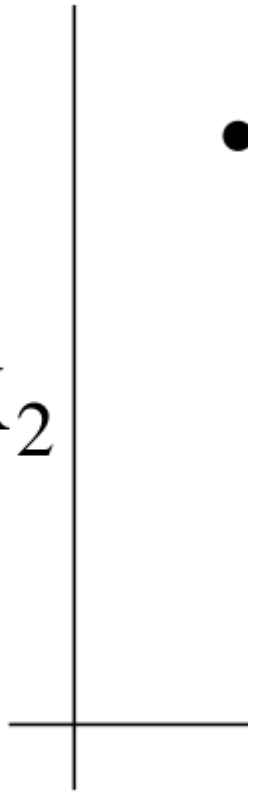
$$|\varepsilon(h) - \hat{\varepsilon}(h)| \le O\left(\sqrt{\frac{D}{n}\log\frac{n}{D} + \frac{1}{n}\log\frac{1}{\delta}}\right).$$

因此，至少以 $1-\delta$ 的概率，也有

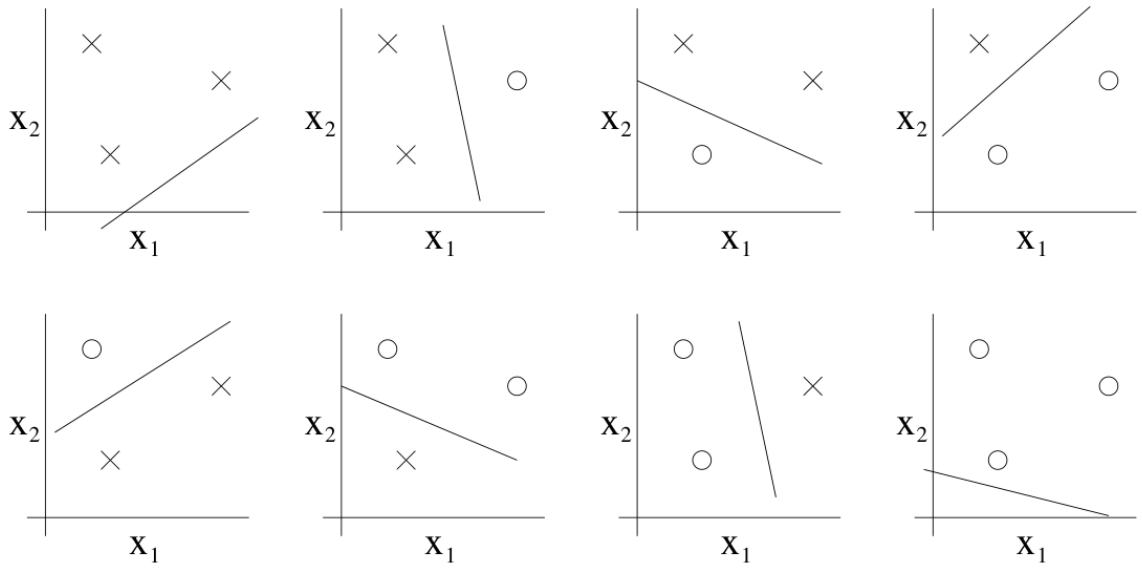
$$\varepsilon(\hat{h}) \le \varepsilon(h^*) + O\left(\sqrt{\frac{D}{n}\log\frac{n}{D} + \frac{1}{n}\log\frac{1}{\delta}}\right).$$

也就是说，VC 维有限时，n 足够大便发生一致收敛，从而可用最优误差界定学得模型的误差。还有：

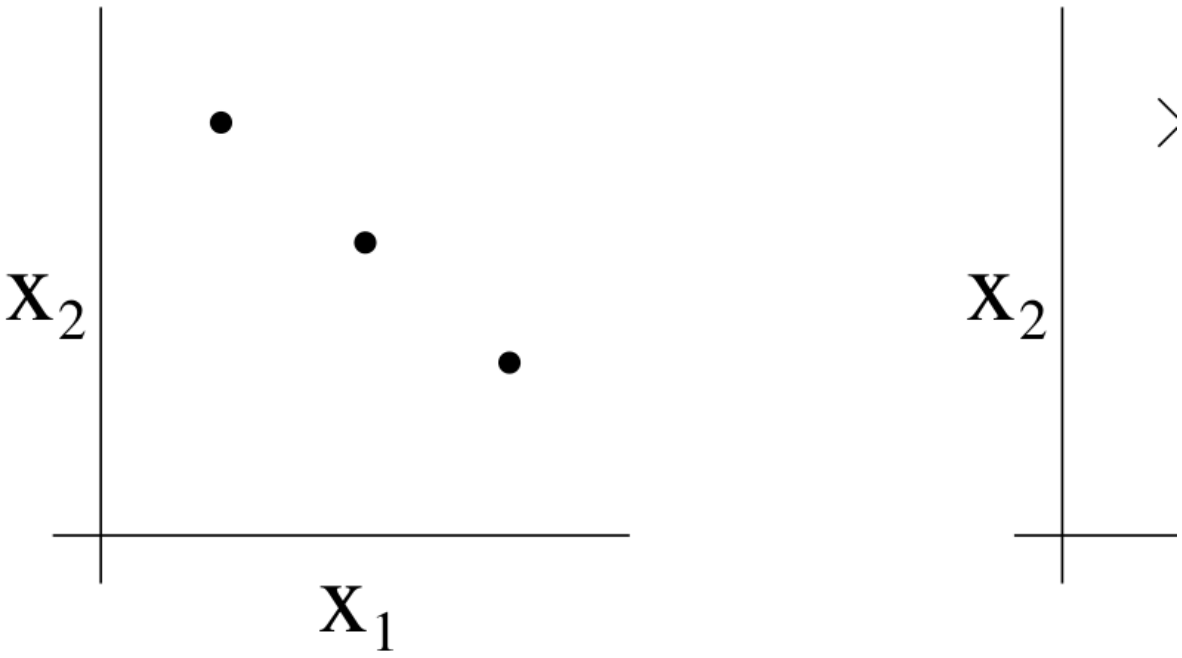
推论。要至少以 $1-\delta$ 的概率，对所有 h 有 $|\varepsilon(h) - \hat{\varepsilon}(h)| \le \gamma$，从而 $\varepsilon(\hat{h}) \le \varepsilon(h^*) + 2\gamma$，只需 $n = O_{\gamma,\delta}(D)$。



三个不共线的点。



三点的八种标记及对应线性分类器。



共线三点及不能线性分隔的一种标记。

换言之，用 $\mathcal{H}$ 学得好所需样本数线性依赖 VC 维。对 “大多数” 假设类，在 “合理” 参数化下，VC 维又大致线性依赖参数数。结合起来，对给定函数类和最小化训练误差的算法，要使泛化误差接近类中最优分类器，所需样本数通常大致线性依赖参数数。

第 9 章　正则化与模型选择

原文印刷页码：137–145。

9.1 正则化

回顾第 8.1 节的讨论，过拟合通常是使用过于复杂的模型造成的，因此我们需要选择适当的模型复杂度，以实现最佳的偏差—方差权衡。如果用参数个数衡量模型复杂度，就可以改变模型的大小，例如神经网络的宽度。不过，对模型复杂度恰当且有信息量的度量，也可能是参数的某个函数（例如参数的 ℓ_2 范数），未必取决于参数个数。在这些情况下，我们将使用机器学习中的一项重要技术——**正则化**——来控制模型复杂度并防止过拟合。

正则化通常是在训练损失 / 成本函数中加入一个额外的项，称为**正则项**，这里记为 $R(\theta)$：

$$J_\lambda(\theta) = J(\theta) + \lambda R(\theta). \tag{9.1}$$

这里，J_λ 通常称为**正则化损失**，$\lambda \geq 0$ 称为**正则化参数**。几乎在所有情况下，正则项 $R(\theta)$ 都是非负函数。在经典方法中，$R(\theta)$ 仅仅是参数 θ 的函数，但一些现代方法允许它依赖训练数据集。[37]

通常，我们选择模型 θ 的某种复杂度度量作为 $R(\theta)$。因此，使用正则化损失时，我们希望找到一个既能拟合数据（损失 $J(\theta)$ 小），又具有较低模型复杂度（$R(\theta)$ 小）的模型。正则化参数 λ 控制这两个目标之间的平衡。当 $\lambda = 0$ 时，正则化损失等价于原始损失。当 λ 是足够小的正数时，最小化正则化损失实际上相当于最小化原始损失，而在出现并列结果时用正则项作出选择。当正则项的作用极强时，原始损失就不再起有效作用，此时模型很可能具有较大的偏差。

最常用的正则化方法或许是 ℓ_2 **正则化**，其中 $R(\theta) = \frac{1}{2}\|\theta\|_2^2$。它鼓励优化器寻找 ℓ_2 范数较小的模型。在深度学习中，它经常被称为**权重衰减**，因为以学习率 η 对正则化损失进行梯度下降，等价于先将 θ 乘以标量 $1 - \eta\lambda$ 使其收缩 / 衰减，然后应用通常的梯度更新：

$$\begin{aligned} \theta &\leftarrow \theta - \eta \nabla J_\lambda(\theta) = \theta - \eta\lambda\theta - \eta\nabla J(\theta) \\ &= \underbrace{(1 - \lambda\eta)\theta}_{\text{权重衰减}} - \eta \nabla J(\theta). \end{aligned} \tag{9.2}$$

除了鼓励更简单的模型，正则化还可以对模型参数施加归纳偏置或结构。例如，假设我们事先相信真实模型参数中非零项的个数很少，[38] 这通常称为模型的**稀疏性**；那么可以对 θ 中非零项的个数施加正则化，以利用这一先验认识。非零项的个数记为 $\|\theta\|_0$。对参数施加额外结构会缩小搜索

[37] 为了简洁，这里的记号通常省略对训练数据集的依赖。例如，即使 $J(\theta)$ 显然必须依赖训练数据集，我们仍只写作 $J(\theta)$。

[38] 对线性模型而言，这意味着模型只需要输入中的少数坐标，就能作出准确预测。

空间，降低模型族的复杂度。例如，可以认为稀疏模型族的复杂度低于全体模型组成的模型族，因此这种做法往往能带来更好的泛化。另一方面，施加额外结构也可能增加偏差。例如，如果强烈地约束模型稀疏性，但没有任何稀疏模型能准确预测标签，就会产生很大的偏差。这类似于第 8.1 节中，用线性模型学习只能由二次函数表示的数据的情况。

参数的稀疏性不是参数的连续函数，因此无法用（随机）梯度下降对其进行优化。一种常见的松弛方式是使用 $R(\theta) = \|\theta\|_1$ 作为连续替代。[39]

$R(\theta) = \|\theta\|_1$（也称为 LASSO）和 $R(\theta) = \frac{1}{2}\|\theta\|_2^2$ 可能是线性模型最常用的正则项。有时也会使用其他范数或范数的幂。在核方法中，ℓ_2 范数正则化要常见得多，因为 ℓ_1 正则化通常与核技巧不兼容：最优解无法写成仅依赖特征内积的函数。

在深度学习中，最常用的正则项是 ℓ_2 正则化，即权重衰减。其他常见方法包括 dropout、数据增强、对权重矩阵的谱范数进行正则化，以及约束模型的 Lipschitz 性等。深度学习中的正则化是一个活跃的研究领域。人们还知道存在另一种隐式的正则化来源，下一节将对此展开讨论。

9.2 隐式正则化效应

优化器的**隐式正则化效应**，也称为**隐式偏置**或**算法正则化**，是深度学习时代观察到的一种新概念 / 现象。它大体上是指：除了正则化损失已经施加的约束外，优化器还会隐式地对参数施加结构。

在大多数经典情形中，损失或正则化损失只有唯一的全局最小值。因此，任何合理的优化器都应收敛到该全局最小值，不可能施加额外的偏好。然而在深度学习中，损失或正则化损失往往有多个（近似的）全局最小值，不同优化器可能收敛到不同的全局最小值。虽然这些全局最小值具有相同或相近的训练损失，但它们的性质可能不同，泛化表现也可能截然不同。图 9.1、图 9.2 及其图注给出了示意和一些实验结果。例如，某个全局最小值可能对应一个 Lipschitz 性更好或更稀疏的模型，从而具有更低的测试误差。事实表明，许多常用优化器或其中的组成部分，会偏好或倾向于找到具有某些性质的全局最小值，因而带来更好的测试表现。

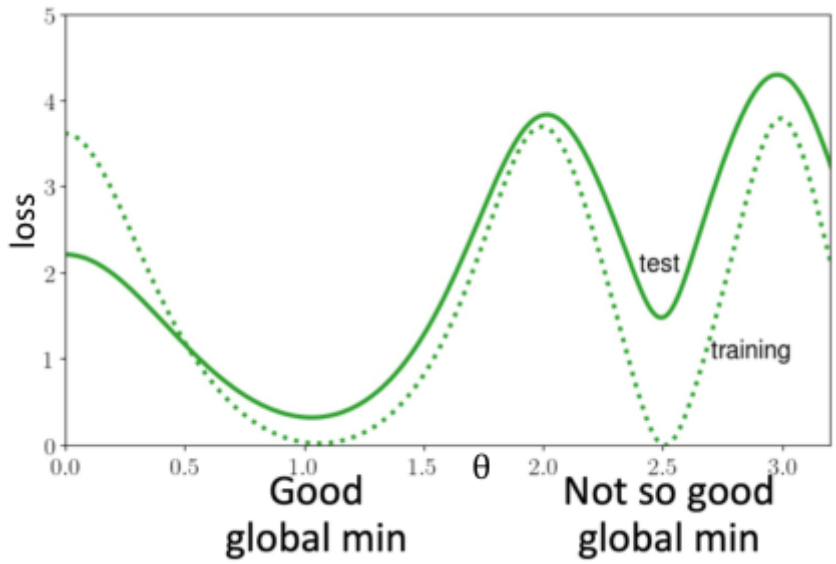


图 9.1

[39]已有大量理论工作解释为什么 $\|\theta\|_1$ 是鼓励稀疏性的良好替代，但这超出了本课程范围。一个直觉是：假设参数位于单位球面上，ℓ_1 范数最小的参数也恰好是最稀疏的参数，只有一个非零坐标。因此，在某种程度上，稀疏性与 ℓ_1 范数具有相同的极值点。

图 9.1：训练损失的不同全局最小值可以具有不同测试表现的示意图。

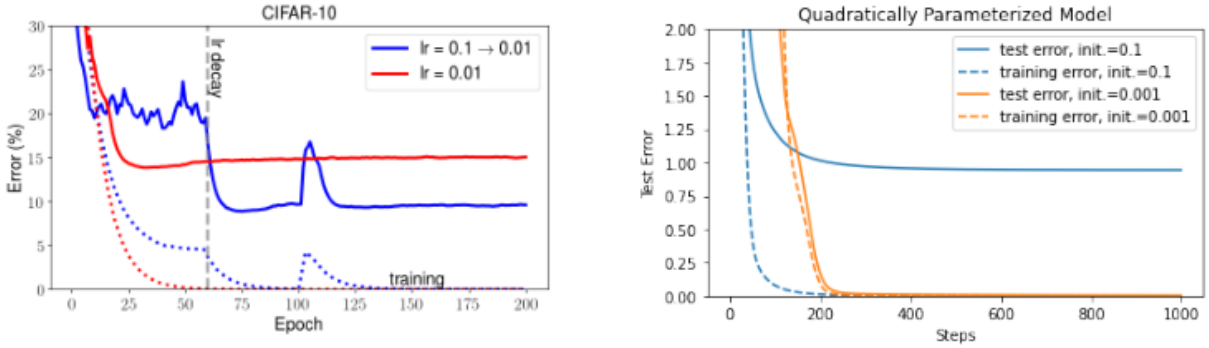


图 9.2

图 9.2：左图：在 CIFAR-10 数据集上，使用两种不同学习率调度训练神经网络的表现。虽然两个实验使用了完全相同的正则化损失，而且优化器都完美拟合了训练数据，但模型的泛化表现相差很大。右图：在另一个合成数据集上，采用不同初始化的优化器具有相同训练误差，却表现出不同的泛化能力。[40]

因此，这里需要记住的是，优化器的选择不仅影响训练损失的最小化，还会施加隐式正则化，影响模型泛化。即使当前优化器已经完美收敛到很小的训练误差，你仍可能需要调整优化器，以获得更好的泛化表现。

人们或许会问：优化器中的哪些组成部分会偏向哪类全局最小值？什么样的全局最小值泛化更好？这些仍是研究人员积极探索的开放问题。实证和理论研究提供了一些线索及经验法则。在许多情形中——但显然远非全部情形——只要优化过程能够成功地最小化训练损失，采用较大的初始学习率、较小的初始化、较小的批量，以及动量，似乎都有助于使优化偏向泛化能力更强的解。一个猜想是，优化过程中的随机性有助于优化器找到更平坦的全局最小值，即损失曲率较小的全局最小值，而平坦的全局最小值往往对应 Lipschitz 性更好的模型和更好的泛化。这个猜想在某些简化情形下可以证明。对隐式正则化效应进行严格刻画，仍是一个富有挑战的开放研究问题。

9.3 通过交叉验证选择模型

假设我们正在为一个学习问题从几个不同的模型中作出选择。例如，我们可能使用多项式回归模型 $h_\theta(x) = g(\theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_k x^k)$，希望确定 k 应取 $0, 1, \ldots, 10$ 中的哪个值。如何自动选出一个模型，在偏差和方差这两个“孪生祸害”之间取得良好权衡？[41] 或者，假设我们想自动选择局部加权回归的带宽参数 τ，或 ℓ_1 正则化 SVM 的参数 C，该怎么做？

为使讨论具体化，本讲义假设我们需要在有限的模型集合 $\mathcal{M} = \{M_1, \ldots, M_d\}$ 中选择。例如，在上面的第一个例子中，模型 M_i 可以是 i 次多项式回归模型。（推广到无限的 $\mathcal{M}$ 并不难。[42]）或者，如果我们正在决定使用 SVM、神经网络还是逻辑回归，$\mathcal{M}$ 就可以包含这些模型。

交叉验证。假设我们像往常一样，给定训练集 S。根据已有的经验风险最小化知识，最初看起来可能合理的模型选择算法是：

[40] 实验设置与 Woodworth 等（2020）、HaoChen 等（2020）相同。

[41] 前面讲义中说过，偏差和方差是两种很不一样的东西，因此有些读者也许会好奇，这里是否适合称它们为“孪生”祸害。或许把它们看作异卵双胞胎更合适，但“偏差和方差这两个异卵孪生祸害”这个说法就没有那么顺口了。

[42] 如果我们试图从无限模型集合中选择，例如模型对应于带宽 $\tau \in \mathbb{R}_+$ 的所有可能取值，可以将 τ 离散化，只考虑有限个值。更一般地，这里介绍的大多数算法都可视为在模型空间中执行优化搜索，而这种搜索也可以在无限模型类上进行。

1. 在 S 上训练每个模型 M_i，得到假设 h_i。
2. 选择训练误差最小的假设。

这个算法行不通。考虑选择多项式次数的问题：多项式的次数越高，对训练集 S 的拟合就越好，训练误差也就越低。因此，这个方法总会选择一个高方差的高次多项式模型，而我们此前已经看到，这往往不是一个好的选择。

下面的算法效果更好。在**留出交叉验证**（也称为**简单交叉验证**）中，我们执行以下步骤：

1. 将 S 随机划分为 S_{train}（例如占数据的 70%）和 S_{cv}（剩余 30%）。其中，S_{cv} 称为**留出交叉验证集**。
2. 仅在 S_{train} 上训练每个模型 M_i，得到假设 h_i。
3. 选择并输出在留出交叉验证集上误差 $\hat{\varepsilon}_{S_{\text{cv}}}(h_i)$ 最小的假设 h_i。这里，$\hat{\varepsilon}_{S_{\text{cv}}}(h)$ 表示 h 在 S_{cv} 中样本上的平均误差。留出验证集上的误差也称为**验证误差**。

通过在模型未用于训练的样本集 S_{cv} 上进行测试 / 验证，我们能够更好地估计每个假设 h_i 的真实泛化 / 测试误差。因此，这种方法实际上是在选择估计泛化 / 测试误差最小的模型。验证集大小取决于可用样本总数。通常将数据的 1/4 至 1/3 用作留出交叉验证集，30% 是一个典型选择。但当数据集非常大时，只要验证样本的绝对数量足够，验证集就可以占较小的比例。例如，ImageNet 数据集约有 100 万张训练图像，验证集有时设为 5 万张，仅约占全部样本的 5%。

也可以将算法的第 3 步替换为：按 $\arg\min_i \hat{\varepsilon}_{S_{\text{cv}}}(h_i)$ 选择模型 M_i，然后在整个训练集 S 上重新训练它。这通常是个好主意，但对初始条件和 / 或数据扰动十分敏感的学习算法是一个例外。对于这些方法，M_i 在 S_{train} 上表现良好，并不一定意味着它在 S_{cv} 上也表现良好，因此省略重新训练这一步可能更合适。

留出交叉验证的缺点是，它“浪费”了大约 30% 的数据。即使选择在整个训练集上重新训练模型，实际寻找的仍仿佛是一个只有 $0.7n$ 个训练样本的学习问题的好模型，而不是具有 n 个样本的问题，因为每次测试的模型都只在 $0.7n$ 个样本上训练。如果数据丰富且 / 或获取成本低，这没有问题；但对于数据稀缺的学习问题（例如 $n = 20$），我们希望做得更好。

下面介绍一种每次留出更少数据的方法，称为 k **折交叉验证**：

1. 将 S 随机划分为 k 个互不相交的子集，每个子集有 m/k 个训练样本。将这些子集记为 $S_1, \ldots, S_k$。
2. 对每个模型 M_i，按以下方式评估。对 $j = 1, \ldots, k$：在 $S_1 \cup \cdots \cup S_{j-1} \cup S_{j+1} \cup \cdots \cup S_k$ 上训练 M_i，也就是使用除 S_j 以外的全部数据，得到假设 h_{ij}；在 S_j 上测试 h_{ij}，得到 $\hat{\varepsilon}_{S_j}(h_{ij})$。随后，对这些误差关于 j 取平均，作为模型 M_i 的估计泛化误差。
3. 选择估计泛化误差最低的模型 M_i，在整个训练集 S 上重新训练，将得到的假设作为最终结果输出。

这里的折数通常取 $k = 10$。虽然每次留出的数据比例变为 $1/k$，比此前小得多，但这种流程可能比留出交叉验证需要更多计算，因为现在每个模型都要训练 k 次。

虽然 $k = 10$ 很常见，但当数据极其稀缺时，有时会采用极端选择 $k = m$，使每次留出的数据尽可能少。此时，我们反复使用 S 中除一个样本之外的所有训练样本来训练，再在留出的那个样本上测试。最后，对得到的 $m = k$ 个误差取平均，作为模型泛化误差的估计。这种方法有自己的名称：由于每次只留出一个训练样本，它称为**留一交叉验证**。

最后，虽然我们把交叉验证的不同形式介绍为模型选择方法，但它们也可以更简单地用于评估单个模型或算法。例如，如果你实现了某个学习算法，想估计它在应用中的表现；或者发明了一种新

的学习算法，想在技术论文中报告它在各个测试集上的表现，那么交叉验证就是一种合理的方法。

9.4 贝叶斯统计与正则化

本节将介绍我们对抗过拟合时可以使用的又一种工具。

本学期开始时，我们讨论过用最大似然估计（MLE）拟合参数，并按以下方式选择参数：

$$\theta_{\text{MLE}} = \arg\max_{\theta} \prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}; \theta).$$

在此后的讨论中，我们始终将 θ 视为世界的一个未知参数。把 θ 看作取值固定但未知的量，是**频率学派统计**的观点。在这种世界观中，θ 不是随机的，只是恰好未知；我们的任务是提出统计过程，例如最大似然法，尝试估计该参数。

处理参数估计问题的另一种方式是采用**贝叶斯**观点，把 θ 看作取值未知的随机变量。在这种方法中，我们为 θ 指定一个**先验分布** $p(\theta)$，用以表达我们对参数的“先验信念”。给定训练集 $S = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$，当需要对新的 x 进行预测时，可以计算参数的**后验分布**：

$$\begin{aligned} p(\theta \mid S) &= \frac{p(S \mid \theta) p(\theta)}{p(S)} \\ &= \frac{\left(\prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, \theta)\right) p(\theta)}{\int_{\theta} \left(\prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, \theta)\right) p(\theta)\, d\theta}. \end{aligned} \tag{9.3}$$

上式中的 $p(y^{(i)} \mid x^{(i)}, \theta)$ 来自你为学习问题采用的模型。例如，使用贝叶斯逻辑回归时，可以选择 $p(y^{(i)} \mid x^{(i)}, \theta) = h_\theta(x^{(i)})^{y^{(i)}}(1 - h_\theta(x^{(i)}))^{1-y^{(i)}}$，其中 $h_\theta(x^{(i)}) = 1/(1 + \exp(-\theta^T x^{(i)}))$。[43]

给定一个新的测试样本 x，需要进行预测时，可以利用 θ 的后验分布，计算类别标签的后验分布：

$$p(y \mid x, S) = \int_{\theta} p(y \mid x, \theta) p(\theta \mid S)\, d\theta. \tag{9.4}$$

上式中的 $p(\theta \mid S)$ 来自式 (9.3)。例如，如果目标是在给定 x 的条件下预测 y 的期望值，我们就输出[44]

$$\mathbb{E}[y \mid x, S] = \int_{y} y p(y \mid x, S)\, dy.$$

上述过程可以看作“完全贝叶斯”的预测：预测结果是相对于 θ 的后验分布 $p(\theta \mid S)$ 求平均得到的。遗憾的是，一般而言计算这个后验分布非常困难，因为它要求像式 (9.3) 那样，对通常处于高维空间中的 θ 积分，而这种积分通常没有闭式解。

因此，实践中会改为近似 θ 的后验分布。一种常见近似是用单个点估计替代式 (9.4) 中的 θ 后验分布。θ 的 **MAP（最大后验）估计**为

[43] 由于现在把 θ 视为随机变量，就可以将其取值作为条件，写作 $p(y \mid x, \theta)$，而不是 $p(y \mid x; \theta)$。

[44] 如果 y 是离散变量，下面的积分应替换为求和。

$$\theta_{\text{MAP}} = \arg\max_{\theta} \left(\prod_{i=1}^{n} p(y^{(i)} \mid x^{(i)}, \theta) \right) p(\theta). \tag{9.5}$$

注意，除了末尾的先验项 $p(\theta)$，这个公式与 θ 的 MLE（最大似然）估计相同。

实际应用中，先验 $p(\theta)$ 的一种常见选择是假设 $\theta \sim \mathcal{N}(0, \tau^2 I)$。采用这种先验后，拟合得到的参数 θ_{MAP} 的范数通常小于最大似然估计所选择的参数范数。实际中，这使贝叶斯 MAP 估计比参数的最大似然估计更不容易过拟合。例如，尽管文本分类中通常有 $d \gg n$，贝叶斯逻辑回归仍是一种有效的文本分类算法。

第四部分　无监督学习

第 10 章　聚类与 k-means 算法

原文印刷页码：146–149。

在聚类问题中，给定训练集 $\{x^{(1)}, \dots, x^{(n)}\}$，希望把数据分成若干内部紧密的“簇”。与前面一样，$x^{(i)} \in \mathbb{R}^d$；但这里没有标签 $y^{(i)}$。因此，这是一个无监督学习问题。

k-means 聚类算法如下：

1. 随机初始化簇质心 $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^d$。
2. 重复以下操作，直到收敛：

   对每个 i，令

$$c^{(i)} := \arg\min_j \|x^{(i)} - \mu_j\|^2.$$

   对每个 j，令

$$\mu_j := \frac{\sum_{i=1}^{n} \mathbf{1}\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^{n} \mathbf{1}\{c^{(i)} = j\}}.$$

在上述算法中，k 是算法的一个参数，表示希望找到的簇的数量；簇质心 μ_j 表示当前对各簇中心位置的估计。初始化簇质心时，也就是上面的第 1 步，可以随机选择 k 个训练样本，把这些样本的值作为簇质心。当然也可以采用其他初始化方法。

算法的内层循环反复执行两步：（i）把每个训练样本 $x^{(i)}$“分配”给最近的簇质心 μ_j；（ii）把每个簇质心 μ_j 移动到分配给它的那些点的均值处。图 10.1 展示了 k-means 的运行过程。

k-means 算法一定会收敛吗？在某种意义下，答案是肯定的。具体来说，定义**失真函数**：

$$J(c, \mu) = \sum_{i=1}^{n} \|x^{(i)} - \mu_{c^{(i)}}\|^2.$$

因此，J 衡量的是每个训练样本 $x^{(i)}$ 到其所属簇质心 $\mu_{c^{(i)}}$ 的距离平方之和。可以证明，k-means 恰好是在 J 上进行坐标下降。具体而言，k-means 的内层循环先固定 μ，对 c 最小化 J；再固定 c，对 μ 最小化 J。所以 J 必然单调下降，其值必然收敛。通常这也意味着 c 和 μ 会收敛。理论上，k-means 可能在若干具有完全相同 J 值的聚类结果之间振荡，即在不同的 c 或 μ 取值之间振荡；但实践中几乎不会遇到这种情况。

失真函数 J 是非凸函数，因此对 J 做坐标下降并不能保证收敛到全局最小值。换句话说，k-means 可能受到局部最优解的影响。即便如此，它通常仍能很好地工作，得到很好的聚类结果。如果担

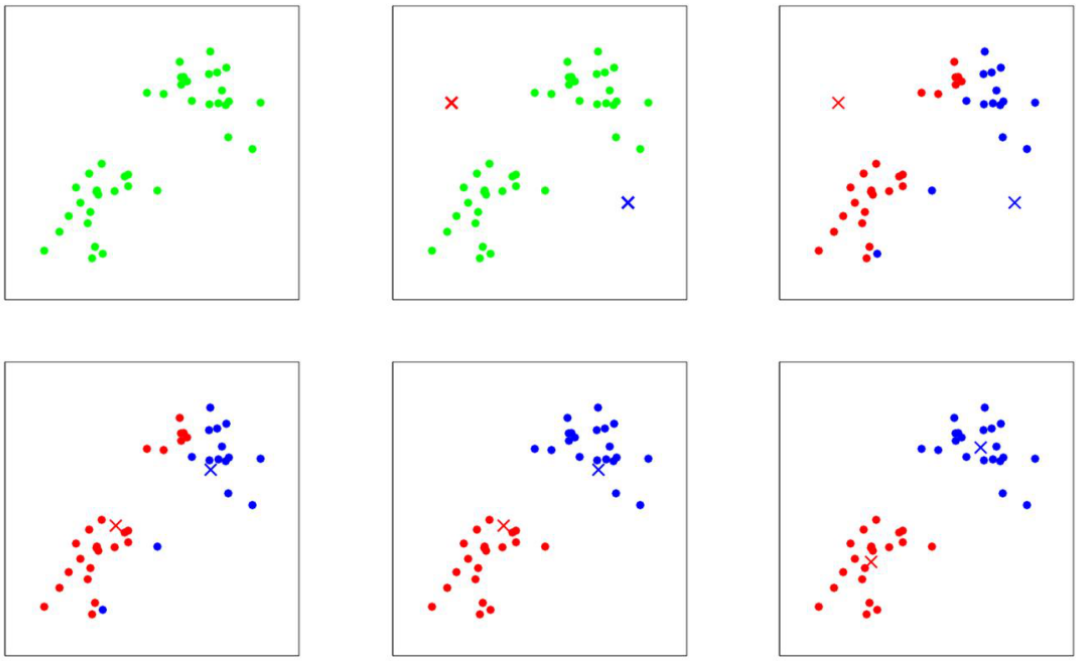


图 10.1：k-means 算法。训练样本用圆点表示，簇质心用叉号表示。按从左到右、从上到下的顺序，(a) 为原始数据集；(b) 为随机初始化的簇质心，此例中初始质心没有直接选为两个训练样本；(c–f) 展示两次迭代。每次迭代先把样本分配给最近的质心，用与该质心相同的颜色表示归属，再把质心移到所分配样本的均值处。建议以彩色查看。图片由 Michael Jordan 提供。

心陷入较差的局部最小值，一种常见方法是多次运行 k-means，每次对簇质心 μ_j 使用不同的随机初值。然后在所有得到的聚类结果中，选择失真 $J(c,\mu)$ 最小的一个。

第 11 章　EM 算法

原文印刷页码：150–166。

本章讨论用于密度估计的 EM（期望最大化，Expectation-Maximization）算法。

11.1 高斯混合模型的 EM 算法

假设我们像往常一样，给定训练集 $\{x^{(1)}, \dots, x^{(n)}\}$。由于这里是无监督学习，这些数据点没有任何标签。

我们希望通过指定联合分布 $p(x^{(i)}, z^{(i)}) = p(x^{(i)} \mid z^{(i)})p(z^{(i)})$ 来对数据建模。其中，$z^{(i)} \sim \text{Multinomial}(\phi)$，满足 $\phi_j \geq 0$、$\sum_{j=1}^{k} \phi_j = 1$，参数 ϕ_j 给出 $p(z^{(i)} = j)$；同时，$x^{(i)} \mid z^{(i)} = j \sim \mathcal{N}(\mu_j, \Sigma_j)$。用 k 表示 $z^{(i)}$ 的可能取值个数。因此，模型假设每个 $x^{(i)}$ 的生成方式如下：先从 $\{1, \dots, k\}$ 随机选择 $z^{(i)}$，再根据 $z^{(i)}$ 从 k 个高斯分布中的某一个抽取 $x^{(i)}$。这称为**高斯混合模型**。还要注意，$z^{(i)}$ 是**潜随机变量**，也就是说，它们是隐藏 / 未观测到的。正是这一点使估计问题变得困难。

因此，模型的参数是 ϕ, μ, Σ。为了估计这些参数，我们可以写出数据的对数似然：

$$
\begin{aligned}
\ell(\phi, \mu, \Sigma) &= \sum_{i=1}^{n} \log p(x^{(i)}; \phi, \mu, \Sigma) \\
&= \sum_{i=1}^{n} \log \sum_{z^{(i)}=1}^{k} p(x^{(i)} \mid z^{(i)}; \mu, \Sigma) p(z^{(i)}; \phi).
\end{aligned}
$$

然而，如果把这个公式对参数的导数设为零并尝试求解，就会发现无法得到参数最大似然估计的闭式解。（请自行尝试。）

随机变量 $z^{(i)}$ 表明每个 $x^{(i)}$ 来自 k 个高斯分布中的哪一个。注意，如果知道 $z^{(i)}$ 的值，最大似然问题就容易了。具体而言，此时对数似然可写为

$$
\ell(\phi, \mu, \Sigma) = \sum_{i=1}^{n} \left[\log p(x^{(i)} \mid z^{(i)}; \mu, \Sigma) + \log p(z^{(i)}; \phi) \right].
$$

对 ϕ, μ, Σ 最大化该式，得到参数：

$$\phi_j = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{z^{(i)} = j\},$$
$$\mu_j = \frac{\sum_{i=1}^{n} \mathbf{1}\{z^{(i)} = j\} x^{(i)}}{\sum_{i=1}^{n} \mathbf{1}\{z^{(i)} = j\}},$$
$$\Sigma_j = \frac{\sum_{i=1}^{n} \mathbf{1}\{z^{(i)} = j\}(x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^{n} \mathbf{1}\{z^{(i)} = j\}}.$$

确实可以看出，如果 $z^{(i)}$ 已知，那么最大似然估计几乎与我们估计高斯判别分析模型参数时的情况完全相同，只是这里由 $z^{(i)}$ 扮演类别标签的角色。[45]

但在密度估计问题中，我们并不知道 $z^{(i)}$。该怎么办?

EM 算法是一种包含两个主要步骤的迭代算法。应用到当前问题时，在 **E** 步中，它尝试 "猜测" $z^{(i)}$ 的值；在 **M** 步中，它根据猜测更新模型参数。由于 M 步中假定前一步的猜测正确，最大化就变得容易。算法如下。

重复以下步骤，直至收敛：

E 步：对每个 i, j，令

$$w_j^{(i)} := p(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma).$$

M 步：更新参数：

$$\phi_j := \frac{1}{n}\sum_{i=1}^{n} w_j^{(i)},$$
$$\mu_j := \frac{\sum_{i=1}^{n} w_j^{(i)} x^{(i)}}{\sum_{i=1}^{n} w_j^{(i)}},$$
$$\Sigma_j := \frac{\sum_{i=1}^{n} w_j^{(i)}(x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^{n} w_j^{(i)}}.$$

在 E 步中，给定 $x^{(i)}$，使用当前参数设置计算 $z^{(i)}$ 的后验概率。也就是说，根据贝叶斯法则，得到

$$p(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma) = \frac{p(x^{(i)} \mid z^{(i)} = j; \mu, \Sigma) p(z^{(i)} = j; \phi)}{\sum_{l=1}^{k} p(x^{(i)} \mid z^{(i)} = l; \mu, \Sigma) p(z^{(i)} = l; \phi)}.$$

这里，$p(x^{(i)} \mid z^{(i)} = j; \mu, \Sigma)$ 是均值为 μ_j、协方差为 Σ_j 的高斯分布在 $x^{(i)}$ 处的密度；$p(z^{(i)} = j; \phi)$ 则等于 ϕ_j，其余以此类推。E 步计算的 $w_j^{(i)}$ 表示我们对 $z^{(i)}$ 取值的 "软" 猜测。[46]

还应该把 M 步的更新与 $z^{(i)}$ 完全已知时的公式比较。二者相同，只不过原来表示每个数据点来自哪个高斯分布的指示函数 $\mathbf{1}\{z^{(i)} = j\}$，现在被 $w_j^{(i)}$ 取代了。

EM 算法也让人联想到 K 均值聚类算法，只不过这里用 "软" 分配 $w_j^{(i)}$ 取代了 "硬" 簇分配 $c^{(i)}$。与

[45]此处公式与习题集 1 中高斯判别分析所得公式还有一些细微差别：首先，$z^{(i)}$ 已从伯努利变量推广为多项变量；其次，这里每个高斯分布都使用不同的 Σ_j。

[46]"软" 是指猜测以概率表示，取值在 $[0, 1]$ 内；相比之下，"硬" 猜测是一个确定的最佳猜测，例如取值在 $\{0, 1\}$ 或 $\{1, \ldots, k\}$ 内。

K 均值类似，它也容易陷入局部最优，因此尝试多组不同初始参数可能是个好主意。

显然，EM 算法具有很自然的解释：反复尝试猜测未知的 $z^{(i)}$。但它究竟是如何推导出来的？我们能否对它作出某些保证，例如关于收敛性的保证？下面将介绍更一般的 EM 观点。借助这一观点，我们能方便地将其应用于其他含潜变量的估计问题，也能给出收敛性保证。

11.2 Jensen 不等式

先介绍一个非常有用的结果：**Jensen 不等式**。

设函数 f 的定义域为实数集。回顾，如果对所有 $x \in \mathbb{R}$ 都有 $f''(x) \geq 0$，则 f 为**凸函数**。当 f 的输入是向量时，这一条件推广为其 Hessian 矩阵 H 半正定，即 $H \geq 0$。如果对所有 x 都有 $f''(x) > 0$，则称 f **严格凸**；对于向量输入，相应条件是 H 正定，记为 $H > 0$。Jensen 不等式可陈述如下。

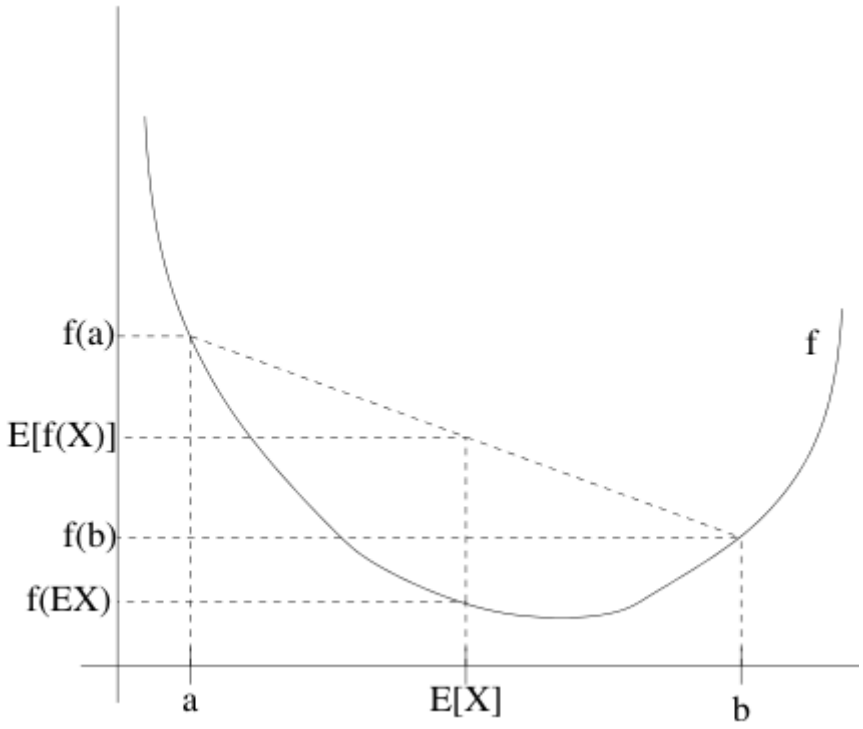
定理。设 f 为凸函数，X 为随机变量，则

$$\mathbb{E}[f(X)] \geq f(\mathbb{E}X).$$

此外，如果 f 严格凸，那么 $\mathbb{E}[f(X)] = f(\mathbb{E}X)$ 成立，当且仅当 $X = \mathbb{E}[X]$ 以概率 1 成立，即 X 是常量。

回顾，我们有时会在写期望时省略括号，因此上面定理中的 $f(\mathbb{E}X) = f(\mathbb{E}[X])$。

为解释这个定理，请看下图。



Jensen 不等式示意图

图示：凸函数上的两点及其期望。实线为 f；先求函数值再取平均，得到的值不小于在输入平均值处计算的函数值。

这里，实线表示凸函数 f。随机变量 X 以 0.5 的概率取值 a，以 0.5 的概率取值 b，这两个点标在

横轴上。因此，X 的期望值就是 a 与 b 的中点。

纵轴上还标出了 $f(a)$、$f(b)$ 和 $f(\mathbb{E}[X])$。而 $\mathbb{E}[f(X)]$ 是纵轴上 $f(a)$ 与 $f(b)$ 的中点。由这个例子可看出，因为 f 是凸函数，必有 $\mathbb{E}[f(X)] \geq f(\mathbb{E}X)$。

顺便说一句，很多人不容易记住不等号的方向，记住这样的图形就能很快判断答案。

备注。回顾，f 为［严格］凹函数，当且仅当 $-f$ 为［严格］凸函数，即 $f''(x) \leq 0$ 或 $H \leq 0$。Jensen 不等式对凹函数 f 也成立，但所有不等号的方向都要反转，例如 $\mathbb{E}[f(X)] \leq f(\mathbb{E}X)$。

11.3 一般的 EM 算法

假设我们要解决一个估计问题，训练集 $\{x^{(1)}, \ldots, x^{(n)}\}$ 包含 n 个独立样本。我们有一个潜变量模型 $p(x, z; \theta)$，其中 z 为潜变量；为简单起见，假设它只有有限个可能取值。对潜变量 z 求边缘分布，即可得到 x 的密度：

$$p(x; \theta) = \sum_z p(x, z; \theta). \tag{11.1}$$

我们希望通过最大化数据的对数似然来拟合参数 θ。对数似然定义为

$$\ell(\theta) = \sum_{i=1}^{n} \log p(x^{(i)}; \theta). \tag{11.2}$$

可以用联合密度 $p(x, z; \theta)$ 改写目标函数：

$$\ell(\theta) = \sum_{i=1}^{n} \log p(x^{(i)}; \theta), \tag{11.3}$$

$$\ell(\theta) = \sum_{i=1}^{n} \log \sum_{z^{(i)}} p(x^{(i)}, z^{(i)}; \theta). \tag{11.4}$$

然而，显式求出参数 θ 的最大似然估计可能很困难，因为这会产生难以求解的非凸优化问题。[47] 这里，$z^{(i)}$ 是潜随机变量；通常如果能观测到它们，最大似然估计就很容易了。

在这样的情形中，EM 算法为最大似然估计提供了一种高效方法。直接最大化 $\ell(\theta)$ 可能很困难，我们的策略是反复构造 ℓ 的下界（E 步），然后优化这个下界（M 步）。[48]

事实上，求和符号 $\sum_{i=1}^{n}$ 在这里并非本质性的。为了更简单地阐述 EM 算法，先考虑只优化单个样本 x 的对数似然 $\log p(x)$。推导出优化 $\log p(x)$ 的算法后，再在每个相关公式中补回求和，转换为适用于 n 个样本的算法。因此，现在要优化的是 $\log p(x; \theta)$，它可以改写为

$$\log p(x; \theta) = \log \sum_z p(x, z; \theta). \tag{11.5}$$

[47] 优化问题难以求解，这主要是一种经验观察。

[48] 经验上，E 步和 M 步通常比直接优化 $\ell(\cdot)$ 更容易高效计算，但这并不意味着交替执行这两步总能收敛到 $\ell(\cdot)$ 的全局最优。即使对于高斯混合模型，EM 算法也可能收敛到全局最优，或陷入困境，这取决于训练数据的性质。对真实数据的经验表明，EM 经常能收敛到似然相对较高的解，即使未达到最优；其背后的理论仍在很大程度上不清楚。

令 Q 是定义在 z 的可能取值上的一个分布，即 $\sum_z Q(z) = 1$、$Q(z) \geq 0$。

考虑以下推导：[49]

$$\begin{aligned} \log p(x;\theta) &= \log \sum_z p(x,z;\theta) \\ &= \log \sum_z Q(z) \frac{p(x,z;\theta)}{Q(z)}. \end{aligned} \tag{11.6}$$

$$\log p(x;\theta) \geq \sum_z Q(z) \log \frac{p(x,z;\theta)}{Q(z)}. \tag{11.7}$$

推导的最后一步使用了 Jensen 不等式。具体来说，$f(x) = \log x$ 是凹函数，因为在其定义域 $x \in \mathbb{R}_+$ 上，$f''(x) = -1/x^2 < 0$。而求和式

$$\sum_z Q(z) \frac{p(x,z;\theta)}{Q(z)}$$

正是当 z 按照分布 Q 抽取时，量 $p(x,z;\theta)/Q(z)$ 的期望。[50] 根据 Jensen 不等式，

$$f\left(\mathbb{E}_{z\sim Q}\left[\frac{p(x,z;\theta)}{Q(z)}\right]\right) \geq \mathbb{E}_{z\sim Q}\left[f\left(\frac{p(x,z;\theta)}{Q(z)}\right)\right].$$

其中，下标 $z \sim Q$ 表示期望是针对从 Q 抽取的 z 计算的。这使我们能从式 (11.6) 得到式 (11.7)。

对于任意分布 Q，式 (11.7) 都给出了 $\log p(x;\theta)$ 的下界。Q 有许多可能选择，应该选哪一个？如果当前有一个参数猜测 θ，自然会希望在这个 θ 值处，让下界尽可能紧。也就是说，我们希望上面的不等式在当前的 θ 处取等号。

要使下界在某个特定的 θ 处取等号，就要求推导中使用 Jensen 不等式的那一步取等号。我们知道，一个充分条件是：被求期望的随机变量取“常数”值。即要求

$$\frac{p(x,z;\theta)}{Q(z)} = c,$$

其中常数 c 不依赖 z。只需选择

$$Q(z) \propto p(x,z;\theta)$$

即可做到。实际上，因为 Q 是概率分布，$\sum_z Q(z) = 1$，所以进一步得到

$$\begin{aligned} Q(z) &= \frac{p(x,z;\theta)}{\sum_z p(x,z;\theta)} \\ &= \frac{p(x,z;\theta)}{p(x;\theta)} \\ &= p(z \mid x;\theta). \end{aligned} \tag{11.8}$$

[49] 如果 z 连续，Q 就是概率密度，讨论中对 z 的求和应替换为积分。

[50] 仅当 $p(x,z;\theta) \neq 0$ 的地方都有 $Q(z) \neq 0$ 时，$p(x,z;\theta)/Q(z)$ 才有意义。这里隐含地假设只考虑具有这一性质的 Q。

因此，只需把 Q 设为给定 x 和当前参数 θ 时，z 的后验分布。

事实上，可以直接验证，当 $Q(z) = p(z \mid x;\theta)$ 时，式 (11.7) 取等号，因为

$$\begin{aligned}\sum_z Q(z)\log\frac{p(x,z;\theta)}{Q(z)} &= \sum_z p(z\mid x;\theta)\log\frac{p(x,z;\theta)}{p(z\mid x;\theta)}\\ &= \sum_z p(z\mid x;\theta)\log\frac{p(z\mid x;\theta)p(x;\theta)}{p(z\mid x;\theta)}\\ &= \sum_z p(z\mid x;\theta)\log p(x;\theta)\\ &= \log p(x;\theta)\sum_z p(z\mid x;\theta)\\ &= \log p(x;\theta),\end{aligned}$$

最后一步使用了 $\sum_z p(z \mid x;\theta) = 1$。

为方便起见，将式 (11.7) 中的表达式称为**证据下界（ELBO）**，记为

$$\mathrm{ELBO}(x;Q,\theta) = \sum_z Q(z)\log\frac{p(x,z;\theta)}{Q(z)}. \tag{11.9}$$

由此，式 (11.7) 可写为

$$\forall Q,\theta,x, \qquad \log p(x;\theta) \geq \mathrm{ELBO}(x;Q,\theta). \tag{11.10}$$

直观地说，EM 算法交替更新 Q 和 θ：第一步，按照式 (11.8) 令 $Q(z) = p(z \mid x;\theta)$，使当前 x,θ 满足 $\mathrm{ELBO}(x;Q,\theta) = \log p(x;\theta)$；第二步，固定 Q，对 θ 最大化 $\mathrm{ELBO}(x;Q,\theta)$。

回顾，上面的所有讨论都假设我们要优化单个样本 x 的对数似然 $\log p(x;\theta)$。当有多个训练样本时，基本思想相同，只需在相关位置对样本求和。下面构造多个训练样本的证据下界，并正式给出 EM 算法。

训练集为 $\{x^{(1)},\dots,x^{(n)}\}$。注意，$Q$ 的最优选择为 $p(z \mid x;\theta)$，它依赖具体样本 x。因此，这里引入 n 个分布 $Q_1,\dots,Q_n$，每个样本 $x^{(i)}$ 对应一个分布。对每个样本，可以构造证据下界

$$\log p(x^{(i)};\theta) \geq \mathrm{ELBO}(x^{(i)};Q_i,\theta) = \sum_{z^{(i)}} Q_i(z^{(i)})\log\frac{p(x^{(i)},z^{(i)};\theta)}{Q_i(z^{(i)})}.$$

对所有样本求和，得到对数似然的一个下界：

$$\begin{aligned}\ell(\theta) &\geq \sum_i \mathrm{ELBO}(x^{(i)};Q_i,\theta)\\ &= \sum_i\sum_{z^{(i)}} Q_i(z^{(i)})\log\frac{p(x^{(i)},z^{(i)};\theta)}{Q_i(z^{(i)})}.\end{aligned} \tag{11.11}$$

对于任意一组分布 $Q_1,\dots,Q_n$，式 (11.11) 都给出 $\ell(\theta)$ 的下界。与式 (11.8) 附近的论证类似，使等号成立的 Q_i 满足

$$Q_i(z^{(i)}) = p(z^{(i)} \mid x^{(i)}; \theta).$$

所以，只需将 Q_i 设为给定 $x^{(i)}$ 和当前参数 θ 时，$z^{(i)}$ 的后验分布。接下来，固定这些 Q_i，对参数最大化式 (11.11) 中的下界，得到新的 θ。反复执行这两个步骤，就得到如下 EM 算法。

重复以下步骤，直至收敛：

E 步：对每个 i，令

$$Q_i(z^{(i)}) := p(z^{(i)} \mid x^{(i)}; \theta).$$

M 步：令

$$\begin{aligned} \theta &:= \arg\max_{\theta} \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i, \theta) \\ &= \arg\max_{\theta} \sum_{i} \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})}. \end{aligned} \tag{11.12}$$

怎样知道算法是否收敛？设 $\theta^{(t)}$ 和 $\theta^{(t+1)}$ 是 EM 连续两次迭代的参数。现在证明 $\ell(\theta^{(t)}) \le \ell(\theta^{(t+1)})$，这说明 EM 总是单调提高对数似然。证明的关键是 Q_i 的选择。具体来说，在参数初始为 $\theta^{(t)}$ 的那次 EM 迭代中，我们选择 $Q_i^{(t)}(z^{(i)}) := p(z^{(i)} \mid x^{(i)}; \theta^{(t)})$。前面已经看到，这个选择确保式 (11.11) 推导中的 Jensen 不等式取等号，因此

$$\ell(\theta^{(t)}) = \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i^{(t)}, \theta^{(t)}). \tag{11.13}$$

参数 $\theta^{(t+1)}$ 通过最大化上式右侧得到。所以，

$$\begin{aligned} \ell(\theta^{(t+1)}) &\ge \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i^{(t)}, \theta^{(t+1)}) \\ &\ge \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i^{(t)}, \theta^{(t)}) \\ &= \ell(\theta^{(t)}). \end{aligned}$$

第一个不等式成立，是因为式 (11.11) 对所有 Q 和 θ 都成立；最后一个等式来自式 (11.13)；中间的不等式则是因为我们明确选择

$$\theta^{(t+1)} = \arg\max_{\theta} \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i^{(t)}, \theta).$$

因此，EM 使似然单调收敛。在描述算法时，我们说要运行到收敛。根据刚证明的结果，一个合理的收敛检验是：检查连续两次迭代之间 $\ell(\theta)$ 的增量是否小于某个容差参数；如果 EM 对 $\ell(\theta)$ 的改善已经很慢，就宣布收敛。

备注。如果重用 ELBO(·) 这个记号，定义

$$\begin{aligned}\mathrm{ELBO}(Q,\theta) &= \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i, \theta) \\ &= \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})},\end{aligned} \tag{11.14}$$

那么根据之前的推导，$\ell(\theta) \geq \mathrm{ELBO}(Q,\theta)$。EM 也可以看作对 $\mathrm{ELBO}(Q,\theta)$ 进行交替最大化的算法：E 步对 Q 最大化它（请自行验证），M 步对 θ 最大化它。

11.3.1 ELBO 的其他解释

令 $\mathrm{ELBO}(x; Q, \theta) = \sum_z Q(z) \log \frac{p(x,z;\theta)}{Q(z)}$，与式 (11.9) 的定义一致。ELBO 还有若干其他形式。首先，可以改写为

$$\begin{aligned}\mathrm{ELBO}(x; Q, \theta) &= \mathbb{E}_{z\sim Q}[\log p(x, z; \theta)] - \mathbb{E}_{z\sim Q}[\log Q(z)] \\ &= \mathbb{E}_{z\sim Q}[\log p(x \mid z; \theta)] - D_{\mathrm{KL}}(Q \| p_z),\end{aligned} \tag{11.15}$$

其中，用 p_z 表示联合分布 $p(x, z; \theta)$ 下 z 的边缘分布，而 D_{KL} 表示 KL 散度：

$$D_{\mathrm{KL}}(Q \| p_z) = \sum_z Q(z) \log \frac{Q(z)}{p(z)}. \tag{11.16}$$

在许多情况下，z 的边缘分布不依赖参数 θ。此时，对 θ 最大化 ELBO 就等价于最大化式 (11.15) 的第一项。这对应于最大化给定 z 时 x 的条件似然，通常比原问题更简单。

ELBO 的另一个形式为（请自行验证）

$$\mathrm{ELBO}(x; Q, \theta) = \log p(x) - D_{\mathrm{KL}}(Q \| p_{z|x}). \tag{11.17}$$

其中，$p_{z|x}$ 是在参数 θ 下，给定 x 时 z 的条件分布。这个形式表明，当 $Q = p_{z|x}$ 时，ELBO 关于 Q 达到最大值，这正是前面式 (11.8) 已经证明的结果。

11.4 再看高斯混合模型

有了一般的 EM 算法定义，再回到之前的例子：拟合高斯混合模型的参数 ϕ, μ, Σ。为简洁起见，我们只推导 ϕ 和 μ_j 的 M 步更新，将 Σ_j 的更新留作读者练习。

E 步很简单。根据上述算法推导，只需计算

$$w_j^{(i)} = Q_i(z^{(i)} = j) = P(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma).$$

这里，$Q_i(z^{(i)} = j)$ 表示在分布 Q_i 下，$z^{(i)}$ 取值为 j 的概率。

接下来，在 M 步中，需要对参数 ϕ, μ, Σ 最大化以下量：

$$
\begin{aligned}
&\sum_{i=1}^{n}\sum_{z^{(i)}} Q_i(z^{(i)})\log\frac{p(x^{(i)},z^{(i)};\phi,\mu,\Sigma)}{Q_i(z^{(i)})}\\
&=\sum_{i=1}^{n}\sum_{j=1}^{k} Q_i(z^{(i)}=j)\log\frac{p(x^{(i)}\mid z^{(i)}=j;\mu,\Sigma)p(z^{(i)}=j;\phi)}{Q_i(z^{(i)}=j)}\\
&=\sum_{i=1}^{n}\sum_{j=1}^{k} w_j^{(i)}\log\frac{\frac{1}{(2\pi)^{d/2}|\Sigma_j|^{1/2}}\exp\Big(-\frac{1}{2}(x^{(i)}-\mu_j)^T\Sigma_j^{-1}(x^{(i)}-\mu_j)\Big)\phi_j}{w_j^{(i)}}.
\end{aligned}
$$

先对 μ_l 最大化。对 μ_l 求导，得到

$$
\begin{aligned}
&\nabla_{\mu_l}\sum_{i=1}^{n}\sum_{j=1}^{k} w_j^{(i)}\log\frac{\frac{1}{(2\pi)^{d/2}|\Sigma_j|^{1/2}}\exp\Big(-\frac{1}{2}(x^{(i)}-\mu_j)^T\Sigma_j^{-1}(x^{(i)}-\mu_j)\Big)\phi_j}{w_j^{(i)}}\\
&=-\nabla_{\mu_l}\sum_{i=1}^{n}\sum_{j=1}^{k} w_j^{(i)}\frac{1}{2}(x^{(i)}-\mu_j)^T\Sigma_j^{-1}(x^{(i)}-\mu_j)\\
&=\frac{1}{2}\sum_{i=1}^{n} w_l^{(i)}\nabla_{\mu_l}\left(2\mu_l^T\Sigma_l^{-1}x^{(i)}-\mu_l^T\Sigma_l^{-1}\mu_l\right)\\
&=\sum_{i=1}^{n} w_l^{(i)}\left(\Sigma_l^{-1}x^{(i)}-\Sigma_l^{-1}\mu_l\right).
\end{aligned}
$$

令该式为零，解出 μ_l，得到更新规则

$$
\mu_l := \frac{\sum_{i=1}^{n} w_l^{(i)}x^{(i)}}{\sum_{i=1}^{n} w_l^{(i)}},
$$

这正是之前得到的公式。

再举一个例子，推导参数 ϕ_j 的 M 步更新。只保留依赖 ϕ_j 的项后，需要最大化

$$
\sum_{i=1}^{n}\sum_{j=1}^{k} w_j^{(i)}\log\phi_j.
$$

然而，还存在额外约束：ϕ_j 之和为 1，因为它们表示概率 $\phi_j = p(z^{(i)}=j;\phi)$。为了处理约束 $\sum_{j=1}^{k}\phi_j = 1$，构造拉格朗日函数

$$
\mathcal{L}(\phi)=\sum_{i=1}^{n}\sum_{j=1}^{k} w_j^{(i)}\log\phi_j+\beta\left(\sum_{j=1}^{k}\phi_j-1\right),
$$

其中，β 为拉格朗日乘子。[51] 求导可得

$$
\frac{\partial}{\partial\phi_j}\mathcal{L}(\phi)=\sum_{i=1}^{n}\frac{w_j^{(i)}}{\phi_j}+\beta.
$$

[51] 不必担心 $\phi_j \geq 0$ 的约束，因为马上就会看到，推导得到的解自动满足这一条件。

令其为零并求解，得到

$$\phi_j = \frac{\sum_{i=1}^{n} w_j^{(i)}}{-\beta}.$$

即 $\phi_j \propto \sum_{i=1}^{n} w_j^{(i)}$。利用约束 $\sum_j \phi_j = 1$，容易得到 $-\beta = \sum_{i=1}^{n} \sum_{j=1}^{k} w_j^{(i)} = \sum_{i=1}^{n} 1 = n$。这里用到了 $w_j^{(i)} = Q_i(z^{(i)} = j)$，以及概率之和为 1，即 $\sum_j w_j^{(i)} = 1$。因此，参数 ϕ_j 的 M 步更新为

$$\phi_j := \frac{1}{n} \sum_{i=1}^{n} w_j^{(i)}.$$

Σ_j 的 M 步更新推导也完全直接。

11.5 变分推断与变分自编码器

粗略地说，**变分自编码器**（Kingma 和 Welling，2013）通常指一类把 EM 算法推广到更复杂的神经网络参数化模型的算法。它通过额外使用下面将介绍的 “重参数化技巧”，扩展了变分推断技术。变分自编码器在许多数据集上或许不能达到最佳表现，但它包含了若干核心思想，说明如何在非线性模型中将 EM 算法推广到高维连续潜变量。理解它，很可能会使你具备理解相关近期论文所需的术语和背景知识。

作为贯穿本节的例子，考虑用神经网络按以下方式参数化 $p(x, z; \theta)$。令 θ 为神经网络 $g(z; \theta)$ 的权重集合，该网络将 $z \in \mathbb{R}^k$ 映射到 $\mathbb{R}^d$。设

$$z \sim \mathcal{N}(0, I_{k\times k}), \tag{11.18}$$

$$x \mid z \sim \mathcal{N}(g(z; \theta), \sigma^2 I_{d\times d}). \tag{11.19}$$

这里，$I_{k\times k}$ 为 $k \times k$ 单位矩阵；为简便起见，假设标量 σ 已知。

对于第 11.4 节的高斯混合模型，每个固定 θ 对应的最优选择 $Q(z) = p(z \mid x; \theta)$，即 z 的后验分布，可以通过解析方式计算。但在式 (11.19) 这样的复杂模型中，精确后验分布 $p(z \mid x; \theta)$ 的计算往往不可行。

回顾式 (11.10)，无论如何选择 Q，ELBO 始终是下界，因此我们也可以寻求真实后验分布的近似。通常必须用某种特定形式来近似真实后验。令 $\mathcal{Q}$ 表示所考虑的 Q 的分布族，我们希望在该族内找到最接近真实后验分布的 Q。为了严格描述，回顾式 (11.14) 中作为 Q, θ 的函数的 ELBO 定义：

$$\begin{aligned}\mathrm{ELBO}(Q, \theta) &= \sum_{i=1}^{n} \mathrm{ELBO}(x^{(i)}; Q_i, \theta) \\ &= \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{p(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})}.\end{aligned}$$

回顾，EM 可以看作对 $\mathrm{ELBO}(Q, \theta)$ 交替最大化。现在改为在 $Q \in \mathcal{Q}$ 的限制下优化 ELBO：

$$\max_{Q\in\mathcal{Q}} \max_{\theta} \mathrm{ELBO}(Q,\theta). \tag{11.20}$$

接下来的问题是，采用什么形式的 $\mathcal{Q}$，或者对 Q 作出什么结构性假设，能让我们高效最大化上述目标？当潜变量 z 是高维离散变量时，一种常见假设是**平均场假设**，即假设 $Q_i(z)$ 各坐标独立。换言之，Q_i 可分解为 $Q_i(z) = Q_i^1(z_1)\cdots Q_i^k(z_k)$。平均场假设在含离散潜变量的生成模型学习中有大量应用。关于这些模型及其在计算生物学、计算神经科学、社会科学等广泛应用中的影响，可参阅 Blei 等（2017）的综述。我们不深入讨论离散潜变量的情形，而是主要关注连续潜变量；这不仅需要平均场假设，还需要其他技术。

当 $z\in\mathbb{R}^k$ 为连续潜变量时，要成功优化式 (11.20)，需要作出若干选择。首先，由于分布 Q_i 定义在无限多个点上，需要为其提供简洁表示。一个自然选择是假设 Q_i 是具有某个均值和方差的高斯分布。我们还希望能更简洁地表示所有样本的 Q_i 均值。注意，$Q_i(z^{(i)})$ 应当近似 $p(z^{(i)}\mid x^{(i)};\theta)$。因此，让所有 Q_i 的均值都是 $x^{(i)}$ 的某个函数，是合理的。具体地，令 $q(\cdot;\phi)$ 和 $v(\cdot;\psi)$ 为两个从 d 维映射到 k 维的函数，分别由 ϕ,ψ 参数化。假设

$$Q_i = \mathcal{N}\big(q(x^{(i)};\phi), \mathrm{diag}(v(x^{(i)};\psi))^2\big). \tag{11.21}$$

这里，$\mathrm{diag}(w)$ 表示以向量 $w\in\mathbb{R}^k$ 的元素为对角元素的 $k\times k$ 矩阵。换言之，假设 Q_i 是各坐标独立的高斯分布，均值和标准差分别由 q 和 v 决定。在变分自编码器中，q 和 v 通常选为神经网络。[52] 在近期深度学习文献中，q,v 常被称为**编码器**，含义是把数据编码为潜在编码；$g(z;\theta)$ 则常被称为**解码器**。

需要指出，这种形式的 Q_i 在许多情况下远不能很好地近似真实后验。但为了使优化可行，必须进行某种近似。事实上，Q_i 的形式还需要满足其他要求，而式 (11.21) 恰好满足这些要求。

在优化 ELBO 之前，先验证对于固定的 θ 和式 (11.21) 形式的 Q，是否可以高效计算 ELBO 的值。将 ELBO 改写为 ϕ,ψ,θ 的函数：

$$\begin{aligned}\mathrm{ELBO}(\phi,\psi,\theta) &= \sum_{i=1}^{n} \mathbb{E}_{z^{(i)}\sim Q_i}\left[\log \frac{p(x^{(i)},z^{(i)};\theta)}{Q_i(z^{(i)})}\right],\\ Q_i &= \mathcal{N}\big(q(x^{(i)};\phi), \mathrm{diag}(v(x^{(i)};\psi))^2\big).\end{aligned} \tag{11.22}$$

注意，为计算期望内部的 $Q_i(z^{(i)})$，需要能够计算 Q_i 的密度。为估计期望 $\mathbb{E}_{z^{(i)}\sim Q_i}$，需要能够从 Q_i 采样，以利用样本构造经验估计量。对于高斯分布 $Q_i = \mathcal{N}(q(x^{(i)};\phi), \mathrm{diag}(v(x^{(i)};\psi))^2)$，这两件事恰好都能高效完成。

现在开始优化 ELBO。事实上，可以对 ϕ,ψ,θ 运行梯度上升，而不必交替最大化。没有必要付出大得多的成本，计算每个变量上的精确最大值。（对于第 11.4 节的高斯混合模型，最大值能解析计算，且成本相对低，因此使用了交替最大化。）在数学上，令 η 为学习率，梯度上升步骤为

$$\begin{aligned}\theta &:= \theta + \eta\nabla_\theta \mathrm{ELBO}(\phi,\psi,\theta),\\ \phi &:= \phi + \eta\nabla_\phi \mathrm{ELBO}(\phi,\psi,\theta),\\ \psi &:= \psi + \eta\nabla_\psi \mathrm{ELBO}(\phi,\psi,\theta).\end{aligned}$$

[52] q 和 v 也可以共享参数。本讲义略去这一层面的细节。

计算关于 θ 的梯度很简单，因为

$$\begin{aligned}\nabla_\theta \mathrm{ELBO}(\phi,\psi,\theta) &= \nabla_\theta \sum_{i=1}^{n} \mathbb{E}_{z^{(i)}\sim Q_i}\left[\log \frac{p(x^{(i)},z^{(i)};\theta)}{Q_i(z^{(i)})}\right] \\ &= \nabla_\theta \sum_{i=1}^{n} \mathbb{E}_{z^{(i)}\sim Q_i}[\log p(x^{(i)},z^{(i)};\theta)] \\ &= \sum_{i=1}^{n} \mathbb{E}_{z^{(i)}\sim Q_i}[\nabla_\theta \log p(x^{(i)},z^{(i)};\theta)].\end{aligned} \tag{11.23}$$

但计算关于 ϕ,ψ 的梯度很棘手，因为采样分布 Q_i 依赖它们。抽象地说，这个问题可简化为：计算 $\mathbb{E}_{z\sim Q_\phi}[f(\phi)]$ 关于 ϕ 的梯度。一般情况下，$\nabla \mathbb{E}_{z\sim Q_\phi}[f(\phi)] \neq \mathbb{E}_{z\sim Q_\phi}[\nabla f(\phi)]$，因为还必须考虑 Q_ϕ 对 ϕ 的依赖。

解决问题的关键是所谓的**重参数化技巧**：将 $z^{(i)} \sim Q_i = \mathcal{N}(q(x^{(i)};\phi), \mathrm{diag}(v(x^{(i)};\psi))^2)$ 等价地写为

$$z^{(i)} = q(x^{(i)};\phi) + v(x^{(i)};\psi)\odot \xi^{(i)}, \qquad \xi^{(i)} \sim \mathcal{N}(0, I_{k\times k}). \tag{11.24}$$

这里，$x \odot y$ 表示两个同维向量的逐元素乘积。我们利用了这样一个事实：$x \sim \mathcal{N}(\mu,\sigma^2)$ 等价于 $x = \mu + \xi\sigma$，其中 $\xi \sim \mathcal{N}(0,1)$。本质上，只是把这个事实同时应用于随机变量 $z^{(i)} \sim Q_i$ 的每个维度。

采用这个重参数化后，有

$$\begin{aligned}&\mathbb{E}_{z^{(i)}\sim Q_i}\left[\log \frac{p(x^{(i)},z^{(i)};\theta)}{Q_i(z^{(i)})}\right] \\ &= \mathbb{E}_{\xi^{(i)}\sim \mathcal{N}(0,I_{k\times k})}\left[\log \frac{p(x^{(i)}, q(x^{(i)};\phi)+v(x^{(i)};\psi)\odot\xi^{(i)};\theta)}{Q_i(q(x^{(i)};\phi)+v(x^{(i)};\psi)\odot\xi^{(i)})}\right].\end{aligned} \tag{11.25}$$

于是，

$$\begin{aligned}&\nabla_\phi \mathbb{E}_{z^{(i)}\sim Q_i}\left[\log \frac{p(x^{(i)},z^{(i)};\theta)}{Q_i(z^{(i)})}\right] \\ &= \nabla_\phi \mathbb{E}_{\xi^{(i)}\sim \mathcal{N}(0,I_{k\times k})}\left[\log \frac{p(x^{(i)}, q(x^{(i)};\phi)+v(x^{(i)};\psi)\odot\xi^{(i)};\theta)}{Q_i(q(x^{(i)};\phi)+v(x^{(i)};\psi)\odot\xi^{(i)})}\right] \\ &= \mathbb{E}_{\xi^{(i)}\sim \mathcal{N}(0,I_{k\times k})}\left[\nabla_\phi \log \frac{p(x^{(i)}, q(x^{(i)};\phi)+v(x^{(i)};\psi)\odot\xi^{(i)};\theta)}{Q_i(q(x^{(i)};\phi)+v(x^{(i)};\psi)\odot\xi^{(i)})}\right].\end{aligned}$$

现在，可以抽取多个 $\xi^{(i)}$ 样本，估计上式右侧的期望。[53] 关于 ψ 的梯度也可类似估计。有了这些结果，就能实现梯度上升算法，对 ϕ,ψ,θ 优化 ELBO。

在密度函数可解析计算的高维分布中，已知能够重参数化的分布并不多。关于可以替代高斯分布的其他几种选择，请参阅 Kingma 和 Welling（2013）。

[53] 实践中，为了最大程度提高计算效率，有时仅使用一个样本来估计。

第 12 章　主成分分析

原文印刷页码：167–172。

本章介绍**主成分分析**（Principal Components Analysis，PCA）。这种方法试图找出数据近似所在的子空间。PCA 的计算效率很高，只需要计算特征向量，在 Matlab 中用 `eig` 函数就很容易实现。

假设给定一个数据集 $\{x^{(i)};\, i = 1, \ldots, n\}$，记录了 n 种汽车的属性，例如最高速度、转弯半径等。对每个 i，都有 $x^{(i)} \in \mathbb{R}^d$，并且 $d \ll n$。但我们不知道，其中两个属性 x_i 和 x_j 实际上分别是以英里/小时和千米/小时为单位表示的同一辆汽车的最高速度。因此，这两个属性几乎线性相关，两者之间只存在取整到最接近的英里/小时或千米/小时时产生的微小差异。这样，数据实际上近似落在一个 $n-1$ 维子空间中。我们怎样自动检测并消除这种冗余呢？

> 译注：这里原文写作 “$n-1$ 维”。按本段 $x^{(i)} \in \mathbb{R}^d$ 的定义，应为 “$d-1$ 维”；正文保留了原文表述。

再看一个没那么人为构造的例子。假设有一份遥控直升机驾驶者的调查数据，$x_1^{(i)}$ 衡量驾驶者 i 的驾驶技能，$x_2^{(i)}$ 衡量其对飞行的喜爱程度。遥控直升机很难驾驶，只有最投入、真正喜爱飞行的学习者才能成为优秀驾驶者。因此，属性 x_1 和 x_2 高度相关。我们可以设想，数据实际上沿着某条对角轴，即 u_1 方向分布；这个方向反映一个人内在的 “飞行禀赋”，只有少量噪声偏离这条轴，如图所示。我们怎样自动计算这个 u_1 方向呢？

下面将推导 PCA 算法。但在运行 PCA 本身之前，通常先对数据进行预处理，将每个特征规范化为均值 0、方差 1。做法是减去均值，再除以经验标准差：

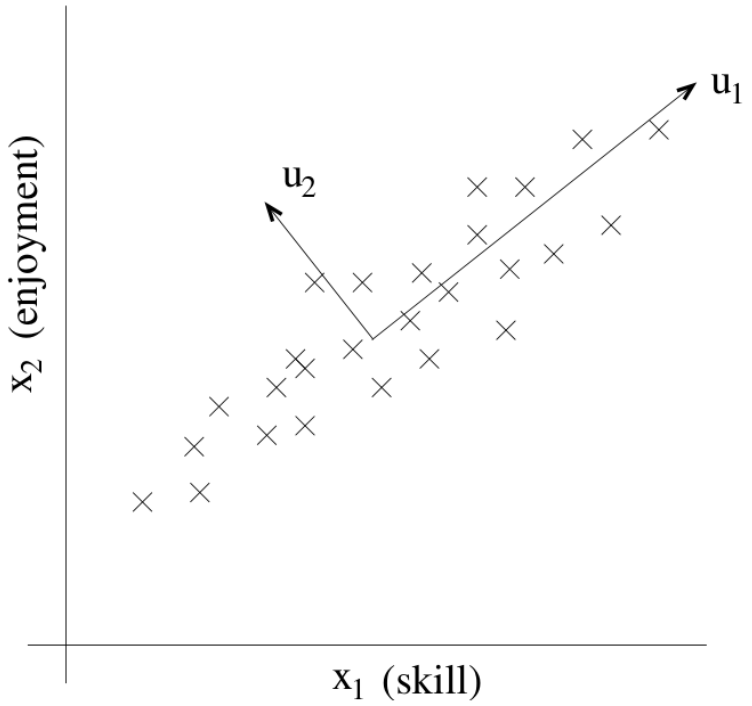
$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \mu_j}{\sigma_j},$$

其中

$$\mu_j = \frac{1}{n}\sum_{i=1}^{n} x_j^{(i)}, \qquad \sigma_j^2 = \frac{1}{n}\sum_{i=1}^{n} (x_j^{(i)} - \mu_j)^2$$

分别是特征 j 的均值和方差。

减去 μ_j 会使均值归零；如果已知数据本来就是零均值，例如语音或其他声学信号的时间序列，就可以省略这一步。除以标准差 σ_j 会把各坐标缩放到单位方差，使不同属性处于相同的 “尺度”。例如，x_1 是以英里/小时计的汽车最高速度，数值通常为几十到一百多，而 x_2 是座位数，通常在 2–4 左右。重新规范化之后，不同属性就更有可比性。如果事先知道所有属性本来就在相同尺度上，则可以省略缩放。例如，每个数据点是一张灰度图像，而 $x_j^{(i)} \in \{0, 1, \ldots, 255\}$ 表示第 i 张图像中像素 j 的亮度值。



驾驶技能与飞行喜爱程度的相关性。横轴 x_1 为技能，纵轴 x_2 为喜爱程度；u_1 为数据变化的主要方向，u_2 为与其正交的方向。

完成规范化后，怎样计算数据的“主要变化轴” u，也就是数据近似分布的方向呢？一种方式是把问题表述为寻找单位向量 u，使数据投影到 u 方向后，投影数据的方差最大。直观地说，原始数据包含一定的方差或信息。我们希望选出一个方向 u，在把数据近似视为位于 u 对应的方向或子空间时，仍尽可能保留这些方差。

考虑下面这个已经完成规范化的数据集：

假设选择下图中的方向作为 u。圆点表示原始数据在这条直线上的投影。

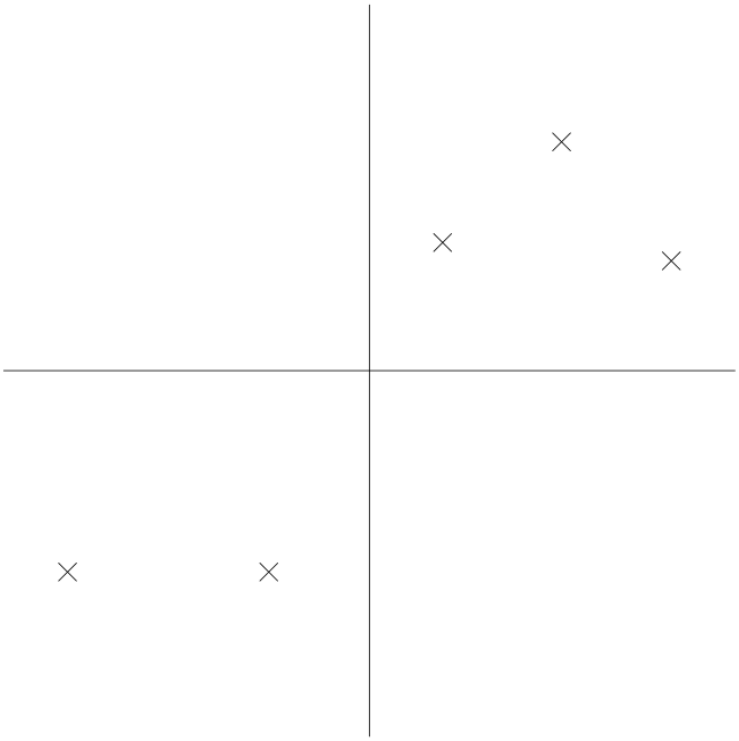
可以看到，投影后的数据仍具有较大的方差，各点通常离零点较远。相反，假设选择下面这个方向：

这时，投影数据的方差明显更小，也更接近原点。

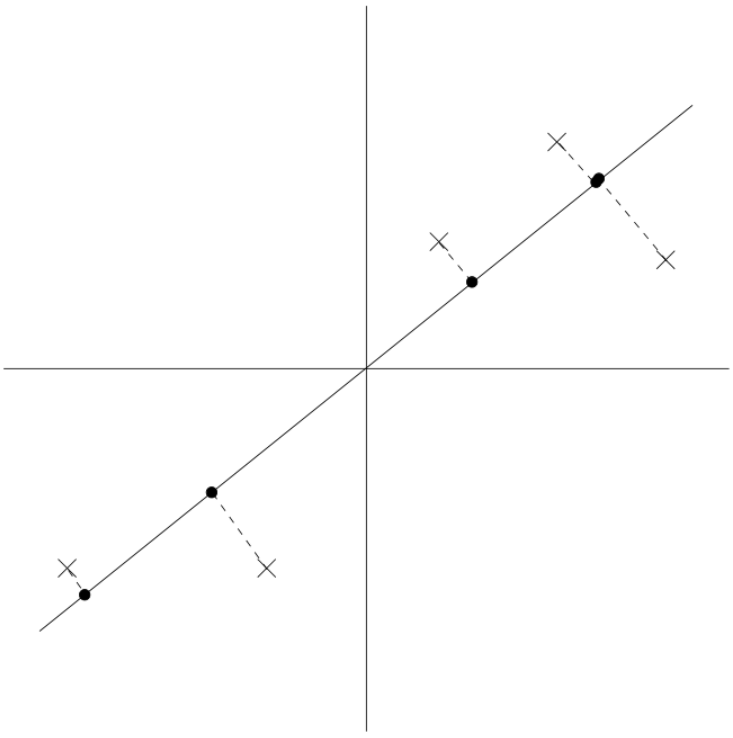
我们希望自动选出对应于前一幅投影图的方向 u。为了形式化这一目标，注意给定单位向量 u 与点 x，x 在 u 上的投影长度为 $x^T u$。也就是说，如果 $x^{(i)}$ 是数据集中的一个点，即图中的一个叉号，那么其在 u 上的投影，即对应的圆点，距原点为 $x^T u$。为了最大化投影的方差，希望选择单位长度的 u，使下面的量最大：

$$\begin{aligned}\frac{1}{n}\sum_{i=1}^{n}\left(x^{(i)T}u\right)^2 &= \frac{1}{n}\sum_{i=1}^{n}u^T x^{(i)}x^{(i)T}u \\ &= u^T\left(\frac{1}{n}\sum_{i=1}^{n}x^{(i)}x^{(i)T}\right)u.\end{aligned}$$

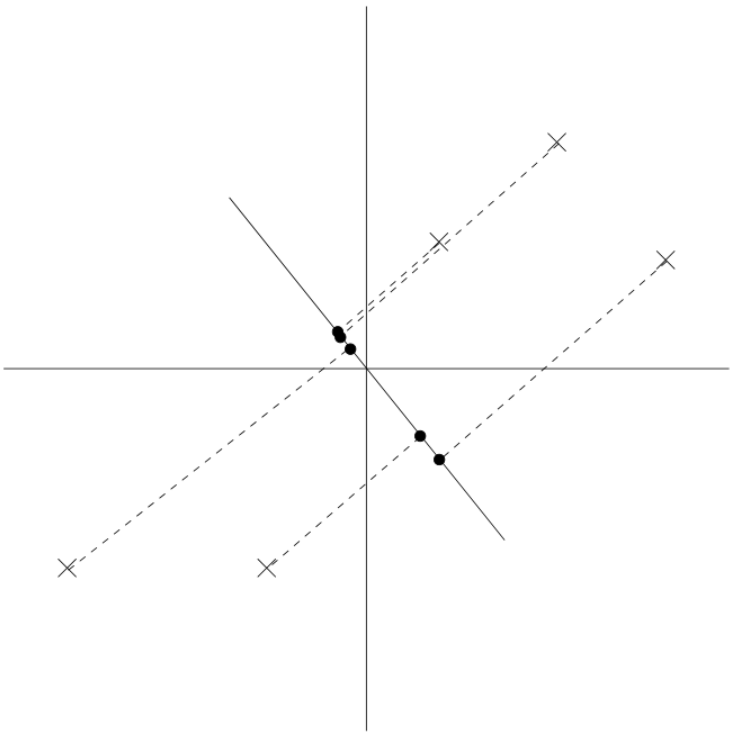
在约束 $\|u\|_2 = 1$ 下最大化这个量，得到的正是



经过规范化的二维数据集。



沿主要变化方向投影：投影后的数据仍具有较大的方差。



沿另一个方向投影：投影点更接近原点，方差较小。

$$\Sigma = \frac{1}{n}\sum_{i=1}^{n} x^{(i)}x^{(i)T}$$

的主特征向量。在数据均值为零的前提下，Σ 就是数据的经验协方差矩阵。[54]

总之，如果希望找出一个一维子空间来近似数据，就应选择 Σ 的主特征向量作为 u。更一般地，如果希望把数据投影到 k 维子空间中，其中 $k < d$，应选择 Σ 的前 k 个特征向量 $u_1, \ldots, u_k$。这些 u_i 构成数据的一组新的正交基。[55]

为了用这组基表示 $x^{(i)}$，只需要计算相应向量

$$y^{(i)} = \begin{bmatrix} u_1^T x^{(i)} \\ u_2^T x^{(i)} \\ \vdots \\ u_k^T x^{(i)} \end{bmatrix} \in \mathbb{R}^k.$$

原来 $x^{(i)} \in \mathbb{R}^d$，而现在 $y^{(i)}$ 给出了 $x^{(i)}$ 的一个较低的 k 维近似或表示。因此，PCA 也被称为一种**降维算法**。向量 $u_1, \ldots, u_k$ 称为数据的前 k 个**主成分**。

说明。上面仅对 $k = 1$ 的情形进行了正式证明，但利用特征向量的标准性质，很容易证明：在所有可能的正交基 $u_1, \ldots, u_k$ 中，我们选择的这组基使 $\sum_i \|y^{(i)}\|_2^2$ 最大。因此，这组基尽可能保留了原始数据的变异性。

[54]如果以前没有见过这个结论，可以尝试用拉格朗日乘子法，在约束 $u^T u = 1$ 下最大化 $u^T \Sigma u$。可以证明，对某个 λ 有 $\Sigma u = \lambda u$，因此 u 是 Σ 的一个特征向量，对应特征值为 λ。

[55]因为 Σ 是对称矩阵，u_i 两两正交，或者总可以选成两两正交。

另一种推导 PCA 的方法是：选择一组基，使数据投影到这些基所张成的 k 维子空间时产生的近似误差最小。更多内容见作业。

PCA 有许多应用，下面举几个例子作为本章的结尾。首先，一个直接应用是压缩，即用低维的 $y^{(i)}$ 表示原来的 $x^{(i)}$。如果把高维数据降到 $k = 2$ 或 3 维，就还能通过绘制 $y^{(i)}$ 来可视化数据。例如，把汽车数据降到二维后，可以将它画出来，让图中的每一个点对应一种车型，从而观察哪些汽车彼此相似，哪些汽车可能聚在一起。

另一种常见应用是：在以 $x^{(i)}$ 为输入运行监督学习算法之前，先对数据集进行降维预处理。除了降低计算成本，降低数据维度还能降低所考虑的假设类的复杂度，帮助避免过拟合。例如，低维输入空间上的线性分类器具有较小的 VC 维。

最后，就像前面的遥控直升机驾驶者例子，我们还可以把 PCA 看作一种去噪算法。在那个例子中，它根据带噪声的驾驶技能和喜爱程度测量值，估计内在的 “飞行禀赋”。课堂上还展示了把这一思想用于人脸图像的例子，得到**特征脸**（eigenfaces）方法。这里，每个点 $x^{(i)} \in \mathbb{R}^{100\times100}$ 是一个 10000 维向量，每个坐标对应一张 100×100 人脸图像中的一个像素亮度。利用 PCA，可以用维度低得多的 $y^{(i)}$ 表示每张图像 $x^{(i)}$。这样做时，我们希望找到的主成分保留人脸之间有意义的、系统性的变化，也就是反映一个人真实相貌的差异，而不保留轻微照明变化、略有不同的成像条件等所引入的 “噪声”。随后，在低维空间中计算 $\|y^{(i)} - y^{(j)}\|_2$，衡量人脸 i 与 j 之间的距离。这样得到的人脸匹配和检索算法，效果出乎意料地好。

第 13 章　独立成分分析

原文印刷页码：173–178。

接下来讨论**独立成分分析**（Independent Components Analysis，ICA）。与 PCA 类似，ICA 会寻找一组新的基来表示数据，但它的目标很不相同。

先考虑一个例子："鸡尾酒会问题"。在聚会上，d 位说话者同时讲话，房间里任意一支麦克风记录到的都只是这 d 个人声音叠加后的结果。假设房间内放置了 d 支不同的麦克风，由于每支麦克风到每位说话者的距离不同，它记录下来的声音组合也不同。能否利用这些麦克风的录音，分离出原来 d 个人各自的语音信号？

为了形式化这一问题，设某个数据向量 $s \in \mathbb{R}^d$ 由 d 个相互独立的源生成，而我们观察到的是

$$x = As,$$

其中 A 是一个未知方阵，称为**混合矩阵**。重复观测后，得到数据集 $\{x^{(i)};\ i = 1, \dots, n\}$。我们的目标是恢复产生这些数据的源 $s^{(i)}$，即 $x^{(i)} = As^{(i)}$ 中的 $s^{(i)}$。

在鸡尾酒会问题中，$s^{(i)}$ 是一个 d 维向量，$s_j^{(i)}$ 表示说话者 j 在时刻 i 发出的声音。同样，$x^{(i)}$ 是一个 d 维向量，$x_j^{(i)}$ 表示麦克风 j 在时刻 i 记录下来的声学读数。

令 $W = A^{-1}$ 为**解混矩阵**。目标是找到 W，这样给定麦克风录音 $x^{(i)}$ 后，就可以通过计算 $s^{(i)} = Wx^{(i)}$ 来恢复各个源。为了简化记号，用 w_i^T 表示 W 的第 i 行，即

$$W = \begin{bmatrix} — \, w_1^T \, — \\ \vdots \\ — \, w_d^T \, — \end{bmatrix}.$$

于是 $w_i \in \mathbb{R}^d$，第 j 个源可以由 $s_j^{(i)} = w_j^T x^{(i)}$ 恢复。

13.1 ICA 的不确定性

我们能在多大程度上恢复 $W = A^{-1}$？如果对源和混合矩阵没有任何先验知识，很容易看出，仅根据 $x^{(i)}$，A 中存在一些无法消除的固有不确定性。

具体来说，令 P 为任意一个 $d \times d$ 的**置换矩阵**，即 P 的每一行和每一列都恰好有一个 "1"。下面是几个置换矩阵的例子：

$$P=\begin{bmatrix}0&1&0\\1&0&0\\0&0&1\end{bmatrix};\qquad P=\begin{bmatrix}0&1\\1&0\end{bmatrix};\qquad P=\begin{bmatrix}1&0\\0&1\end{bmatrix}.$$

如果 z 是一个向量，那么 Pz 就是把 z 的坐标重新排列后得到的向量。仅根据 $x^{(i)}$，无法区分 W 和 PW。也就是说，原始各源的排列顺序是不确定的，这并不奇怪。幸运的是，对大多数应用来说，这一点无关紧要。

此外，也无法恢复 w_i 的正确尺度。例如，把 A 换成 $2A$，同时把每个 $s^{(i)}$ 换成 $0.5s^{(i)}$，那么观测到的 $x^{(i)}=2A\cdot(0.5s^{(i)})$ 仍然相同。更一般地，如果将 A 的某一列乘以 α，再把相应的源乘以 $1/\alpha$，也无法仅凭 $x^{(i)}$ 判断是否发生过这种变换。因此，我们不能恢复各源的“正确”尺度。但对我们关心的应用，包括鸡尾酒会问题，这种不确定性同样无关紧要。具体来说，把某人的语音信号 $s_j^{(i)}$ 乘以正数 α，只会改变其声音的音量。符号变化也没有影响，$s_j^{(i)}$ 和 $-s_j^{(i)}$ 经扬声器播放时听起来完全相同。因此，如果算法找到的 w_i 被乘以任意非零实数，相应恢复出的源 $s_i=w_i^Tx$ 也会按同一因子缩放，但这通常没有关系。这些讨论也适用于课堂上讲过的大脑或脑磁图（MEG）数据中的 ICA。

这些就是 ICA 中所有的不确定性吗？事实证明，只要源 s_i 是非高斯的，答案就是肯定的。为了理解高斯数据带来的困难，考虑原文所述的一个 $n=2$、$s\sim\mathcal{N}(0,I)$ 的例子，其中 I 是 2×2 单位矩阵。标准正态分布 $\mathcal{N}(0,I)$ 的密度等高线是以原点为中心的圆，密度具有旋转对称性。

译注：这里的二维情形对应源维数 $d=2$；原文写作 $n=2$。

现在假设观察到 $x=As$，其中 A 是混合矩阵。那么 x 服从高斯分布 $\mathcal{N}(0,AA^T)$，因为

$$\mathbb{E}_{s\sim\mathcal{N}(0,I)}[x]=\mathbb{E}[As]=A\mathbb{E}[s]=0,$$

$$\begin{aligned}\mathrm{Cov}[x]&=\mathbb{E}_{s\sim\mathcal{N}(0,I)}[xx^T]\\&=\mathbb{E}[Ass^TA^T]=A\mathbb{E}[ss^T]A^T\\&=A\,\mathrm{Cov}[s]A^T=AA^T.\end{aligned}$$

令 R 为任意正交矩阵，不太正式地说就是旋转或反射矩阵，满足 $RR^T=R^TR=I$，并令 $A'=AR$。如果数据是用 A' 而不是 A 混合的，那么观察到的将是 $x'=A's$。x' 也服从高斯分布 $\mathcal{N}(0,AA^T)$，因为

$$\begin{aligned}\mathbb{E}_{s\sim\mathcal{N}(0,I)}[x'(x')^T]&=\mathbb{E}[A'ss^T(A')^T]\\&=\mathbb{E}[ARss^T(AR)^T]=ARR^TA^T=AA^T.\end{aligned}$$

因此，不论混合矩阵是 A 还是 A'，我们观察到的数据都来自 $\mathcal{N}(0,AA^T)$。无法判断源究竟是用 A 还是 A' 混合的。混合矩阵中存在一个无法由数据确定的任意旋转成分，因此不能恢复原始各源。

上述论证建立在多元标准正态分布具有旋转对称性这一事实上。尽管这说明 ICA 对高斯数据无能为力，但事实证明，只要数据是非高斯的，在数据充分时就有可能恢复 d 个独立源。

13.2 密度与线性变换

在正式推导 ICA 算法之前，先简要讨论线性变换对概率密度的影响。

假设随机变量 s 按密度 $p_s(s)$ 生成。为简单起见，先假设 $s \in \mathbb{R}$ 是实数。定义随机变量 $x = As$，其中 $x \in \mathbb{R}$、$A \in \mathbb{R}$。令 p_x 表示 x 的密度，那么 p_x 是什么?

令 $W = A^{-1}$。计算某个特定 x 值的 "概率" 时，很容易先求出 $s = Wx$，再计算该点的 p_s，然后得出 "$p_x(x) = p_s(Wx)$"。但这是错误的。例如，设 $s \sim \text{Uniform}[0,1]$，那么 $p_s(s) = \mathbf{1}\{0 \le s \le 1\}$。令 $A = 2$，于是 $x = 2s$。显然，x 在区间 $[0,2]$ 上均匀分布，所以它的密度为 $p_x(x) = 0.5\mathbf{1}\{0 \le x \le 2\}$。这不等于 $p_s(Wx)$，其中 $W = 0.5 = A^{-1}$。正确公式是 $p_x(x) = p_s(Wx)|W|$。

更一般地，若 s 是具有密度 p_s 的随机向量，$x = As$，且 A 是可逆方阵，那么 x 的密度为

$$p_x(x) = p_s(Wx)\,|W|,$$

其中 $W = A^{-1}$，$|W|$ 在这里表示行列式的绝对值。

说明。如果见过 A 将 $[0,1]^d$ 映射到一个体积为 $|A|$ 的集合这一结论，就可以用另一种方式记住上面的 p_x 公式，同时把前面的一维例子推广到高维。具体地，给定 $A \in \mathbb{R}^{d\times d}$，仍令 $W = A^{-1}$。令 $C_1 = [0,1]^d$ 为 d 维超立方体，并定义

$$C_2 = \{As : s \in C_1\} \subseteq \mathbb{R}^d$$

为 C_1 在 A 所定义映射下的像。线性代数的一个标准结果是，C_2 的体积等于 $|A|$；事实上，这也是定义行列式的一种方式。现在假设 s 在 $[0,1]^d$ 上均匀分布，密度为 $p_s(s) = \mathbf{1}\{s \in C_1\}$。显然，$x$ 会在 C_2 上均匀分布。因此，$p_x(x) = \mathbf{1}\{x \in C_2\}/\,\text{vol}(C_2)$，因为它在 C_2 上的积分必须为 1。又因为逆矩阵的行列式等于原矩阵行列式的倒数，

$$\frac{1}{\text{vol}(C_2)} = \frac{1}{|A|} = |A^{-1}| = |W|.$$

从而

$$\begin{aligned} p_x(x) &= \mathbf{1}\{x \in C_2\}|W| \\ &= \mathbf{1}\{Wx \in C_1\}|W| = p_s(Wx)|W|. \end{aligned}$$

13.3 ICA 算法

现在可以推导 ICA 算法了。这里介绍 Bell 和 Sejnowski 提出的算法，并将它解释为一种最大似然估计方法。这不同于他们最初基于一个称为信息最大化原理（infomax principle）的复杂思想所给出的解释；有了现代对 ICA 的理解，那种解释已不再必要。

假设每个源 s_j 的分布由密度 p_s 给出，各源的联合分布为

$$p(s) = \prod_{j=1}^{d} p_s(s_j).$$

把联合分布建模为边缘分布的乘积，就表达了各源相互独立的假设。根据上一节的公式，$x = As = W^{-1}s$ 的密度为

$$p(x) = \left(\prod_{j=1}^{d} p_s(w_j^T x) \right) |W|.$$

剩下的只需指定单个源的密度 p_s。

回顾一下，实值随机变量 z 的累积分布函数（cdf）F 定义为

$$F(z_0) = P(z \le z_0) = \int_{-\infty}^{z_0} p_z(z)\, dz,$$

其密度是累积分布函数的导数：$p_z(z) = F'(z)$。

因此，要指定 s_i 的密度，只需要指定它的某个累积分布函数。累积分布函数必须是从 0 单调增加到 1 的函数。根据前面的讨论，不能选择高斯分布的累积分布函数，因为 ICA 对高斯数据不起作用。这里选择一个合理的 "默认" 累积分布函数：从 0 平缓增加到 1 的 sigmoid 函数

$$g(s) = \frac{1}{1 + e^{-s}}.$$

于是 $p_s(s) = g'(s)$。[56]

方阵 W 是模型的参数。给定训练集 $\{x^{(i)};\ i = 1, \ldots, n\}$，对数似然为

$$\ell(W) = \sum_{i=1}^{n} \left(\sum_{j=1}^{d} \log g'(w_j^T x^{(i)}) + \log |W| \right).$$

希望关于 W 最大化这个量。求导，并使用前面讲义中的结论 $\nabla_W |W| = |W|(W^{-1})^T$，很容易得到随机梯度上升学习规则。对一个训练样本 $x^{(i)}$，更新规则为

$$W := W + \alpha \left(\begin{bmatrix} 1 - 2g(w_1^T x^{(i)}) \\ 1 - 2g(w_2^T x^{(i)}) \\ \vdots \\ 1 - 2g(w_d^T x^{(i)}) \end{bmatrix} x^{(i)T} + (W^T)^{-1} \right),$$

其中 α 是学习率。

算法收敛后，计算 $s^{(i)} = W x^{(i)}$，即可恢复原始各源。

说明。写出数据似然时，我们隐含地假定不同 i 对应的 $x^{(i)}$ 相互独立。注意，这与同一个 $x^{(i)}$ 的不同坐标是否独立，是两个不同的问题。由此，训练集的似然写成 $\prod_i p(x^{(i)}; W)$。对语音数据及其他时间序列来说，这一假设显然不成立，因为不同时间的 $x^{(i)}$ 存在依赖关系。但可以证明，只要数据足够多，训练样本之间的相关性不会损害算法的表现。不过，在连续训练样本存在相关性的问题中，实现随机梯度上升时，按随机排列的顺序访问训练样本有时能加快收敛。也就是说，在训练集的随机打乱副本上运行随机梯度上升。

[56] 如果事先知道各源的密度具有某种特定形式，最好直接在这里使用该密度。缺乏这种知识时，sigmoid 函数可以视为一种合理的默认选择，在许多问题上表现良好。此外，这里的推导假设 $x^{(i)}$ 已经经过零均值预处理，或者本来就可以合理地认为它是零均值的，例如声学信号。这一条件是必要的：$p_s(s) = g'(s)$ 意味着 $\mathbb{E}[s] = 0$，因为 logistic 函数的导数是对称函数，给出一个零均值随机变量的密度；因此 $\mathbb{E}[x] = \mathbb{E}[As] = 0$。

第五部分　生成模型与基础模型

第 14 章　扩散模型

原文印刷页码：179–190。

生成建模要求我们对数据本身的分布建模。监督学习的目标是根据输入 x 预测目标 y，而这里的目标是生成新的、合理的样本 x。我们将主要以图像为贯穿本章的例子。

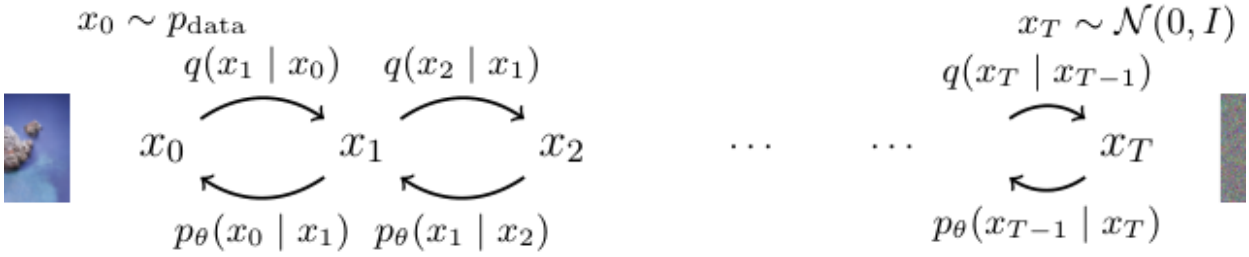
扩散模型首先定义一个简单的过程，逐步向数据添加噪声，然后学习一步一步地逆转这一过程。这一基本思想可以追溯到基于非平衡热力学的模型 [Sohl-Dickstein et al., 2015]；它的现代形式形成于去噪扩散概率模型（DDPM）[Ho et al., 2020]，之后又与基于得分的建模建立了联系 [Song et al., 2021]。除图像外，扩散模型在视频生成以及机器人视觉—语言—动作（VLA）模型中也占有重要地位，将扩散式方法用于语言的早期研究也已取得有希望的结果。

14.1　扩散过程

设 p_{data} 为数据（例如图像或归一化图像）的分布，我们最终希望能从中生成样本。

前向扩散过程定义了一条固定的马尔可夫链，逐步破坏来自 p_{data} 的干净数据。这条马尔可夫链由随机变量序列 $x_0, x_1, \dots, x_T$ 组成，其中 $x_0 \sim p_{\text{data}}$。变量 $x_1, x_2, \dots, x_T$ 是 x_0 的噪声逐渐增多的版本。用 q 表示该马尔可夫链的联合分布。它的单步转移密度为

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I). \tag{14.1}$$



前向扩散与反向去噪

图 14.1：前向扩散过程逐步破坏干净样本，而学习到的反向过程则逐步去噪。

这里 $\beta_1, \dots, \beta_T \in (0,1)$ 是标量，表示每一步的噪声水平。[57]

[57] 遵照扩散模型文献中的惯用记号，我们将方差写为 β_t，而不是 β_t^2。

关于概率记号的说明。与本讲义其他部分一样，$\mathcal{N}(\mu,\Sigma)$ 表示均值为 μ、协方差为 Σ 的高斯分布。这里用 $\mathcal{N}(x;\mu,\Sigma)$ 表示相应高斯密度在 x 处的值：

$$\mathcal{N}(x;\mu,\Sigma)=\frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}}\exp\left[-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right].$$

机器学习中经常采用简化且具有多重含义的记号。例如，x_t 既可以表示随机变量本身，也可以表示该随机变量取到的某个具体数值。因此，记号 $x_t \mid x_{t-1}$ 表示一个条件分布，此时将 x_t, x_{t-1} 理解为随机变量；而 $q(x_t \mid x_{t-1})$ 表示条件密度在写作 x_t 和 x_{t-1} 的具体数值处的值。我们也采用这种标准的多义记号。因此，式（14.1）表示：在 q 下，给定 x_{t-1} 后的条件随机变量 x_t 服从均值为 $\sqrt{1-\beta_t}x_{t-1}$、协方差为 $\beta_t I$ 的高斯分布，尽管该式本身写成了密度函数之间的等式。[58] 也可以写成下面的形式，更明确地强调我们在定义转移 q 下 $x_t \mid x_{t-1}$ 的分布，记为 $\mathrm{Law}_q(x_t \mid x_{t-1})$：

$$\mathrm{Law}_q(x_t \mid x_{t-1})=\mathcal{N}(\sqrt{1-\beta_t}\,x_{t-1},\beta_t I). \tag{14.2}$$

我们将学习一个沿反方向运行此加噪过程的模型。在此之前，先了解前向过程的一些性质。为方便起见，定义 $\alpha_t=1-\beta_t$。于是，前向过程中的样本可写为

$$x_t=\sqrt{\alpha_t}x_{t-1}+\sqrt{\beta_t}\epsilon_t,\qquad \epsilon_1,\dots,\epsilon_T \text{ 相互独立，且}\epsilon_t\sim\mathcal{N}(0,I). \tag{14.3}$$

每一步先将前一个带噪变量 x_{t-1} 乘以因子 $\sqrt{\alpha_t}$，使其略微收缩，再加入新的高斯噪声。实践中通常将 β_t 取为较小的值，例如 10^{-4} 至 10^{-2} 的数量级，因此 α_t 接近 1，每一步只对数据施加很小的扰动。但经过很多步后，原有结构会逐渐被抹去。

我们还可以考察协方差的演化。由于加入的噪声与 x_{t-1} 独立，

$$\mathrm{Cov}(x_t)=\alpha_t\,\mathrm{Cov}(x_{t-1})+\beta_t I. \tag{14.4}$$

因为 $\alpha_t+\beta_t=1$，$\mathrm{Cov}(x_t)$ 是 $\mathrm{Cov}(x_{t-1})$ 与 I 之间的线性插值。因此，假定 x_0 的每个维度均已归一化，每一步都会让分布稍微靠近球形高斯分布，同时保持其尺度。

由于每一步只加入高斯噪声，累积噪声也服从高斯分布；也就是说，对任意 t，给定 x_0 后 x_t 的条件分布仍是高斯分布。具体地，令 $\bar{\alpha}_t=\prod_{s=1}^{t}\alpha_s$，则

$$x_t=\sqrt{\bar{\alpha}_t}x_0+\sqrt{1-\bar{\alpha}_t}\,\hat{\epsilon}_t,\qquad \hat{\epsilon}_t\sim\mathcal{N}(0,I). \tag{14.5}$$

式（14.5）的证明。展开递推式可得

$$x_t=\sqrt{\alpha_t}x_{t-1}+\sqrt{1-\alpha_t}\epsilon_t, \tag{14.6}$$

$$=\sqrt{\alpha_t\alpha_{t-1}}x_{t-2}+\sqrt{\alpha_t(1-\alpha_{t-1})}\epsilon_{t-1}+\sqrt{1-\alpha_t}\epsilon_t, \tag{14.7}$$

[58] 机器学习中还经常出现含义更加重叠的记号 $q(x_t \mid x_{t-1})=\mathcal{N}(\sqrt{1-\beta_t}x_{t-1},\beta_t I)$，将左侧的密度等同于右侧的分布。本讲义避免这种用法。

$$
\begin{aligned}
&= \sqrt{\alpha_t\alpha_{t-1}\alpha_{t-2}}x_{t-3} + \sqrt{\alpha_t\alpha_{t-1}(1-\alpha_{t-2})}\epsilon_{t-2} \\
&\quad + \sqrt{\alpha_t(1-\alpha_{t-1})}\epsilon_{t-1} + \sqrt{1-\alpha_t}\epsilon_t,
\end{aligned} \tag{14.8}
$$

$$
= \sqrt{\bar{\alpha}_t}x_0 + \sum_{s=1}^{t}\sqrt{(1-\alpha_s)\prod_{r=s+1}^{t}\alpha_r}\,\epsilon_s. \tag{14.9}
$$

式（14.9）的第二项是独立高斯变量的线性组合，因此也是高斯变量。其协方差必为 $(1-\bar{\alpha}_t)I$：事实上，如果 x_0 的协方差为单位矩阵，式（14.4）的协方差递推就给出 $\mathrm{Cov}(x_t) = I$，而干净数据项 $\sqrt{\bar{\alpha}_t}x_0$ 对协方差的贡献为 $\bar{\alpha}_t I$。也可以将协方差求和式望远镜消去，验证相同的结论。因此，整个噪声项与 $\sqrt{1-\bar{\alpha}_t}\hat{\epsilon}_t$ 同分布，其中 $\hat{\epsilon}_t \sim \mathcal{N}(0, I)$，于是得到（14.5）。

式（14.5）说明 $\bar{\alpha}_t$ 控制干净数据分量：随着 $\bar{\alpha}_t$ 减小，x_t 中关于 x_0 的信息越来越少，而高斯噪声越来越多。在 $T \to \infty$ 且 $\bar{\alpha}_T \to 0$ 的极限情形下，对任意 x_0 都有

$$
q(x_T \mid x_0) \to \mathcal{N}(x_T; 0, I), \qquad \text{即 } \mathrm{Law}_q(x_T \mid x_0) \to \mathcal{N}(0, I). \tag{14.10}
$$

因此，x_T 最终收敛到白高斯噪声。对 x_0 取平均，也就意味着：当 T 很大时，q 下 x_T 的边缘分布接近 $\mathcal{N}(0, I)$。本章余下部分假定 $q(x_T)$ 恰好是球形高斯分布。

注意，前向过程 q 是固定的，而不是学习得到的。这是扩散模型的一个吸引人之处。学习问题不是发现如何破坏数据，而只是学会逆转一个由我们自己选择的破坏过程。

14.2 反向过程的参数化

上一节定义了一个固定的前向过程。现在我们考察逆转它意味着什么，然后用学习得到的参数化模型替换未知的反向条件分布。由于前向过程具有马尔可夫性，同一联合分布也可以沿反方向作马尔可夫分解。更准确地说，在分布 q 下，[59]

$$
q(x_{0:T}) = q(x_T)\prod_{t=1}^{T} q(x_{t-1} \mid x_t), \tag{14.11}
$$

其中条件分布 $q(x_{t-1} \mid x_t)$ 定义了真正的反向马尔可夫链。上一节说明，q 下 x_T 的边缘分布是标准球形高斯分布 $\mathcal{N}(0, I)$。

我们将学习近似未知的反向条件分布 $q(x_{t-1} \mid x_t)$，从而可以先采样 $x_T \sim \mathcal{N}(0, I)$，再反复向后采样 $x_T \to x_{T-1} \to \cdots \to x_1 \to x_0$，以从 p_{data} 生成样本。

我们用参数由神经网络产生的高斯分布来近似每个反向核 $q(x_{t-1} \mid x_t)$。具体地，定义

$$
\begin{aligned}
p_\theta(x_{t-1} \mid x_t) &= \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I), \\
\text{即 } \mathrm{Law}_{p_\theta}(x_{t-1} \mid x_t) &= \mathcal{N}(\mu_\theta(x_t, t), \sigma_t^2 I).
\end{aligned} \tag{14.12}
$$

这些条件分布与先验密度 $p_\theta(x_T) = \mathcal{N}(x_T; 0, I)$（等价地，$\mathrm{Law}_{p_\theta}(x_T) = \mathcal{N}(0, I)$）一起，定义了轨迹上的完整联合分布：

[59] 用简写 $x_{0:T}$ 表示整条轨迹 $(x_0, x_1, \ldots, x_T)$。类似地，$x_{1:T}$ 表示 $(x_1, \ldots, x_T)$。

$$p_\theta(x_{0:T}) = p_\theta(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t). \tag{14.13}$$

均值 $\mu_\theta(x_t, t)$ 由参数为 θ 的神经网络产生；时间步 t 经嵌入后，与 x_t 一起输入网络。标量方差 σ_t^2 是固定的，稍后将确定其取值，但也可以学习该方差 [Nichol and Dhariwal, 2021]。

这种高斯反向参数化不仅是一种方便的建模选择，也得到连续时间反向扩散定理的支持。我们将在第 14.4 节回到这一视角。

14.3 通过最大化 ELBO 训练扩散模型

现在采用标准的变分方法，通过最大化似然拟合参数化反向过程。目标是对参数 θ 最大化似然 $p_\theta(x_0)$：[60]

$$p_\theta(x_0) = \int p_\theta(x_{0:T})\, dx_1 \cdots dx_T. \tag{14.14}$$

由于积分难以处理，我们最大化 $p_\theta(x_0)$ 的变分下界。利用标准的 ELBO 恒等式，即（11.9）的定义和（11.10）的下界结论，令隐变量 z 对应路径 $x_{1:T}$、辅助分布 $Q(z)$ 对应 $q(x_{1:T} \mid x_0)$，得到

$$\begin{aligned} \log p_\theta(x_0) \geq L_{\mathrm{ELBO}}(x_0; q) &= \mathbb{E}_{x_{1:T}\sim \mathrm{Law}_q(x_{1:T}|x_0)}\left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)}\right] \\ &= \mathbb{E}_{x_{1:T}\sim \mathrm{Law}_q(x_{1:T}|x_0)}[\log p_\theta(x_0 \mid x_{1:T})] \\ &\quad - \mathrm{KL}(\mathrm{Law}_q(x_{1:T} \mid x_0) \| \,\mathrm{Law}_{p_\theta}(x_{1:T})). \end{aligned} \tag{14.15}$$

由于反向模型具有马尔可夫性，$p_\theta(x_0 \mid x_{1:T}) = p_\theta(x_0 \mid x_1)$。所以第一项是最终重构项。利用 KL 的链式法则（引理 A.1.4）以及 p_θ 和 q 的反向分解，第二个 KL 项可分解为

$$\begin{aligned} &\mathrm{KL}(q(x_{1:T} \mid x_0) \| p_\theta(x_{1:T})) \\ &= \underbrace{\mathrm{KL}(q(x_T \mid x_0) \| p_\theta(x_T))}_{\triangleq L_T} + \sum_{t=2}^{T} \underbrace{\mathbb{E}_q[\mathrm{KL}(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t))]}_{\triangleq L_{t-1}}. \end{aligned} \tag{14.16}$$

可以从训练目标中去掉 $L_T = \mathrm{KL}(q(x_T \mid x_0) \| p_\theta(x_T))$，因为 $p_\theta(x_T) = \mathcal{N}(0, I)$ 固定，故 L_T 与学习的反向转移无关。此外，假定前向加噪计划使 $\bar\alpha_T$ 非常接近零，就有 $q(x_T \mid x_0) = \mathcal{N}(\sqrt{\bar\alpha_T} x_0, (1-\bar\alpha_T) I) \approx \mathcal{N}(0, I)$，因此 L_T 也很小。由（14.15）和（14.16），

$$\log p_\theta(x_0) \geq L_{\mathrm{ELBO}}(x_0; q) = -L_0 - \sum_{t=2}^{T} L_{t-1} = -\sum_{t=0}^{T-1} L_t. \tag{14.17}$$

> 译注：原式（14.17）在去掉与参数无关的 L_T 后仍使用等号。严格而言，完整 ELBO 还包含 $-L_T$；这里应理解为沿用前文理想化假设，或在优化目标中忽略该常数。

$L_1, L_2, \ldots, L_{T-1}$ 是主要的去噪项。在每个时间步，它们要求模型的反向核 $p_\theta(x_{t-1} \mid x_t)$ 匹配真正的高斯后验 $q(x_{t-1} \mid x_t, x_0)$。L_0 是最终重构项的负期望，控制如何将几乎干净的样本 x_1 变回 x_0。这是

[60] 更准确地说，目标是 $\max_\theta \frac{1}{n}\sum_{i=1}^{n} \log p_\theta(x_0^{(i)})$，其中 $x^{(i)}$ 是经验样本。这里为简单起见，只考虑单个样本 x_0。

生成与去噪之间的概率联系：如果每个反向步骤都匹配相应的真实后验，那么将这些步骤串联起来就会得到一个好的生成模型。

接下来将去噪项 $L_1, \dots, L_{T-1}$ 推导为 θ 的显式函数，以得到具体的优化目标。首先，根据高斯分布的基本性质，给定 x_0 时 $x_1, \dots, x_T$ 联合服从高斯分布，因此真实的单步反向转移也是具有特定均值和方差的高斯分布。更具体地，在引理 A.1.2 的条件高斯公式中取 $A = x_{t-1}$、$B = x_t$，同时将 x_0 视为固定值。这说明 $q(x_{t-1} \mid x_t, x_0)$ 是高斯分布。对前向高斯链作代数运算可得

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I), \tag{14.18}$$

其中

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t, \qquad \tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t. \tag{14.19}$$

回顾我们的假设，p_θ 下的反向转移也是高斯分布，具有类似形式：$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$。自然可以选择 $\sigma_t^2 = \tilde{\beta}_t$，使 KL 项 L_{t-1} 中的两个分布具有相同的协方差。

具有相同协方差的两个高斯分布，其 KL 散度具有闭式表达式

$$\mathrm{KL}(\mathcal{N}(m_1, \Sigma) \| \mathcal{N}(m_2, \Sigma)) = \frac{1}{2}(m_1 - m_2)^T \Sigma^{-1}(m_1 - m_2). \tag{14.20}$$

因此，对于 $t = 2, \dots, T$，$L_{t-1} = \mathbb{E}_q[\mathrm{KL}(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t))]$ 等于

$$L_{t-1} = \mathbb{E}_q\left[\frac{1}{2\tilde{\beta}_t}\|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|_2^2\right]. \tag{14.21}$$

换言之，模型 $\mu_\theta(x_t, t)$ 应当重构 $\tilde{\mu}_t(x_t, x_0)$，而根据（14.19），后者是 x_0 与 x_t 的线性组合。

简化为重构噪声。根据（14.5），x_t 是 x_0 与 $\hat{\epsilon}_t$ 的线性组合，因此后验均值 $\tilde{\mu}_t(x_t, x_0)$ 也能用 x_t 和 $\hat{\epsilon}_t$ 表示。所以，重构 $\tilde{\mu}_t(x_t, x_0)$ 等价于重构 $\hat{\epsilon}_t$。

因此可以进一步简化。首先，由（14.5）得到 $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}_t)/\sqrt{\bar{\alpha}_t}$。将其代入精确后验均值（14.19），并利用 $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$ 和 $\alpha_t = 1 - \beta_t$，对 $t \geq 2$ 得到

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\hat{\epsilon}_t\right). \tag{14.22}$$

因此，也可以用参数为 θ 的神经网络 $\epsilon_\theta(x_t, t)$ 预测 $\hat{\epsilon}_t$，然后令

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right). \tag{14.23}$$

结合（14.21）至（14.23），对于 $t \geq 2$，

$$L_{t-1} = \frac{\beta_t^2}{2\tilde{\beta}_t \alpha_t (1-\bar{\alpha}_t)}\mathbb{E}_{\hat{\epsilon}_t}[\|\hat{\epsilon}_t - \epsilon_\theta(x_t, t)\|_2^2]. \tag{14.24}$$

ELBO 由加权的噪声预测损失项组成。

注意，不同时间步的 $\hat{\epsilon}_t$ 并不独立。然而，在单个采样时间步上训练时，只需要边缘分布满足 $\hat{\epsilon}_t \sim \mathcal{N}(0, I)$；因此，其他资料常将 $\hat{\epsilon}_t$ 简写为 $\epsilon \sim \mathcal{N}(0, I)$。此外，上述推导仅适用于 $t \geq 2$ 的去噪项 L_{t-1}，因为 L_0 的定义不同。

L_0 项。在（14.13）中，我们将反向过程参数化为马尔可夫过程，所以 $p_\theta(x_0 \mid x_{1:T}) = p_\theta(x_0 \mid x_1)$。一种自然的选择是用高斯分布建模 $p_\theta(x_0 \mid x_1)$，即 $p_\theta(x_0 \mid x_1) = \mathcal{N}(x_0; \mu_\theta(x_1, 1), \sigma_1^2 I)$。因此，

$$L_0 = \frac{1}{2\sigma_1^2} \mathbb{E}_q[\|x_0 - \mu_\theta(x_1, 1)\|_2^2] + \frac{d}{2} \log(2\pi\sigma_1^2). \tag{14.25}$$

当 $t = 1$ 时，$\bar{\alpha}_1 = \alpha_1$，且 $x_0 = (x_1 - \sqrt{1-\bar{\alpha}_1}\hat{\epsilon}_1)/\sqrt{\bar{\alpha}_1} = (x_1 - \sqrt{\beta_1}\hat{\epsilon}_1)/\sqrt{\alpha_1}$。若定义 $\tilde{\mu}_1(x_1, x_0) = x_0$，则（14.22）在 $t = 1$ 时也成立。因此可以采用与（14.23）相同的噪声预测参数化，即 $\mu_\theta(x_1, 1) = (x_1 - \sqrt{\beta_1}\epsilon_\theta(x_1, 1))/\sqrt{\alpha_1}$。结合这些表达式，

$$L_0 = \frac{\beta_1}{2\sigma_1^2 \alpha_1} \mathbb{E}_{\hat{\epsilon}_1}[\|\hat{\epsilon}_1 - \epsilon_\theta(x_1, 1)\|_2^2] + \frac{d}{2} \log(2\pi\sigma_1^2). \tag{14.26}$$

该权重与将 $t = 1$ 代入（14.24）得到的形式相同，最终一步的方差 σ_1^2 充当分母中的方差项。因此，忽略与 θ 无关的常数后，$t = 1$ 的重构项与 $t \geq 2$ 的去噪项具有相同的加权噪声预测形式。[61]

实践中往往去掉这些系数，使用不加权的噪声预测损失训练。

训练算法。简化后的目标带来一个格外简单的训练循环：

1. 从数据集中采样一个干净样本 x_0。
2. 采样时间步 t（通常从 $\{1, \ldots, T\}$ 均匀采样）。
3. 采样高斯噪声 $\epsilon \sim \mathcal{N}(0, I)$。
4. 构造带噪样本：

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon. \tag{14.27}$$

5. 对下列损失作梯度更新，要求网络 $\epsilon_\theta(x_t, t)$ 预测实际使用的噪声 ϵ：

$$L_t(\theta) = \|\epsilon - \epsilon_\theta(x_t, t)\|_2^2. \tag{14.28}$$

测试时采样。采样沿反向时间使用同一个学习到的去噪器：

1. 采样 $x_T \sim \mathcal{N}(0, I)$。
2. 对 $t = T, T-1, \ldots, 1$，先计算 $\epsilon_\theta(x_t, t)$，再用（14.23）构造均值 $\mu_\theta(x_t, t)$。然后采样

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t \xi, \qquad \xi \sim \mathcal{N}(0, I), \tag{14.29}$$

其中 $t > 1$ 时使用上述随机噪声，最后一步则令 $\xi = 0$。

[61] 实践中，图像数据实际由离散像素值表示，在连续观测分布之上还会进行一些截断和离散化；详情参见 Ho et al. [2020] 第 3.3 节。

14.4 反向扩散的连续时间视角

连续时间视角提供了另一种理解方式，解释为什么高斯反向核与得分函数会自然地出现在扩散模型中。我们不使用离散马尔可夫链，而用随机微分方程描述前向加噪过程：

$$dX_t = f(X_t, t)\,dt + g(t)\,dW_t, \qquad 0 \le t \le T, \tag{14.30}$$

其中 W_t 是维纳过程，即标准布朗运动。非正式地说，$f(X_t, t)dt$ 是无穷小时间区间中的确定性漂移，类似于离散更新中的条件均值增量 $(\sqrt{\alpha_t} - 1)x_{t-1}$；$g(t)^2 dt$ 是无穷小噪声方差，类似于离散前向转移中 β_t 的作用。增量 dW_t 提供标准高斯随机性。

令 p_t 表示 X_t 的密度。Anderson [1982] 的反向时间定理给出了反向过程的形式，并指出它也是扩散过程。为陈述该定理，定义反向时间过程

$$Y_\tau = X_{T-\tau}, \qquad 0 \le \tau \le T. \tag{14.31}$$

因此，τ 增大时，原来的时间 $T - \tau$ 减小。

定理 14.4.1（反向时间扩散定理，非正式表述）。在适当的正则性条件下，反向时间过程 $Y_\tau = X_{T-\tau}$ 仍是扩散过程。其动力学为

$$dY_\tau = [-f(Y_\tau, T-\tau) + g(T-\tau)^2 \nabla_x \log p_{T-\tau}(Y_\tau)]\,d\tau + g(T-\tau)\,d\bar{W}_\tau, \tag{14.32}$$

其中 $\bar{W}_\tau$ 是相对于反向时间滤过的维纳过程。[62]

(14.32) 中的正负号来自将 τ 用作反向过程的正向变量。原来的漂移 f 被反向经过，因而产生 $-f(Y_\tau, T-\tau)$ 项。额外的得分项 $\nabla_x \log p_{T-\tau}$ 出现，是因为反向转移还必须考虑：在 $T - \tau$ 时刻的边缘密度下，哪些先前状态更有可能。

下面给出一个简短、非正式的一维推导，以说明主要直觉。考虑特例

$$dX_t = f(X_t, t)\,dt + dW_t. \tag{14.34}$$

当步长 h 很小时，从 $T - \tau - h$ 时刻到 $T - \tau$ 时刻的前向转移近似为高斯分布：

$$\mathrm{Law}_X(X_{T-\tau} \mid X_{T-\tau-h} = z) \approx \mathcal{N}(z + hf(z, T-\tau), h). \tag{14.35}$$

现在以 $Y_\tau = y$ 为条件，这与事件 $X_{T-\tau} = y$ 相同。根据贝叶斯公式，将其视为关于 z 的密度（从而省略贝叶斯公式分母中仅依赖 y 的乘性因子），

$$p(Y_{\tau+h} = z \mid Y_\tau = y) \propto p(X_{T-\tau} = y \mid X_{T-\tau-h} = z)\, p_{T-\tau-h}(z). \tag{14.36}$$

[62] 等价地，人们常保留原来的时间变量 t，但从 T 向 0 积分。若 $\bar{X}_t$ 表示用这一递减原时间参数写出的同一反向过程，则

$$d\bar{X}_t = [f(\bar{X}_t, t) - g(t)^2 \nabla_x \log p_t(\bar{X}_t)]\,dt + g(t)\,d\bar{W}_t, \qquad t : T \to 0. \tag{14.33}$$

此处 dt 理解为负的时间增量，$d\bar{W}_t$ 是反向时间中的维纳增量。令 $t = T - \tau$，即可恢复 (14.32)。

（14.36）的第一个因子迫使 $z-y$ 的范数以高概率处于 $\sqrt{h}$ 的数量级。对对数密度作泰勒展开，舍去小于 $O(h)$ 的各项：

$$\log p_{T-\tau-h}(z) = \log p_{T-\tau}(y) + \partial_x \log p_{T-\tau}(y)(z-y) + O(h). \tag{14.37}$$

注意，对大多数 z，第二项处于 $\sqrt{h}$ 的数量级。令 $u = z-y$，并注意 u 大多处于 $\sqrt{h}$ 的数量级，可由泰勒展开推得 $p(Y_{\tau+h} = z \mid Y_\tau = y)$ 的对数：

$$\log p(Y_{\tau+h} = z \mid Y_\tau = y) \tag{14.38}$$

$$\begin{aligned} &= -\frac{(y - z - hf(y, T-\tau))^2}{2h} + \log p_{T-\tau-h}(z) + \text{const} \\ &= -\frac{u^2}{2h} - f(y, T-\tau)u + \partial_x \log p_{T-\tau}(y)u + \text{const} + O(h) \\ &= -\frac{1}{2h}\left(u - h[-f(y, T-\tau) + \partial_x \log p_{T-\tau}(y)]\right)^2 + \text{const} + O(h). \end{aligned} \tag{14.39}$$

这里 const 包含所有仅为 y 的函数的项。换句话说，

$$Y_{\tau+h} \mid Y_\tau = y \approx \mathcal{N}\left(y + h[-f(y, T-\tau) + \partial_x \log p_{T-\tau}(y)], h\right). \tag{14.40}$$

这就是 $g \equiv 1$ 时（14.32）的小步长形式。一般情形的无穷小方差为 $g(T-\tau)^2 h$，因此得分的贡献变为 $g(T-\tau)^2 \nabla_x \log p_{T-\tau}$。这解释了为什么反向过程在局部仍为高斯，以及为什么扩散模型用高斯分布参数化反向核。

第 15 章　基础模型概述

原文印刷页码：191–195。

尽管取得了巨大成功，以监督学习训练的神经网络通常仍依赖规模可观的标注数据集，而收集此类数据可能成本高昂。大约自 2018 年起，随着 BERT [Devlin et al., 2019]、GPT-3 [Brown et al., 2020] 等模型兴起，人工智能与机器学习正在经历范式转变：先在广泛的数据上进行大规模预训练，再适配到各种下游任务。Bommasani et al. [2021] 将这些模型称为**基础模型**。它们通常利用海量无标注数据，从而能以更少的、甚至不使用标注样本来解决一大类下游任务。此外，虽然基础模型仍基于深度学习和神经网络，其规模可能带来新的涌现能力。这些模型通常通过自监督学习训练，监督信号由输入的一部分构造而来。

基础模型范式包括两个阶段：**预训练**（也简称训练）和**适配**。首先，在通常无标注的海量数据集上预训练一个大型模型。[63] 然后将预训练模型适配到下游任务，通常仅使用有限的、甚至完全不使用标注数据。直觉上，在多样化数据上预训练，可以教会模型广泛的结构，而这些结构能够迁移到许多下游任务。下面形式化描述这两个阶段。

预训练。预训练数据集通常是大规模无标注数据集

$$\{x^{(1)}, \dots, x^{(n)}\}.$$

根据模态不同，每个 $x^{(i)}$ 可以是一张图像、一个词语序列，或文字、图像、音频、视频的混合。最常见的情况下，预训练数据没有标注；有时也可以有标签。

令 θ 为模型参数，ℓ_{pre} 为损失函数。预训练损失通常写为预训练样本上的平均值：

$$L_{\text{pre}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_{\text{pre}}(x^{(i)}, \theta). \tag{15.1}$$

这里 ℓ_{pre} 是单个数据点 $x^{(i)}$ 上的自监督损失，因为正如后面的例子所示，“监督” 可以由数据点本身构造出来。预训练损失也可能不恰好是单个样本损失的求和。通常使用 SGD 或 Adam [Kingma and Ba, 2014] 等优化器最小化 $L_{\text{pre}}(\theta)$。

用 $\hat{\theta}$ 表示得到的预训练模型。

适配。我们可以适配预训练模型来解决下游任务。在典型的监督下游任务中，有一个带标签的数据集

$$\{(x_{\text{task}}^{(1)}, y_{\text{task}}^{(1)}), \dots, (x_{\text{task}}^{(n_{\text{task}})}, y_{\text{task}}^{(n_{\text{task}})})\}.$$

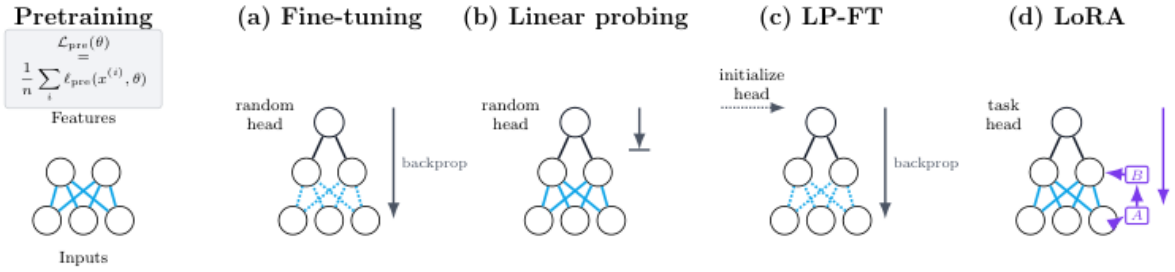
[63] 有时预训练也会使用大规模标注数据集，例如 ImageNet。

通常 n_{task} 远小于预训练数据集的规模。如果只有少量标注样本，例如 5 至 10 个样本，这种设置通常称为**少样本学习**。如果下游任务没有任何标注样本，则称为**零样本学习**；在现代语言模型应用中，这可能意味着模型只收到任务的自然语言描述。目前零样本设置最为常见，但本章主要讨论下游样本较多的设置，将零样本设置留到后续章节。

15.1 表示学习中的线性探测与微调

表示学习是一类预训练方法。由 θ 参数化的模型 ϕ_θ 将原始输入 x 映射为向量 $\phi_\theta(x)$，称为**特征向量**、**嵌入**或**表示**。好的表示应捕捉 x 中对许多下游任务有用的语义信息。用预训练损失训练这一模型后，得到预训练表示函数 $\phi_{\hat{\theta}}$。

适配随后利用下游数据，决定如何从这些表示中提取任务特定的信息。



预训练与各种适配方法

图 15.1：预训练与适配示意图。预训练最小化（15.1）中的通用预训练损失；对于自监督样本，单样本损失 ℓ_{pre} 可以来自下一个词元预测、掩码图像块预测，或其他由 $x^{(i)}$ 本身导出的预测任务。全量微调更新复制过来的权重和预测头；线性探测冻结复制的权重，只训练预测头；LP-FT 先用线性探测的结果初始化，再进行微调；LoRA 冻结复制的基础权重 W_0，只训练低秩更新。

线性探测保持 $\phi_{\hat{\theta}}$ 不变，只在表示 $\phi_{\hat{\theta}}(x)$ 之上学习一个简单的预测头。具体地，下游预测模型具有 $w^\top \phi_{\hat{\theta}}(x)$ 的形式，其中训练 $w \in \mathbb{R}^m$，而 $\hat{\theta}$ 固定。利用下游标注数据集学习 w。例如，回归任务可求解

$$\min_{w \in \mathbb{R}^m} \frac{1}{n_{\text{task}}} \sum_{i=1}^{n_{\text{task}}} \left(y_{\text{task}}^{(i)} - w^\top \phi_{\hat{\theta}}(x_{\text{task}}^{(i)}) \right)^2. \tag{15.2}$$

更一般地，可以用任务损失 ℓ_{task} 替换（15.2）中的平方损失。

微调。微调使用相同的下游预测模型结构，但也更新预训练表示。预测模型为 $w^\top \phi_\theta(x)$，参数为 w 和 θ。在下游数据上同时优化两者，将 θ 初始化为预训练模型 $\hat{\theta}$，通常随机初始化 w：

$$\underset{w,\theta}{\text{minimize}} \quad \frac{1}{n_{\text{task}}} \sum_{i=1}^{n_{\text{task}}} \ell_{\text{task}} \left(y_{\text{task}}^{(i)}, w^\top \phi_\theta(x_{\text{task}}^{(i)}) \right), \tag{15.3}$$

$$\text{初始化为} w \leftarrow \text{随机向量}, \tag{15.4}$$

$$\theta \leftarrow \hat{\theta}. \tag{15.5}$$

一种相关情况是**继续预训练**，或采用与预训练相同形式的损失函数（但数据不同）进行微调。此时从 $\hat{\theta}$ 出发，在另一个无标注数据集上继续训练，通常使用与（15.1）相同类型的预训练损失。当新数据分布比原始预训练数据更接近下游领域时，这种做法很有用。

先线性探测后微调（LP-FT）。LP-FT 是一种简单的两阶段适配方法。首先，进行线性探测：冻结预训练表示 $\phi_{\hat{\theta}}$，只学习预测头 w，如（15.2）。其次，进行全量微调，如（15.5），以线性探测得到的预测头以及 $\theta = \hat{\theta}$ 初始化。第一阶段找到从预训练表示中读取任务特定信息的方法，第二阶段则允许表示本身适应下游数据。提出这一流程的目的，是减轻微调中的不良特征扭曲；在某些设置下，它可以提高分布外表现 [Kumar et al., 2022]。

还有许多其他适配方法，有些针对特定的预训练方法或模型类别设计。第 17 章将讨论这些方法。

15.2 低秩适配（LoRA）

全量微调更新预训练模型的全部参数。当下游任务或下游数据分布与预训练相差不远时，得到的微调模型可能也不会偏离预训练模型太多，因此未必需要动用预训练参数的全部自由度。**低秩适配**，即 LoRA [Hu et al., 2022]，是一种参数高效的微调方法：冻结预训练权重，只训练权重矩阵的低秩更新。

对于一个线性层，设预训练权重矩阵为 $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$，该层计算 $h = W_0 x$。从 W_0 出发对参数作全量微调，等价于寻找一个无约束的更新 ΔW，并使用 $h = (W_0 + \Delta W)x$。LoRA 将更新限制为低秩。对所选秩 $r \ll \min(d_{\text{out}}, d_{\text{in}})$，写为

$$\Delta W = BA, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r}, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}}, \tag{15.6}$$

通常将该更新缩放为

$$h = W_0 x + \frac{\alpha}{r} BAx. \tag{15.7}$$

关键在于，W_0 冻结，只训练 A 和 B。稠密更新有 $d_{\text{out}} d_{\text{in}}$ 个可训练参数，而 LoRA 更新只有

$$r(d_{\text{out}} + d_{\text{in}}) \tag{15.8}$$

个可训练参数。这里 r 是 LoRA 的秩，α 通常称为 LoRA alpha，α/r 是 LoRA 的缩放因子。

秩控制更新的维度，α 控制更新相对于冻结预训练矩阵的整体大小。一种常见初始化方式是将一个因子设为零，例如 $B = 0$，使初始化时 $BA = 0$，于是适配模型起初计算的函数与预训练模型完全相同。不能将低秩限制理解为最终适配权重矩阵 $W_0 + (\alpha/r)BA$ 是低秩的。通常 W_0 是满秩的；只有相对于预训练矩阵的变化是低秩的。

LoRA 减少了可训练参数的数量，因此减少了存储可训练更新的梯度与优化器状态所需的内存。然而，冻结的基础权重 W_0 仍必须存储，并用于前向与反向传播；反向传播所需的激活值也不会因低秩参数化而减少。因此，在基础模型权重和激活值占据主要内存的设置下，例如采用数据并行或模型并行的大模型训练，训练时节省的内存可能有限。同样，LoRA 通常很少节省计算量，因为仍要执行基础模型中的大部分前向与反向计算。

因此，LoRA 最大的好处往往出现在多个用户共享推理或训练计算资源的场景。由于每个适配器都

很小，可以在 CPU 内存、甚至 GPU 内存中，为同一基础模型存放许多任务特定的适配器，以便快速切换。这让系统可以在同一组机器上近乎同时服务多个微调模型，瞬间切换定制模型；即使某些用户只是偶尔使用，所有用户也能获得低延迟，计算资源仍能保持很高利用率。它也可能用于多租户训练系统：不同训练作业共享相同基础权重，同时保留各自的 LoRA 更新，从而共享计算并提高 GPU 利用率。

最后，LoRA 也是一种表达能力上的权衡：它将每个更新限制在低秩族内。这对许多适配任务很有效，但当分布变化很大或适配需要较高容量时，全参数训练仍可能更合适。

第 16 章　表示学习

原文印刷页码：196–201。

本章介绍两种具体的表示学习预训练方法：监督预训练与对比学习。

16.1 监督预训练

这里，预训练数据集是一个大规模带标签的数据集，例如 ImageNet；预训练模型就是通过普通监督学习训练出的神经网络，并去掉最后一层。具体来说，假设将学到的神经网络写为 $U\phi_{\hat{\theta}}(x)$，其中 U 为最后一个全连接层的参数，$\hat{\theta}$ 对应其余所有层的参数，$\phi_{\hat{\theta}}(x)$ 是倒数第二层的激活值，用作表示。只需丢弃 U，将 $\phi_{\hat{\theta}}(x)$ 作为预训练的表示模型。

16.2 对比学习

对比学习是一种只使用无标签数据的自监督预训练方法。其核心直觉是：一个好的表示函数 $\phi_\theta(\cdot)$ 应把语义相似的图像 / 文本映射到相似的表示，而随机选取的一对图像 / 文本通常应具有不同表示。例如，我们可能希望把两只哈士奇的图像映射到相似表示，但哈士奇与大象应具有不同表示。对比学习的思想广泛适用于图像、文本、视频等，不过后续几节主要以图像为例展开讨论。

对比学习的一个关键问题是：如何获得一对相似图像？一种相似性定义是，属于同一类别的图像相似。采用这个定义，就得到所谓的监督对比算法；在有带标签预训练数据集时，这类算法效果很好。

如果没有带标签数据，可以利用数据增强，从一张原始图像 x 生成一对 “相似” 的增强图像。数据增强通常是对原始图像 x 随机裁剪、翻转和 / 或进行颜色变换，生成其变体。对同一个原始图像 x 进行两次随机增强，记为 $\hat{x}$ 和 $\tilde{x}$，将其称为**正样本对**。可以观察到，正样本对中的图像往往在语义上相关，因为它们是同一张图像的增强结果。我们将为 θ 设计一个损失函数，使正样本对的表示 $\phi_\theta(\hat{x})$ 和 $\phi_\theta(\tilde{x})$ 尽可能接近。

另一方面，也可以从预训练数据集中随机选择另一张图像 z，并从中生成增强图像 $\hat{z}$。注意，$(\hat{x},\hat{z})$ 来自不同图像，因此很可能在语义上不相关。将 $(\hat{x},\hat{z})$ 称为**负样本对**或**随机样本对**。[64] 我们将设计一种损失，把随机样本对的表示 $\phi_\theta(\hat{x})$ 与 $\phi_\theta(\hat{z})$ 推得更远。

近期有许多基于对比学习原理的算法，这里以 SIMCLR［Chen 等，2020］为具体例子。损失函数定义在一个包含 B 个样本的批次 $(x^{(1)},\dots,x^{(B)})$ 上。算法为批次中的每个样本 $x^{(i)}$ 计算两次随机增强，记为 $\hat{x}^{(i)}$ 和 $\tilde{x}^{(i)}$。由此得到包含 $2B$ 个样本的增强批次：$\hat{x}^{(1)},\dots,\hat{x}^{(B)},\tilde{x}^{(1)},\dots,\tilde{x}^{(B)}$。SIMCLR 损

[64] “随机样本对” 或许是更准确的术语，因为 x 与 z 仍有可能在语义上相关，尽管可能性不大，$\hat{x}$ 与 $\hat{z}$ 也就可能相关。不过，“负样本对” 在文献中似乎同样常见。

失定义为[65]

$$L_{\text{pre}}(\theta) = -\sum_{i=1}^{B} \log \frac{\exp\left(\phi_\theta(\hat{x}^{(i)})^T \phi_\theta(\tilde{x}^{(i)})\right)}{\exp\left(\phi_\theta(\hat{x}^{(i)})^T \phi_\theta(\tilde{x}^{(i)})\right) + \sum_{j \neq i} \exp\left(\phi_\theta(\hat{x}^{(i)})^T \phi_\theta(\tilde{x}^{(j)})\right)}.$$

直觉如下：损失随 $\phi_\theta(\hat{x}^{(i)})^T \phi_\theta(\tilde{x}^{(j)})$ 增大而增大，因此最小化损失会促使该内积变小，使 $\phi_\theta(\hat{x}^{(i)})$ 与 $\phi_\theta(\tilde{x}^{(j)})$ 彼此远离。另一方面，损失随 $\phi_\theta(\hat{x}^{(i)})^T \phi_\theta(\tilde{x}^{(i)})$ 增大而减小，因此最小化损失会促使该内积增大，使 $\phi_\theta(\hat{x}^{(i)})$ 与 $\phi_\theta(\tilde{x}^{(i)})$ 彼此接近。[66]

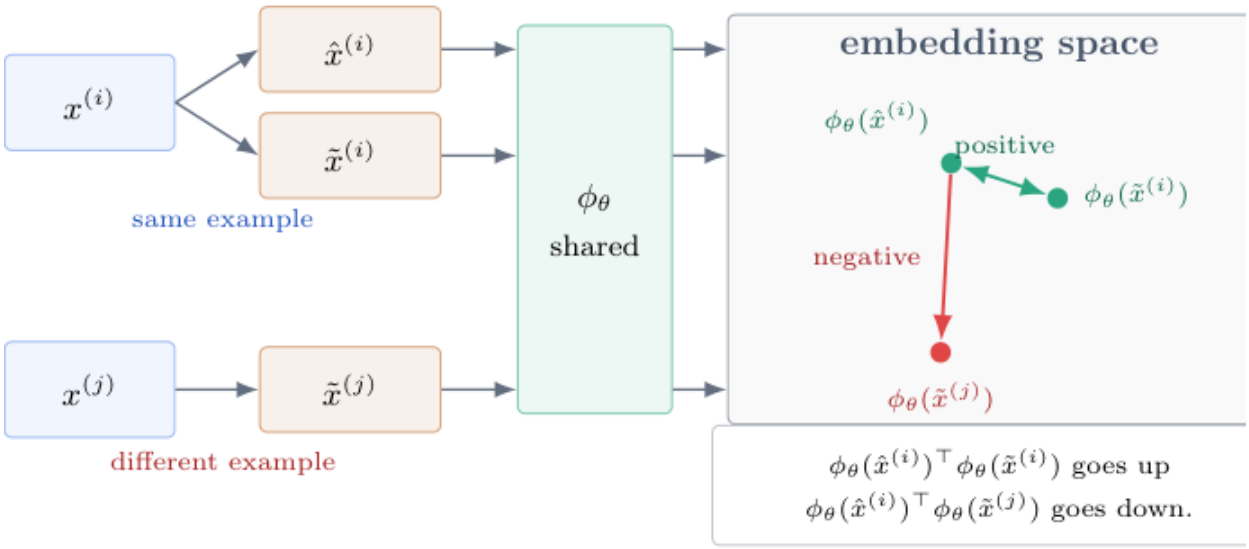


图 16.1

图 16.1：对比学习示意图。同一样本的两种增强构成正样本对，损失鼓励其表示具有较大的内积。不同样本的增强构成随机或负样本对，损失鼓励其表示具有较小的内积。

16.3 语义检索

在第 15 章中，表示主要用作下游监督任务的特征或输入，通过线性探测来使用。随着现代大语言模型的发展，下游任务对线性探测的需求大大减少，但表示或嵌入仍广泛用于**语义检索**：在大型语料库中搜索与查询相关的对象，即使这种相关性不能由精确的词语重合来体现。查询可以是自然语言问题、产品描述、图像或一小段代码，语料库则可以包含文档、段落、图像或其他对象。例如，查询“怎样防止模型死记训练集？”应当检索出有关过拟合与正则化的文档，即使文档中没有出现“死记”一词。

更正式地，令 $\mathcal{D} = \{d_1, \ldots, d_N\}$ 表示由文档、段落、图像或其他对象组成的集合。给定查询 q，检索系统返回一个较短的有序列表 $d_{i_1}, d_{i_2}, \ldots, d_{i_k}$，其中排名靠前的条目应与 q 更相关。许多检索系统还会为每个查询—对象对计算分数 $s(q, d)$，分数越大，表示系统预测 d 与 q 越相关。

使用嵌入进行语义检索。嵌入把检索转化为几何搜索。设 ϕ_θ 是嵌入模型，$\phi_\theta(q), \phi_\theta(d) \in \mathbb{R}^m$ 分别为查询 q 和文档 d 的嵌入。如果嵌入经过归一化、具有单位范数，那么内积 $\phi_\theta(q)^T \phi_\theta(d)$ 也就是余弦相似度 $\cos \angle(\phi_\theta(q), \phi_\theta(d))$，常被用作相关性分数。

[65]这是原始损失的一种变体和简化，不改变本质，但可能略微改变效率。
[66]为理解这一点，可以验证：当 $p, q > 0$ 时，函数 $-\log \frac{p}{p+q}$ 关于 p 单调递减，关于 q 单调递增。

事先计算整个语料库的嵌入 $\{\phi_\theta(d_1), \dots, \phi_\theta(d_N)\}$，通常将它们存储在向量数据库中。查询时，语义检索就变成一个向量搜索问题：计算 $\phi_\theta(q)$，再找出已存储嵌入 $\phi_\theta(d_i)$ 与其内积最大的对象。暴力遍历全部 N 个对象，每次查询的成本为 $O(Nm)$。对于大型语料库，这通常太慢，因此实际系统使用近似最近邻索引，在接受少量漏掉精确最近邻的概率的同时，显著提高搜索速度。常见方法包括 HNSW［Malkov 和 Yashunin，2020］等基于图的索引、量化以及倒排文件索引。另一种方法是在大多数数据存放于对象存储时很有用的基于质心的 ANN 索引［Chen 等，2021］：系统先搜索一个小型簇质心索引，再通过少数几次大块读取获取最有希望的簇，并对候选结果重新排序。与图遍历相比，这减少了随机访问存储的往返次数，同时仍能避免扫描整个语料库。

检索评估。检索评估数据集通常包含一组查询和一个语料库。对于每个查询，都有一组作为金标准的相关文档，记为 $R(q)$。

假设检索方法返回有序列表 $\hat{R}(q) = (d_{i_1}(q), \dots, d_{i_k}(q))$。一个简单指标是前 k 项召回率，定义为

$$\text{Recall@}\,k(q) = \frac{|R(q) \cap \{d_{i_1}(q), \dots, d_{i_k}(q)\}|}{|R(q)|}.$$

这个指标不考虑 $R(q)$ 和 $\hat{R}(q)$ 中各文档的相对排名。

另一个常见的排序指标是**归一化折损累计增益**，即 NDCG［Yilmaz 等，2008］。[67] 对这一指标，评估数据集为每个查询—文档对指定一个真实相关性等级 $s^*(q,d) \geq 0$。对于检索列表 $\hat{R}(q) = (d_{i_1}(q), \dots, d_{i_k}(q))$，定义

$$\text{DCG@}\,k(q) = \sum_{j=1}^{k} \frac{s^*(q, d_{i_j}(q))}{\log_2(j+1)}. \tag{16.1}$$

若 $s^*_{(1)}(q) \geq s^*_{(2)}(q) \geq \cdots$ 是将语料库文档对查询 q 的相关性等级按降序排列后的结果，则

$$\text{IDCG@}\,k(q) = \sum_{j=1}^{k} \frac{s^*_{(j)}(q)}{\log_2(j+1)}, \tag{16.2}$$

$$\text{NDCG@}\,k(q) = \frac{\text{DCG@}\,k(q)}{\text{IDCG@}\,k(q)}. \tag{16.3}$$

因此，分母用理想排序的得分对 DCG 进行归一化。[68] 与前 k 项召回率不同，NDCG 会奖励把更相关文档排在列表更靠前的位置。

图 16.2：检索增强生成示意图。系统用查询从语料库中检索相关文档，生成器在产生回答时，同时以查询和检索到的文档为条件。

16.4 检索增强生成

除了典型的搜索应用，在 AI 模型需要使用大量专有或定制文档时，检索尤其有用。**检索增强生成**，即 **RAG**，结合了两个步骤：先从给定语料库中检索相关信息，再以检索到的信息为条件生成答案。在最简单的形式中，若 $\hat{R}(q)$ 表示对查询 q 检索到的前 k 个条目，语言模型生成

[67] 维基百科 “折损累计增益” 条目介绍了一种常见的指数增益版本。该版本把相关性等级 s 替换为 $2^s - 1$；等价地，也可以通过将相关性分数重新定义为 $2^s - 1$ 来得到。

[68] 一个边界情况是 $\text{IDCG@}\,k(q) = 0$，例如查询 q 的所有相关性等级都为零。由于没有相关文档可供检索，这类查询通常会从检索评估数据集中移除。如果仍保留这样的查询，一种常见约定是定义 $\text{NDCG@}\,k(q) = 0$。

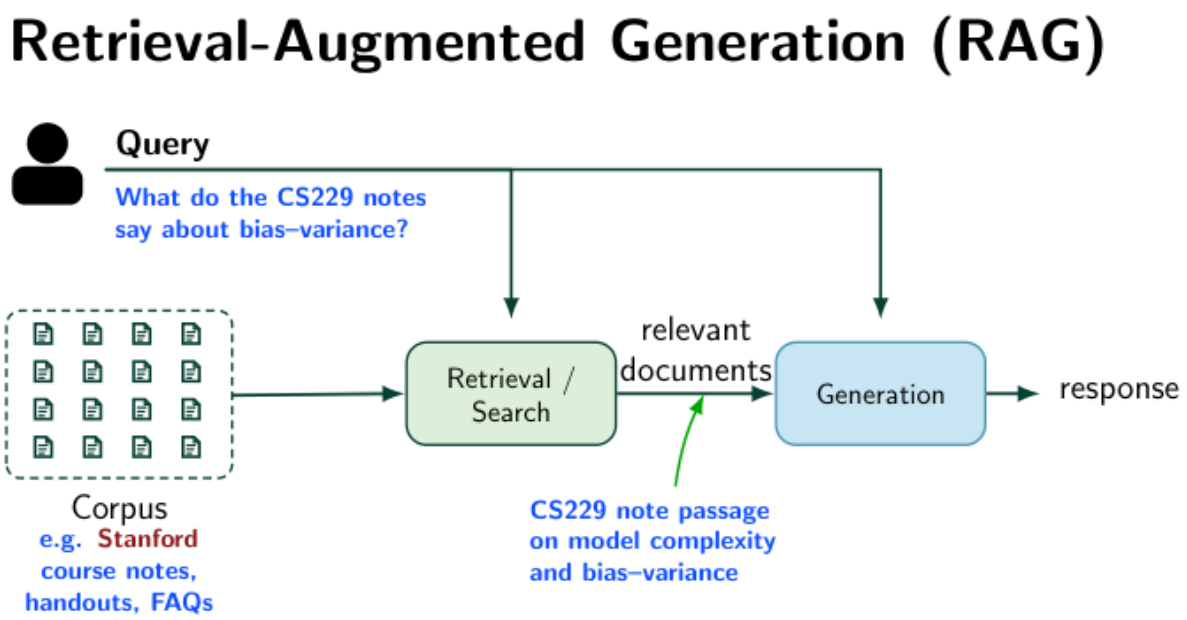


图 16.2

$$y \sim p_\psi(y \mid q, \hat{R}(q)).$$

这能使输出更有来源语料库的依据，并允许通过修改语料库来更新可访问的知识，无须重新训练语言模型。

第 17 章　大语言模型

原文印刷页码：202–219。

自然语言处理是预训练模型尤其成功的另一个领域。我们想要学习的基本对象是语言的分布：哪些句子是合理的，一段文字接下来可能怎样展开。如果模型能为文本序列赋予概率，并从条件分布中采样后续内容，那么许多任务就能表述为以提示为条件的生成任务。这就是语言建模，尤其是下一个词元预测，适合作为语言数据预训练目标的原因。

17.1 词元化

在定义语言建模目标之前，我们需要明确如何把原始文本表示成模型输入。语言模型并不是把一句话作为一个原始字符串来读取，而是先把文本拆成称为**词元**的小片段。词元就是模型视为一个符号的文本单位：它可以是 `hello` 这样的完整单词、`un` 这样的单词片段、标点、空白、ASCII 编码中的一个字节，或者文本结束词元这样的特殊标记。词元化器是一套预处理规则，把字符串转换为词元序列；我们用离散编号表示这些词元：

$$x = (x_1, \dots, x_T), \qquad x_t \in V,$$

其中 V 是词表，即词元化器能够输出的全部词元的集合。例如，句子 `A dog runs.` 可能被表示成大致对应于 `A`、`dog`、`runs` 和 `.` 的词元；`unhappiness` 这样的低频词可能被拆为 `un`、`h` 和 `appiness` 等片段。

现代语言模型通常采用子词词元化，往往以字节对编码（byte-pair encoding，BPE）为基础 [Sennrich et al., 2016, Kudo and Richardson, 2018]。子词词元化是两种更简单方案的折中：字符级词元化的词表很小，却会让每个句子都很长；词级词元化能让常见句子很短，却难以处理低频词或新词。例如，如果词级词表不包含 `glioblastoma` 这样的低频生物医学术语，或 `LLMification` 这样的新造词，它可能不得不把整个字符串映射成未知词元。子词词元化器则可以用可复用的片段编码它们，例如 `glio+blast+oma` 或 `LL+M+ification`；具体拆分取决于学到的切分方式。字节对编码的直觉是，从字节或字符等很小的片段开始，不断把语料库中频繁出现的相邻片段对添加为新词元。经过多次合并后，`ing`、`tion` 等常见字符串，甚至经常出现的完整单词，都能用一个词元表示，而低频词仍可由更小的片段组成。词元化器固定后，预训练就在词元序列而不是原始文本上定义；词元化器的选择会影响词表大小 $|V|$ 和最终的序列长度 T。词表大小 $|V|$ 可以达到 10^5 的量级；例如，Qwen3.5 的词表包含 248,320 个词元。

17.2 自回归模型与下一个词元预测损失

预训练通常在序列层面进行，也就是说，每个“样本”都是一个序列。文档（或多个文档的拼接）首先被词元化为词元序列，记为 $x = (x_1, \dots, x_T)$，其中每个 x_i 都是词表 V 中的词元。为方便起见，我们也会把 V 中的词元与索引 $\{1, \dots, |V|\}$ 等同起来。

语言模型是表示序列概率的概率模型，记为 $p(x_1, \dots, x_T)$。这个概率分布非常复杂，因为其支撑集为 V^T，大小为 $|V|^T$。词表大小 $|V|$ 可达到 10^5 的量级，例如 Qwen3.5 的词表包含 248,320 个词元；而 T 很容易达到 1 万，甚至 100 万，因此 $|V|^T$ 是一个天文数字。所以，我们不直接建模整个序列的分布，而是利用条件概率的链式法则分解它：

$$p(x_1, \dots, x_T) = p(x_1)p(x_2 \mid x_1) \cdots p(x_T \mid x_1, \dots, x_{T-1}). \tag{17.1}$$

这样，每个条件概率 $p(x_t \mid x_1, \dots, x_{t-1})$ 的支撑集大小就是 $|V|$。

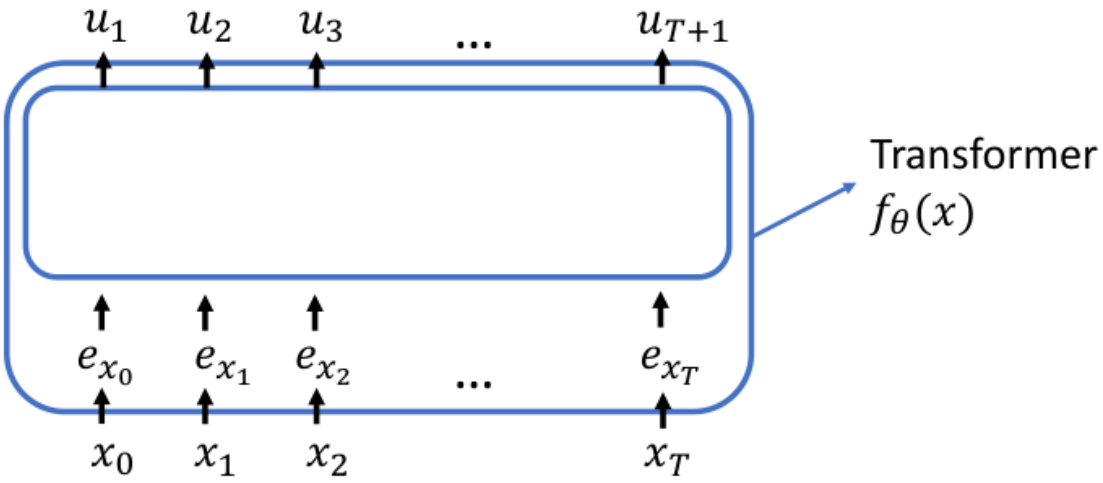


图 17.1 Transformer 模型的输入与输出。

我们把条件概率 $p(x_t \mid x_1, \dots, x_{t-1})$ 建模成以 $x_1, \dots, x_{t-1}$ 为输入、由参数 θ 参数化的函数。

参数化模型接受数值输入，因此我们先引入词元的嵌入或表示。设 $e_i \in \mathbb{R}^{1\times d}$ 是第 i 个词元的行向量嵌入。令

$$E = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_{|V|} \end{bmatrix} \in \mathbb{R}^{|V|\times d} \tag{17.2}$$

表示所有嵌入的集合。

最常用的模型是 Transformer [Vaswani et al., 2017] 的自回归版本及其变体。本节介绍 Transformer 的输入输出接口，把其内部计算视为黑箱；第 17.3 节将讨论 Transformer 及其他变体的细节。

如图 17.1 所示，给定词元化后的文档 $(x_1, \dots, x_T)$，首先把这一离散变量序列转换为相应的词元

嵌入序列 $(e_{x_1}, \dots, e_{x_T})$。[69] 我们还在词表中引入固定的特殊词元 $x_0 = \perp$，其嵌入为 $e_\perp$，用于标记文档开头。接着把词元嵌入送入 Transformer 模型；该模型接收向量序列 $(e_{x_0}, e_{x_1}, \dots, e_{x_T})$，输出向量序列 $(u_1, u_2, \dots, u_{T+1})$，其中 $u_t \in \mathbb{R}^{|V|}$ 被解释为下一个词元概率分布的 logits。这里使用自回归 Transformer，其设计保证 u_t 只依赖 $x_1, \dots, x_{t-1}$（Transformer 的定义见第 17.3 节；注意，掩码语言模型 [Devlin et al., 2019] 不具备这一性质，其损失函数也不同）。本小节把从 x 到 u 的整个映射视为黑箱，称其为 Transformer，记为 f_θ；θ 同时包含 Transformer 内部参数和输入嵌入。写作 $u_t = f_\theta(x_0, x_1, \dots, x_{t-1})$，其中 f_θ 表示从输入到输出的映射。

条件概率 $p_\theta(x_t \mid x_1, \dots, x_{t-1})$ 是 logits 的 softmax：

$$\begin{bmatrix} p_\theta(x_t = 1 \mid x_1, \dots, x_{t-1}) \\ p_\theta(x_t = 2 \mid x_1, \dots, x_{t-1}) \\ \vdots \\ p_\theta(x_t = |V| \mid x_1, \dots, x_{t-1}) \end{bmatrix} = \text{softmax}(f_\theta(x_0, \dots, x_{t-1})) \in \mathbb{R}^{|V|}. \tag{17.3}$$

自回归生成／采样／推断。给定自回归 Transformer，我们可以依次从中采样文本。给定前缀 $x_1, \dots, x_t$，利用条件分布逐个生成词元补全 $x_{t+1}, \dots, x_T$：

$$x_{t+1} \sim \text{softmax}(f_\theta(x_0, x_1, \dots, x_t)) \tag{17.4}$$
$$x_{t+2} \sim \text{softmax}(f_\theta(x_0, x_1, \dots, x_{t+1})) \tag{17.5}$$
$$\dots \tag{17.6}$$
$$x_T \sim \text{softmax}(f_\theta(x_0, x_1, \dots, x_{T-1})). \tag{17.7}$$

注意，生成后续词元时，每个已经生成的词元都会成为模型的输入。实践中，人们常引入一个称为**温度**的参数 $\tau > 0$，进一步调节生成分布的熵或尖锐程度：

$$x_{t+1} \sim \text{softmax}(f_\theta(x_0, x_1, \dots, x_t)/\tau) \tag{17.8}$$
$$x_{t+2} \sim \text{softmax}(f_\theta(x_0, x_1, \dots, x_{t+1})/\tau) \tag{17.9}$$
$$\dots \tag{17.10}$$
$$x_T \sim \text{softmax}(f_\theta(x_0, x_1, \dots, x_{T-1})/\tau). \tag{17.11}$$

当 $\tau = 1$ 时，文本按模型定义的原始条件概率采样。随着 τ 减小，生成文本逐渐变得更“确定”。当 $\tau \to 0$ 时，就退化为贪心解码，即生成条件概率最大的下一个词元。

另外两种常见的解码启发式方法是 top-k 采样和 top-p 采样，后者也称核采样。top-k 采样不从完整词表分布中采样，而只保留概率最大的 k 个词元，并在这个较小集合上重新归一化。top-p 采样则保留由高概率词元组成、总概率至少为 p 的最小集合，并在该集合上重新归一化。这些方法属于推断时的选择，不会改变参数化概率分布或相应的训练损失；它们经常与温度结合，避免从极不

[69] 原始文本到 $(x_1, \dots, x_T)$ 的映射由第 17.1 节介绍的词元化器确定。特殊边界词元的处理也取决于模型和词元化器：有些模型在开头添加序列起始词元，另一些模型把文档打包成长序列流，并在每篇文档后使用文本结束词元，不另行插入起始词元。这个输入输出接口还省略了位置编码；实际 Transformer 会添加位置信息以区分词元顺序，我们将在后续修订中补充讨论。

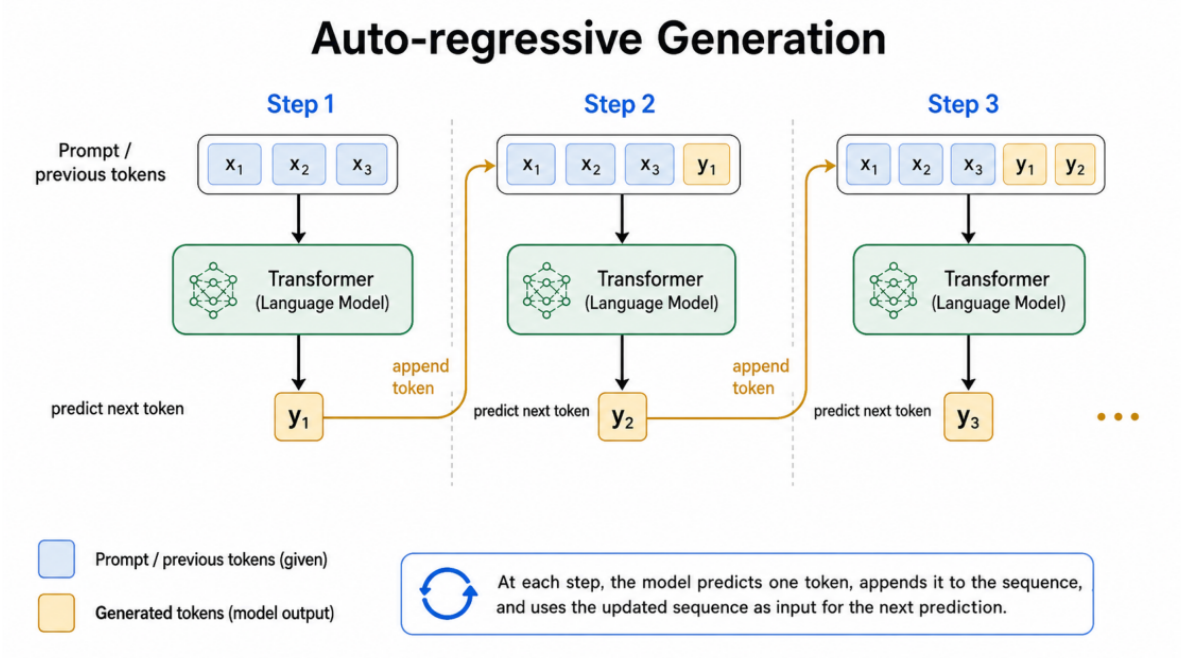


图 17.2 自回归生成：每个新生成的词元都会先回填到前缀中，再生成下一个词元。

可能词元组成的长尾中采样，同时仍允许非贪心生成。例如，一种常见选择是采用 $p = 0.9$ 的 top-p 采样，配合 0.7–1.0 范围内的温度，从而避开极低概率词元的长尾，同时保留非贪心生成。

自回归大语言模型的预训练。训练自回归 Transformer 参数 θ 所用的预训练损失就是最大似然估计，即最小化在 θ 定义的概率模型下观察到数据的负对数似然；这也是 logits 上的交叉熵损失。为方便起见，通常用序列长度 T 归一化。对于一个序列，写作

$$\begin{aligned}
\text{loss}(x_1, \dots, x_T; \theta) &= -\frac{1}{T} \log p_\theta(x_1, \dots, x_T) && (17.12)\\
&= \frac{1}{T} \sum_{t=1}^{T} -\log p_\theta(x_t \mid x_0, \dots, x_{t-1}) && (17.13)\\
&= \frac{1}{T} \sum_{t=1}^{T} \ell_{\text{ce}}(f_\theta(x_0, x_1, \dots, x_{t-1}), x_t)\\
&= \frac{1}{T} \sum_{t=1}^{T} -\log\big(\text{softmax}(f_\theta(x_0, x_1, \dots, x_{t-1}))_{x_t}\big).
\end{aligned}$$

实践中有大量训练序列，总训练损失是所有序列损失的平均值。现代语言模型训练中常用的优化器是 AdamW [Loshchilov and Hutter, 2019]：取一小批这样的序列，计算梯度，然后应用更新规则。

17.3 Transformer 架构

Transformer 是语言建模的主流模型架构，由多头自注意力和 MLP 的多个模块构成。MLP 通常使用第 7.3 节提到的 GELU 或 SwiGLU 激活函数。本节详细介绍自注意力：从单头自注意力开始，再介绍多头自注意力，以及它的两个变体——多查询注意力和分组查询注意力；随后说明如何把多头自注意力和 MLP 连接为 Transformer 的构件。

因为语言模型的输入是词元序列，模型需要从所有先前词元中收集信息，以预测下一个词元。标准前馈神经网络不容易做到这一点，因为输入维度会随 T 改变。本节介绍的自注意力机制是当今融合输入序列不同位置的信息的核心设计。[70]

单头自注意力。单头自注意力记为 $\mathrm{att}^{1h}(\cdot)$，是一个把向量序列映射为向量序列的函数。输入 $(h_1^{\mathrm{in}}, \dots, h_T^{\mathrm{in}})$ 是嵌入或隐藏激活，输出 $(h_1^{\mathrm{out}}, \dots, h_T^{\mathrm{out}})$ 始终是激活。每个隐藏状态表示为行向量：对所有 $t \in \{1, \dots, T\}$，有 $h_t^{\mathrm{in}} \in \mathbb{R}^{1\times d}$、$h_t^{\mathrm{out}} \in \mathbb{R}^{1\times d_h}$。等价地，把输入输出向量堆叠为矩阵，则 att^{1h} 把输入矩阵 $H^{\mathrm{in}} \in \mathbb{R}^{T\times d}$ 映射为输出矩阵 $H^{\mathrm{out}} \in \mathbb{R}^{T\times d_h}$。注意力层参数为 $W^Q, W^K, W^V \in \mathbb{R}^{d\times d_h}$，通常称为查询、键和值投影矩阵。

单头自注意力的第一步，是把输入向量分别乘以查询、键和值投影矩阵，线性变换为查询、键和值。直观上，可以把查询向量看作在问“我在寻找什么信息？”，键向量看作在声明“我拥有什么信息？”，值向量则提供查询与键匹配时实际向前传递的信息。

$$q_1 = h_1^{\mathrm{in}} W^Q, \dots, q_T = h_T^{\mathrm{in}} W^Q, \tag{17.14}$$

$$k_1 = h_1^{\mathrm{in}} W^K, \dots, k_T = h_T^{\mathrm{in}} W^K, \tag{17.15}$$

$$v_1 = h_1^{\mathrm{in}} W^V, \dots, v_T = h_T^{\mathrm{in}} W^V. \tag{17.16}$$

接着，对序列中的每个位置 t，计算其查询与所有词元的键的内积 $q_t k_1^\top, \dots, q_t k_T^\top$。在不使用任何掩码（后文会介绍这一概念）时，把 softmax 应用于这些内积，就定义了注意力分数 $p_{t,1}, \dots, p_{t,T}$：

$$p_{t,1}, \dots, p_{t,T} = \mathrm{softmax}(q_t k_1^\top / c, \dots, q_t k_T^\top / c), \tag{17.17}$$

其中 c 是缩放因子。这里 q_t 与 k_j 的内积常称为“注意力”，因为它直观上试图确定词元 t 的输出应该在多大程度上依赖或关注词元 j。softmax 把总注意力归一化为 1。

Vaswani 等人 [2017] 采用 $c = \sqrt{d_h}$。这种缩放的简单直觉如下：假设初始化时，查询向量 q 和键向量 k 的各坐标近似为均值 0、方差 1（更一般地为 $O(1)$）的独立随机变量。那么点积

$$q \cdot k = \sum_{i=1}^{d_h} q_i k_i$$

的均值为 0，方差在常数因子意义下为 d_h。若不缩放，注意力 logits 就会随头维度增长；当 d_h 很大时，logits 的绝对值会过大，使注意力分数进入饱和区，导致梯度非常小。除以 $\sqrt{d_h}$ 可以让 logits 的方差保持在 1 的量级。这只是启发式解释，因为实际 Transformer 中查询和键通常并不独立。位置 t 的自注意力输出，是以注意力分数为系数的值向量的线性组合：

$$h_t^{\mathrm{out}} = \sum_{j=1}^{T} p_{t,j} v_j. \tag{17.18}$$

这样，自注意力就按注意力分数混合各位置的值向量。

相同计算可以紧凑地写成矩阵形式。把查询、键和值行向量堆叠为矩阵 $Q, K, V \in \mathbb{R}^{T\times d_h}$：

[70]本节不讨论位置嵌入。它们在实践中很重要：自注意力本身具有置换等变性，因此 Transformer 需要位置讯号来区分词元顺序，例如学习得到的绝对位置嵌入、正弦嵌入、旋转位置嵌入或相关变体。我们将在后续修订中补充讨论。

$$Q = \begin{bmatrix} q_1 \\ \vdots \\ q_T \end{bmatrix} = H^{\text{in}} W^Q, \quad K = \begin{bmatrix} k_1 \\ \vdots \\ k_T \end{bmatrix} = H^{\text{in}} W^K, \quad V = \begin{bmatrix} v_1 \\ \vdots \\ v_T \end{bmatrix} = H^{\text{in}} W^V. \tag{17.19}$$

因此，Q, K, V 的第 t 行分别为 q_t, k_t, v_t。没有掩码（稍后介绍）时，单头注意力可写为

$$H^{\text{out}} = \text{softmax}_{\text{row}}(QK^\top/c)V. \tag{17.20}$$

矩阵形式并不是新模型，而是把同一逐位置规则写成能并行计算所有查询与键两两比较的形式。尽管两两分数的数量是平方级的，这仍是注意力适合现代加速器的一个原因。其中 $\text{softmax}_{\text{row}}$ 对矩阵的每一行（对应一个查询向量）分别应用 softmax。可以验证，矩阵 $\text{softmax}_{\text{row}}(QK^\top/c)$ 的第 t 行恰好就是式（17.17）定义的注意力分数 $p_{t,1}, \ldots, p_{t,T}$，因此 H^{out} 的第 t 行等于式（17.18）定义的 h_t^{out}。

掩码。在自回归 Transformer 中，位置 t 的最终输出，即第 17.2 节定义的 $f_\theta(x_0, \ldots, x_t)$，不应依赖任何 $j > t$ 的 x_j。后文把注意力与 MLP 结合成 Transformer 时会看到，这一自回归性质对应于对注意力模块的要求：位置 t 的输出向量不应依赖任何位置 $j > t$ 的输入向量。为此，修改注意力定义，在 softmax 运算之前引入注意力掩码 $M \in \mathbb{R}^{T \times T}$。掩码使我们能在整个序列上并行训练，同时保留从左到右的预测规则。所有位置可在同一次前向传播中处理，但掩码阻止位置 t 使用自回归预测下一个词元时尚不可见的词元。

$$H^{\text{out}} = \text{softmax}_{\text{row}}(QK^\top/c + M)V. \tag{17.21}$$

最常见的掩码称为因果掩码，专门用来实现自回归性质：$j \le t$ 时 $M_{t,j} = 0$，$j > t$ 时 $M_{t,j} = -\infty$。$-\infty$ 项使对应的 softmax 概率严格为 0，因此每个输出隐藏状态只依赖当前和先前的位置。具体来说，可以验证，第 t 个位置或第 t 行在注意力计算前的 logits 为

$$q_t k_1^\top/c, \ldots, q_t k_t^\top/c, -\infty, -\infty, \ldots, -\infty. \tag{17.22}$$

因此注意力分数为

$$p_{t,1}, \ldots, p_{t,T} = \text{softmax}(q_t k_1^\top/c, \ldots, q_t k_t^\top/c, -\infty, -\infty, \ldots, -\infty), \tag{17.23}$$

对所有 $j > t$ 都有 $p_{t,j} = 0$。位置 t 的最终输出为

$$h_t^{\text{out}} = \sum_{j=1}^{t} p_{t,j} v_j, \tag{17.24}$$

它不依赖第 t 步之后的任何信息。注意力掩码还有其他用途，例如在批处理序列中忽略填充词元，或当输入的不同部分不应相互关注时施加分块结构的可见性约束。后续章节需要时会讨论这些用法。

（多头）注意力。一个注意力层通过组合多个单头注意力模块，并行使用多个注意力头。具体而言，注意力输出是把多个单头自注意力输出拼接后进行线性变换的结果。假设有 n_h 个注意力头，更准

确地说，有 n_h 个单头注意力模块，记为 $\text{att}_1^{1h}, \ldots, \text{att}_{n_h}^{1h}$。实践中通常选择满足 $n_h d_h = d$ 的 n_h, d_h。回忆这些模块的输出为 $\text{att}_1^{1h}(h_1, \ldots, h_T), \ldots, \text{att}_{n_h}^{1h}(h_1, \ldots, h_T)$，每个都是向量序列。设 $\text{att}_j^{1h}(h_1, \ldots, h_T)_t$ 表示第 j 个模块输出中的第 t 个向量，令 z_t 是 $\text{att}_1^{1h}(h_1, \ldots, h_T)_t, \ldots, \text{att}_{n_h}^{1h}(h_1, \ldots, h_T)_t$ 的拼接，把它视为 $n_h d_h$ 维行向量。多头自注意力再引入输出权重矩阵 $W^O \in \mathbb{R}^{(n_h d_h) \times d}$，将其作用于 z_t，得到多头注意力的第 t 个输出。精确地说，

$$\text{att}(h_1, \ldots, h_T) = (z_1 W^O, \ldots, z_T W^O). \tag{17.25}$$

为明确起见，具有 n_h 个头的注意力层，输入是 T 个 d 维向量的序列，输出也是 T 个 d 维向量的序列。多个头使该层能并行运行若干注意力规则。不同头可以专门处理不同类型的词元交互，再由输出投影把这些不同视角重新组合为各位置上的一个 d 维表示。

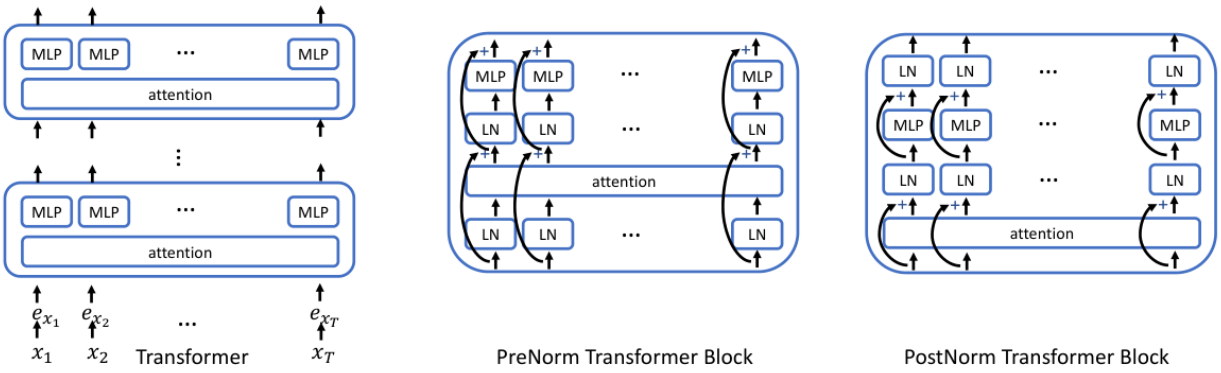


图 17.3 Transformer 和 Transformer 模块的结构。左：Transformer；中：PreNorm 模块；右：PostNorm 模块。

自回归生成与 KV 缓存。式（17.21）描述所有词元都可用时的模型行为。以下情况需要这种运算：（1）训练模型时，词元已经给定（来自预训练数据集，或如强化学习那样由模型上一次迭代自行生成），我们希望计算当前模型权重生成给定词元序列的似然；（2）词元序列是用户的提示。

要从模型生成新词元，就必须逐个生成 x_t，再按式（17.7）将生成结果回送给 Transformer。注意式（17.7）中的计算共享前缀。例如，当 $j < k$ 时，$f_\theta(x_0, \ldots, x_j)$ 和 $f_\theta(x_0, \ldots, x_k)$ 都涉及计算位置 j 及之前的键和值，因此可以缓存这些计算结果，无需重新计算。这些缓存的键和值向量集合称为 KV 缓存。特别地，在时刻 t，运行中的 KV 缓存是截至 t 的所有键和值：

$$K_{1:t} = \begin{bmatrix} k_1 \\ \vdots \\ k_t \end{bmatrix}, \qquad V_{1:t} = \begin{bmatrix} v_1 \\ \vdots \\ v_t \end{bmatrix}, \tag{17.26}$$

第 t 个位置的注意力输出可计算为

$$h_t^{\text{out}} = \text{softmax}\left(\frac{q_t K_{1:t}^\top}{c}\right) V_{1:t}. \tag{17.27}$$

随着 t 增大，KV 缓存分别追加新行 k_{t+1}, v_{t+1}，但已缓存的键和值不需要重新计算。这避免了在每一步解码时重新计算整个前缀的键和值。上面只描述单个注意力头的缓存；标准多头注意力中，每个头都有一组需要保存在 GPU 显存中的键和值。

Transformer。Transformer 主要由注意力和 MLP 交替组合而成；MLP 以共享权重分别作用于每个

位置，如图 17.3 所示。

如何组合这些部分，有一些具体选择。Transformer 中的一层通常指注意力层与 MLP 层的组合。设 $(h_1^\ell, \dots, h_T^\ell)$ 是进入第 ℓ 层的行向量隐藏状态，其中 $h_t^\ell \in \mathbb{R}^{1\times d}$；$(h_1^{\ell+1}, \dots, h_T^{\ell+1})$ 是离开该层的隐藏状态。主要有 PostNorm 和 PreNorm 两种结构。PreNorm 在 MLP 和自注意力之前应用层归一化：

$$r_t = h_t^\ell + [\text{att}(\text{LN}(h_1^\ell), \dots, \text{LN}(h_T^\ell))]_t \tag{17.28}$$

$$h_t^{\ell+1} = r_t + \text{MLP}(\text{LN}(r_t)). \tag{17.29}$$

PostNorm 则在残差连接之后应用层归一化：

$$r_t = \text{LN}(h_t^\ell + [\text{att}(h_1^\ell, \dots, h_T^\ell)]_t) \tag{17.30}$$

$$h_t^{\ell+1} = \text{LN}(r_t + \text{MLP}(r_t)). \tag{17.31}$$

图 17.3 展示了 PreNorm 和 PostNorm 的 Transformer 模块结构。现代大语言模型通常使用 PreNorm，并用 RMSNorm 替代层归一化。

存储与计算。下面讨论注意力的存储和计算成本。

计算。训练和提示预填充时，所有词元已给定，每个注意力头计算式（17.21）中的掩码注意力，需要 $O(T^2 d_h)$ 次算术运算。对于 n_h 个头的多头注意力，n_h 次式（17.21）需要 $O(T^2 d_h n_h)$ 次运算；式（17.25）的输出投影则需要 $O(T n_h d_h d)$ 次运算，当 T 较大时，该项较小，不占主导。关键结论是：注意力计算关于 T 为平方复杂度；这是长上下文场景面临困难的主要来源，在这些场景中 T 大于 1 万，有时高达 100 万甚至更多。

生成时，生成 T 个词元所需注意力运算的总量仍为同一量级，但必须逐个词元顺序生成。这往往更难充分利用 GPU 的并行能力，达到最佳利用率。

存储。注意力的 GPU 显存使用量取决于具体实现。训练和预填充时，式（17.21）的朴素实现每个头需要 T^2 的存储，因为注意力分数矩阵 $QK^\top$ 有 T^2 个元素。然而，FlashAttention [Dao et al., 2022] 等方法可让注意力计算的显存用量关于 T 呈线性增长；本质上，它保存 Q, K, V，并通过一系列等价运算重新计算或流式处理注意力分数，而不显式构造完整的 $QK^\top$ 矩阵。

生成和解码时，KV 缓存需要保存在 GPU 显存中，对于长度为 T 的上下文，每个头需要 $O(T d_h)$ 存储；除非愿意把部分 KV 缓存移到相对较慢的 CPU 内存。

下一节讨论通过改变注意力架构来提高效率的方法。

17.4 注意力的变体

GQA、MQA 与滑动窗口注意力。若干注意力变体可以减小 KV 缓存的存储量。多查询注意力（MQA）保留多个查询头，但让它们共享一个键头和一个值头 [Shazeer, 2019]。分组查询注意力（GQA）是介于两者之间的设计：把查询头分组，每组共享一个键头和一个值头 [Ainslie et al., 2023]。因此，普通多头注意力、GQA 和 MQA 构成了在模型质量与解码效率之间权衡的一系列方案：键值头越少，KV 缓存越小。

数学上，GQA 意味着第 t 个位置有 n_h 个查询 $q_{t,1},\dots,q_{t,n_h}$，以及 n_g 个键和值 $k_{t,1},\dots,k_{t,n_g}$、$v_{t,1},\dots,v_{t,n_g}$，其中 n_g 整除 n_h。令 $\tau = n_h/n_g$ 为分配给每个键值头的查询头数量。对查询头 $j \in \{1,\dots,n_h\}$，定义组索引

$$g(j) = \left\lfloor \frac{j-1}{\tau} \right\rfloor + 1 \in \{1,\dots,n_g\}. \tag{17.32}$$

于是查询头 j 使用键值头 $g(j)$。对 $j \in \{1,\dots,n_h\}$，令 Q_j 是第 t 行为 $q_{t,j}$ 的矩阵。对 $s \in \{1,\dots,n_g\}$，令 K_s, V_s 是第 t 行分别为 $k_{t,s}, v_{t,s}$ 的矩阵：

$$Q_j = \begin{bmatrix} q_{1,j} \\ \vdots \\ q_{T,j} \end{bmatrix}, \qquad K_s = \begin{bmatrix} k_{1,s} \\ \vdots \\ k_{T,s} \end{bmatrix}, \qquad V_s = \begin{bmatrix} v_{1,s} \\ \vdots \\ v_{T,s} \end{bmatrix}. \tag{17.33}$$

第 j 个输出头记为 H_j^{out}，按下式计算：

$$H_j^{\text{out}} = \text{softmax}_{\text{row}}\left(\frac{Q_j K_{g(j)}^\top}{c} + M \right) V_{g(j)}, \tag{17.34}$$

其中 M 是注意力掩码，例如自回归解码中的因果掩码。标准多头注意力是 $n_g = n_h$ 的特例，而 MQA 是 $n_g = 1$ 的特例。

另一种方法是滑动窗口注意力：每个词元只关注最近的 w 个词元，而非整个前缀。这对应因果局部注意力掩码

$$M_{t,j} = \begin{cases} 0, & \max(1, t-w+1) \le j \le t, \\ -\infty, & \text{其他情况}, \end{cases} \tag{17.35}$$

所以窗口之前的键和值不影响当前输出，无需继续保留在活跃的 KV 缓存中。

查询—键归一化。另一个注意力变体是查询—键归一化，即 QK-Norm [Henry et al., 2020]。标准注意力中的 logits 是点积 $q_t k_j^\top / c$，所以其大小取决于查询和键向量的范数。如果这些范数变大，softmax 分布可能变得非常尖锐或饱和。QK-Norm 在求点积前先归一化每个查询和键向量，例如用缩放后的余弦相似度 $\text{RMSNorm}(q_t)\,\text{RMSNorm}(k_j)^\top$ 替换 $q_t k_j^\top$，其中 RMSNorm 由式（7.51）定义。其动机是控制注意力 logits 的尺度、提高训练稳定性，同时仍允许模型学习注意力分布的整体尖锐程度。若干近期开放权重模型采用了这种方法，例如 Qwen3 和 OLMo2 [Yang et al., 2025, OLMo Team et al., 2024]。

17.5 混合专家层

增大 Transformer 规模的另一种方式，是让某些层变得稀疏。最常见的例子是混合专家（Mixture-of-Experts，MoE）层。普通 Transformer 模块中的 MLP 是稠密模块：每个词元都由同一个 MLP 处理，因此使用相同参数，并产生相同的 MLP 计算量。在 MoE 模块中，模型改用多个称为“专家”的 MLP，由一个小型路由网络决定哪些专家处理每个词元。

具体而言，设该层有专家 $E_1,\dots,E_m$，每个 E_s 都是 MLP。给定词元 t 的隐藏状态 h_t，路由器计算路由权重

$$r_t = \sigma(h_t W^R) \in \mathbb{R}^m,$$

其中 σ 是路由函数，通常是对所有专家做 softmax，或对每个专家分数分别做 sigmoid。随后，模型选择一个较小的专家集合 S_t，其大小通常为常数 k，并组合其输出：

$$\mathrm{MoE}(h_t) = \sum_{s \in S_t} \alpha_{t,s} E_s(h_t), \qquad \alpha_{t,s} = \frac{r_{t,s}}{\sum_{j \in S_t} r_{t,j}}.$$

例如，DeepSeek-V3 的每个 MoE 层有 256 个路由专家，对每个词元激活排名最高的 8 个路由专家，此外还有一个共享专家 [DeepSeek-AI, 2024]。共享专家应用于每个词元，提供公共的前馈通路；路由专家则提供依词元而定的专门化处理。因此，虽然层中包含大量专家参数，每个词元只激活少量专家。这是 MoE 的主要吸引力：它增加模型总参数量和表示能力，却不同比例地增加每个词元的计算量。可以把路由器理解为选择最适合当前词元及上下文的专门化 MLP。MoE 层通常替代 Transformer 模块中部分或全部稠密 MLP 层。

17.6 上下文学习

上下文学习主要用于少样本情形：我们只有少量带标签样本 $(x_{\text{task}}^{(1)}, y_{\text{task}}^{(1)}), \ldots, (x_{\text{task}}^{(n_{\text{task}})}, y_{\text{task}}^{(n_{\text{task}})})$。给定测试样本 x_{test}，把这些带标签样本和测试样本按某种格式拼接，构造提示 $(x_1, \ldots, x_T)$。例如，可构造如下提示：

$$\begin{aligned} x_1, \ldots, x_T = \text{“Q: } 2 \sim 3 = ? \quad & x_{\text{task}}^{(1)} \\ \text{A: } 5 \quad & y_{\text{task}}^{(1)} \\ \text{Q: } 6 \sim 7 = ? \quad & x_{\text{task}}^{(2)} \\ \text{A: } 13 \quad & y_{\text{task}}^{(2)} \\ \cdots & \\ \text{Q: } 15 \sim 2 = ?\text{”} \quad & x_{\text{test}}. \end{aligned}$$

然后，让预训练模型生成最可能的 $x_{T+1}, x_{T+2}, \ldots$。如果模型能够从少量示例中 “学会” 符号 ~ 表示加法，就会得到如下结果，表明答案为 17：

$$x_{T+1}, x_{T+2}, \ldots = \text{“A: 17”}.$$

同样的思路可以用于更实际的提示。例如，假设希望语言模型对客户支持消息进行分类，同时按固定的机器可读格式返回答案。可以给模型如下示例：

任务：把每条消息分类为 `{billing, technical, account}`。返回 `JSON`。

消息：我的订阅被重复扣费了。
输出：`{"label": "billing"}`

消息：上传文件时应用会崩溃。
输出：`{"label": "technical"}`

消息：请修改我个人资料中的电子邮箱地址。
输出：`{"label": "account"}`

消息：我无法重置密码。
输出：

期望的续写是 `{"label": "account"}` 这样的 JSON 对象。这个例子中的上下文示例有两个作用：明确任务和标签空间，并教会模型输出格式。更复杂的提示还可以演示工作流程，例如先提取相关事实，再产生最终答案。Brown 等人 [2020] 使上下文学习作为大语言模型的一项能力广为人知；后续工作研究了示例提供什么信息，例如输入分布、标签空间和格式惯例 [Min et al., 2022]，以及演示中间推理步骤如何改善某些推理任务 [Wei et al., 2022]。

17.7 零样本学习／提示

对于自回归语言模型，最轻量的适配方式是提示：改变提供给模型的文本，不改变模型参数。上一节的上下文学习示例就是提示的一种形式。零样本情形中，没有下游任务的输入输出示例，所以提示通常用自然语言描述任务，并要求模型按指定格式产生答案。例如，可以问：

$$\begin{aligned} x_{\text{task}} &= (x_{\text{task},1}, \dots, x_{\text{task},T}) \\ &= \text{“问题：光速是一个普适常数吗？回答是或否。”} \end{aligned}$$

然后从 $p_\theta(\cdot \mid x_{\text{task}})$ 解码续写。如果模型续写 “No”，它就给出了此提示期望的标签。[71] 更一般地，输出可以采用贪心解码、采样、约束在标签集合中，或从较长的生成答案中解析出来。提示之所以有吸引力，是因为它不需要训练数据或梯度更新；但性能可能取决于预训练模型和提示措辞。

17.8 监督微调（SFT）

微调则通过在下游样本上更新参数来适配模型。对于自回归语言模型，通常不需要添加新的预测头。每个监督样本可格式化为提示—补全对 $(x^{(i)}, y^{(i)})$，其中 $x^{(i)}$ 由词元序列 $x_1^{(i)}, \dots, x_{L_i}^{(i)}$ 组成，$y^{(i)}$ 由词元序列 $y_1^{(i)}, \dots, y_{T_i}^{(i)}$ 组成。把 θ 初始化为预训练参数 $\hat{\theta}$，然后继续最小化条件下一个词元损失：

$$\min_\theta \frac{1}{n} \sum_{i=1}^{n} \left[-\sum_{t=1}^{T_i} \log p_\theta(y_t^{(i)} \mid x^{(i)}, y_{<t}^{(i)}) \right]. \tag{17.36}$$

例如，对分类任务，$y^{(i)}$ 可以只是类别标签的文本，或包含该标签的简短 JSON 对象。这里 $x^{(i)}$ 是问题、指令（见下段）或用户提示，$y^{(i)}$ 是期望答案或助手回答。一种常见实现把提示和回答拼接成一个因果语言模型序列：

$$z^{(i)} = (x_1^{(i)}, \dots, x_{L_i}^{(i)}, y_1^{(i)}, \dots, y_{T_i}^{(i)}),$$

但只在回答位置应用下一个词元损失。等价地，对于小批量 B，使用损失掩码 $m_r^{(i)}$，提示词元处取 0，回答词元处取 1，然后最小化小批量中所有回答词元位置上的平均损失：

[71] 译注：此处 “No” 按原文保留。原文用该例说明输出标签的格式，并未限定 “光速” 是否指真空中的光速。

$$-\frac{1}{\sum_{i\in B}\sum_{r=1}^{L_i+T_i} m_r^{(i)}}\sum_{i\in B}\sum_{r=1}^{L_i+T_i} m_r^{(i)}\log p_\theta(z_r^{(i)} \mid z_{<r}^{(i)}). \tag{17.37}$$

损失掩码与注意力掩码不同：仍使用通常的因果注意力掩码，因此回答词元可以关注提示及之前的回答词元，但提示词元本身不对 SFT 目标作出贡献。

指令微调。指令微调是微调的一种重要特例。这里的指令是自然语言任务描述，例如 “用一句话总结以下段落” “把这条客户消息分类为账单、技术或账户相关问题”，或 “编写一个对列表排序的 Python 函数”。我们不只在一个下游任务上训练，而是收集许多以自然语言指令及期望回答写成的任务，再在它们的混合数据上微调模型 [Wei et al., 2021, Ouyang et al., 2022]。目标不仅是拟合这些训练任务，还要教会模型遵循一个惯例：提示中的指令应当被执行。这就是指令微调后的模型通常能成为更好的零样本助手的原因：测试时，用户可以用文字描述新任务，而模型已受训把这种描述当作可执行的指令。

第 18 章　大语言模型中的推理

原文印刷页码：220–225。

18.1 思维链

动机。考虑一个小学算术例子：Roger 原有 5 个网球，又买了 2 罐网球，每罐 3 个。求解时，我们自然会分步骤进行：先计算新买的球数，再加上原来的 5 个球：

$$z: \quad 2\times 3 = 6, \qquad 5 + 6 = 11,$$

因此最终答案为 $a = 11$。

典型的大语言模型提示只会要求模型解决问题。思维链（chain-of-thought，CoT）提示则要求模型先给出中间计算，再给出答案。

其动机是：许多推理任务用一小串中间步骤加上答案来表达，比直接进行一次预测更容易。推理轨迹让模型有更多时间思考（更多计算），也提供了记录中间结果的更多存储空间。这些额外词元之所以有用，是因为自回归模型可以把先前的部分计算作为后续预测的条件，从而把一个困难预测转化为一系列较小的预测。

少样本思维链。在少样本 CoT 中，每个上下文示例不仅包含输入和答案，还包含用自然语言写出的中间解答。提示的形式为

$$(x^{(1)}, z^{(1)}, a^{(1)}), \ldots, (x^{(m)}, z^{(m)}, a^{(m)}), x,$$

我们期望模型接着为问题 x 生成新的轨迹 z 和答案 a。这相当于用提示要求模型 “展示解题过程”。Wei 等人 [2022] 表明：对于足够大的预训练语言模型，与只包含最终答案的标准少样本提示相比，少样本 CoT 能显著改善算术、符号和常识推理。

零样本思维链。零样本 CoT 去掉手写示例，改用鼓励生成中间轨迹的简短指令。典型做法是在要求回答前附上 “让我们一步一步地思考” 之类的短语 [Kojima et al., 2022]。当编写优质的少样本推理说明成本很高，或希望同一个提示模板能迁移到许多任务类型时，这种方法很有用。

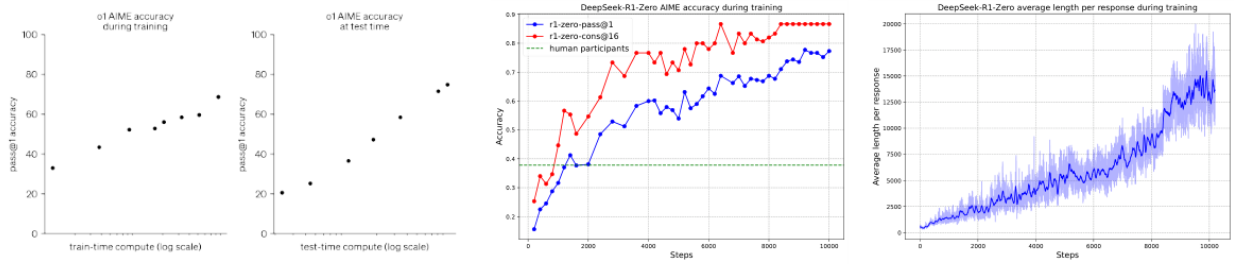
18.2 采用长思维链推理的 RLVR

上一节把思维链描述为一种推断时的格式：通过提示，让模型先写出中间步骤，再回答问题。如果这种格式能使预测更容易，那么自然会想到：能否通过训练让模型产生有用的推理轨迹，而不只是用提示把这种能力引导出来？挑战在于，高质量的逐步监督成本很高。不过在很多领域，最

终答案可以自动检查：数学题可能有已知答案，编程题可能有单元测试，定理证明任务可能有证明检查器，而结构化输出任务可能有确定性的验证器。

可验证奖励强化学习（reinforcement learning with verifiable rewards，RLVR）恰好针对这一情形进行训练。模型采样出包含思维链和最终答案的补全，然后自动验证器根据最终答案是否通过检查来给予奖励。这使 RLVR 尤其适合长思维链推理：监督整条轨迹很困难，但验证最终结果相对容易。

OpenAI 的 o1 报告表明，推理准确率会随着强化学习训练时计算量和测试时思考计算量的增加而提高 [OpenAI, 2024]。随后，DeepSeek-R1 给出了更详细的公开方案：DeepSeek-R1-Zero 在类似 RLVR 的训练过程中不断改善，并随训练推进产生更长的推理轨迹 [DeepSeek-AI, 2025]。这一经验现象如今常被称为推理模型的规模扩展：性能不仅取决于预训练规模，也取决于训练和采样显式推理轨迹所投入的计算量。图 18.1 展示了这两个经验性的扩展维度。



(a) OpenAI o1 scaling.　　(b) DeepSeek-R1-Zero RL trajectory.

图 18.1 推理模型的两个计算维度。左为 OpenAI o1 的规模扩展，右为 DeepSeek-R1-Zero 的强化学习轨迹。

图 18.1：推理模型的两个计算维度。o1 报告绘制了 AIME pass@1 准确率随额外强化学习训练计算量和额外测试计算量变化的曲线。DeepSeek-R1 报告绘制了 DeepSeek-R1-Zero 对应的强化学习训练轨迹，其准确率与模型回答的平均长度一同上升。这些图共同说明，我们可以把长思维链推理既视为推断时可投入的资源，也视为可以训练的行为。来源：OpenAI [OpenAI, 2024] 和 DeepSeek-AI [DeepSeek-AI, 2025]。

形式化。设 $x \sim \mathcal{D}$ 是问题提示，$y = (y_1, \dots, y_T)$ 是给定 x 后的回答，其中可以同时包含思维链和最终答案。验证器产生标量奖励 $R(x, y) \in [0, 1]$。例如，如果提取出的最终答案正确，就令 $R(x, y) = 1$，否则为 0；也可以额外施加格式惩罚。

对于固定提示 x，可以把自回归生成视为一个有限时域马尔可夫决策过程（MDP，见第 21 章）。在词元位置 t，也就是 MDP 的时间步 t，状态是前缀

$$s_t = (x, y_{<t}),$$

动作是下一个词元 $a_t = y_t$，按策略采样；该策略就是下一个词元的概率：

$$\pi_\theta(a_t \mid s_t) = p_\theta(y_t \mid x, y_{<t}).$$

选择 y_t 后，转移是确定性的：

$$s_{t+1} = (x, y_{\le t}).$$

当模型输出序列结束词元或达到长度限制时，这一回合结束。验证器提供终端奖励 $R(x, y)$。因此，唯一非零的任务奖励出现在生成补全的末尾。奖励目标为

$$J_R(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}[R(x, y)]. \tag{18.1}$$

一种常见的正则化目标从奖励中减去 KL 惩罚，以鼓励策略不要偏离参考策略太远：

$$J_\beta(\theta) = J_R(\theta) - \beta\, \mathbb{E}_{x\sim\mathcal{D}}[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid x)\|\pi_{\mathrm{ref}}(\cdot \mid x))], \tag{18.2}$$

训练问题是最大化 $J_\beta(\theta)$。这里 π_{ref} 通常是初始的监督微调或指令微调模型。KL 项阻止策略偏离一个能力良好的语言模型太远，而验证器奖励则推动它生成能解决任务的补全。

奖励和验证器。最简单的验证器提取最终答案，并与真实答案比较。对于代码，验证器可以运行单元测试；对于符号任务，它可以调用解析器、定理检查器或代数系统。实践中，奖励通常还会检查答案是否遵循要求的格式。

策略梯度。利用上述 MDP 表述，把第 21 章的策略梯度恒等式应用于奖励项（式（18.1）中的 J_R），得到

$$\nabla_\theta J_R(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\left[\sum_{t=1}^{T}(R(x, y) - b_t)\nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t})\right], \tag{18.3}$$

其中 b_t 是序列级基线。因此，如果采样出的思维链获得的奖励高于基线，更新就会增加该补全中词元的概率；如果奖励低于基线，则会减小这些概率。

PPO。通用 PPO 更新见第 21.2 节。在上述语言模型 MDP 中，PPO 应用于词元状态

$$s_t^{(i)} = (x, y_{<t}^{(i)}), \qquad a_t^{(i)} = y_t^{(i)}.$$

假设已从旧策略 $\pi_{\theta_{\mathrm{old}}}(\cdot \mid x)$ 中采样出一批补全 $y^{(1)}, \ldots, y^{(G)}$，其中 $y^{(i)} = (y_1^{(i)}, \ldots, y_{T_i}^{(i)})$。下文用 $\mathbb{E}_{i,t}$ 表示对采样补全及词元位置的经验平均：

$$\mathbb{E}_{i,t}[\cdot] = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_i}\sum_{t=1}^{T_i}[\cdot].$$

当小批量中包含多个提示时，这个记号还包括对提示的经验平均。定义词元级似然比

$$r_t^{(i)}(\theta) = \frac{\pi_\theta(y_t^{(i)} \mid x, y_{<t}^{(i)})}{\pi_{\theta_{\mathrm{old}}}(y_t^{(i)} \mid x, y_{<t}^{(i)})}. \tag{18.4}$$

给定词元优势 $\widehat{A}_t^{(i)}$，词元级裁剪替代目标为

$$C_t^{(i)}(\theta) = \min\left\{r_t^{(i)}(\theta)\widehat{A}_t^{(i)},\ \mathrm{clip}_{[1-\epsilon_{\mathrm{clip}},1+\epsilon_{\mathrm{clip}}]}(r_t^{(i)}(\theta))\widehat{A}_t^{(i)}\right\}, \tag{18.5}$$

相应的词元级 PPO 目标是

$$\hat{J}^{\mathrm{LM}}_{\mathrm{PPO}}(\theta) = \mathbb{E}_{i,t}[C_t^{(i)}(\theta)]. \tag{18.6}$$

更新要最大化 $\hat{J}^{\mathrm{LM}}_{\mathrm{PPO}}(\theta)$。裁剪项防止新策略在一次更新中，赋予任意采样词元大得多或小得多的概率。

GRPO。群组相对策略优化（Group Relative Policy Optimization，GRPO）由 DeepSeekMath [Shao et al., 2024] 提出，随后用于 DeepSeek-R1 [DeepSeek-AI, 2025] 等推理模型的训练方案。它将 PPO 模板中学习得到的价值模型替换为群组相对基线。对于每个提示 x，从旧策略 $\pi_{\theta_{\mathrm{old}}}(\cdot \mid x)$ 采样一组 G 个补全 $y^{(1)}, \ldots, y^{(G)}$，并定义 $R_i = R(x, y^{(i)})$。群组统计量给出序列级优势：

$$\widehat{A}_i = \frac{R_i - \overline{R}}{s_R + \epsilon}, \qquad \overline{R} = \mathbb{E}_j[R_j], \qquad s_R^2 = \mathbb{E}_j[(R_j - \overline{R})^2], \tag{18.7}$$

其中 $\mathbb{E}_j$ 表示对同一提示下的 G 个补全取经验平均。GRPO 对补全 i 中的每个词元设置 $\widehat{A}_t^{(i)} = \widehat{A}_i$，并在词元级应用 PPO 式裁剪：

$$J_{\mathrm{GRPO}}(\theta) = \mathbb{E}_{i,t}[C_t^{(i)}(\theta)], \tag{18.8}$$

其中 $r_t^{(i)}$ 由式（18.4）定义，$C_t^{(i)}$ 是式（18.5）中的词元裁剪替代目标。更新最大化 $J_{\mathrm{GRPO}}(\theta)$。裁剪表达式与 PPO 相同；关键区别在于，GRPO 通过比较同一问题的多个答案获得优势，而不是训练单独的评价器。

CISPO。 MiniMax-M1 [MiniMax, 2025] 提出的裁剪重要性采样策略优化（Clipped Importance Sampling Policy Optimization，CISPO），针对长推理链修改了这类更新。在 PPO 式裁剪中，概率比变化非常大的词元，其梯度可能会被裁剪替代目标抑制。一个简化的 CISPO 式词元目标，改为裁剪作为系数的重要性权重，同时仍通过对数概率项求梯度：

$$L_{\mathrm{CISPO}} = -\mathbb{E}_{i,t}\left[\mathrm{stopgrad}\Big(\min\{r_t^{(i)}(\theta), \epsilon_{\mathrm{high}}\}\Big)\widehat{A}_i \log \pi_\theta(y_t^{(i)} \mid x, y_{<t}^{(i)})\right]. \tag{18.9}$$

这里 $r_t^{(i)}(\theta)$ 由式（18.4）定义，期望对采样补全及词元位置取平均。CISPO 裁剪重要性采样权重，同时为每个采样词元保留直接的对数概率梯度，并使用与 GRPO 相同的序列级优势 $\widehat{A}_i$。[72]

[72]这里给出的是单侧 CISPO 版本。MiniMax-M1 论文还给出了双侧裁剪重要性采样版本，把重要性权重限制在上下界之间；细节参见原论文。但在实验中，作者没有施加下界，而只调节上界，因此这里为简洁起见介绍单侧版本。

第六部分 强化学习与控制

第 19 章　强化学习

原文印刷页码：226–243。

现在开始研究强化学习与自适应控制。

在监督学习中，算法试图让输出模仿训练集给定的标签 y。对于每个输入 x，这些标签提供明确的“正确答案”。相比之下，许多序贯决策与控制问题很难向学习算法提供这种显式监督。例如，刚造好一台四足机器人，想编程让它行走时，最初并不知道什么动作才是使它行走的“正确”动作，因此也不知道如何提供供算法模仿的显式监督。

在强化学习框架中，我们只给算法提供奖励函数，用它告诉学习智能体什么时候表现好、什么时候表现差。在四足行走的例子中，奖励函数可以在机器人向前移动时给正奖励，在向后移动或摔倒时给负奖励。学习算法的任务就是找出如何随时间选择动作，以获得较大奖励。

强化学习已成功应用于自主直升机飞行、机器人足式运动、移动电话网络路由、营销策略选择、工厂控制、高效网页索引等多种领域。我们从马尔可夫决策过程（MDP）的定义开始；它是通常用于表述强化学习问题的形式化框架。

19.1 马尔可夫决策过程

马尔可夫决策过程是元组 $(S, A, \{P_{sa}\}, \gamma, R)$，其中：

- S 是状态集合。例如，自主直升机飞行中的所有可能位置和姿态。
- A 是动作集合。例如，操纵直升机控制杆的所有可能方向。
- P_{sa} 是状态转移概率。对每个状态 $s \in S$ 和动作 $a \in A$，P_{sa} 是状态空间上的分布。稍后会进一步说明；简言之，它给出在状态 s 采取动作 a 后会转移到哪些状态的分布。
- $\gamma \in [0, 1)$ 称为折扣因子。
- $R : S \times A \mapsto \mathbb{R}$ 是奖励函数。奖励有时也只写成状态的函数，此时 $R : S \mapsto \mathbb{R}$。

MDP 如下演化：从某个状态 s_0 开始，选择动作 $a_0 \in A$。因这一选择，MDP 随机转移到按 $s_1 \sim P_{s_0 a_0}$ 采样的后继状态 s_1。随后再选动作 a_1，状态转移为 $s_2 \sim P_{s_1 a_1}$；接着选 a_2，以此类推。图示为

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} \cdots .$$

用动作 $a_0, a_1, \ldots$ 访问状态序列 $s_0, s_1, \ldots$ 后，总收益为

$$R(s_0, a_0) + \gamma R(s_1, a_1) + \gamma^2 R(s_2, a_2) + \cdots .$$

若奖励只依赖状态，则变为

$$R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots .$$

下面大部分推导采用较简单的状态奖励 $R(s)$；推广到状态—动作奖励 $R(s,a)$ 没有特别的困难。

强化学习的目标是随时间选择动作，使总收益的期望最大：

$$\mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots].$$

时间步 t 的奖励乘有折扣因子 γ^t。因此，要使期望较大，我们希望尽早积累正奖励，并尽量推迟负奖励。在 $R(\cdot)$ 表示赚到的钱的经济应用中，γ 还有与利率相关的自然解释：今天的一美元比明天的一美元更值钱。

策略是任意把状态映射到动作的函数 $\pi : S \mapsto A$。如果每当处于状态 s 时都采取动作 $a = \pi(s)$，就称为执行策略 π。策略的**价值函数**定义为

$$V^\pi(s) = \mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots \mid s_0 = s, \pi].$$

它就是从 s 开始并依照 π 行动时，折扣奖励总和的期望。[73]

给定固定策略 π，其价值函数满足贝尔曼方程：

$$V^\pi(s) = R(s) + \gamma \sum_{s' \in S} P_{s\pi(s)}(s') V^\pi(s').$$

这表示从 s 开始的期望折扣奖励总和分为两项：一是仅因起始于状态 s 就立即获得的奖励 $R(s)$；二是未来折扣奖励总和的期望。仔细看第二项，其中的和可写为 $\mathbb{E}_{s' \sim P_{s\pi(s)}}[V^\pi(s')]$：这是从 s' 开始的期望折扣奖励总和，而 s' 服从 $P_{s\pi(s)}$，即从 s 执行第一个动作 $\pi(s)$ 后到达状态的分布。因此第二项给出 MDP 第一步之后所获折扣奖励的期望总和。

贝尔曼方程能高效求出 V^π。具体地，在有限状态 MDP（$|S| < \infty$）中，可对每个状态 s 的 $V^\pi(s)$ 写出一个方程。这就得到含 $|S|$ 个变量（每个状态对应一个未知的 $V^\pi(s)$）的 $|S|$ 个线性方程，可以高效求解。

最优价值函数定义为

$$V^*(s) = \max_\pi V^\pi(s). \tag{19.1}$$

它表示使用任意策略能达到的最佳期望折扣奖励总和。最优价值函数也有相应的贝尔曼方程：

$$V^*(s) = R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') V^*(s'). \tag{19.2}$$

第一项仍是即时奖励。第二项是在所有动作 a 中，选择执行后未来期望折扣奖励总和最大的那个。请确保理解这个方程以及它为什么合理。

再定义策略 $\pi^* : S \mapsto A$：

[73] 严格来说，以 π 为条件的记号并不准确，因为 π 不是随机变量，但文献中普遍这样使用。

$$\pi^*(s) = \arg\max_{a \in A} \sum_{s' \in S} P_{sa}(s') V^*(s'). \tag{19.3}$$

$\pi^*(s)$ 给出使式（19.2）中 “max” 达到最大值的动作。事实上，对每个状态 s 和每个策略 π，都有

$$V^*(s) = V^{\pi^*}(s) \geq V^{\pi}(s).$$

第一个等号说明：对所有状态，π^* 的价值函数都等于最优价值函数；不等号则说明 π^* 的价值至少与任何其他策略一样大。换言之，式（19.3）定义的 π^* 是最优策略。

π^* 有一个有趣性质：它同时对所有状态最优。并不是从某个 s 开始有一个最优策略，从另一个 s' 开始又有另一个最优策略。同一个 π^* 对所有状态都达到式（19.1）的最大值。因此，无论 MDP 的初始状态是什么，都可以使用同一策略 π^*。

19.2 价值迭代与策略迭代

现在介绍两个高效求解有限状态 MDP 的算法。暂时只考虑有限状态与动作空间，即 $|S| < \infty, |A| < \infty$，并假设已知状态转移概率 $\{P_{sa}\}$ 和奖励函数 R。

第一个算法是价值迭代。

算法 4：价值迭代

1. 对每个状态 s，初始化 $V(s) := 0$。
2. 重复直到收敛：对每个状态更新

$$V(s) := R(s) + \max_{a \in A} \gamma \sum_{s'} P_{sa}(s') V(s'). \tag{19.4}$$

这可以理解为反复用贝尔曼方程（19.2）更新价值函数估计。

内层循环有两种更新方式。一种先计算所有状态的新 $V(s)$，再统一覆盖旧值，称为**同步更新**。此时算法相当于实现了 “贝尔曼回溯算子”，将当前价值函数估计映射为一个新估计，详情见作业题。另一种是**异步更新**：按某种次序遍历状态，逐个更新其价值。

无论同步还是异步，都可以证明价值迭代会使 V 收敛到 V^*。求得 V^* 后，使用式（19.3）即可找到最优策略。

除了价值迭代，还有第二种求 MDP 最优策略的标准算法——策略迭代。

算法 5：策略迭代

1. 随机初始化 π。
2. 重复直到收敛：令 $V := V^{\pi}$（通常用线性方程组求解器）；随后，对每个状态 s，令

$$\pi(s) := \arg\max_{a \in A} \sum_{s'} P_{sa}(s') V(s').$$

即在内层循环中反复计算当前策略的价值函数，再用当前价值函数更新策略。第二步所得策略也称为相对于 V 的贪心策略。计算 V^{π} 可按前述方法求解贝尔曼方程：固定策略时，它只是 $|S|$ 个变

量的 $|S|$ 个线性方程。

最多经过有限次迭代，V 就会收敛到 V^*，π 收敛到 π^*。[74]

两者都是标准的 MDP 求解算法，目前没有普遍共识认为哪一种更好。对于小 MDP，策略迭代往往很快，少量迭代就能收敛。但对于状态空间很大的 MDP，显式求 V^π 涉及大型线性方程组，可能很困难，而且策略迭代中需要多次求解。此时价值迭代可能更合适，因此实践中似乎使用得更频繁。关于两者比较与联系的更多讨论见第 19.5 节。

19.3 学习 MDP 的模型

至此，讨论都假定状态转移概率和奖励已知。许多实际问题不会显式给出它们，必须从数据估计；通常 S, A, γ 已知。

例如，在倒立摆问题（见习题集 4）中，进行若干次如下试验：

$$
\begin{aligned}
&s_0^{(1)} \xrightarrow{a_0^{(1)}} s_1^{(1)} \xrightarrow{a_1^{(1)}} s_2^{(1)} \xrightarrow{a_2^{(1)}} s_3^{(1)} \xrightarrow{a_3^{(1)}} \cdots \\
&s_0^{(2)} \xrightarrow{a_0^{(2)}} s_1^{(2)} \xrightarrow{a_1^{(2)}} s_2^{(2)} \xrightarrow{a_2^{(2)}} s_3^{(2)} \xrightarrow{a_3^{(2)}} \cdots \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \cdots
\end{aligned}
$$

$s_i^{(j)}$ 是第 j 次试验在时刻 i 的状态，$a_i^{(j)}$ 是在该状态采取的动作。实践中，每次试验可以运行到 MDP 终止，例如倒立摆的杆倒下，也可以运行一个较大但有限的步数。

给定这些试验组成的“经验”，很容易推出状态转移概率的最大似然估计：

$$
P_{sa}(s') = \frac{\text{在状态 } s \text{ 采取动作 } a \text{ 并到达 } s' \text{ 的次数}}{\text{在状态 } s \text{ 采取动作 } a \text{ 的次数}}. \tag{19.5}
$$

如果上述比值为“0/0”，即从未在 s 采取过 a，可简单估计为 $P_{sa}(s') = 1/|S|$，也就是全部状态上的均匀分布。

随着在 MDP 中积累更多经验、观察更多试验，可以高效更新这些估计：保留式（19.5）分子和分母的计数，观察新试验时继续累加，再取比值，就得到 P_{sa} 的估计。

类似地，若 R 未知，可把状态 s 的期望即时奖励 $R(s)$ 估计为在该状态观察到的平均奖励。

学到 MDP 模型后，就可用估计出的转移概率和奖励，通过价值迭代或策略迭代求解。例如，将模型学习与价值迭代结合，可得到处理未知转移概率 MDP 的一种算法：

1. 随机初始化 π。
2. 反复执行以下过程：在 MDP 中用 π 运行若干次试验；根据累积经验更新 P_{sa}（必要时也更新 R）；用估计的转移概率和奖励运行价值迭代，得到新的价值函数估计 V；将 π 更新为相对于 V 的贪心策略。

这个算法有一个简单加速方法：内层运行价值迭代时，不从 $V = 0$ 开始，而用算法上一轮求得的解初始化。这样能提供好得多的起点，令价值迭代更快收敛。

[74] 价值迭代不能在有限次迭代内达到精确的 V^*，而使用精确线性方程组求解器的策略迭代可以。原因是动作空间和策略空间离散且有限；一旦策略迭代达到最优策略，策略就完全不再改变。价值迭代虽然会收敛到 V^*，但学到的价值函数中始终存在非零误差。

19.4 连续状态 MDP

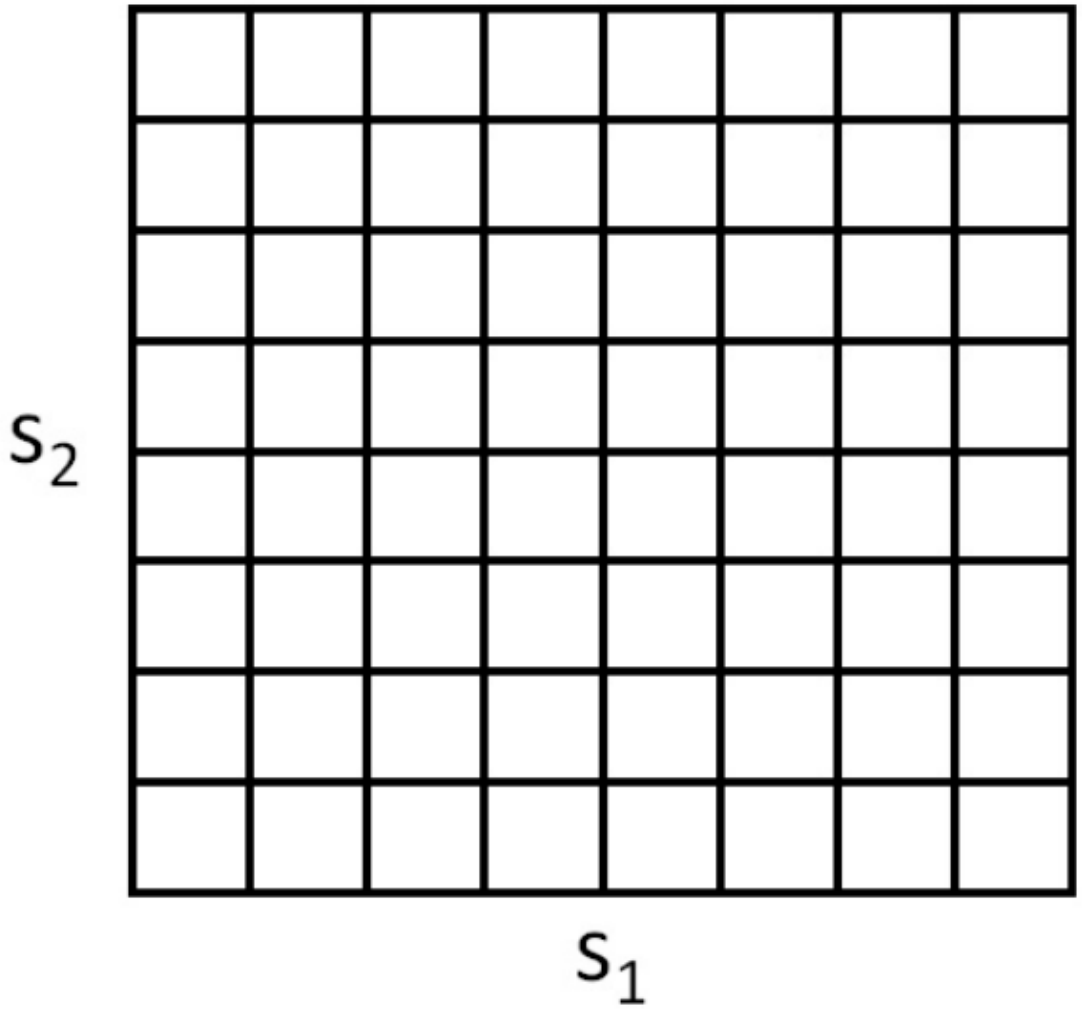
此前聚焦于有限状态的 MDP。现在讨论可能具有无限多个状态的 MDP。汽车状态可以表示为 $(x, y, \theta, \dot{x}, \dot{y}, \dot{\theta})$，包含位置 (x, y)、朝向 θ、沿 x, y 方向的速度 $\dot{x}, \dot{y}$，以及角速度 $\dot{\theta}$。因此 $S = \mathbb{R}^6$ 是无限状态集合，因为汽车有无限多种可能的位置和朝向。[75] 类似地，习题集 4 中的倒立摆状态为 $(x, \theta, \dot{x}, \dot{\theta})$，其中 θ 为杆的角度。在三维空间飞行的直升机状态为 $(x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi})$，横滚角 ϕ、俯仰角 θ 和偏航角 ψ 指定其三维姿态。

本节考虑状态空间 $S = \mathbb{R}^d$，并介绍求解方法。

19.4.1 离散化

求解连续状态 MDP 最简单的方法，也许是先离散化状态空间，再使用此前的价值迭代或策略迭代。

例如，对于二维状态 (s_1, s_2)，可用网格离散化：



二维状态空间的网格离散化（原文无编号图）。

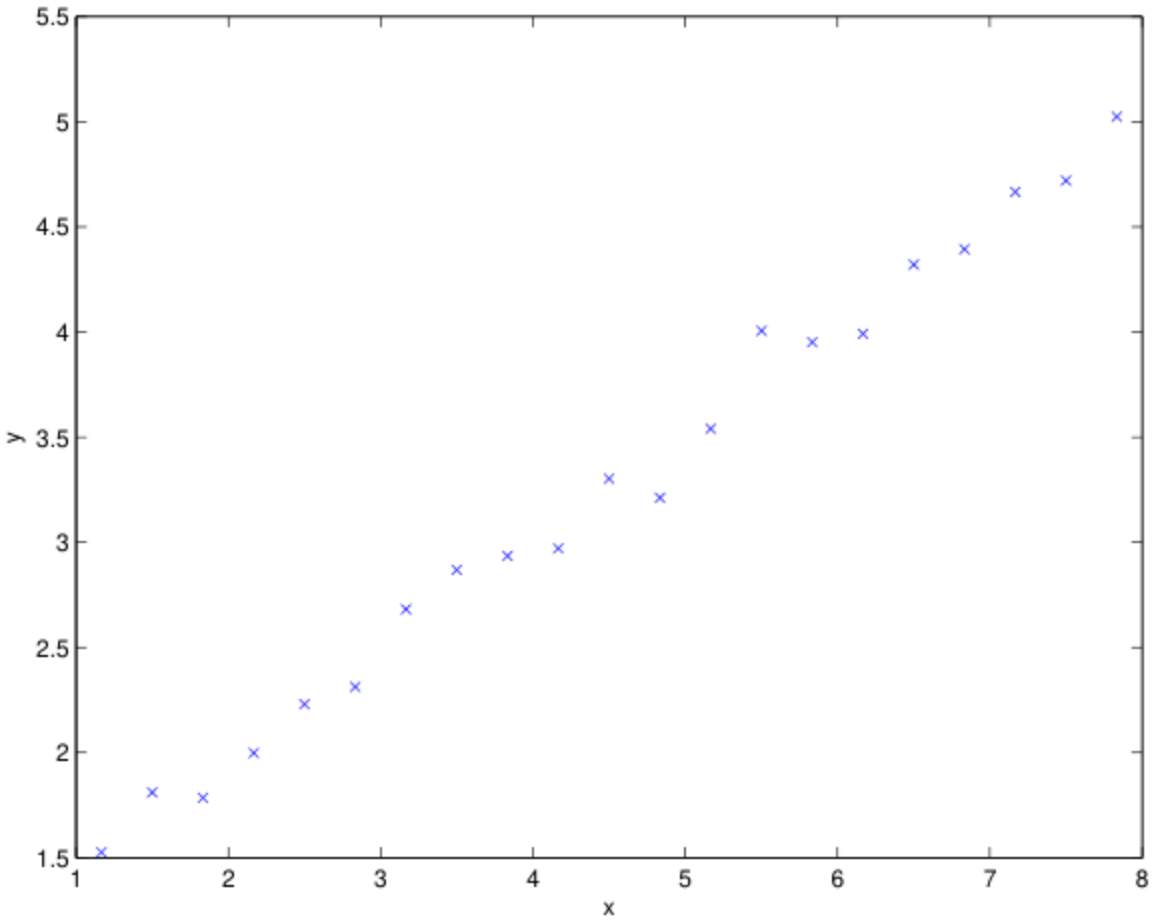
每个网格单元表示一个独立的离散状态 $\bar{s}$。由此，用离散状态 MDP $(\bar{S}, A, \{P_{\bar{s}a}\}, \gamma, R)$ 近似连续状态

[75] 严格说，θ 是方向，其取值范围应写为 $[-\pi, \pi)$ 而非 $\mathbb{R}$，但这里这一区别不重要。

MDP，其中 $\bar{S}$ 是离散状态集合，$\{P_{\bar{s}a}\}$ 为离散状态上的转移概率，等等。再用价值迭代或策略迭代求该离散 MDP 中的 $V^*(\bar{s}), \pi^*(\bar{s})$。当实际系统处于连续状态 $s \in S$，需要选动作时，先计算对应离散状态 $\bar{s}$，再执行 $\pi^*(\bar{s})$。

离散化对许多问题有效，但有两个缺点。首先，它对 V^* 和 π^* 的表示相当简单：假设价值函数在每个离散区间内取常数，即在各网格单元内分段恒定。

为理解局限，考虑对下图数据拟合函数的监督学习问题：



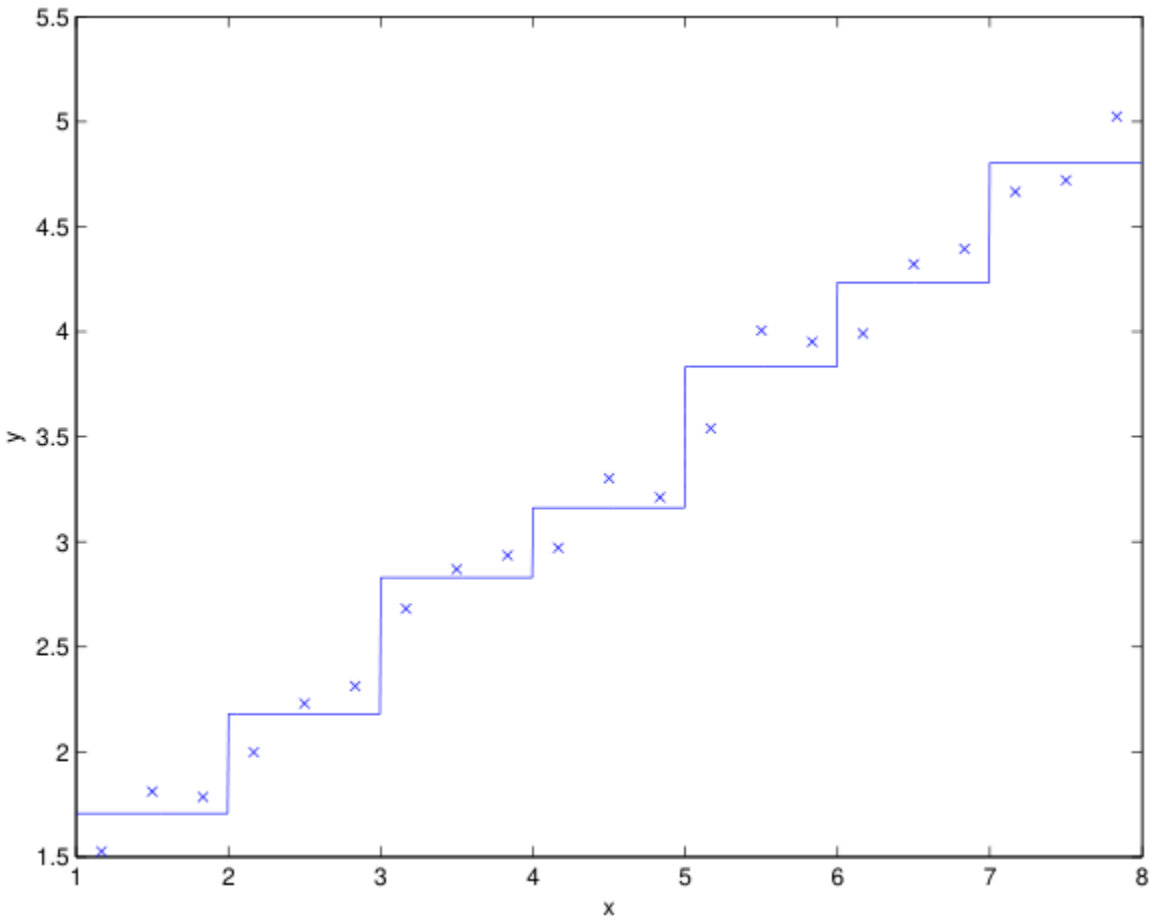
监督学习拟合示例中的数据（原文无编号图）。

显然，线性回归能很好地处理它。但如果离散化 x 轴，并在每个区间使用分段常数表示，拟合就会如下图所示：

分段常数并不适合表示很多平滑函数。它对输入几乎不作平滑，也不能在不同网格单元间泛化。要得到好的近似，还需要很细的离散化，也就是很小的网格单元。

第二个缺点称为**维数灾难**。设 $S = \mathbb{R}^d$，把每一维离散为 k 个值，总离散状态数就是 k^d，随维度 d 指数增长，因此不适合大问题。例如，10 维状态中若每个变量离散为 100 个值，就有 $100^{10} = 10^{20}$ 个状态，即使现代台式电脑也远远无法表示。

经验上，离散化通常非常适合一维和二维问题，还具有实现简单快速的优点。稍动脑筋并谨慎选择离散化方式，通常也能很好处理最高四维的状态。如果非常巧妙又有些幸运，甚至可处理某些六维问题；但更高维度时几乎很少有效。



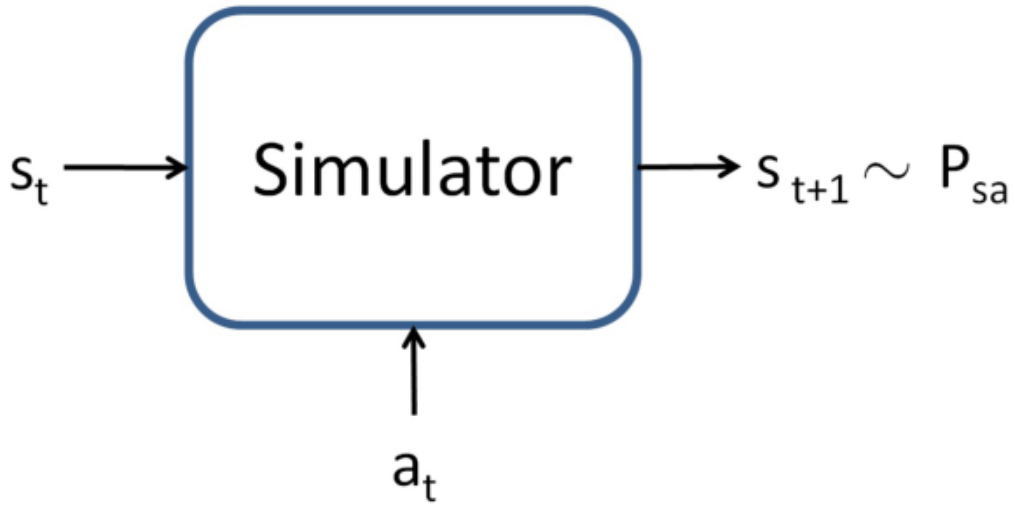
把横轴离散化后得到的分段常数拟合（原文无编号图）。

19.4.2 价值函数近似

现在介绍一种在连续状态 MDP 中寻找策略的替代方法：直接近似 V^*，不进行离散化。这称为**价值函数近似**，已经成功用于许多强化学习问题。

使用模型或模拟器

为构造价值函数近似算法，假设已有 MDP 的模型或模拟器。非正式地说，模拟器是一个黑箱：输入任意连续状态 s_t 和动作 a_t，输出按状态转移概率 $P_{s_t a_t}$ 采样的下一状态 s_{t+1}。



模拟器接受当前状态和动作，输出下一个状态（原文无编号图）。

获得模型有几种途径。一是物理仿真。例如，习题集 4 的倒立摆模拟器根据物理定律，在已知杆长、杆质量等全部系统参数的情况下，由当前时刻 t 的状态和动作 a 计算时刻 $t+1$ 小车与杆的位置和朝向。也可使用现成的物理仿真软件包：输入机械系统的完整物理描述、当前状态 s_t 和动作 a_t，计算稍后一小段时间系统的状态 s_{t+1}。[76]

另一途径是从 MDP 中收集的数据学习模型。假设进行 n 次试验，每次在 MDP 中连续行动 T 个时间步。动作可随机选择，也可执行特定策略，或用其他方式选择。于是观察到 n 条状态序列：

$$
\begin{aligned}
&s_0^{(1)} \xrightarrow{a_0^{(1)}} s_1^{(1)} \xrightarrow{a_1^{(1)}} s_2^{(1)} \to \cdots \xrightarrow{a_{T-1}^{(1)}} s_T^{(1)} \\
&s_0^{(2)} \xrightarrow{a_0^{(2)}} s_1^{(2)} \xrightarrow{a_1^{(2)}} s_2^{(2)} \to \cdots \xrightarrow{a_{T-1}^{(2)}} s_T^{(2)} \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \cdots \\
&s_0^{(n)} \xrightarrow{a_0^{(n)}} s_1^{(n)} \xrightarrow{a_1^{(n)}} s_2^{(n)} \to \cdots \xrightarrow{a_{T-1}^{(n)}} s_T^{(n)}.
\end{aligned}
$$

然后应用学习算法，把 s_{t+1} 预测为 s_t, a_t 的函数。

例如，可用类似线性回归的算法学习线性模型

$$s_{t+1} = As_t + Ba_t. \tag{19.6}$$

[76] Open Dynamics Engine（原文网址 http://www.ode.com）是免费的开源物理模拟器示例，可模拟倒立摆等系统，曾是强化学习研究者中相当流行的选择。

参数是矩阵 A, B，可通过 n 次试验的数据估计：

$$\arg\min_{A,B} \sum_{i=1}^{n} \sum_{t=0}^{T-1} \left\| s_{t+1}^{(i)} - (As_t^{(i)} + Ba_t^{(i)}) \right\|_2^2.$$

也可用其他损失函数学习模型。例如，Luo 等人 [2018] 的近期工作发现，在某些情形下使用不平方的 $\|\cdot\|_2$ 范数可能有帮助。

学到 A, B 后，可以构建确定性模型：给定 s_t, a_t，输出 s_{t+1} 完全确定，始终按式（19.6）计算。也可构建随机模型，使 s_{t+1} 是输入的随机函数：

$$s_{t+1} = As_t + Ba_t + \epsilon_t,$$

其中噪声项通常建模为 $\epsilon_t \sim \mathcal{N}(0, \Sigma)$，协方差矩阵 Σ 也容易从数据估计。

这里把下一状态写为当前状态和动作的线性函数，当然也可使用非线性函数。例如学习 $s_{t+1} = A\phi_s(s_t) + B\phi_a(a_t)$，其中 ϕ_s, ϕ_a 为状态和动作的非线性特征映射。也可用局部加权线性回归等非线性学习算法，把 s_{t+1} 估计为 s_t, a_t 的函数。这些方法都能构建确定性或随机的 MDP 模拟器。

拟合价值迭代

下面介绍近似连续状态 MDP 价值函数的**拟合价值迭代**算法。假设状态空间 $S = \mathbb{R}^d$ 连续，动作空间 A 较小且离散。[77]

回忆价值迭代要执行的更新：

$$V(s) := R(s) + \gamma \max_a \int_{s'} P_{sa}(s')V(s')\, ds' \tag{19.7}$$

$$= R(s) + \gamma \max_a \mathbb{E}_{s' \sim P_{sa}}[V(s')]. \tag{19.8}$$

第 19.2 节使用对状态求和的形式 $V(s) := R(s) + \gamma \max_a \sum_{s'} P_{sa}(s')V(s')$；现在改为积分，是因为状态由离散变为连续。

拟合价值迭代的核心，是在有限状态样本 $s^{(1)}, \ldots, s^{(n)}$ 上近似执行这一步。具体而言，用监督学习算法——下文采用线性回归——把价值函数近似为状态的线性或非线性函数：

$$V(s) = \theta^\top \phi(s),$$

其中 ϕ 为适当的状态特征映射。

对于有限样本中的每个状态，先计算 $y^{(i)}$，近似 $R(s) + \gamma \max_a \mathbb{E}_{s' \sim P_{sa}}[V(s')]$，即式（19.8）右侧；再应用监督学习，努力让 $V(s)$ 接近这一量，也就是接近 $y^{(i)}$。详细算法如下。

1. 随机采样 n 个状态 $s^{(1)}, s^{(2)}, \ldots, s^{(n)} \in S$。
2. 初始化 $\theta := 0$。

[77] 实践中，多数 MDP 的动作空间远小于状态空间。例如，汽车有六维状态和二维动作（转向与速度控制）；倒立摆有四维状态和一维动作；直升机有十二维状态和四维动作。因此，离散化动作通常不像离散化状态那么困难。

3. 重复以下过程。

对每个 $i = 1, \dots, n$，以及每个动作 $a \in A$，用 MDP 模型采样 $s'_1, \dots, s'_k \sim P_{s^{(i)}a}$，令

$$q(a) = \frac{1}{k} \sum_{j=1}^{k} [R(s^{(i)}) + \gamma V(s'_j)].$$

因此 $q(a)$ 估计 $R(s^{(i)}) + \gamma \mathbb{E}_{s' \sim P_{s^{(i)}a}}[V(s')]$。遍历动作后，令

$$y^{(i)} = \max_a q(a),$$

它估计 $R(s^{(i)}) + \gamma \max_a \mathbb{E}_{s' \sim P_{s^{(i)}a}}[V(s')]$。

原始离散状态价值迭代直接更新 $V(s^{(i)}) := y^{(i)}$。这里希望 $V(s^{(i)}) \approx y^{(i)}$，通过监督学习，即线性回归实现：

$$\theta := \arg\min_\theta \frac{1}{2} \sum_{i=1}^{n} (\theta^\top \phi(s^{(i)}) - y^{(i)})^2.$$

以上使用线性回归让 $V(s^{(i)})$ 接近 $y^{(i)}$。这一步与标准监督回归完全类似：给定训练集 $(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})$，学习从 x 到 y 的映射；区别仅在于这里由 s 扮演 x 的角色。虽然描述采用线性回归，显然也可用局部加权线性回归等其他回归算法。

与离散状态价值迭代不同，无法证明拟合价值迭代总会收敛。但实践中它常常收敛或近似收敛，在很多问题上效果很好。此外，若 MDP 模拟器或模型是确定性的，可令 $k = 1$ 简化算法。这是因为式（19.8）的期望针对确定性分布，一个样本就足以精确计算。否则，需要抽取 k 个样本取平均来近似期望，正如伪代码中 $q(a)$ 的定义。

最后，拟合价值迭代输出 V 作为 V^* 的近似，这隐式定义了策略。具体而言，系统处于 s 且需要选动作时，希望选择

$$\arg\max_a \mathbb{E}_{s' \sim P_{sa}}[V(s')]. \tag{19.9}$$

计算或近似它的方法类似拟合价值迭代内层循环：对每个动作采样 $s'_1, \dots, s'_k \sim P_{sa}$ 以近似期望。模拟器确定时，仍可取 $k = 1$。

实践中通常还有其他近似方式。很常见的是模拟器具有形式 $s_{t+1} = f(s_t, a_t) + \epsilon_t$，其中 f 为确定性函数，例如 $f(s_t, a_t) = As_t + Ba_t$，$\epsilon_t$ 为零均值高斯噪声。此时可选择

$$\arg\max_a V(f(s, a)).$$

也就是设 $\epsilon_t = 0$，忽略模拟器噪声，并取 $k = 1$。等价地，可以从式（19.9）用如下近似推导：

$$\mathbb{E}_{s'}[V(s')] \approx V(\mathbb{E}_{s'}[s']) \tag{19.10}$$

$$= V(f(s, a)), \tag{19.11}$$

期望针对随机状态 $s' \sim P_{sa}$。只要噪声项较小，这通常是合理近似。

但对不适用此类近似的问题，为近似上述期望而使用模型采样 $k|A|$ 个状态，计算成本可能很高。

19.5 策略迭代与价值迭代的联系（选读）

算法 5 第 3 行通常用线性方程组求解器计算 V^π。另一种方式是类似价值迭代，使用迭代贝尔曼更新评估 V^π，如下方算法 6 中的过程 VE(·)。如果使用 VE 初始化中的选项 1，VE 与价值迭代（算法 4）的区别在于：更新时使用策略 π 指定的动作，而不是贪心动作。

算法 6：策略迭代的变体

过程 VE(π, k)：**评估** V^π。

- 初始化：选项 1，$V(s) := 0$；选项 2，从主算法当前的 V 初始化。
- 对 $i = 0, \ldots, k-1$，对所有状态 s 更新

$$V(s) := R(s) + \gamma \sum_{s'} P_{s\pi(s)}(s')V(s'). \tag{19.12}$$

- 返回 V。

主算法。给定超参数 k，随机初始化 π。重复直到收敛：令 $V = \mathrm{VE}(\pi, k)$，然后对每个状态 s 令

$$\pi(s) := \arg\max_{a \in A} \sum_{s'} P_{sa}(s')V(s'). \tag{19.13}$$

利用 VE 就得到算法 6；它是策略迭代的变体，也是连接策略迭代与价值迭代的中间算法。这里采用选项 2，以最大程度复用此前学到的知识。可以验证，当 $k = 1$ 且使用选项 2 时，算法 6 在语义上等价于价值迭代（算法 4）。两者都交替执行式（19.13）与（19.12）的更新：算法 6 交替执行 k 次（19.12）和一次（19.13），价值迭代则交替执行各一次。因此，一般而言，算法 6 不应比价值迭代更快：若两种更新同样有用且耗时相同，最佳更新频率平衡可能就是 $k = 1$ 或 $k \approx 1$。

另一方面，如果成组执行 k 次式（19.12）比逐次执行快得多，那么额外执行一组这样的更新可能有用。这正是策略迭代利用的性质：线性方程组求解器给出相当于 VE 取 $k = \infty$ 的结果，比以很大的 k 运行 VE 快得多。反过来，若没有这种加速，例如状态空间很大而线性求解器也不快，价值迭代就更合适。

第 20 章　LQR、DDP 与 LQG

原文印刷页码：244–257。

20.1 有限时域 MDP

第 19 章定义了马尔可夫决策过程（MDP），并在简化情形下介绍了价值迭代和策略迭代。具体而言，我们介绍了定义最优策略 π^* 的最优价值函数 V^{π^*} 的最优贝尔曼方程：

$$V^{\pi^*}(s) = R(s) + \max_{a \in A} \gamma \sum_{s' \in S} P_{sa}(s') V^{\pi^*}(s').$$

回忆，可由最优价值函数恢复最优策略：

$$\pi^*(s) = \arg\max_{a \in A} \sum_{s' \in S} P_{sa}(s') V^*(s').$$

本章考虑更一般的情形。

1. 我们希望写出的方程同时适用于离散和连续情况，因此把 $\sum_{s' \in S} P_{sa}(s') V^{\pi^*}(s')$ 写为 $\mathbb{E}_{s' \sim P_{sa}}[V^{\pi^*}(s')]$，表示下一个状态价值函数的期望。有限状态情形下，期望可写为对状态求和；连续情形下，可写为积分。$s' \sim P_{sa}$ 表示状态 s' 从分布 P_{sa} 中采样。
2. 假设奖励同时依赖状态和动作，即 $R : S \times A \to \mathbb{R}$。于是最优动作的计算变为

$$\pi^*(s) = \arg\max_{a \in A} \left(R(s,a) + \gamma \mathbb{E}_{s' \sim P_{sa}}[V^{\pi^*}(s')] \right).$$

3. 不再考虑无限时域 MDP，而是假设有限时域 MDP，定义为元组 (S, A, P_{sa}, T, R)，其中 $T > 0$ 是时域长度（例如 $T = 100$）。此时收益定义略有变化：

$$R(s_0, a_0) + R(s_1, a_1) + \cdots + R(s_T, a_T),$$

而不是无限时域中的

$$R(s_0, a_0) + \gamma R(s_1, a_1) + \gamma^2 R(s_2, a_2) + \cdots = \sum_{t=0}^{\infty} R(s_t, a_t) \gamma^t.$$

折扣因子 γ 去哪了？回忆，引入它的部分原因是保证无穷和有限且定义良好。如果奖励的绝对值以常数 $\overline{R}$ 为界，则

$$\left|\sum_{t=0}^{\infty} R(s_t)\gamma^t\right| \le \overline{R}\sum_{t=0}^{\infty}\gamma^t,$$

右侧是几何级数。这里收益是有限和，因此不再必须使用折扣因子。

在新情形下，行为有很大不同。首先，最优策略 π^* 可能是非平稳的，即随时间变化。现在有

$$\pi^{(t)} : S \to A,$$

上标 (t) 表示时间步 t 的策略。遵循 $\pi^{(t)}$ 的有限时域 MDP 如下演化：从状态 s_0 开始，按时刻 0 的策略选择 $a_0 := \pi^{(0)}(s_0)$，再按 $P_{s_0 a_0}$ 采样后继状态 s_1；然后依照时刻 1 的新策略选择 $a_1 := \pi^{(1)}(s_1)$，以此类推。

为什么有限时域下的最优策略可能非平稳？直觉上，可采取的动作数有限，我们可能需要根据环境中的位置和剩余时间采用不同策略。想象一个网格中有两个目标，奖励分别为 +1 和 +10。开始时可能想朝 +10 目标前进；但若经过几步，动力学把我们推到更接近 +1 的目标，而且剩余步数不足以到达 +10，那么更好的策略就是改为追求 +1 目标。

4. 这一观察使我们可以采用随时间变化的动力学：

$$s_{t+1} \sim P^{(t)}_{s_t, a_t},$$

即转移分布 $P^{(t)}_{s_t,a_t}$ 随时间变化。奖励 $R^{(t)}$ 也可以如此。这更符合现实：汽车油箱会逐渐变空，交通状况也会改变。综合以上说明，使用以下一般的有限时域 MDP 表述：

$$\left(S, A, P^{(t)}_{sa}, T, R^{(t)}\right).$$

说明：上述表述等价于把时间纳入状态。

策略 π 在时刻 t 的价值函数与此前一样，定义为从状态 s 开始、按照策略 π 生成轨迹的期望：

$$V_t(s) = \mathbb{E}[R^{(t)}(s_t, a_t) + \cdots + R^{(T)}(s_T, a_T) \mid s_t = s, \pi].$$

问题是：在有限时域下，如何找到最优价值函数

$$V_t^*(s) = \max_{\pi} V_t^{\pi}(s)\ ?$$

价值迭代中的贝尔曼方程正适合动态规划。这并不奇怪：Bellman 是动态规划的奠基人之一，贝尔曼方程与该领域密切相关。为理解迭代如何简化问题，作如下观察。

1. 游戏结束时，即时间步 T，最优价值显而易见：

$$\forall s \in S: \quad V_T^*(s) := \max_{a \in A} R^{(T)}(s, a). \tag{20.1}$$

2. 对其他时间步 $0 \le t < T$，若知道下一时间步的最优价值函数 V_{t+1}^*，则

$$\forall t < T,\ s \in S: \quad V_t^*(s) := \max_{a \in A} \Big[R^{(t)}(s,a) + \mathbb{E}_{s' \sim P_{sa}^{(t)}}[V_{t+1}^*(s')] \Big]. \tag{20.2}$$

于是得到求解最优价值函数的巧妙算法：先用式（20.1）计算 V_T^*；再对 $t = T-1, \dots, 0$，用式（20.2）由 V_{t+1}^* 计算 V_t^*。

补充说明。标准价值迭代可看作这一一般情形的特例，只是不跟踪时间。在标准设定中，运行 T 步价值迭代会得到误差按 γ^T 缩减的最优价值近似，即几何收敛。以下结论的证明见习题集 4。

定理。令 B 表示贝尔曼更新，$\|f(x)\|_\infty := \sup_x |f(x)|$。若 V_t 是第 t 步的价值函数，则

$$\begin{aligned} \|V_{t+1} - V^*\|_\infty &= \|B(V_t) - V^*\|_\infty \\ &\le \gamma \|V_t - V^*\|_\infty \\ &\le \gamma^t \|V_1 - V^*\|_\infty. \end{aligned}$$

也就是说，贝尔曼算子 B 是一个 γ 压缩算子。

20.2 线性二次调节（LQR）

本节介绍第 20.1 节有限时域情形的一个特例，可以容易地求出精确解。这个模型广泛用于机器人领域；很多问题的一种常见方法就是把原问题归约到这个框架。

先描述模型假设。我们考虑连续情形，

$$S = \mathbb{R}^d, \qquad A = \mathbb{R}^d,$$

假设转移是带噪声的线性形式：

$$s_{t+1} = A_t s_t + B_t a_t + w_t,$$

其中 $A_t, B_t \in \mathbb{R}^{d \times d}$，$w_t \sim \mathcal{N}(0, \Sigma_t)$ 为零均值高斯噪声。下面将看到，只要噪声均值为 0，它就不影响最优策略！同时假设奖励是二次函数：

$$R^{(t)}(s_t, a_t) = -s_t^\top U_t s_t - a_t^\top W_t a_t,$$

其中 $U_t \in \mathbb{R}^{d \times n}$、$W_t \in \mathbb{R}^{d \times d}$ 为正定矩阵，意味着奖励总是负的。[78]

说明。二次奖励意味着希望状态接近原点，那里奖励更高。例如 $U_t = I_d, W_t = I_d$ 时，$R_t = -\|s_t\|^2 - \|a_t\|^2$，表示希望以平缓动作（$a_t$ 的范数小）返回原点（s_t 的范数小）。这可以模拟汽车尽量保持在车道中央、避免突兀操作。

明确 LQR 模型假设后，介绍算法的两个步骤。

步骤 1：估计模型。假设不知道矩阵 A, B, Σ。可以采用强化学习讲义 “价值近似” 部分的思路估计它们：先用任意策略收集转移，再通过线性回归求

[78] 译注：原文此处把 U_t 的维度写为 $d \times n$；要使该二次型与正定条件成立，应为 $d \times d$。在零状态、零动作处奖励为 0，严格说奖励非正。

$$\arg\min_{A,B} \sum_{i=1}^{n} \sum_{t=0}^{T-1} \left\| s_{t+1}^{(i)} - (As_t^{(i)} + Ba_t^{(i)}) \right\|^2 .$$

最后，用高斯判别分析中介绍的技术学习 Σ。

步骤 2：求解策略。假设模型参数已知（给定，或由步骤 1 估计），用动态规划推导最优策略。也就是说，给定

$$\begin{cases} s_{t+1} = A_t s_t + B_t a_t + w_t, \\ R^{(t)}(s_t, a_t) = -s_t^\top U_t s_t - a_t^\top W_t a_t, \end{cases} \qquad A_t, B_t, U_t, W_t, \Sigma_t \text{ 已知,}$$

我们希望计算 V_t^*。应用第 20.1 节的动态规划，有以下步骤。

初始化。对最后一个时间步 T，

$$\begin{aligned} V_T^*(s_T) &= \max_{a_T \in A} R_T(s_T, a_T) \\ &= \max_{a_T \in A} (-s_T^\top U_T s_T - a_T^\top W_T a_T) \\ &= -s_T^\top U_T s_T \qquad \text{（在 } a_T = 0 \text{ 时最大）.} \end{aligned}$$

递推。令 $t < T$，假设已知 V_{t+1}^*。

事实 1：可以证明，如果 V_{t+1}^* 是状态的二次函数，那么 V_t^* 也是二次函数。也就是说，存在矩阵 Φ 和标量 Ψ，使得

$$\begin{aligned} & V_{t+1}^*(s_{t+1}) = s_{t+1}^\top \Phi_{t+1} s_{t+1} + \Psi_{t+1} \\ \Longrightarrow \quad & V_t^*(s_t) = s_t^\top \Phi_t s_t + \Psi_t. \end{aligned}$$

时刻 T 有 $\Phi_T = -U_T, \Psi_T = 0$。

事实 2：可以证明，最优策略只是状态的线性函数。知道 V_{t+1}^* 等价于知道 Φ_{t+1}, Ψ_{t+1}，所以只需说明如何根据它们和其他问题参数计算 Φ_t, Ψ_t。

$$\begin{aligned} V_t^*(s_t) &= s_t^\top \Phi_t s_t + \Psi_t \\ &= \max_{a_t} \left[R^{(t)}(s_t, a_t) + \mathbb{E}_{s_{t+1} \sim P_{s_t, a_t}^{(t)}} [V_{t+1}^*(s_{t+1})] \right] \\ &= \max_{a_t} \left[-s_t^\top U_t s_t - a_t^\top V_t a_t + \mathbb{E}_{s_{t+1} \sim \mathcal{N}(A_t s_t + B_t a_t, \Sigma_t)} [s_{t+1}^\top \Phi_{t+1} s_{t+1} + \Psi_{t+1}] \right]. \end{aligned}$$

第二行是最优价值函数的定义，第三行代入了模型动力学和二次函数假设。最后的表达式是 a_t 的二次函数，因此很容易优化。[79] 得到最优动作（以下按原文公式保留）：

$$\begin{aligned} a_t^* &= (B_t^\top \Phi_{t+1} B_t - V_t)^{-1} B_t \Phi_{t+1} A_t \cdot s_t \\ &= L_t \cdot s_t, \end{aligned}$$

其中

[79] 使用恒等式 $\mathbb{E}[w_t^\top \Phi_{t+1} w_t] = \mathrm{Tr}(\Sigma_t \Phi_{t+1})$，其中 $w_t \sim \mathcal{N}(0, \Sigma_t)$。

$$L_t := (B_t^\top \Phi_{t+1} B_t - W_t)^{-1} B_t \Phi_{t+1} A_t.$$

这是一个很漂亮的结果：最优策略对 s_t 是线性的。给定 a_t^*，可解出 Φ_t, Ψ_t，最终得到离散 Riccati 方程（按原文保留）：

$$\Phi_t = A_t^\top \left(\Phi_{t+1} - \Phi_{t+1} B_t (B_t^\top \Phi_{t+1} B_t - W_t)^{-1} B_t \Phi_{t+1}\right) A_t - U_t,$$

$$\Psi_t = -\operatorname{tr}(\Sigma_t \Phi_{t+1}) + \Psi_{t+1}.$$

译注：上述原文 LQR 推导有记号、转置和符号错误。二次动作惩罚中的 V_t 应为 W_t。在本章采用的 $V_t^*(s) = s^\top \Phi_t s + \Psi_t$ 约定下，令 $M_t = B_t^\top \Phi_{t+1} B_t - W_t$，一致的公式应为

$$L_t = -M_t^{-1} B_t^\top \Phi_{t+1} A_t,$$

$$\Phi_t = A_t^\top \left(\Phi_{t+1} - \Phi_{t+1} B_t M_t^{-1} B_t^\top \Phi_{t+1}\right) A_t - U_t,$$

$$\Psi_t = \operatorname{tr}(\Sigma_t \Phi_{t+1}) + \Psi_{t+1}.$$

事实 3：注意，Φ_t 既不依赖 Ψ，也不依赖噪声 Σ_t！因为 L_t 是 A_t, B_t, Φ_{t+1} 的函数，所以最优策略也不依赖噪声！（但 Ψ_t 依赖 Σ_t，因此 V_t^* 依赖 Σ_t。）

总结起来，LQR 算法如下：

1. 如有必要，估计参数 A_t, B_t, Σ_t。
2. 初始化 $\Phi_T := -U_T, \Psi_T := 0$。
3. 从 $t = T-1$ 迭代到 0，用离散 Riccati 方程根据 Φ_{t+1}, Ψ_{t+1} 更新 Φ_t, Ψ_t。若存在把状态驱向零的策略，则可以保证收敛！

利用事实 3，还能让算法稍快一些：最优策略不依赖 Ψ_t，而 Φ_t 的更新只依赖 Φ，所以只更新 Φ_t 就足够了。

20.3 从非线性动力学到 LQR

即使动力学是非线性的，很多问题仍可归约为 LQR。LQR 的优点是有漂亮的精确解，但它远不是一般性的框架。以倒立摆为例，状态转移形式为

$$\begin{pmatrix} x_{t+1} \\ \dot{x}_{t+1} \\ \theta_{t+1} \\ \dot{\theta}_{t+1} \end{pmatrix} = F\left(\begin{pmatrix} x_t \\ \dot{x}_t \\ \theta_t \\ \dot{\theta}_t \end{pmatrix}, a_t\right),$$

函数 F 依赖角度的余弦等量。我们可以问：能否把这个系统线性化？

20.3.1 动力学的线性化

假设在时刻 t，系统大部分时间位于某个状态 $\bar{s}_t$ 附近，执行的动作位于 $\bar{a}_t$ 附近。对倒立摆，如果已达到某种最优状态，这确实成立：动作很小，偏离竖直位置也不多。

我们用泰勒展开线性化动力学。在状态是一维且转移函数 F 不依赖动作的简单情形，可写为

$$s_{t+1} = F(s_t) \approx F(\bar{s}_t) + F'(\bar{s}_t)(s_t - \bar{s}_t).$$

一般情况下形式相同，只是用梯度代替普通导数：

$$s_{t+1} \approx F(\bar{s}_t, \bar{a}_t) + \nabla_s F(\bar{s}_t, \bar{a}_t)(s_t - \bar{s}_t) + \nabla_a F(\bar{s}_t, \bar{a}_t)(a_t - \bar{a}_t). \tag{20.3}$$

现在 s_{t+1} 对 s_t, a_t 是线性的，因为可重写为

$$s_{t+1} \approx As_t + Ba_t + \kappa,$$

其中 $A = \nabla_s F(\bar{s}_t, \bar{a}_t)$、$B = \nabla_a F(\bar{s}_t, \bar{a}_t)$，$\kappa = F(\bar{s}_t, \bar{a}_t) - A\bar{s}_t - B\bar{a}_t$ 是常数。这与 LQR 假设非常相似，只需处理常数项 κ！通过人为增加一维，可以把它吸收到 s_t 中。这就是课程开头在线性回归中用过的技巧。

20.3.2 微分动态规划（DDP）

当目标是保持在状态 s^* 附近时，上一方法效果很好，例如倒立摆或保持在车道中央的汽车。但有时目标更复杂。

下面介绍适用于系统需要沿某条轨迹运动（例如火箭）的方法。它把轨迹离散为时间步，并设立中间目标，从而可以在各目标附近使用上一节的技术。这称为微分动态规划，主要步骤如下。

步骤 1。用简单控制器得到近似目标轨迹的标称轨迹。也就是说，控制器能把理想轨迹近似为

$$s_0^*, a_0^* \to s_1^*, a_1^* \to \cdots.$$

步骤 2。在每个轨迹点 s_t^* 附近线性化动力学：

$$s_{t+1} \approx F(s_t^*, a_t^*) + \nabla_s F(s_t^*, a_t^*)(s_t - s_t^*) + \nabla_a F(s_t^*, a_t^*)(a_t - a_t^*),$$

其中 s_t, a_t 是当前状态和动作。有了各点附近的线性近似，就可用上一节的方法重写为

$$s_{t+1} = A_t \cdot s_t + B_t \cdot a_t.$$

注意，这里采用了本章开头介绍的非平稳动力学设定。

说明。对奖励 $R^{(t)}$，也可用二阶泰勒展开作类似推导：

$$
\begin{aligned}
R(s_t, a_t) \approx\ & R(s_t^*, a_t^*) + \nabla_s R(s_t^*, a_t^*)(s_t - s_t^*) + \nabla_a R(s_t^*, a_t^*)(a_t - a_t^*) \\
& + \frac{1}{2}(s_t - s_t^*)^\top H_{ss}(s_t - s_t^*) + (s_t - s_t^*)^\top H_{sa}(a_t - a_t^*) \\
& + \frac{1}{2}(a_t - a_t^*)^\top H_{aa}(a_t - a_t^*).
\end{aligned}
$$

其中 H_{xy} 表示 R 关于 x, y 的 Hessian 块，在 (s_t^*, a_t^*) 处取值；为清晰起见省略了取值位置。通过同样增加一个恒为 1 的维度的技巧，原文把此式重写为

$$
R_t(s_t, a_t) = -s_t^\top U_t s_t - a_t^\top W_t a_t,
$$

其中 U_t, W_t 为某些矩阵。为理解这个技巧，注意

$$
\begin{bmatrix} 1 & x \end{bmatrix} \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} 1 \\ x \end{bmatrix} = a + 2bx + cx^2.
$$

> 译注：一般的二阶展开还含有状态—动作交叉项；单独给状态增加常数维度并不能消去该交叉项。严格处理需采用允许交叉项的一般二次代价 LQR，或进一步变换变量。

步骤 3。现在可以看出，问题已重写在 LQR 框架中。用 LQR 找到最优策略 π_t，希望新控制器更好！

注意：如果 LQR 轨迹偏离用于线性化的近似轨迹过远，可能出现问题，但可以通过奖励塑形处理。

步骤 4。得到新控制器（新策略 π_t）后，用它生成新轨迹：

$$
s_0^*, \pi_0(s_0^*) \to s_1^*, \pi_1(s_1^*) \to \cdots \to s_T^*.
$$

注意，生成新轨迹时，用真实的 F 而不是线性近似来计算转移，即

$$
s_{t+1}^* = F(s_t^*, a_t^*).
$$

随后回到步骤 2，反复执行直到满足某个停止条件。

20.4 线性二次高斯（LQG）

现实中，我们往往无法观测完整状态 s_t。例如，自动驾驶汽车从相机接收图像，但图像只是观测，并非世界的完整状态。此前假设状态可获得，但多数现实问题可能不满足，因此需要新工具来建模：部分可观测 MDP。

POMDP 是增加了观测层的 MDP。即引入新变量 o_t，给定当前状态 s_t 时服从某个条件分布：

$$
o_t \mid s_t \sim O(o \mid s).
$$

形式上，有限时域 POMDP 由元组 (S, O, A, P_{sa}, T, R) 给定。在这个框架中，一般策略是根据观测 $o_1, \ldots, o_t$ 维护信念状态，即状态上的分布；POMDP 中的策略再把信念状态映射为动作。

本节介绍 LQR 在这一设定中的扩展。假设观测 $y_t \in \mathbb{R}^n$，且 $m < n$（原文如此），满足

$$\begin{cases} y_t = C \cdot s_t + v_t, \\ s_{t+1} = A \cdot s_t + B \cdot a_t + w_t, \end{cases}$$

其中 $C \in \mathbb{R}^{n \times d}$ 是压缩矩阵，v_t 是传感器噪声，和 w_t 一样为高斯噪声。[80] 奖励函数 $R^{(t)}$ 不变，仍是状态（不是观测）和动作的函数。此外，因为分布是高斯分布，信念状态也将是高斯分布。先概述求最优策略的方法。

步骤 1。根据已有观测计算可能状态的分布，即信念状态。具体来说，求均值 $s_{t|t}$ 和协方差 $\Sigma_{t|t}$，使

$$s_t \mid y_1, \dots, y_t \sim \mathcal{N}(s_{t|t}, \Sigma_{t|t}).$$

为了随时间高效计算，采用卡尔曼滤波算法——阿波罗登月舱也用过它！

步骤 2。得到分布后，用均值 $s_{t|t}$ 作为 s_t 的最佳近似。

步骤 3。设动作 $a_t := L_t s_{t|t}$，其中 L_t 来自普通 LQR 算法。

直观地说，$s_{t|t}$ 是 s_t 的有噪近似，相当于给 LQR 增加了噪声；而我们已证明 LQR 的策略与噪声无关，所以这种方法成立。

还需要详细说明步骤 1。下面考虑动力学不依赖动作的简单情形；一般情形的思路相同。假设

$$\begin{cases} s_{t+1} = A \cdot s_t + w_t, & w_t \sim \mathcal{N}(0, \Sigma_s), \\ y_t = C \cdot s_t + v_t, & v_t \sim \mathcal{N}(0, \Sigma_y). \end{cases}$$

由于噪声是高斯的，很容易证明联合分布也是高斯的：

$$\begin{pmatrix} s_1 \\ \vdots \\ s_t \\ y_1 \\ \vdots \\ y_t \end{pmatrix} \sim \mathcal{N}(\mu, \Sigma), \qquad \text{对某些} \mu, \Sigma.$$

随后，使用高斯分布的边缘公式（参见因子分析讲义），可得到

$$s_t \mid y_1, \dots, y_t \sim \mathcal{N}(s_{t|t}, \Sigma_{t|t}).$$

不过，直接用这些公式计算分布参数成本很高，需要处理尺寸随 $t \times t$ 增长的矩阵。矩阵求逆需要 $O(t^3)$，且各时间步都要重复，因此总成本达到 $O(t^4)$！

卡尔曼滤波提供了更好的均值和方差计算方式：随时间递推，每步计算成本关于 t 为常数。算法基于两个基本步骤。假设已知 $s_t \mid y_1, \dots, y_t$ 的分布：

- **预测步骤**：计算 $s_{t+1} \mid y_1, \dots, y_t$。

[80] 译注：原文中的 m 没有定义。按这里状态维度 d、观测维度 n 和“压缩矩阵”的叙述，通常应为 $n < d$。高斯信念结论还隐含初始状态为高斯以及通常的噪声独立性假设。

- **更新步骤**：计算 $s_{t+1} \mid y_1, \dots, y_{t+1}$。

随后按时间迭代！预测和更新相结合，不断更新信念状态：

$$(s_t \mid y_1, \dots, y_t) \xrightarrow{\text{预测}} (s_{t+1} \mid y_1, \dots, y_t) \xrightarrow{\text{更新}} (s_{t+1} \mid y_1, \dots, y_{t+1}) \xrightarrow{\text{预测}} \cdots.$$

预测步骤。假设已知

$$s_t \mid y_1, \dots, y_t \sim \mathcal{N}(s_{t|t}, \Sigma_{t|t}),$$

则下一状态的分布仍为高斯：

$$s_{t+1} \mid y_1, \dots, y_t \sim \mathcal{N}(s_{t+1|t}, \Sigma_{t+1|t}),$$

其中

$$\begin{cases} s_{t+1|t} = A \cdot s_{t|t}, \\ \Sigma_{t+1|t} = A \cdot \Sigma_{t|t} \cdot A^\top + \Sigma_s. \end{cases}$$

更新步骤。给定 $s_{t+1|t}, \Sigma_{t+1|t}$，使

$$s_{t+1} \mid y_1, \dots, y_t \sim \mathcal{N}(s_{t+1|t}, \Sigma_{t+1|t}),$$

可以证明

$$s_{t+1} \mid y_1, \dots, y_{t+1} \sim \mathcal{N}(s_{t+1|t+1}, \Sigma_{t+1|t+1}),$$

其中

$$\begin{cases} s_{t+1|t+1} = s_{t+1|t} + K_t(y_{t+1} - C s_{t+1|t}), \\ \Sigma_{t+1|t+1} = \Sigma_{t+1|t} - K_t \cdot C \cdot \Sigma_{t+1|t}, \end{cases}$$

且

$$K_t := \Sigma_{t+1|t} C^\top (C \Sigma_{t+1|t} C^\top + \Sigma_y)^{-1}.$$

矩阵 K_t 称为**卡尔曼增益**。

仔细观察公式会发现，不需要保留时刻 t 之前的观测！更新步骤只依赖此前的分布。综合起来，算法先前向计算 $K_t, \Sigma_{t|t}, s_{t|t}$（文献中有时把后者写为 $\hat{s}$），再通过后向计算，即 LQR 更新，求得 Φ_t, Ψ_t, L_t，最后恢复最优策略 $a_t^* = L_t s_{t|t}$。

第 21 章　策略梯度及其变体

原文印刷页码：258–265。

21.1 REINFORCE

我们将介绍一种称为 REINFORCE 的无模型算法，它不需要价值函数和 Q 函数的概念。在有限时域情形下介绍 REINFORCE 更方便，本章始终采用这一假设：用 $\tau = (s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T)$ 表示一条轨迹，其中 $T < \infty$ 是轨迹长度。此外，REINFORCE 仅适用于学习随机策略。用 $\pi_\theta(a \mid s)$ 表示策略 π_θ 在状态 s 输出动作 a 的概率。其他记号与此前讲义相同。

REINFORCE 的优点是：只需要假设能够从转移概率 $\{P_{sa}\}$ 中采样，并能在状态 s、动作 a 处查询奖励函数 $R(s,a)$，[81] 不需要知道转移概率或奖励函数的解析形式，也不显式学习它们。

设 s_0 从分布 μ 中采样。考虑对参数 θ 优化策略 π_θ 的期望总收益，定义为

$$\eta(\theta) \triangleq \mathbb{E}\left[\sum_{t=0}^{T-1} \gamma^t R(s_t, a_t)\right]. \tag{21.1}$$

回忆 $s_t \sim P_{s_{t-1}a_{t-1}}$，$a_t \sim \pi_\theta(\cdot \mid s_t)$。如果忽略有限时域与无限时域的区别，也有 $\eta(\theta) = \mathbb{E}_{s_0 \sim \mu}[V^{\pi_\theta}(s_0)]$。

我们希望用梯度上升最大化 $\eta(\theta)$。主要挑战在于：不知道奖励函数和转移概率的形式时，如何计算或估计 $\eta(\theta)$ 的梯度。

令 $P_\theta(\tau)$ 表示策略 π_θ 生成的轨迹 τ 的分布，并令 $f(\tau) = \sum_{t=0}^{T-1} \gamma^t R(s_t, a_t)$。于是

$$\eta(\theta) = \mathbb{E}_{\tau \sim P_\theta}[f(\tau)]. \tag{21.2}$$

此前介绍变分自编码器（VAE）时遇到过类似情形：需要对出现在期望内部的变量求梯度，因为分布 P_θ 依赖 θ。回忆 VAE 使用重参数化技巧解决这一问题。然而这里不能使用，因为我们不知道如何计算 f 的梯度。（虽然可以对观测到的奖励加权求和，高效计算 f 的值，但未必知道奖励函数本身，因而无法求梯度。）

REINFORCE 用另一种方式估计 $\eta(\theta)$ 的梯度。先作如下推导：

[81] 本章采用奖励同时依赖状态和动作的一般情形。

$$\begin{aligned}\nabla_\theta \mathbb{E}_{\tau\sim P_\theta}[f(\tau)] &= \nabla_\theta \int P_\theta(\tau) f(\tau)\, d\tau \\ &= \int \nabla_\theta (P_\theta(\tau) f(\tau))\, d\tau && \text{（交换积分与梯度）} \\ &= \int (\nabla_\theta P_\theta(\tau)) f(\tau)\, d\tau && \text{（}f\text{ 不依赖 }\theta\text{）} \\ &= \int P_\theta(\tau)(\nabla_\theta \log P_\theta(\tau)) f(\tau)\, d\tau \\ &= \mathbb{E}_{\tau\sim P_\theta}[(\nabla_\theta \log P_\theta(\tau)) f(\tau)].\end{aligned} \tag{21.3}$$

倒数第二步使用 $\nabla \log P_\theta(\tau) = \nabla P_\theta(\tau)/P_\theta(\tau)$。

现在得到了 $\nabla_\theta \mathbb{E}_{\tau\sim P_\theta}[f(\tau)]$ 的基于样本的估计量。设 $\tau^{(1)}, \ldots, \tau^{(n)}$ 是来自 P_θ 的 n 个经验样本，即运行策略 π_θ 共 n 次，每次运行 T 步。可估计梯度为

$$\nabla_\theta \mathbb{E}_{\tau\sim P_\theta}[f(\tau)] = \mathbb{E}_{\tau\sim P_\theta}[(\nabla_\theta \log P_\theta(\tau)) f(\tau)] \tag{21.4}$$

$$\approx \frac{1}{n}\sum_{i=1}^{n} (\nabla_\theta \log P_\theta(\tau^{(i)})) f(\tau^{(i)}). \tag{21.5}$$

下一个问题是如何计算 $\log P_\theta(\tau)$。我们推导它的解析式，再用自动微分求它关于 θ 的梯度。根据轨迹定义，

$$P_\theta(\tau) = \mu(s_0)\pi_\theta(a_0 \mid s_0) P_{s_0 a_0}(s_1) \pi_\theta(a_1 \mid s_1) P_{s_1 a_1}(s_2) \cdots P_{s_{T-1} a_{T-1}}(s_T). \tag{21.6}$$

这里 μ 表示 s_0 的分布密度。因此

$$\begin{aligned}\log P_\theta(\tau) = \log \mu(s_0) &+ \log \pi_\theta(a_0 \mid s_0) + \log P_{s_0 a_0}(s_1) + \log \pi_\theta(a_1 \mid s_1) \\ &+ \log P_{s_1 a_1}(s_2) + \cdots + \log P_{s_{T-1} a_{T-1}}(s_T).\end{aligned} \tag{21.7}$$

对 θ 求梯度，得到

$$\nabla_\theta \log P_\theta(\tau) = \nabla_\theta \log \pi_\theta(a_0 \mid s_0) + \nabla_\theta \log \pi_\theta(a_1 \mid s_1) + \cdots + \nabla_\theta \log \pi_\theta(a_{T-1} \mid s_{T-1}).$$

很多项因不依赖 θ 而消失，其梯度为 0。（这一点很重要：我们无法访问转移概率，因此不知道如何计算 $\log P_{s_0 a_0}(s_1)$ 等项，但幸好它们的梯度为 0 !）

代入式（21.4），得到

$$\begin{aligned}\nabla_\theta \eta(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim P_\theta}[f(\tau)] \\ &= \mathbb{E}_{\tau\sim P_\theta}\left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right) f(\tau)\right] \\ &= \mathbb{E}_{\tau\sim P_\theta}\left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right)\left(\sum_{t=0}^{T-1} \gamma^t R(s_t, a_t)\right)\right].\end{aligned} \tag{21.8}$$

用经验轨迹样本估计右侧，可得到无偏估计。基本 REINFORCE 算法用估计出的梯度进行梯度上升，迭代更新参数。

策略梯度公式（21.8）的解释。 $\nabla_\theta \log P_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$ 是轨迹的得分函数。它指向局部增大轨迹 τ 的对数概率、从而增大其概率的参数变化方向（也就是增大沿该轨迹选择 $a_0, \dots, a_{T-1}$ 的概率）。标量 $f(\tau)$ 是这条轨迹的总收益。因此，直观上，一次梯度更新试图提高采样轨迹的可能性，但对各条 τ 给予不同的侧重或权重。如果 τ 的奖励很高，则 $f(\tau)$ 很大，更新会强烈强化增大该轨迹概率的方向；如果收益较低，同一方向得到的权重就较小。

式（21.3）蕴含一个有趣事实：

$$\mathbb{E}_{\tau\sim P_\theta}\left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right] = 0. \tag{21.9}$$

为看出这一点，取 $f(\tau) = 1$（即奖励始终为常数），式（21.8）左侧为 0，因为收益始终是固定常数 $\sum_{t=0}^{T} \gamma^t$。所以右侧也为 0，从而得到式（21.9）。

事实上，对任何固定的 t, s_t，都可验证 $\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)} \nabla_\theta \log \pi_\theta(a_t \mid s_t) = 0$。[82] 这有两个推论。第一，可以简化式（21.8）：

$$\begin{aligned}\nabla_\theta \eta(\theta) &= \sum_{t=0}^{T-1} \mathbb{E}_{\tau\sim P_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\left(\sum_{j=0}^{T-1} \gamma^j R(s_j, a_j)\right)\right] \\ &= \sum_{t=0}^{T-1} \mathbb{E}_{\tau\sim P_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\left(\sum_{j=t}^{T-1} \gamma^j R(s_j, a_j)\right)\right].\end{aligned} \tag{21.10}$$

第二个等号来自

$$\begin{aligned}&\mathbb{E}_{\tau\sim P_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{0\le j<t} \gamma^j R(s_j, a_j)\right] \\ &= \mathbb{E}\left[\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \mid s_0, a_0, \dots, s_{t-1}, a_{t-1}, s_t] \sum_{0\le j<t} \gamma^j R(s_j, a_j)\right] \\ &= 0,\end{aligned}$$

因为 $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \mid s_0, a_0, \dots, s_{t-1}, a_{t-1}, s_t] = 0$。这里使用了全期望定律。第二行的外层期望针对 $s_0, a_0, \dots, a_{t-1}, s_t$ 的随机性，内层期望则针对给定这些变量后的 a_t 的随机性。这样估计量略微简化了。

上述零期望性质的第二个推论是：对任何只依赖 s_t 的值 $B(s_t)$，都有

$$\begin{aligned}&\mathbb{E}_{\tau\sim P_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) B(s_t)] \\ &= \mathbb{E}[\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \mid s_0, a_0, \dots, s_{t-1}, a_{t-1}, s_t] B(s_t)] \\ &= 0,\end{aligned}$$

因为内层条件期望为 0。这里又使用全期望定律：外层期望针对 $s_0, a_0, \dots, a_{t-1}, s_t$ 的随机性，内层期望针对给定这些变量后的 a_t 的随机性。由此和式（21.10）可得

[82] 更一般地，有 $\mathbb{E}_{x\sim p_\theta}[\nabla \log p_\theta(x)] = 0$。

$$\begin{aligned}\nabla_\theta \eta(\theta) &= \sum_{t=0}^{T-1} \mathbb{E}_{\tau\sim P_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\left(\sum_{j=t}^{T-1}\gamma^j R(s_j,a_j) - \gamma^t B(s_t)\right)\right] \\ &= \sum_{t=0}^{T-1} \mathbb{E}_{\tau\sim P_\theta}\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\gamma^t\left(\sum_{j=t}^{T-1}\gamma^{j-t} R(s_j,a_j) - B(s_t)\right)\right].\end{aligned} \tag{21.11}$$

因此，不同的 $B(\cdot)$ 会给出不同的 $\nabla\eta(\theta)$ 估计量。引入合适的 $B(\cdot)$——通常称为基线——可以降低估计量的方差。[83] 一个接近最优的选择是期望未来收益 $\mathbb{E}[\sum_{j=t}^{T-1}\gamma^{j-t}R(s_j,a_j) \mid s_t]$；若忽略有限与无限时域的差别，它基本就是价值函数 $V^{\pi_\theta}(s_t)$。这里可以粗略估计 $V^{\pi_\theta}(\cdot)$，因为其精确值不影响估计量的均值，只影响方差。由此得到算法 7 所述带基线的策略梯度算法。[84]

算法 7：带基线的基本策略梯度

对 $i = 1, 2, \ldots$ 重复：

1. 执行当前策略，收集一组轨迹。用 $R_{\geq t}$ 简记 $\sum_{j=t}^{T-1}\gamma^{j-t}R(s_j,a_j)$。
2. 寻找使下式最小的函数 B，以拟合基线：

$$\sum_\tau \sum_t (R_{\geq t} - B(s_t))^2. \tag{21.12}$$

3. 用以下梯度估计量更新策略参数 θ：

$$\sum_\tau \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)(R_{\geq t} - B(s_t)). \tag{21.13}$$

21.2 PPO

上面的基本策略梯度算法是同策略算法：梯度估计所用的轨迹，来自即将更新的同一个策略。这可能导致数据效率低下，因为每采样一批轨迹，实际上只能做一次梯度更新；更新之后，这批轨迹就来自旧策略。轨迹采样通常很昂贵，在采样和梯度更新之间切换也很昂贵，所以我们希望从固定的一组采样轨迹中学习更多，即进行多次更新。

近端策略优化（Proximal Policy Optimization，PPO）[Schulman et al., 2017] 是基本策略梯度的改进版，允许一定程度的异策略更新，即重复使用近期旧策略采样的轨迹进行若干梯度步骤。

首先，从原始的有限时域奖励目标开始：

$$\eta(\theta) = \mathbb{E}_{\tau\sim P_\theta}\left[\sum_{t=0}^{T-1}\gamma^t R(s_t,a_t)\right]. \tag{21.14}$$

为便于分别理解 PPO 中的思想，可以先忽略基线和优势，只关注如何复用旧策略采样的数据。令 θ_{old} 为生成采样轨迹的参数。对一般采样轨迹 $\tau = (s_0, a_0, \ldots, s_{T-1}, a_{T-1}, s_T)$，定义后续回报 $R_{\geq t} = \sum_{j=t}^{T-1}\gamma^{j-t}R(s_j,a_j)$。

[83] 一个有启发性的例子是：固定 t，未来奖励 $\sum_{j=t}^{T-1}\gamma^{j-t}R(s_j,a_j)$ 等概率取 $1000+1$ 和 $1000-2$，对应的 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ 为向量 z 和 $-z$。（因为它的期望为 0，若只能等概率取两个值，两者必然互为相反向量。）不减基线时，估计量取 $(1000+1)z$ 和 $-(1000-2)z$；减去基线 1000 后，则取 z 和 $2z$。后者的方差远小于原估计量。

[84] 算法中的梯度估计量与式（21.11）不完全一致。如果给式（21.13）的求和项乘上 γ^t，两者就完全一致。经验上，去掉这个折扣因子效果不错，因为它给出了较大的更新。

假设已经做了若干次梯度更新，到达与 θ_{old} 相距不远的策略参数 θ。我们想用 $\pi_{\theta_{\text{old}}}$ 采样的轨迹估计 π_θ 的更新。在策略梯度估计量中，每个时间步贡献一个后续回报项。如果状态 s_t 固定，那么把动作分布从 $\pi_{\theta_{\text{old}}}$ 改为 π_θ 所需的基本修正就是似然比

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}. \tag{21.15}$$

令 $d_{\theta_{\text{old}},t}$ 表示旧策略下 s_t 的分布。相应的重要性加权局部替代目标为

$$J_{\text{IS}}(\theta) = \sum_{t=0}^{T-1} \mathbb{E}_{s_t \sim d_{\theta_{\text{old}},t}} \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}(\cdot|s_t)}[r_t(\theta) R_{\geq t}]. \tag{21.16}$$

这里外层期望仍针对旧策略访问的状态，而似然比把内层动作期望从 $\pi_{\theta_{\text{old}}}$ 转换为 π_θ。因此应把它理解为局部替代目标：它修正采样动作，却仍使用旧策略产生的状态分布和轨迹展开。

简单的重要性采样梯度算法会在 $\pi_{\theta_{\text{old}}}$ 下收集轨迹，计算后续回报，然后对局部目标（21.16）执行多次梯度更新。PPO 在此基础上增加两个变化。第一，从原始后续回报中减去价值函数基线，得到方差较小的优势估计，并如式（18.2）那样加入 KL 散度正则项，惩罚偏离旧策略。第二，裁剪似然比：如果 θ 在某个采样动作上偏离旧策略过远，更新就不再持续增加沿该方向进一步偏移的激励。

回忆，从后续回报中减去依赖状态的基线，不会改变期望策略梯度方向，却可降低估计量方差。特别地，可减去旧策略下的近似价值函数基线，使用优势估计

$$\widehat{A}_t = R_{\geq t} - V_{\text{old}}(s_t).$$

由此得到基于优势的损失

$$J_{\text{adv}}(\theta) = -\sum_{t=0}^{T-1} \mathbb{E}_{s_t \sim d_{\theta_{\text{old}},t}} \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}(\cdot|s_t)}[r_t(\theta) \widehat{A}_t], \tag{21.17}$$

它与式（21.16）的重要性加权结构相同，只是把 $R_{\geq t}$ 替换为 $\widehat{A}_t$。随后 PPO 将逐时间步的贡献 $r_t(\theta)\widehat{A}_t$ 替换为裁剪形式

$$C_t(\theta) = \min\left\{ r_t(\theta)\widehat{A}_t, \text{clip}_{[1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}]}(r_t(\theta))\widehat{A}_t \right\}. \tag{21.18}$$

PPO 的替代损失为

$$J_{\text{PPO}}(\theta) = -\sum_{t=0}^{T-1} \mathbb{E}_{s_t \sim d_{\theta_{\text{old}},t}} \mathbb{E}_{a_t \sim \pi_{\theta_{\text{old}}}(\cdot|s_t)}[C_t(\theta)]. \tag{21.19}$$

可以根据优势的符号理解这里的最小值运算。如果 $\widehat{A}_t > 0$，采样动作带来的后续回报优于基线，更新希望增加该动作的概率。当 $r_t(\theta) \leq 1 + \epsilon_{\text{clip}}$ 时，PPO 使用普通贡献 $r_t(\theta)\widehat{A}_t$；当 $r_t(\theta) > 1 + \epsilon_{\text{clip}}$ 时，裁剪项把增益限制在 $(1 + \epsilon_{\text{clip}})\widehat{A}_t$，目标不再奖励继续增加该动作概率。这是合理的，因为当前策略下该动作的概率已显著高于旧策略下的概率，没必要继续增加。

另一方面，如果 $\widehat{A}_t < 0$，采样动作带来的后续回报差于基线，更新希望减小其概率。当 $r_t(\theta) \geq 1 - \epsilon_{\text{clip}}$

时，PPO 仍用普通贡献；当 $r_t(\theta) < 1 - \epsilon_{\text{clip}}$ 时，裁剪项限制了继续降低这一不良动作概率所带来的改进。同样，这能避免沿该方向不必要的过度更新，因为该动作概率已经低于旧策略下的概率。我们对 θ 最小化这一替代损失。裁剪项阻止目标继续奖励那些已经让动作相较于旧策略变得可能得多或不可能得多的变化。

最后，在很多实际情形中，会用广义优势估计（GAE）[Schulman et al., 2016] 改善优势估计。在第 18.2 节的 RLVR 场景中，优势估计经常进一步简化，例如使用群组相对基线或平均奖励基线，而不是学习得到的价值函数。

附录 A　高斯分布与 KL 散度的基本结论

原文印刷页码：266–268。

本附录汇集前面推导所用到的一些基本事实。

A.1 高斯分布与 KL 散度的基本恒等式

引理 A.1.1（独立高斯变量的线性组合）。设 $\epsilon_1, \dots, \epsilon_t$ 是 $\mathbb{R}^d$ 中相互独立的标准高斯随机向量，$a_1, \dots, a_t$ 为实数。则

$$\sum_{s=1}^{t} a_s \epsilon_s \sim \mathcal{N}\left(0, \left(\sum_{s=1}^{t} a_s^2\right) I\right). \tag{A.1}$$

证明。这个和是联合高斯随机向量的线性组合，因此仍服从高斯分布。其均值为零，协方差为

$$\operatorname{Cov}\left(\sum_{s=1}^{t} a_s \epsilon_s\right) = \sum_{s=1}^{t} a_s^2 \operatorname{Cov}(\epsilon_s) = \left(\sum_{s=1}^{t} a_s^2\right) I, \tag{A.2}$$

其中独立性消去了交叉协方差项。

引理 A.1.2（联合高斯分布的条件分布）。假设

$$\begin{bmatrix} A \\ B \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_A \\ \mu_B \end{bmatrix}, \begin{bmatrix} \Sigma_{AA} & \Sigma_{AB} \\ \Sigma_{BA} & \Sigma_{BB} \end{bmatrix}\right), \tag{A.3}$$

且 Σ_{BB} 可逆。那么

$$A \mid B = b \sim \mathcal{N}\big(\mu_A + \Sigma_{AB}\Sigma_{BB}^{-1}(b - \mu_B),\ \Sigma_{AA} - \Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BA}\big). \tag{A.4}$$

类似地，如果 Σ_{AA} 可逆，则

$$B \mid A = a \sim \mathcal{N}\big(\mu_B + \Sigma_{BA}\Sigma_{AA}^{-1}(a - \mu_A),\ \Sigma_{BB} - \Sigma_{BA}\Sigma_{AA}^{-1}\Sigma_{AB}\big). \tag{A.5}$$

证明。令

$$R_A = A - \mu_A - \Sigma_{AB}\Sigma_{BB}^{-1}(B - \mu_B). \tag{A.6}$$

因为 (A,B) 联合服从高斯分布，(R_A,B) 也联合服从高斯分布。并且

$$\mathrm{Cov}(R_A,B)=\Sigma_{AB}-\Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BB}=0. \tag{A.7}$$

对于联合高斯随机变量，协方差为零意味着相互独立，所以 R_A 与 B 独立。R_A 的均值为零，协方差为

$$\mathrm{Cov}(R_A)=\Sigma_{AA}-\Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BA}. \tag{A.8}$$

因此，对 $B=b$ 取条件，只会把 A 的仿射表示中的 B 换成 b，从而得到上面 $A\mid B=b$ 的公式。交换 A 与 B 的角色，用同样的论证即可得到 $B\mid A=a$ 的公式。

引理 A.1.3（具有相同协方差的高斯分布之间的 KL 散度）。若 $P=\mathcal{N}(m_1,\Sigma)$、$Q=\mathcal{N}(m_2,\Sigma)$ 具有相同的正定协方差矩阵 Σ，则

$$\mathrm{KL}(P\|Q)=\frac{1}{2}(m_1-m_2)^T\Sigma^{-1}(m_1-m_2). \tag{A.9}$$

证明。写出两个高斯分布的对数密度并相减，归一化常数以及关于 x 的二次项相互抵消，只留下由均值差异产生的项。在 P 下取期望，得到

$$\begin{aligned}\mathbb{E}_P\left[\log\frac{dP}{dQ}\right]=\frac{1}{2}\mathbb{E}_P\big[&(X-m_2)^T\Sigma^{-1}(X-m_2)\\&-(X-m_1)^T\Sigma^{-1}(X-m_1)\big].\end{aligned} \tag{A.10}$$

由 $\mathbb{E}_P[X]=m_1$，即可化简为所述结论。

引理 A.1.4（KL 散度的链式法则）。假设 $q(x,y)=q(x)q(y\mid x)$、$p(x,y)=p(x)p(y\mid x)$。则

$$\begin{aligned}\mathrm{KL}(q(x,y)\|p(x,y))=&\mathrm{KL}(q(x)\|p(x))\\&+\mathbb{E}_{q(x)}\big[\mathrm{KL}(q(y\mid x)\|p(y\mid x))\big].\end{aligned} \tag{A.11}$$

更一般地，如果

$$\begin{aligned}q(x_{1:T})&=q(x_T)\prod_{t=2}^{T}q(x_{t-1}\mid x_{t:T}),\\p(x_{1:T})&=p(x_T)\prod_{t=2}^{T}p(x_{t-1}\mid x_{t:T}),\end{aligned} \tag{A.12}$$

那么

$$\begin{aligned}\mathrm{KL}(q(x_{1:T})\|p(x_{1:T}))=&\mathrm{KL}(q(x_T)\|p(x_T))\\&+\sum_{t=2}^{T}\mathbb{E}_q\big[\mathrm{KL}\big(q(x_{t-1}\mid x_{t:T})\|p(x_{t-1}\mid x_{t:T})\big)\big].\end{aligned} \tag{A.13}$$

证明。把分解式代入对数似然比：

$$\mathrm{KL}(q(x,y)\|p(x,y)) = \mathbb{E}_q\left[\log \frac{q(x)q(y \mid x)}{p(x)p(y \mid x)}\right] \tag{A.14}$$

$$= \mathbb{E}_q\left[\log \frac{q(x)}{p(x)}\right] + \mathbb{E}_q\left[\log \frac{q(y \mid x)}{p(y \mid x)}\right], \tag{A.15}$$

这正是要证明的恒等式。

参考文献

原文印刷页码：269–277。为保留可检索的作者、论文题名、刊物和链接信息，本节沿用原文书目。

参考文献原文 PDF

Bibliography

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

Brian D. O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Science*, 116(32), 2019.

Mikhail Belkin, Daniel Hsu, and Ji Xu. Two models of double descent for weak features. *SIAM Journal on Mathematics of Data Science*, 2(4):1167–1180, 2020.

David M Blei, Alp Kucukelbir, and Jon D McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Kohd, Mark Krass, Ranjay Krishna, Rohith Kuditipudi, Ananya

Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avanika Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighbor search. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL `https://proceedings.neurips.cc/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html`.

Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1597–1607. PMLR, PMLR, 13–18 Jul 2020.

Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022. URL https://arxiv.org/abs/2205.14135.

Yann N Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. *arXiv preprint arXiv:1612.08083*, 2016.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Association for Computational Linguistics (ACL)*, pages 4171–4186, 2019.

Jeff Z HaoChen, Colin Wei, Jason D Lee, and Tengyu Ma. Shape matters: Understanding the implicit bias of the noise covariance. *arXiv preprint arXiv:2006.08680*, 2020.

Trevor Hastie, Andrea Montanari, Saharon Rosset, and Ryan J Tibshirani. Surprises in high-dimensional ridgeless least squares interpolation. 2019.

Trevor Hastie, Andrea Montanari, Saharon Rosset, and Ryan J Tibshirani. Surprises in high-dimensional ridgeless least squares interpolation. *The Annals of Statistics*, 50(2):949–986, 2022.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4246–4253, 2020.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=nZeVKeeFYf9`.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 448–456. JMLR.org, 2015. URL `http://jmlr.org/proceedings/papers/v37/ioffe15.html`.

Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An introduction to statistical learning, second edition*, volume 112. Springer, 2021.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213, 2022. URL `https://arxiv.org/abs/2205.11916`.

Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. *arXiv preprint arXiv:2202.10054*, 2022.

Zhimin Li, Jianwei Zhang, Qin Lin, Jiangfeng Xiong, Yanxin Long, Xinchi Deng, Yingfang Zhang, Xingchao Liu, Minbin Huang, Zedong Xiao, et al. Hunyuan-dit: A powerful multi-resolution diffusion transformer with fine-grained chinese understanding. *arXiv preprint arXiv:2405.08748*, 2024.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025. URL `https://arxiv.org/abs/2503.20783`.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Yuping Luo, Huazhe Xu, Yuanzhi Li, Yuandong Tian, Trevor Darrell, and Tengyu Ma. Algorithmic framework for model-based deep reinforcement learning with theoretical guarantees. *arXiv preprint arXiv:1807.03858*, 2018.

Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. URL `https://arxiv.org/abs/1603.09320`.

Vincent Mazet. Convolution. Basics of Image Processing, 2026. Online; accessed April 22, 2026. `https://vincmazet.github.io/bip/filtering/convolution.html`.

Song Mei and Andrea Montanari. The generalization error of random features regression: Precise asymptotics and the double descent curve. *Communications on Pure and Applied Mathematics*, 75(4):667–766, 2022.

Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11048–11064, 2022.

MiniMax. MiniMax-M1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025. URL `https://arxiv.org/abs/2506.13585`.

Preetum Nakkiran. More data can hurt for linear regression: Sample-wise double descent. 2019.

Preetum Nakkiran, Prayaag Venkat, Sham Kakade, and Tengyu Ma. Optimal regularization can mitigate double descent. *arXiv preprint arXiv:2003.01897*, 2020.

Huy Nguyen, Thong T. Doan, Quang Pham, Nghi D. Q. Bui, Nhat Ho, and Alessandro Rinaldo. On deepseekmoe: Statistical benefits of shared experts and normalized sigmoid gating. *arXiv preprint arXiv:2505.10860*, 2025.

Alex Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8162–8171, 2021.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021. URL `https://arxiv.org/abs/2112.00114`.

OLMo Team, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*, 2024.

OpenAI. Learning to reason with LLMs. `https://openai.com/index/learning-to-reason-with-llms/`, September 2024.

OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL `https://arxiv.org/abs/2508.10925`.

Manfred Opper. Statistical mechanics of learning: Generalization. *The Handbook of Brain Theory and Neural Networks,*, pages 922–925, 1995.

Manfred Opper. Learning to generalize. *Frontiers of Life*, 3(part 2):763–775, 2001.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022. URL `https://arxiv.org/abs/2203.02155`.

Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. *arXiv preprint arXiv:2505.06708*, 2025.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL `https://arxiv.org/abs/2305.18290`.

Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897, 2015.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016. URL `https://arxiv.org/abs/1506.02438`.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Association for Computational Linguistics (ACL)*, 2016.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL `https://arxiv.org/abs/2402.03300`.

Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

David R So, Wojciech Mańke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V Le. Primer: Searching for efficient transformers for language modeling. In *Advances in Neural Information Processing Systems*, 2021.

Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics.

In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2256–2265, 2015.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021, 2020. URL `https://arxiv.org/abs/2009.01325`.

Mingxing Tan and Quoc V Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, pages 6105–6114. PMLR, 2019.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL `https://arxiv.org/abs/2203.11171`.

Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. *arXiv*, 2021.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

Blake Woodworth, Suriya Gunasekar, Jason D Lee, Edward Moroshko, Pedro Savarese, Itay Golan, Daniel Soudry, and Nathan Srebro. Kernel and rich regimes in overparametrized models. *arXiv preprint arXiv:2002.09277*, 2020.

Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Emine Yilmaz, Evangelos Kanoulas, and Javed A Aslam. A simple and efficient sampling method for estimating AP and NDCG. In *ACM Special Interest Group on Information Retreival (SIGIR)*, pages 603–610, 2008.

Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models in the wild. In *Conference on Language Modeling*, 2025. URL `https://arxiv.org/abs/2503.18892`.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL `https://proceedings.neurips.cc/paper_files/paper/2019/file/1e8a19426224ca89e83cef47f1e7f53b-Paper.pdf`.

Zhengyan Zhang, Yixin Song, Guanghui Yu, Xu Han, Yankai Lin, Chaojun Xiao, Chenyang Song, Zhiyuan Liu, Zeyu Mi, and Maosong Sun. Relu^2 wins: Discovering efficient activation functions for sparse llms. *arXiv preprint arXiv:2402.03804*, 2024.

Denny Zhou, Nathanael Scharli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023. URL `https://arxiv.org/abs/2205.10625`.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019. URL `https://arxiv.org/abs/1909.08593`.